

An Honest Benchmark for Glioma Survival Prediction: Quantifying Data Leakage on the MU-Glioma-Post Dataset

Joshua Alliet

Supervisor: Dr. Fumie Costen

School of Engineering, Department of Computer Science
The University of Manchester

2026

Abstract

Gliomas are the most common primary malignant brain tumours in adults. Survival outcomes range from months to over a decade depending on molecular subtype. Accurate prediction at diagnosis informs surgical and treatment decisions. Machine learning on clinical and molecular data can improve prognostication, but many published studies inflate their metrics through data leakage, where future events slip into the training set.

This project builds a survival prediction pipeline on the MU-Glioma-Post dataset ($n = 203$) from The Cancer Imaging Archive. The primary contribution is a four-step leakage detection methodology combining correlation analysis ($|r| > 0.25$ threshold) with temporal reasoning. It flagged seven progression-related features unavailable at diagnosis. After their removal, XGBoost classification dropped from $\text{AUC } 0.730 \pm 0.091$ to 0.674 ± 0.111 , a 7.7% relative reduction that quantifies the cost of honest evaluation.

Across five binary classifiers and three survival models on clean feature sets, logistic regression achieved $\text{AUC } 0.692 \pm 0.119$ and Random Survival Forest a concordance index of 0.706 ± 0.069 . IDH1 mutation status carried 19.4% of binary classification importance, matching its definitional role in the 2021 WHO classification. Imaging features failed to improve prediction beyond clinical and molecular markers (pre-extracted: $\Delta\text{AUC} = -0.002$, $p = 0.936$; PyRadiomics: $\text{AUC } 0.598 \pm 0.151$ in isolation). Tumour volume change prediction ($n = 105$) yielded only negative R^2 values, and a seven-axis treatment-exposure check ruled out confounding by indication.

Achievable performance on this cohort sits at $\text{AUC } 0.67$ to 0.72 , matching the EORTC nomogram C-index of 0.66 . The work delivers a leakage-aware benchmark for tabular glioma prediction and a generalisable detection methodology for medical machine learning.

Contents

1	Introduction	8
1.1	Motivation	8
1.2	Aims and Objectives	9
1.2.1	Aims	9
1.2.2	Objectives	9
1.3	Report Structure	10
2	Background	11
2.1	Glioma Biology and Clinical Context	11
2.1.1	Classification and Grading	11
2.1.2	Molecular Markers	12
2.1.3	Prognostic Factors	13
2.1.4	Clinical Endpoints	14
2.2	Machine Learning Fundamentals	14
2.2.1	Supervised Learning Pipeline	14
2.2.2	Classification Models	14
2.2.3	Survival Analysis	15
2.2.4	Evaluation Metrics	17
2.2.5	Feature Importance	17
2.3	Data Leakage in Medical Machine Learning	18
2.3.1	Foundations	18
2.3.2	Leakage Taxonomy	18
2.3.3	Detection Methods	19
2.3.4	Consequences of Undetected Leakage	19
3	Previous Work	20
3.1	Classical Prognostic Models	20
3.1.1	Recursive Partitioning Analysis	20
3.1.2	EORTC Nomograms	20
3.2	Machine Learning for Glioma Survival Prediction	21
3.2.1	Systematic Review Evidence	21
3.2.2	Representative Machine Learning Studies	22
3.3	Data Leakage in Medical Machine Learning	22
3.3.1	Empirical Evidence of Leakage Impact	23
3.4	Radiomics and Imaging-Based Features	24
3.4.1	The Radiomics Paradigm	24
3.4.2	Radiomics in Glioma Survival Prediction	25
3.5	Tumour Growth Modelling and Progression Prediction	25
3.6	Research Gaps	25

4	Research Methods	26
4.1	Dataset	26
4.1.1	MU-Glioma-Post Dataset	26
4.1.2	Data Modalities	26
4.1.3	Target Variables	27
4.2	Preprocessing	27
4.2.1	Data Cleaning	27
4.3	Exploratory Data Analysis	27
4.4	Model Architectures	31
4.4.1	Binary Classification Models	31
4.4.2	Survival Analysis Models	31
4.4.3	Hyperparameter Optimisation	32
4.5	Radiomics Analysis	32
4.6	Tumour Growth Prediction	32
4.6.1	Target Construction	32
4.6.2	Feature Sets and Models	33
4.7	Evaluation Protocol	33
4.7.1	Cross-Validation Strategy	33
4.7.2	Metrics	33
4.8	Software and Implementation	34
5	Leakage Detection	35
5.1	Detection Methodology	35
5.2	Detection Results	37
5.2.1	Correlation Analysis	37
5.2.2	Identified Leakage Features	37
5.2.3	Impact on Feature Count	38
5.3	Impact Quantification	38
6	Prediction Modelling	40
6.1	Binary Classification	40
6.1.1	Models	40
6.1.2	Results	40
6.2	Survival Analysis	41
6.2.1	Models	41
6.2.2	Results	43
6.3	Feature Importance	44
6.3.1	Binary Classification Feature Importance	44
6.3.2	Survival Analysis Feature Importance	47
6.3.3	Biological Interpretation	47
6.4	Radiomics Analysis	48
6.4.1	Pre-Extracted Imaging Features	48
6.4.2	Comprehensive Radiomic Feature Extraction	50
6.5	Tumour Growth Prediction	51
6.5.1	Dataset Characteristics	51
6.5.2	Leakage Re-Detection	53
6.5.3	Model Performance	53
6.5.4	Feature Importance	55
6.5.5	Mortality by Growth Tertile	55
6.6	Summary of Results	58

7	Discussion	59
7.1	Discussion and Appraisal	59
7.2	Evaluation of Objectives	61
7.3	Limitations	62
8	Conclusion and Future Work	64
8.1	Conclusions	64
8.2	Future Work	64
	Appendix A Additional Mathematical Background	66
A.1	Decision Trees	66
A.2	XGBoost Gain Formula	66
A.3	Extended Molecular Marker Biology	66
	Appendix B Growth Prediction Feature Importance	67
	Appendix C Hyperparameter Search Spaces and Selected Values	68
	Appendix D MIMUC 2026 Presentation and Award	69
	Appendix E Source Code Notebooks	83
E.1	Notebook 01: Preprocessing and EDA	83
E.2	Notebook 02: Binary Classification with Leakage Detection	113
E.3	Notebook 03: Survival Analysis	154
E.4	Notebook 04: Pre-extracted Radiomics	179
E.5	Notebook 05: Tumour Growth Prediction	207
E.6	Notebook 06: Comprehensive PyRadiomics Feature Extraction	242
	Bibliography	285

List of Figures

2.1	XGBoost’s learned default directions for missing values. When a feature value is absent, the algorithm routes the instance along a default branch determined during training to minimise loss. Adapted from Chen and Guestrin [1].	15
3.1	Prognostic nomogram for median survival in newly diagnosed GBM patients. Each clinical variable is mapped to a point score on the top axis; total points are converted to median survival (months) and 2-year survival probability on the bottom axes. Reproduced from Gorlia et al. [2].	21
3.2	Survey of data leakage across 17 research fields. Each row represents a review paper; circles indicate which leakage types were identified. The breadth of affected fields demonstrates that data leakage is a systemic problem in ML-based science, not confined to any single domain. Reproduced from Kapoor and Narayanan [3].	23
3.3	Reported versus leakage-corrected results for four civil war prediction studies. After correction, complex ML models (Random Forests, AdaBoost, GBT) no longer outperform logistic regression, paralleling the pattern observed in the present work where XGBoost AUC fell from 0.730 to 0.674 after leakage removal. Reproduced from Kapoor and Narayanan [3].	24
4.1	Patient demographic characteristics including age distribution, sex distribution, racial composition, and previous brain tumour history ($n = 203$). . . .	28
4.2	Tumour characteristics including primary diagnosis distribution, WHO grade classification, and stereotactic biopsy status.	29
4.3	Molecular biomarker status across the cohort. Bars show positive, negative, and not tested counts for each marker.	30
4.4	Clinical outcome distribution showing survival status, progression rates, and a heatmap cross-tabulating progression with overall survival.	30
5.1	Temporal validity framework for healthcare ML predictions. Features from the post-outcome period (orange) constitute label leakage. For cross-sectional models (top), all features must precede the prediction timepoint; for cohort models with repeated predictions (bottom), statistical validity ends at outcome determination. This framework motivates the temporal reasoning applied in Step 3 of the detection pipeline. Adapted from Davis et al. [4].	36
5.2	Correlation analysis identifying potential leakage features. Left: feature-target correlations with the outcome variable, with red bars indicating features flagged above the $ r > 0.25$ threshold. Right: inter-correlation matrix among the most suspicious features, revealing clustering of progression-related variables separately from biological markers.	38

6.1	ROC curves comparing all five binary classification models. The dashed diagonal represents random chance ($AUC = 0.5$). The baseline model (with leakage) demonstrates visibly inflated performance relative to the clean models, which cluster together in the AUC 0.62–0.69 range.	41
6.2	Confusion matrices for all five binary classification models, aggregated across 5-fold cross-validation. The baseline model (with leakage) is shown for reference alongside the four clean models.	42
6.3	Distribution of survival times (days from diagnosis to death or last follow-up). Left: histogram with separate shading for deceased and censored patients. Right: box plot comparing survival times between outcome groups.	43
6.4	Kaplan-Meier survival curves stratified by RSF-predicted risk group (low, medium, high risk terciles). Clear separation between groups validates the discriminative capacity of the survival model.	44
6.5	Comparison of C-index scores across survival analysis methods. Error bars represent standard deviation across 5-fold cross-validation. The dashed line at $C\text{-index} = 0.5$ indicates random chance.	45
6.6	Cox Proportional Hazards model coefficients with 95% confidence intervals. Red bars indicate features that increase mortality risk; blue bars indicate protective features.	45
6.7	Feature importance scores (XGBoost gain) for the clean binary classification model across three model variants: baseline (no leakage), regularised, and feature-engineered. Molecular markers dominate the top predictive features consistently across variants.	46
6.8	Tumour volume analysis showing distribution by survival outcome (left: box plot; right: log-transformed histogram). Deceased patients showed a trend towards larger tumour volumes, but with substantial overlap between groups.	48
6.9	Feature importance in the combined clinical + imaging model. Clinical and molecular features (blue) dominate over imaging features (green), with the highest-ranked imaging feature (T2w mean intensity) appearing at rank 8.	49
6.10	Distribution of imaging-derived features stratified by survival outcome (alive vs. deceased). Overlapping distributions across all nine features confirm the limited discriminative power of pre-extracted intensity statistics.	50
6.11	Representative T1ce axial slices with tumour segmentation overlay for three patients, illustrating the range of tumour sizes and morphologies in the cohort. Segmentation masks (coloured overlay) delineate tumour subregions used as input for radiomic feature extraction.	51
6.12	Radiomic feature correlation matrix (post-QC). Shape features form a tightly correlated cluster; texture features exhibit high within-class redundancy.	52
6.13	Selected radiomic features (Elastic Net, L1 ratio = 0.7). Green bars indicate positive coefficients (associated with survival); red bars indicate negative coefficients (associated with mortality).	52
6.14	Kaplan-Meier survival curves stratified by tumour surface area (median split). Larger tumours are associated with worse prognosis (log-rank $p = 0.003$).	53
6.15	Left: distribution of tumour volume change between first and second MRI timepoints ($n = 105$). The majority of patients cluster near zero change, with long tails in both directions. Right: baseline tumour volume versus subsequent volume change, showing no clear linear relationship.	54

6.16	Cross-validated R^2 by model and feature set for tumour volume change prediction. The dashed line at $R^2 = 0$ indicates mean-prediction baseline performance. All configurations fall below this threshold; linear regression on combined features performed worst due to overfitting in the high-dimensional, small-sample regime.	55
6.17	Feature importance (gain) for the XGBoost combined model after leakage removal. All contributing features are imaging-derived; no clinical feature provided meaningful predictive signal for volume change. T2-weighted intensity standard deviation ranked highest (20.4%), reflecting the prognostic relevance of tissue heterogeneity captured by T2 signal variability.	56
6.18	Mortality rate by tumour volume change tertile. The three rates fall within sampling noise at $n = 35$ per tertile (chi-squared $p > 0.05$).	56
6.19	Balance check across growth tertiles for chemotherapy and radiation. Left: receipt at any point during the patient's trajectory. Right: treatment overlapping the TP1–TP2 scan window. The three growth groups are indistinguishable on all six measurements ($p > 0.10$).	57
6.20	Balance check across growth tertiles for surgical intervention. Left: any re-resection ($p = 0.85$). Right: first surgery inside the TP1–TP2 scan window. One patient across the 105-patient cohort had a first surgery in that window.	57
B.1	Feature importance (gain) for XGBoost combined model after leakage removal. All contributing features are imaging-derived.	67

List of Tables

2.1	Summary of molecular markers used in glioma classification and survival prediction. Frequencies reflect those reported in the literature and observed in the MU-Glioma-Post dataset.	13
6.1	Binary classification model comparison (stratified 5-fold CV). All values reported as mean $\pm$ standard deviation. The baseline model includes leakage features and is presented for reference only; it should not be considered a valid benchmark.	40
6.2	Survival analysis model comparison (5-fold CV). C-index reported as mean $\pm$ standard deviation. Individual fold scores illustrate per-fold variability. .	43
6.3	Top 10 features ranked by XGBoost importance (gain) in the clean binary classification model.	46
6.4	Comparison of feature importance rankings between Random Survival Forest (permutation importance) and Gradient Boosted Survival Analysis (gain importance).	47
6.5	Comparison of clinical-only, imaging-only, and combined (clinical + imaging) models for binary survival classification. The clinical-only model has 27 features (versus 21 in Table 6.1) because one-hot encoding of nominal variables (diagnosis, race) expands the column count. All values reported as mean $\pm$ standard deviation from 5-fold cross-validation.	48
6.6	Top 15 features in the combined model ranked by XGBoost importance. Imaging features are indicated with asterisks (*).	49
6.7	Comprehensive radiomic model comparison (stratified 5-fold CV). Radiomics-only features are the 8 selected by Elastic Net.	51
6.8	Cross-validated regression performance for tumour volume change prediction ($n = 105$, 5-fold $\times$ 5-repeat CV). All R^2 values are negative, indicating no model outperformed a mean-prediction baseline.	54
6.9	Summary of key experimental findings across all analysis components. . . .	58
7.1	Summary of achievement against project objectives.	61
C.1	XGBoost hyperparameter search space and selected values (randomised search, 50 iterations, 5-fold stratified CV).	68
C.2	Survival model hyperparameter configurations.	68

Chapter 1

Introduction

1.1 Motivation

Gliomas are the most common primary malignant brain tumours in adults, accounting for approximately 80% of all malignant central nervous system neoplasms [5]. They encompass a biologically diverse group ranging from indolent, low-grade lesions to glioblastoma (GBM), which carries a median survival of only 12–15 months despite maximal treatment [6]. Survival outcomes span from months to over a decade, and accurate prognostication at diagnosis remains clinically challenging because it informs surgical extent, adjuvant therapy selection, clinical trial eligibility, and patient counselling.

Machine learning can integrate clinical, molecular, and imaging features into unified predictive models, but reported ML performance consistently exceeds what is achievable in clinical deployment. Many published studies report AUC values above 0.85 for glioma survival prediction, well above the C-index of approximately 0.66 achieved by validated prognostic tools on large trial populations [2]. Recent systematic reviews attribute much of this inflation to data leakage: 37% of reviewed studies included features encoding treatment response or progression status unavailable at diagnosis [7, 8].

Data leakage occurs when information unavailable at the time of prediction enters the training process, producing artificially inflated performance estimates [3]. Glioma datasets are particularly susceptible because clinical databases record the complete patient trajectory from diagnosis through progression and death, creating numerous opportunities for temporal contamination. The consequences extend beyond individual studies: inflated metrics set unrealistic deployment expectations, undermine confidence in medical AI, and skew the published literature toward artificially high baselines [9].

We develop a survival prediction pipeline that incorporates leakage detection as a core methodological component. By identifying, removing, and documenting temporal leakage features, we establish the honest achievable performance for tabular clinical and molecular data on a cohort of approximately 200 patients.

Accurate survival prediction has clinical consequences at multiple decision points. Prognostic estimates inform the extent of surgical resection, where more aggressive intervention is justified for patients with favourable predicted outcomes [10]. They guide treatment stratification, as demonstrated by the Radiation Therapy Oncology Group’s recursive partitioning analysis, which grouped patients into prognostic classes to tailor radiotherapy protocols [11]. They also determine eligibility for the standard-of-care Stupp protocol of concurrent temozolomide and radiotherapy [6], where expected survival must be sufficient to justify treatment toxicity. Unreliable prognostic models risk misallocating patients to treatment pathways that are either insufficiently aggressive or unnecessarily harmful.

Glioma biology adds a further layer of difficulty. The 2021 WHO Classification of Central Nervous System Tumours made molecular markers (IDH mutation, 1p/19q codeletion, TERT promoter status) definitional criteria for tumour classification rather

than supplementary descriptors [12]. This molecular taxonomy creates a combinatorial space of prognostically distinct entities: Eckel-Passow et al. [13] identified five principal molecular groups with significantly different survival trajectories based on IDH, TERT, and 1p/19q status alone. Glioblastomas also exhibit extensive intra-tumoural heterogeneity at both the cellular and molecular levels [14], so that even within a single tumour, different regions may harbour distinct genetic profiles and treatment sensitivities. Any predictive model must contend with this multi-scale variability using features captured at a single diagnostic timepoint.

Practical constraints compound these biological difficulties. Institutional glioma cohorts typically comprise 100–500 patients, far below the sample sizes available in other ML domains. The feature space combines continuous variables (age, tumour volume), binary molecular markers, and multi-level categorical features (tumour location, histological grade), requiring careful preprocessing and encoding. Class imbalance is common, as survival outcomes depend on the follow-up duration and endpoint definition. Post-treatment oncology databases also record the full patient trajectory, creating abundant opportunities for temporal leakage that inflates apparent performance. Systematic reviews of the glioma ML literature reflect these difficulties: whilst random forest is the most commonly applied method, reported AUC values range from 0.65 to above 0.95 across studies, with much of this variance attributable to methodological differences in data handling rather than algorithmic superiority [7]. This wide performance spread motivates the leakage-aware approach adopted in this project.

1.2 Aims and Objectives

1.2.1 Aims

This project has six overarching aims:

1. Develop a clinically relevant survival prediction model for glioma patients using baseline features available at diagnosis.
2. Establish a systematic, reproducible methodology for detecting and removing data leakage in medical ML datasets.
3. Quantify the impact of data leakage on model performance.
4. Investigate the relative contributions of clinical, molecular, and imaging feature modalities to survival prediction.
5. Compare binary classification and survival analysis frameworks for glioma prognostication.
6. Investigate whether baseline features can predict tumour volume change between consecutive MRI scans.

1.2.2 Objectives

1. Characterise the MU-Glioma-Post dataset ($n = 203$) through exploratory data analysis.
2. Implement a four-step leakage detection pipeline combining correlation analysis with temporal reasoning.
3. Train and evaluate five binary classification model variants using stratified 5-fold cross-validation.

4. Implement Cox Proportional Hazards, Random Survival Forest, and Gradient Boosted Survival Analysis models.
5. Assess the marginal contribution of pre-extracted imaging features via paired statistical comparison.
6. Construct a regression framework predicting tumour volume change from baseline features.
7. Extract and compare feature importance rankings across all model types, validating alignment with established glioma biology.
8. Contextualise model performance against validated clinical tools and published ML studies.

1.3 Report Structure

Chapter 2 provides the theoretical background covering glioma biology, machine learning methods, and data leakage theory. Chapter 3 reviews relevant literature and identifies research gaps. Chapter 4 describes the dataset, preprocessing, and evaluation protocol. Chapter 5 presents the leakage detection methodology and its results. Chapter 6 details the prediction modelling experiments and their outcomes. Chapter 7 interprets the findings and discusses limitations. Chapter 8 summarises contributions and outlines future work.

Chapter 2

Background

This chapter reviews glioma biology and classification (Section 2.1), machine learning methods for survival analysis (Section 2.2), and data leakage in medical ML (Section 2.3).

2.1 Glioma Biology and Clinical Context

2.1.1 Classification and Grading

Over the past two decades, brain tumour classification has shifted from a system based on histological appearance to one that integrates molecular diagnostics as defining criteria. The 2021 World Health Organisation Classification of Tumours of the Central Nervous System (WHO CNS5) restructured glioma taxonomy around molecular markers rather than morphological features alone [12].

The WHO CNS5 classification addresses these limitations by recognising only three adult-type diffuse glioma entities [12]:

1. **Astrocytoma, IDH-mutant** (grades 2, 3, or 4): defined by the presence of an IDH1 or IDH2 mutation, typically with concurrent ATRX loss and TP53 mutation. The entity formerly known as “glioblastoma, IDH-mutant” no longer exists; such tumours are now classified as astrocytoma, IDH-mutant, grade 4.
2. **Oligodendroglioma, IDH-mutant and 1p/19q-codeleted** (grades 2 or 3): defined by the co-occurrence of IDH mutation and whole-arm codeletion of chromosomes 1p and 19q.
3. **Glioblastoma, IDH-wildtype** (grade 4 only): the most common and aggressive subtype, defined by the absence of IDH mutation. An IDH-wildtype diffuse astrocytic glioma is classified as glioblastoma if it demonstrates microvascular proliferation or necrosis histologically, or if molecular testing reveals TERT promoter mutation, EGFR gene amplification, or combined chromosome 7 gain and chromosome 10 loss.

Under WHO CNS5, molecular classification supersedes histology when the two are discordant. For example, a tumour that appears histologically as a low-grade astrocytoma but harbours CDKN2A/B homozygous deletion is automatically upgraded to grade 4, regardless of its microscopic features [12]. These molecular markers define tumour grade rather than supplement it, making them natural candidates as predictive features in machine learning models.

The grading system itself was revised to use Arabic numerals (grades 2, 3, 4) and to assign grades within tumour types rather than across them. This means that a “grade 3” astrocytoma and a “grade 3” oligodendroglioma, despite sharing a numerical grade, represent biologically distinct entities with different molecular profiles, treatment responses, and survival expectations.

2.1.2 Molecular Markers

The molecular markers that define the WHO CNS5 classification are also the strongest prognostic and predictive features in glioma survival modelling. The biology, measurement, and clinical significance of each marker relevant to the present work are described below.

IDH Mutations. Heterozygous point mutations in *IDH1* (most commonly R132H) and *IDH2* (R172 variants) were identified as defining molecular events in lower-grade gliomas and secondary glioblastomas by Yan et al. [15]. These mutations confer a neomorphic enzymatic activity that leads to genome-wide DNA hypermethylation (the glioma CpG island methylator phenotype, G-CIMP), impairing cellular differentiation. The detailed biochemistry is described in Appendix A.3.

IDH mutations are present in over 70% of WHO grade 2 and 3 diffuse gliomas but in only approximately 5% of primary glioblastomas [15]. Detection is routinely performed by immunohistochemistry (IHC) for the IDH1 R132H protein, with DNA sequencing used for IHC-negative cases to identify non-canonical IDH1 and IDH2 variants. The prognostic impact is large: Yan et al. [15] reported median survival of 65 months for IDH-mutant anaplastic astrocytomas versus 20 months for IDH-wildtype ($p < 0.001$), and 31 months versus 15 months in glioblastoma ($p = 0.002$). Given this survival gap, IDH mutation status should rank among the top predictive features in any well-specified model.

MGMT Promoter Methylation. MGMT promoter methylation epigenetically silences the *MGMT* DNA repair gene, rendering tumour cells unable to repair damage induced by temozolomide (TMZ) chemotherapy [6]. MGMT methylation is therefore both a predictive biomarker (indicating TMZ sensitivity) and a prognostic biomarker (methylated patients have better outcomes even without TMZ) [16].

Detection is typically performed via methylation-specific PCR (MSP) or pyrosequencing, with quantitative assays demonstrating high reproducibility ($R^2 = 0.94$ on retesting) [17]. MGMT methylation is found in approximately 35–45% of glioblastoma patients and is now used as a stratification factor in all GBM clinical trials involving alkylating agents. In pooled analyses of over 4,000 patients, methylated status conferred a hazard ratio of 0.35 relative to unmethylated status ($p < 0.0001$) [17]. In the present dataset, MGMT methylation was detected in 41.4% of tested patients, mirroring these population-level frequencies.

1p/19q Codeletion. Whole-arm codeletion of chromosomes 1p and 19q is the defining molecular feature of oligodendroglioma under WHO CNS5 [12]. Patients with codeleted tumours have an excellent prognosis, with median survival exceeding 10 years for grade 2 oligodendrogliomas. Codeletion is found in approximately 5–10% of all diffuse gliomas; in the present dataset, 11.3% of patients.

TERT Promoter Mutations. Telomerase reverse transcriptase (TERT) promoter mutations enable reactivation of telomerase, which maintains telomere length and confers cellular immortalisation. In gliomas, TERT promoter mutations are associated with two distinct molecular contexts: they occur in the majority of IDH-wildtype glioblastomas (where they indicate aggressive behaviour and poor prognosis) and in IDH-mutant, 1p/19q-codeleted oligodendrogliomas (where prognosis is favourable despite the mutation). Eckel-Passow et al. [13] demonstrated that molecular groups defined by combinations of 1p/19q codeletion, IDH mutation, and TERT promoter mutation status identify five clinically distinct glioma groups with markedly different survival distributions. In the present dataset, TERT promoter mutation was detected in 52.7% of cases, reflecting its high prevalence in predominantly GBM cohorts.

TP53 and ATRX. TP53 mutations and ATRX loss are primarily lineage markers rather than independent prognostic features. TP53 is mutated in approximately 80% of IDH-mutant astrocytomas [15], while ATRX loss is associated with the alternative lengthening of telomeres (ALT) pathway. Together, they support astrocytoma lineage diagnosis and are largely absent in oligodendrogliomas.

Table 2.1 summarises the molecular markers relevant to this project.

Marker	Locus	Function	Freq.	Detection	Prognostic Role
IDH1/2	2q34 / 15q26	TCA cycle enzyme	70–90% ^a	IHC, seq.	Favourable
MGMT	10q26	DNA repair	35–45% ^b	MSP, pyroseq.	Favourable (pred.)
1p/19q	Chr 1p, 19q	Tumour suppression	5–10%	FISH, array	Favourable
TERT	5p15	Telomere maintenance	50–80% ^b	Sequencing	Context-dep.
TP53	17p13	Cell cycle control	60–80% ^a	IHC, seq.	Lineage marker
ATRX	Xq21	Chromatin re-modelling	60–70% ^a	IHC	Lineage marker
CDKN2A/B	9p21	Cell cycle inhibitor	50–60% ^b	FISH, MLPA	Unfavourable
EGFR	7p12	Growth factor receptor	40–50% ^b	FISH, IHC	Unfavourable
Chr 7/10	Chr 7, 10	Multiple genes	80–90% ^b	Microarray	Unfavourable

^aAmong IDH-mutant tumours. ^bAmong IDH-wildtype GBM.

Table 2.1: Summary of molecular markers used in glioma classification and survival prediction. Frequencies reflect those reported in the literature and observed in the MU-Glioma-Post dataset.

2.1.3 Prognostic Factors

Beyond molecular markers, several clinical and treatment-related variables influence glioma survival.

Age at diagnosis is the most consistent prognostic factor across glioma studies. Older age is associated with a higher prevalence of IDH-wildtype tumours, reduced tolerance for aggressive treatment (including both surgical resection and chemoradiation), and cumulative comorbidities that affect overall survival independently of tumour biology. In the EORTC/NCIC nomograms, age contributed the largest share of prognostic points [2].

Performance status, measured by the Karnofsky Performance Status (KPS) scale, captures functional capacity at diagnosis and independently predicts survival. KPS was not available in the MU-Glioma-Post dataset.

Extent of surgical resection influences survival, with gross total resection (GTR) associated with improved outcomes compared to subtotal resection or biopsy, although confounded by tumour location and operability [2].

The standard treatment protocol for glioblastoma, established by the landmark trial of Stupp et al. [6], comprises maximal safe surgical resection followed by concurrent temozolomide chemotherapy and radiotherapy (60 Gy in 30 fractions), then adjuvant temozolomide for six cycles. This “Stupp regimen” improved median overall survival from 12.1 months to 14.6 months ($p < 0.001$), and remains the standard of care. The survival

benefit from temozolomide is concentrated in patients with MGMT promoter methylation [16], an interaction between molecular markers and treatment that multivariate prediction models must account for.

2.1.4 Clinical Endpoints

Two principal clinical endpoints are used in glioma outcome studies. Overall survival (OS) is defined as the time from diagnosis (or from the start of treatment) to death from any cause. Progression-free survival (PFS) measures the time from diagnosis to the first documented tumour progression, as assessed by the Response Assessment in Neuro-Oncology (RANO) criteria, or death. OS is generally preferred as the primary endpoint for prediction modelling because it is unambiguous (death is a definitive event), whereas PFS depends on the frequency and timing of surveillance imaging, which varies across institutions and clinical protocols.

Both endpoints involve right-censored observations: patients alive (for OS) or progression-free (for PFS) at last follow-up are censored rather than excluded. Failure to account for censoring introduces systematic bias, as it selectively removes longer-surviving patients. This distinction between binary classification (alive vs. deceased at a fixed timepoint) and survival analysis (modelling the full time-to-event distribution with censoring) motivates the dual analytical framework adopted in this project.

2.2 Machine Learning Fundamentals

2.2.1 Supervised Learning Pipeline

Supervised machine learning aims to learn a mapping from input features $\mathbf{x} \in \mathbb{R}^p$ to an output y (a class label in classification, or a continuous value in regression) using a training dataset of n labelled examples $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$. The general pipeline comprises data collection, preprocessing (handling missing values, encoding categorical variables, normalising features), model training on a designated training set, evaluation on held-out data, and, in clinical settings, prospective deployment.

The central challenge is ensuring that development performance generalises to unseen patients. Overfitting (capturing noise rather than signal) is amplified in small-sample settings ($n \approx 200$), where models can memorise individual examples. Cross-validation, repeatedly splitting data into training and validation subsets, provides less biased performance estimates than a single split and is the standard strategy for datasets of this size.

2.2.2 Classification Models

Logistic Regression. Logistic regression models the probability of a binary outcome as a logistic (sigmoid) function of a linear combination of input features:

$$P(y = 1 \mid \mathbf{x}) = \frac{1}{1 + \exp(-(\beta_0 + \sum_{i=1}^p \beta_i x_i))} \quad (2.1)$$

where β_0 is the intercept and β_i are feature coefficients. The model is trained by maximising the log-likelihood (equivalently, minimising the binary cross-entropy loss). Each coefficient β_i has a direct interpretation as the log-odds ratio: a one-unit increase in feature x_i multiplies the odds of the positive outcome by $\exp(\beta_i)$, holding other features constant.

L1 and L2 regularisation prevent overfitting by penalising coefficient magnitudes, with L1 additionally performing implicit feature selection by driving some coefficients to zero.

Logistic regression’s principal advantage is interpretability; its principal limitation is the assumption of linearity in the log-odds.

Decision Trees. Decision trees partition the feature space through hierarchical binary splits, selecting the feature and threshold that maximise a purity criterion at each node. Individual trees are interpretable but prone to overfitting on small datasets, motivating ensemble methods that aggregate multiple trees to improve generalisation.

Gradient Boosting and XGBoost. Gradient boosting constructs an ensemble of decision trees sequentially, with each tree fitted to the residual errors of the preceding ensemble. At each iteration t , a new tree f_t is added to minimise the regularised objective:

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell\left(y_i, \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)\right) + \Omega(f_t) \quad (2.2)$$

where ℓ is the loss function and $\Omega(f_t) = \gamma T + \frac{1}{2} \lambda \|\mathbf{w}\|^2$ penalises tree complexity through the number of leaves T and the L2 norm of leaf weights $\mathbf{w}$ [1].

XGBoost (*eXtreme Gradient Boosting*) extends this framework with several innovations suited to tabular medical data [1]. It uses a second-order Taylor approximation of the loss function, incorporating both first-order gradients $g_i = \partial \ell / \partial \hat{y}_i^{(t-1)}$ and second-order Hessians $h_i = \partial^2 \ell / \partial (\hat{y}_i^{(t-1)})^2$ for more accurate optimisation. The regularisation term Ω provides built-in L1 (γ) and L2 (λ) penalties that control tree complexity. XGBoost also natively handles missing values through a sparsity-aware split-finding algorithm that learns the optimal default direction for missing entries during training (Figure 2.1), which suits clinical datasets where missingness patterns may themselves carry prognostic information.

The optimal split at each node is determined by a gain formula (Appendix A.2) that compares the reduction in loss between child nodes; splits with negative gain are pruned. XGBoost was selected as the primary architecture for its native missing value handling, built-in regularisation, and interpretable gain-based feature importance.

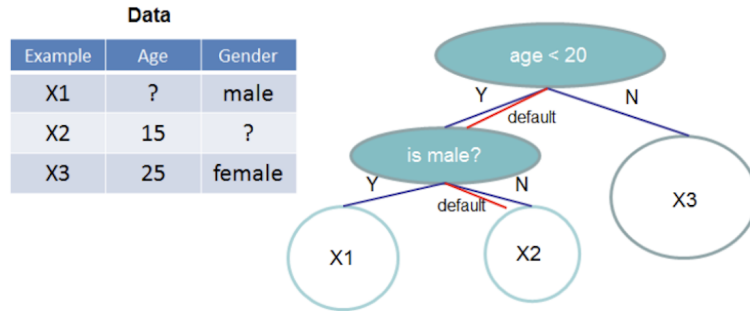


Figure 2.1: XGBoost’s learned default directions for missing values. When a feature value is absent, the algorithm routes the instance along a default branch determined during training to minimise loss. Adapted from Chen and Guestrin [1].

2.2.3 Survival Analysis

Time-to-Event Data. Survival analysis addresses a class of problems where the outcome is the time until an event occurs. Each observation is a tuple (t_i, δ_i) , where t_i is the observed time and δ_i is an event indicator ($\delta_i = 1$ if the event occurred, $\delta_i = 0$ if the observation is right-censored). Right-censoring arises when the study ends before the event is observed, or

when a patient is lost to follow-up. Standard classification and regression methods cannot handle censored observations without introducing bias, as excluding censored patients systematically removes longer-surviving individuals from the analysis, and treating them as non-events underestimates mortality.

Kaplan-Meier Estimator. The Kaplan-Meier (KM) estimator is a non-parametric method for estimating the survival function $S(t) = P(T > t)$, the probability of surviving beyond time t :

$$\hat{S}(t) = \prod_{t_i \leq t} \left(1 - \frac{d_i}{n_i}\right) \quad (2.3)$$

where d_i is the number of events at time t_i and n_i is the number of patients at risk immediately before t_i . The resulting step function decreases at each event time and remains flat between events. Censored observations reduce the risk set without contributing events. The log-rank test compares KM curves between groups (for example, IDH-mutant versus IDH-wildtype) by testing the null hypothesis of equal survival distributions.

Cox Proportional Hazards Model. The Cox Proportional Hazards (Cox PH) model is a semi-parametric regression model for time-to-event data [18]. It models the hazard function as:

$$h(t | \mathbf{x}) = h_0(t) \exp\left(\sum_{i=1}^p \beta_i x_i\right) \quad (2.4)$$

where $h_0(t)$ is an unspecified baseline hazard function and $\exp(\beta_i)$ is the hazard ratio associated with a one-unit increase in feature x_i . The model is “semi-parametric” because it makes no assumption about the shape of $h_0(t)$; estimation proceeds via partial likelihood, which depends only on the ordering of event times and not on the baseline hazard. The proportional hazards assumption requires that the hazard ratio between any two patients remains constant over time. When the assumption holds, Cox model coefficients provide directly interpretable hazard ratios: $\exp(\beta_i) > 1$ indicates that higher values of x_i are associated with increased mortality risk, whilst $\exp(\beta_i) < 1$ indicates a protective effect.

Random Survival Forest. The Random Survival Forest (RSF) adapts the random forest ensemble method to censored data. An ensemble of survival trees is grown, each using a random bootstrap sample and a random subset of features at each split node. Unlike the Cox model, RSF makes no assumptions about the functional form of the hazard and can capture non-linear relationships and complex interactions. Ensemble predictions are obtained by averaging the cumulative hazard functions across all trees. Feature importance is assessed via permutation importance: for each feature, the decrease in concordance index when that feature’s values are randomly shuffled measures its contribution to predictive accuracy.

Gradient Boosted Survival Analysis. Gradient boosted survival analysis applies the sequential boosting framework to the negative log partial likelihood loss (the Cox loss). This combines the interpretability of the Cox partial likelihood with the flexibility of gradient boosting, enabling the model to capture non-linear effects and interactions whilst maintaining the survival analysis framework. As with XGBoost for classification, early stopping on a validation set is used to prevent overfitting.

2.2.4 Evaluation Metrics

Classification Metrics. Binary classification models are evaluated using metrics derived from the confusion matrix. Accuracy = $(TP + TN) / (TP + TN + FP + FN)$ is intuitive but can be misleading when class distributions are imbalanced. Precision = $TP / (TP + FP)$ measures the proportion of positive predictions that are correct, whilst recall (sensitivity) = $TP / (TP + FN)$ measures the proportion of actual positives that are correctly identified. The F1 score = $2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$ balances both error types.

The area under the receiver operating characteristic curve (AUC-ROC) is the primary metric in this project. The ROC curve plots the true positive rate against the false positive rate across all classification thresholds. AUC-ROC has the interpretation of the probability that a randomly selected deceased patient receives a higher predicted risk score than a randomly selected surviving patient: 0.5 indicates random performance and 1.0 indicates perfect discrimination. AUC-ROC is preferred for survival prediction tasks because it is threshold-independent and insensitive to class imbalance.

Survival Metrics. The concordance index (C-index) is the standard discrimination metric for survival models. It represents the proportion of all evaluable patient pairs for which the model correctly orders survival times: if patient A dies before patient B , a concordant model assigns a higher risk score to A . Formally, C-index = $P(\hat{r}_i > \hat{r}_j \mid t_i < t_j)$ for all pairs (i, j) where patient i experienced the event before patient j . The C-index is analogous to the AUC-ROC in the survival context, with 0.5 indicating random performance and 1.0 indicating perfect concordance.

Cross-Validation. Stratified K -fold cross-validation ($K = 5$ in this project) partitions the dataset into K folds, training on $K - 1$ folds and evaluating on the remaining fold, repeating K times. Stratification ensures that the class distribution (alive/deceased ratio) is preserved across all folds. The same fold assignments are used across all models to enable paired statistical comparisons. Performance is reported as the mean $\pm$ standard deviation across folds.

2.2.5 Feature Importance

Feature importance quantifies the contribution of each input variable to model predictions and is both a model interpretation tool and a means of biological validation.

For XGBoost models, the native gain-based importance measures the total reduction in the loss function attributable to splits on each feature across all trees in the ensemble. Features that appear in many splits and produce large loss reductions are assigned high importance. Gain-based importance can be biased towards high-cardinality features, but this concern is mitigated in the present dataset where most molecular markers are binary.

Permutation importance provides a model-agnostic alternative. For each feature, the values are randomly shuffled across patients, breaking the association between that feature and the outcome. The resulting decrease in model performance (measured by C-index or AUC) quantifies the feature’s contribution. Permutation importance is used here for Random Survival Forest models, where gain-based importance is not available.

Feature importance reflects predictive association, not causal relationships. Correlated features share importance unpredictably, and importance depends on which other features are present in the model.

2.3 Data Leakage in Medical Machine Learning

2.3.1 Foundations

Data leakage occurs when information that would not be available at the time of prediction inadvertently enters the model training process, producing artificially inflated performance estimates [3]. Unlike overfitting, which reflects a model’s failure to generalise from noisy training data, leakage introduces a genuine but spurious signal that systematically biases performance estimates upward. The distinction matters because standard regularisation techniques (early stopping, L2 penalties, dropout) can mitigate overfitting but cannot correct for leakage, since the spurious signal is present in both training and validation data when cross-validation is used.

Data leakage poses a heightened risk in medical machine learning. Retrospective clinical datasets contain longitudinal information spanning diagnosis, treatment, follow-up, and outcomes, yet clinical deployment requires prediction at a single timepoint (typically diagnosis). This mismatch between the richness of the training data and the constraints of the deployment setting creates many opportunities for temporal contamination. When leakage goes undetected, the consequences compound: models that appear highly accurate during development fail in clinical deployment, wasting resources and eroding clinician trust in medical AI [9].

2.3.2 Leakage Taxonomy

Kapoor and Narayanan [3] provide a systematic taxonomy of eight leakage types arising during data collection, preprocessing, modelling, and evaluation. Three types are relevant to this project.

Target Leakage (Type L2.2). Target leakage occurs when a feature directly encodes or is derived from the prediction target. In survival prediction, features that describe the outcome itself (or close proxies) inflate performance because the model effectively learns to “read” the answer from the input. For example, including “disease progression” as a predictor of “death” creates circular logic, since progression is a strong proxy for mortality.

Temporal Leakage (Type L3.1). Temporal leakage is the most relevant type for this project. It occurs when features measured after the intended prediction timepoint are included as predictors. In glioma datasets, this often appears as progression-related variables: “time to first progression,” “second progression/recurrence,” and “number of days from diagnosis to first surgery or procedure” all describe events that occur after diagnosis. Including these features in a model trained at the diagnostic timepoint is equivalent to using the future to predict the future. Davis et al. [4] describe this pattern as label leakage in the healthcare context, where features encoding future events act as proxies for the outcome variable.

The subtlety of temporal leakage lies in its plausibility. Features such as “time to first progression” appear to be legitimate clinical variables, and their high correlation with survival is easily mistaken for genuine predictive power rather than an artefact of temporal contamination. This distinguishes temporal leakage from more obvious forms (such as including the target variable directly) and explains why it often goes undetected in published studies.

Train-Test Contamination (Type L2.1). Train-test contamination occurs when preprocessing steps (normalisation, imputation, feature selection) are applied to the combined training and test data before splitting. This allows statistical properties of the test set to influence the training process. Whilst cross-validation reduces this risk compared

to a single train-test split, certain operations (such as computing global imputation statistics or selecting features based on the full dataset) can still introduce contamination. In the present project, imputation statistics for logistic regression are computed exclusively on each training fold to prevent this form of leakage.

2.3.3 Detection Methods

Detecting data leakage requires a combination of statistical screening and domain-specific reasoning, as no purely automated method can reliably distinguish legitimate predictive associations from leakage-induced artefacts.

Correlation-Based Screening. Computing feature-target Pearson correlations provides a first-pass screen: features with $|r|$ exceeding a threshold are flagged for review. However, high correlation does not constitute proof of leakage, as legitimate biological markers (e.g., IDH mutation status) often exhibit strong target correlations because they are prognostic. The threshold is a screening step that must be followed by temporal reasoning.

Temporal Reasoning. For each flagged feature, the critical test is whether the information would be available at the time of diagnosis. This requires domain expertise to evaluate whether each variable represents a baseline measurement (available at the prediction timepoint) or a follow-up measurement (collected after the prediction timepoint). Features describing post-diagnosis events, disease progression, or treatment timing are identified as temporal leakage and removed from the clean feature set. Features with high target correlation that pass the temporal test (such as IDH mutation status and age at diagnosis) are retained.

Four-Step Detection Pipeline. Building on the healthcare leakage framework of Davis et al. [4], the present project implements a four-step leakage detection pipeline. Step 1 computes feature-target correlation coefficients. Step 2 applies a threshold ($|r| > 0.25$) to flag suspicious features. Step 3 performs temporal reasoning for each flagged feature, classifying it as appropriate (available at diagnosis) or inappropriate (requiring future information). Step 4 removes confirmed leakage features and documents each removal decision with clinical justification. This methodology is described in full in Chapter 5.

2.3.4 Consequences of Undetected Leakage

The consequences of undetected data leakage are both scientific and practical. Kapoor and Narayanan [3] documented that leakage affects at least 294 papers across 17 scientific fields, with a median AUC inflation of 0.12 in affected studies. In their case study of civil war prediction, correcting for leakage eliminated the claimed superiority of machine learning models over decades-old logistic regression, demonstrating that the entire performance advantage was attributable to data contamination rather than genuine algorithmic innovation.

In the medical context, leakage creates unrealistic deployment expectations (a model reporting AUC 0.90 may achieve only 0.65–0.70 prospectively), wastes development resources, and propagates inflated baselines that mislead subsequent studies [3, 9].

These considerations motivate the central design principle of the present project: honest performance reporting, even at the cost of lower headline metrics, provides more value to the clinical and research communities than inflated results that cannot be reproduced or deployed.

Chapter 3

Previous Work

Machine learning has been applied extensively to glioma prognosis, yet methodological rigour varies across studies. The following review covers classical prognostic tools (Section 3.1), machine learning approaches (Section 3.2), data leakage prevalence (Section 3.3), radiomics (Section 3.4), and tumour growth modelling (Section 3.5).

3.1 Classical Prognostic Models

Prior to ML approaches, glioma prognostication relied on clinically validated tools developed through traditional statistical methods. These models establish performance benchmarks against which ML approaches should be evaluated.

3.1.1 Recursive Partitioning Analysis

The Radiation Therapy Oncology Group (RTOG) developed Recursive Partitioning Analysis (RPA) classification through pooled analysis of three prospective clinical trials enrolling 1,578 patients with high-grade gliomas [11]. RPA stratifies patients into six prognostic classes based on age, performance status, histology, neurological function, and extent of surgery. The model achieved clear separation between risk groups, with median survivals ranging from 4.6 months (Class VI) to 58.6 months (Class I).

Whilst RPA classes stratify patients at the population level, their discriminative capacity at the individual patient level is limited. The categorical nature of the classification scheme (six discrete bins) precludes continuous risk estimation, and the model predates molecular diagnostics entirely, rendering it less relevant in the contemporary era where IDH mutation status and other genetic markers dominate prognostic hierarchies. Nevertheless, RPA remains clinically relevant as a validated baseline; machine learning models that fail to outperform RPA’s discrimination cannot claim clinical utility.

3.1.2 EORTC Nomograms

Gorlia et al. [2] developed continuous prognostic nomograms for newly diagnosed glioblastoma patients using data from two phase III EORTC clinical trials ($n = 832$ patients). The nomograms incorporate six prognostic factors: age, performance status, extent of resection, mini-mental state examination score, corticosteroid use, and presence of neurological deficits (Figure 3.1). The models were internally validated using bootstrap resampling and demonstrated a concordance index of approximately 0.66 for predicting survival at 6, 12, and 18 months.

The EORTC nomograms anchor the comparison for this project. A C-index of 0.66 establishes the performance ceiling for models using only baseline clinical features without molecular profiling. ML approaches reporting AUC or C-index values well above 0.70–0.75

on similarly sized cohorts warrant scrutiny for potential data leakage, especially if molecular markers are absent or sparse. The EORTC models’ transparent validation methodology and modest performance metrics set the standard of honest evaluation for ML studies.

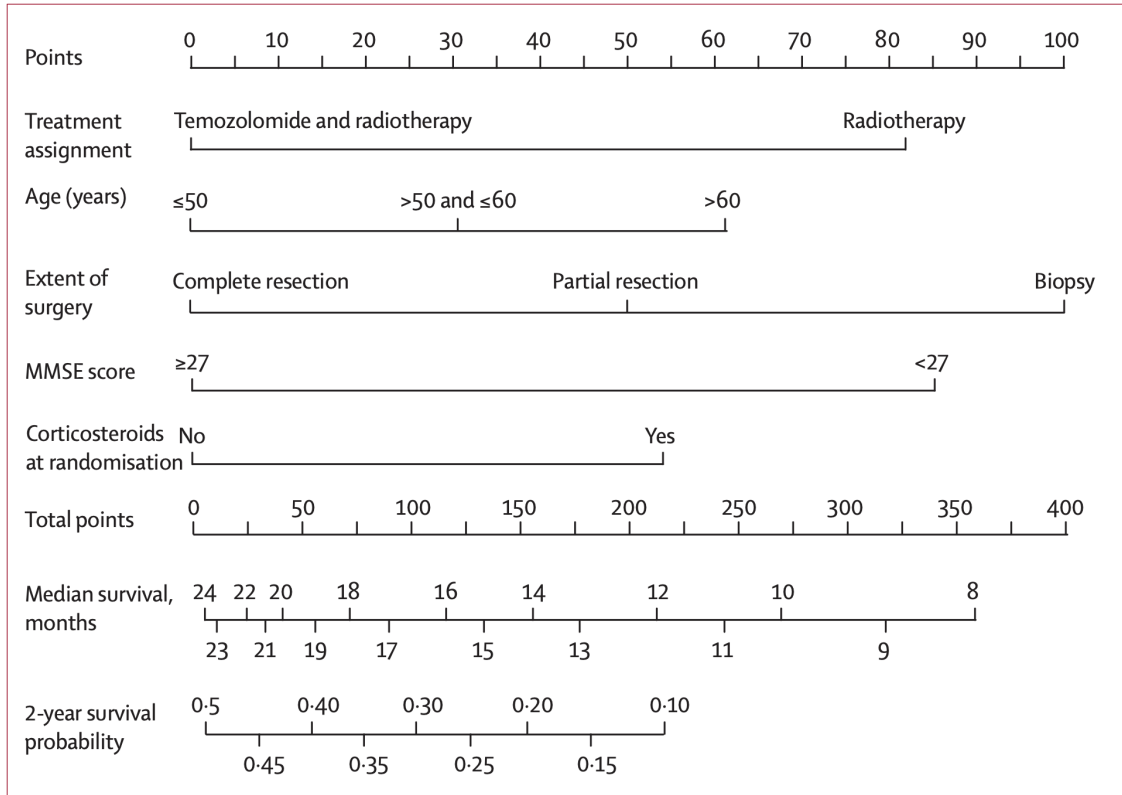


Figure 3.1: Prognostic nomogram for median survival in newly diagnosed GBM patients. Each clinical variable is mapped to a point score on the top axis; total points are converted to median survival (months) and 2-year survival probability on the bottom axes. Reproduced from Gorlia et al. [2].

3.2 Machine Learning for Glioma Survival Prediction

Machine learning for glioma survival prediction now spans hundreds of published studies, driven by increasing availability of large-scale clinical and molecular datasets. Systematic reviews, however, reveal wide heterogeneity in reported performance and methodological rigour.

3.2.1 Systematic Review Evidence

Yan et al. [8] conducted a systematic review of 45 studies published between 2010 and 2023 predicting glioblastoma survival using machine learning. The review identified substantial methodological variation: cross-validation strategies ranged from simple train-test splits (63% of studies) to nested cross-validation (11%), with many studies omitting external validation entirely. Reported AUC values spanned 0.65 to 0.95 across studies using ostensibly similar feature sets (clinical, molecular, imaging). The authors noted that studies reporting exceptionally high performance ($AUC > 0.90$) frequently failed to provide sufficient methodological detail to assess potential sources of optimism bias, including data leakage.

Poursaeed et al. [7] performed a parallel systematic review focusing on both machine learning and deep learning approaches ($n = 52$ studies). This analysis revealed that whilst deep learning methods achieved marginally higher mean performance (AUC 0.82 ± 0.11) compared to classical ML (AUC 0.78 ± 0.13), the performance distributions overlapped, and the best-performing classical models matched or exceeded many deep learning implementations. The review identified data leakage as a pervasive concern, noting that 37% of reviewed studies included features encoding treatment response or progression status that would be unavailable at diagnosis. The authors also highlighted publication bias towards positive results, with few studies reporting negative or null findings, despite the stochastic nature of glioma biology placing hard limits on predictive accuracy.

Taken together, these reviews indicate that reported glioma survival prediction performance likely overestimates what is achievable in prospective deployment. Glioma survival is stochastic and shaped by unmeasured factors (microenvironmental heterogeneity, treatment tolerance, comorbidities), so the wide spread in reported results is difficult to reconcile with biological reality. Models reporting near-perfect discrimination (AUC > 0.90) on retrospective cohorts should be treated with scepticism absent rigorous external validation.

3.2.2 Representative Machine Learning Studies

Babaei Rikan et al. [19] evaluated survival prediction using the SEER (Surveillance, Epidemiology, and End Results) database ($n = 9,154$ GBM patients) with XGBoost, random forest, and neural network architectures. The study reported AUC values of 0.84 for 1-year survival and 0.82 for 2-year survival using 17 clinical features (age, race, marital status, tumour site, extent of resection, radiation, chemotherapy). The feature set included treatment modality (radiation yes/no, chemotherapy yes/no) as predictors. Whilst these are ostensibly baseline decisions rather than outcomes, treatment allocation is itself influenced by prognostic factors and disease characteristics, introducing potential for confounding. The model’s high performance may partially reflect correlation between aggressive treatment (indicative of good performance status and younger age) and survival, rather than purely predictive biological information.

Kickingreder et al. [20] applied 3D convolutional neural networks to preoperative MRI scans from 1,501 patients across 38 institutions to predict progression and overall survival. The CNN-based approach achieved a C-index of 0.71 for overall survival prediction, significantly outperforming models using only clinical and molecular features (C-index 0.63, $p < 0.001$). This study has several methodological strengths: multi-institutional validation, use of raw imaging data rather than hand-crafted features, and transparent reporting of negative results (imaging features did not improve prediction in IDH-mutant gliomas). The C-index of 0.71 establishes a realistic performance ceiling for imaging-integrated models and aligns closely with the EORTC benchmarks, suggesting minimal optimism bias. However, even this rigorous study shows that deep learning on volumetric imaging requires large sample sizes ($n > 1,000$) to achieve modest improvements over tabular clinical models, limiting applicability to smaller institutional cohorts.

3.3 Data Leakage in Medical Machine Learning

Section 2.3 introduced the theoretical foundations and taxonomy of data leakage. The empirical evidence of its prevalence and impact across scientific disciplines is reviewed here.

Kapoor and Narayanan [3] demonstrated the scale of the problem through a meta-analysis of 20 highly cited ML studies: leakage-corrected replication attempts resulted in a median AUC reduction of 0.12 (range: 0.03–0.31), with larger reported effect sizes exhibiting greater performance drops upon correction. Their survey (Figure 3.2) identified

leakage across 17 research fields, establishing it as a systemic problem in ML-based science rather than a domain-specific concern.

Field	Paper	Number of papers reviewed Number of papers with pitfalls [L1.1] No test set [L1.2] Pre-proc. on train-test [L1.3] Feature sel. on train-test [L1.4] Duplicates [L2] Illegitimate features [L3.1] Temporal leakage [L3.2] Non-ind. b/w train-test [L3.3] Sampling bias Comput. reproducibility issues Data quality issues Metric choice issues Standard dataset used?										
Medicine	Bouwmeester et al. (2012)	71	27	○							○	
Neuroimaging	Whelan & Garavan (2014)	–	14	○		○						
Bioinformatics	Blagus & Lusa (2015)	–	6		○							
Autism Diagnostics	Bone et al. (2015)	–	3			○		○		○	○	
Nutrition Research	Ivanescu et al. (2016)	–	4	○						○	○	
Software Eng.	Tu et al. (2018)	58	11				○		○	○	○	
Toxicology	Alves et al. (2019)	–	1			○			○	○		
Clinical Epidem.	Christodoulou et al. (2019)	71	48			○						
Satellite Imaging	Nalepa et al. (2019)	17	17					○		○	○	
Tractography	Poulin et al. (2019)	4	2	○					○	○	○	
Brain-computer Int.	Nakanishi et al. (2020)	–	1	○							○	
Histopathology	Oner et al. (2020)	–	1					○				
Neuropsychiatry	Poldrack et al. (2020)	100	53	○	○					○	○	
Neuroimaging	Ahmed et al. (2021)	–	1					○				
Neuroimaging	Li et al. (2021)	122	18					○				
IT Operations	Lyu et al. (2021)	9	3				○				○	
Medicine	Filho et al. (2021)	–	1				○					
Radiology	Roberts et al. (2021)	62	16	○		○			○	○	○	
Neuropsychiatry	Shim et al. (2021)	–	1			○				○		
Medicine	Vandewiele et al. (2021)	24	21		○			○	○	○	○	
Computer Security	Arp et al. (2022)	30	22	○	○	○	○	○	○	○	○	
Genomics	Barnett et al. (2022)	41	23		○					○		

Figure 3.2: Survey of data leakage across 17 research fields. Each row represents a review paper; circles indicate which leakage types were identified. The breadth of affected fields demonstrates that data leakage is a systemic problem in ML-based science, not confined to any single domain. Reproduced from Kapoor and Narayanan [3].

Davis et al. [4] operationalise this framework for healthcare ML specifically, introducing a four-category taxonomy distinguishing directly measured outcomes, proxy measures, consequence measures, and clinician knowledge leakage. Their main point is that detecting leakage requires domain expertise alongside statistical methods: correlation thresholding can flag suspicious features, but only clinical understanding of the temporal sequence of measurements can definitively classify them.

3.3.1 Empirical Evidence of Leakage Impact

McDermott et al. [9] conducted a reproducibility analysis of 511 published ML studies in healthcare, finding that fewer than 10% provided sufficient methodological detail to assess temporal validity of features. When the authors attempted replication of 27 prominent studies with suspected leakage, correction reduced mean AUC from 0.89 to 0.71 (a 20% relative reduction). Studies in oncology and critical care exhibited the highest leakage rates, attributed to the longitudinal nature of these datasets where multiple measurements occur over extended timeframes.

This evidence has three implications for the present work. First, honest models using baseline clinical and molecular features should achieve C-index or AUC in the range of 0.65–0.75, consistent with validated clinical tools; performance exceeding 0.80 on cohorts of $n < 500$ warrants scrutiny. Second, documenting the leakage detection process and quantifying its impact (before vs. after correction) is more informative than reporting only

optimised metrics. Third, every included feature requires explicit temporal reasoning to confirm its availability at the prediction timepoint.

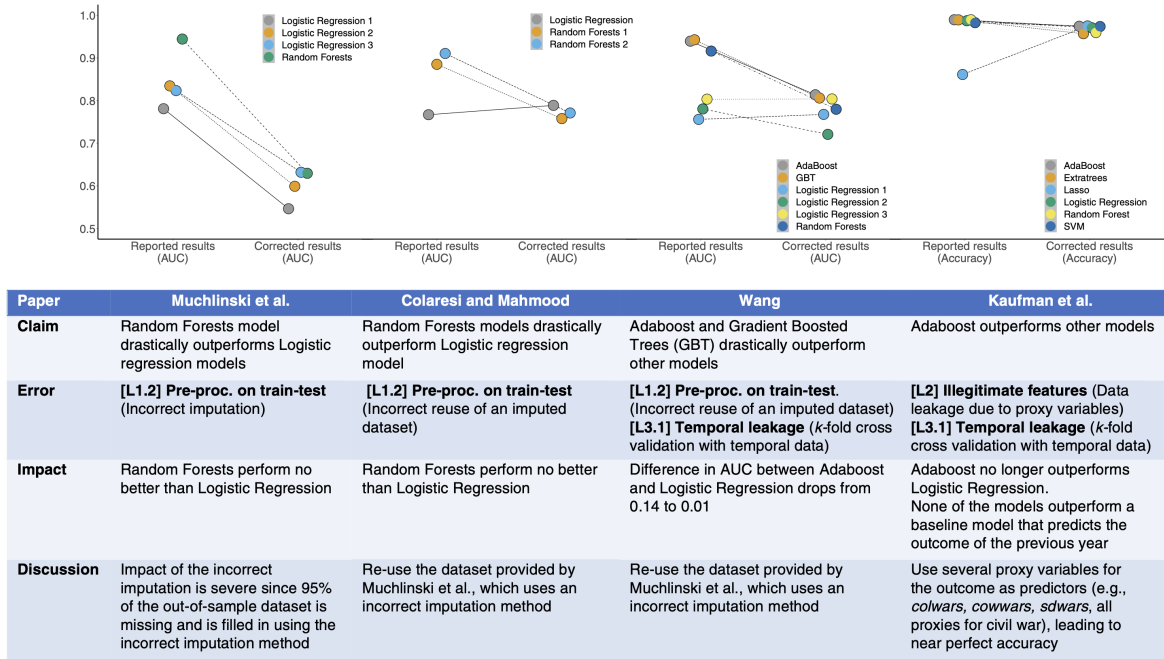


Figure 3.3: Reported versus leakage-corrected results for four civil war prediction studies. After correction, complex ML models (Random Forests, AdaBoost, GBT) no longer outperform logistic regression, paralleling the pattern observed in the present work where XGBoost AUC fell from 0.730 to 0.674 after leakage removal. Reproduced from Kapoor and Narayanan [3].

Given the systematic nature of leakage in the medical ML literature, this project treats methodological transparency and leakage detection as a primary contribution rather than pursuing state-of-the-art performance claims.

3.4 Radiomics and Imaging-Based Features

Radiomics refers to the high-throughput extraction of quantitative features from medical images, transforming images into mineable data for predictive modelling [21]. This section reviews the approach and its application to glioma prognostication, with emphasis on distinguishing clinically meaningful patterns from methodological artefacts.

3.4.1 The Radiomics Paradigm

Radiomics extracts four categories of quantitative features from medical images: first-order statistics, shape descriptors, texture features, and wavelet-transformed features [21]. The hypothesis is that these descriptors capture tumour phenotypic characteristics that correlate with underlying biology and prognosis. However, two significant challenges limit clinical translation. First, radiomics features are sensitive to imaging acquisition parameters, necessitating harmonisation across multi-institutional datasets [22]. Second, the high dimensionality of radiomics feature spaces (hundreds to thousands of features) combined with modest sample sizes introduces substantial overfitting risk. Standardised workflows using validated software such as PyRadiomics [23] can achieve C-index improvements of

0.05–0.10 beyond clinical models, but these improvements require large datasets ($n > 500$) and rigorous cross-validation to avoid overestimation.

3.4.2 Radiomics in Glioma Survival Prediction

Jain et al. [24] investigated whether imaging features derived from the non-enhancing (FLAIR/T2 hyperintense) component of glioblastoma predict outcomes beyond clinical and molecular markers. In a cohort of 65 patients with comprehensive molecular profiling, the authors extracted volumetric and intensity features from the infiltrative tumour margin. Multivariate Cox models demonstrated that the volume of non-enhancing tumour independently predicted survival (HR 1.8 per 10 cm³, $p = 0.004$) even after adjusting for age, extent of resection, and MGMT methylation status. This finding suggests that infiltrative growth patterns captured by FLAIR/T2 imaging reflect aggressive tumour biology not fully captured by enhancing tumour characteristics or genetic markers.

Jain et al. [24] used first-order intensity statistics and volumetric measures rather than complex texture features, and their modest effect size (C-index improvement from 0.68 to 0.72) reflects realistic expectations for imaging-derived features. The distinction between first-order statistics and comprehensive radiomics matters for the present work: first-order features (mean intensity, standard deviation, volume) are interpretable, stable across institutions, and computationally efficient, but capture only coarse tissue characteristics. In cohorts of $n < 200$, first-order features may represent the optimal trade-off between information content and overfitting risk.

3.5 Tumour Growth Modelling and Progression Prediction

Predicting tumour volume change from baseline features represents a distinct challenge from survival prediction. Mathematical reaction-diffusion models [25, 26] demonstrate that observed volume change between imaging timepoints reflects a composite of proliferation, invasion, treatment-induced necrosis, and oedema dynamics rather than a single biological process. Post-treatment imaging is further confounded by pseudoprogression (15–30% prevalence in the first 12 weeks post-chemoradiation [27]) and pseudoresponse [28], where imaging changes reflect treatment effects rather than tumour biology. For ML models, this means that volume change between post-treatment scans does not represent a clean biological signal. To the author’s knowledge, no published study has successfully predicted quantitative tumour volume change from baseline tabular features in glioma [29], likely because the requisite information resides in spatial imaging patterns rather than scalar summary statistics. This project includes an exploratory analysis of this task to empirically test this hypothesis.

3.6 Research Gaps

This review establishes that validated clinical tools achieve C-index ≈ 0.66 , setting a realistic ML target of 0.65–0.75 on similarly sized cohorts. Empirical evidence demonstrates widespread leakage in medical ML (10–30% performance reductions upon correction), with glioma datasets particularly vulnerable due to longitudinal treatment and progression features. First-order imaging features provide modest incremental value on small cohorts, whilst comprehensive radiomics requires $n > 500$ to avoid overfitting.

The present work addresses these gaps by implementing systematic leakage detection, benchmarking against validated clinical tools, evaluating first-order radiomics on a modest cohort, and exploring the limits of growth prediction from tabular features.

Chapter 4

Research Methods

The experimental design covers four areas: dataset selection, preprocessing, model architecture, and evaluation protocol. Leakage detection methodology is presented separately in Chapter 5, with results in Chapter 6.

4.1 Dataset

4.1.1 MU-Glioma-Post Dataset

The MU-Glioma-Post dataset was obtained from The Cancer Imaging Archive (TCIA), a publicly available repository of medical images and associated clinical data. The dataset comprises 203 patients with histologically confirmed glioma diagnoses from University of Missouri Health Care, with both clinical annotations and multimodal MRI imaging. We selected it for three reasons: molecular profiling across IDH, MGMT, TERT, 1p/19q, TP53, and ATRX, pre-computed tumour segmentations enabling both tabular and imaging-based analysis, and sufficient sample size for cross-validated model evaluation.

As the dataset is publicly available and fully de-identified under institutional review board approval at the source institution, no additional ethics approval was required for this secondary analysis. All patient identifiers have been removed, with only coded study IDs retained.

4.1.2 Data Modalities

The dataset contains three data modalities.

Clinical Variables. Demographic information (age at diagnosis, sex, race), tumour characteristics (primary diagnosis, WHO grade, tumour location), treatment details (extent of resection, stereotactic biopsy status), and clinical outcomes (vital status, follow-up duration). Previous brain tumour history was also recorded.

Molecular Markers. Eleven molecular markers were available: IDH1 and IDH2 mutation status, MGMT promoter methylation, TERT promoter mutation, 1p/19q co-deletion, TP53 alteration, ATRX mutation, EGFR amplification, CDKN2A/B deletion, chromosome 7 gain/chromosome 10 loss, and H3-3A mutation. Missing data rates varied across markers, with some (e.g. 1p/19q) tested primarily in suspected oligodendrogliomas.

Imaging Data. Multimodal MRI volumes (T1 native, T1 post-contrast, T2-weighted, T2-FLAIR) with expert-annotated tumour segmentations. The dataset contains 596 post-treatment MR time points across 202 of the 203 patients. One patient lacks imaging.

Analyses use the first available timepoint per patient to avoid temporal leakage. Pre-extracted imaging features include tumour volume (mm^3) and first-order intensity statistics (mean and standard deviation) for each modality, yielding nine imaging-derived variables in total.

4.1.3 Target Variables

Two prediction tasks were defined.

Binary Classification. The target variable was vital status at last follow-up (alive or deceased), coded as a binary indicator. Of 203 patients, 97 (47.8%) had died and 106 (52.2%) were alive at last contact. The near-balanced class distribution reduces the need for class-weighting strategies, although stratified cross-validation was still employed to maintain consistent class ratios across folds.

Survival Analysis. The target comprised a pair: survival time (days from diagnosis to death or last follow-up) and a censoring indicator (1 if the patient died, 0 if censored). Survival times were constructed by taking the maximum follow-up timepoint for each patient, rather than the sum of timepoints, to avoid double-counting in patients with multiple follow-up records. One patient with missing survival time data was excluded, yielding $n = 202$ for survival analysis.

4.2 Preprocessing

4.2.1 Data Cleaning

The preprocessing pipeline comprised the following steps. Features with greater than 70% missing values were excluded, as imputation at this level would introduce excessive uncertainty and risk fabricating signal where none exists. Categorical variables were encoded using ordinal encoding for ordered categories (WHO grade: $1 < 2 < 3 < 4$) and one-hot encoding for nominal categories (tumour location, primary diagnosis). We identified feature types automatically using dtype inspection and verified them against the dataset documentation to ensure appropriate handling of numerical and categorical variables.

Missing values in remaining features were handled differently depending on the model type. For XGBoost models, the native missing value mechanism was used, which learns optimal default split directions during training. This approach preserves information about missingness patterns and avoids the risk of imputation-induced leakage. For logistic regression, median imputation was applied for continuous features and mode imputation for categorical features, with imputation statistics computed exclusively on each training fold to prevent information leakage from the validation set.

4.3 Exploratory Data Analysis

We conducted exploratory analysis prior to model development to characterise the cohort, identify data quality issues, and inform preprocessing decisions. The cohort was predominantly male (58.6%), with median age 61 years (IQR: 45–68). GBM was the most common diagnosis (67.5%), and WHO Grade IV tumours dominated (83.7%). IDH1 mutation was present in 24.1%, MGMT methylation in 41.4%, and TERT mutation in 52.7% of tested patients, aligning with expected frequencies for a predominantly GBM cohort. Overall, 47.8% of patients had died by end of follow-up; 82.5% of deceased patients

had documented progression, confirming the expected association between progression and mortality. Figures 4.1, 4.2, 4.3, and 4.4 present the cohort distributions across demographics, tumour characteristics, molecular markers, and clinical outcomes.

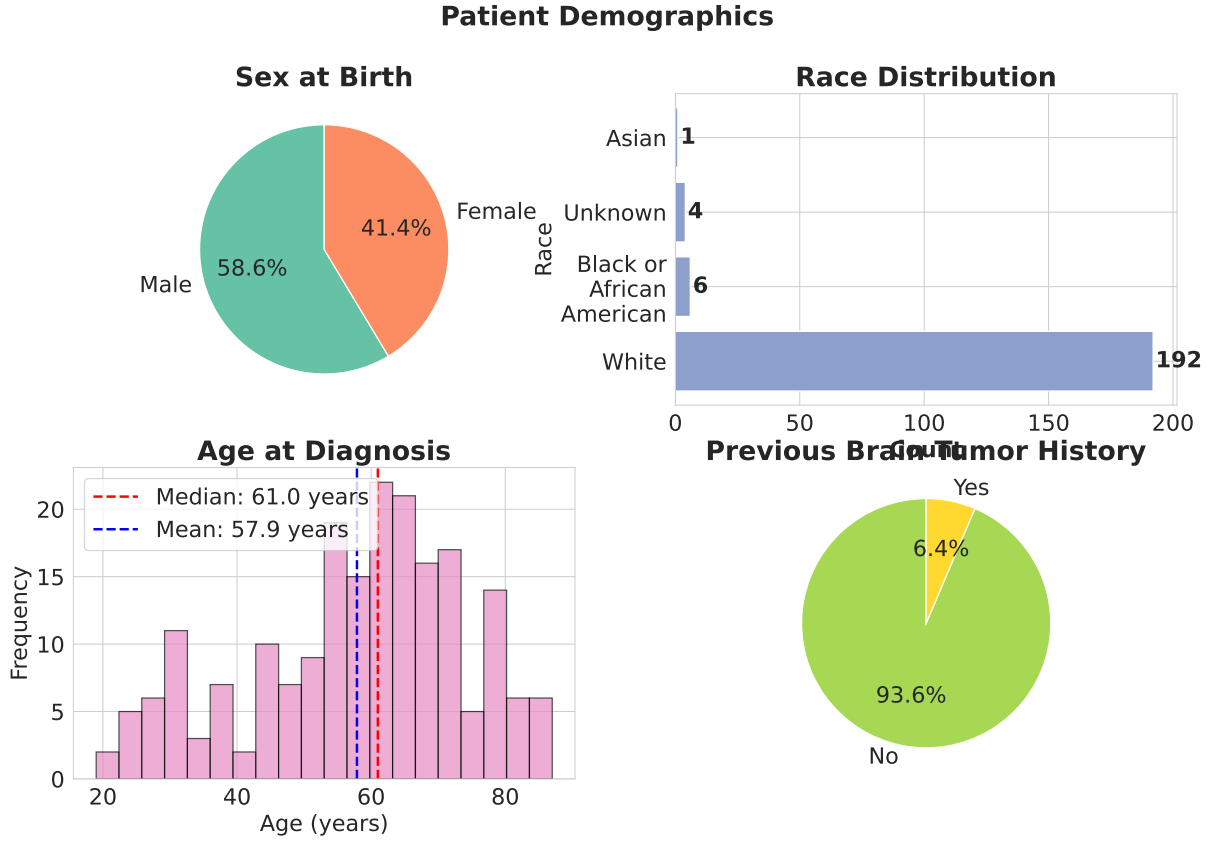


Figure 4.1: Patient demographic characteristics including age distribution, sex distribution, racial composition, and previous brain tumour history ($n = 203$).

The choice of missing-value strategy was informed by the probable mechanism generating each pattern of absence. Under the standard taxonomy [30], data are Missing Completely At Random (MCAR) when absence is independent of all observed and unobserved variables, Missing At Random (MAR) when absence depends only on observed variables, and Missing Not At Random (MNAR) when absence depends on the unobserved value itself. These distinctions matter because standard imputation is only unbiased under MCAR or MAR; under MNAR, imputing a value fabricates information that was deliberately not collected, potentially introducing a systematic bias that is invisible to cross-validation. In the MU-Glioma-Post dataset, molecular marker missingness is best characterised as MNAR or, at minimum, MAR conditional on diagnosis. The decision to test for 1p/19q co-deletion, ATRX, or TP53 is a clinical judgement made on the basis of tumour morphology and suspicion of histological subtype, none of which is captured by a missingness indicator column. Consequently, the absence of a test result is not random noise; it is an informative signal about the tumour’s likely subtype and therefore its prognosis. Imputing these absent values with population-level statistics would destroy this signal by replacing structured, clinically meaningful absence with the average of patients for whom the test was conducted. Preserving missingness through XGBoost’s sparsity-aware split-finding, which learns a separate optimal routing direction for missing entries at each node, is therefore the methodologically appropriate response to the data-generating mechanism.

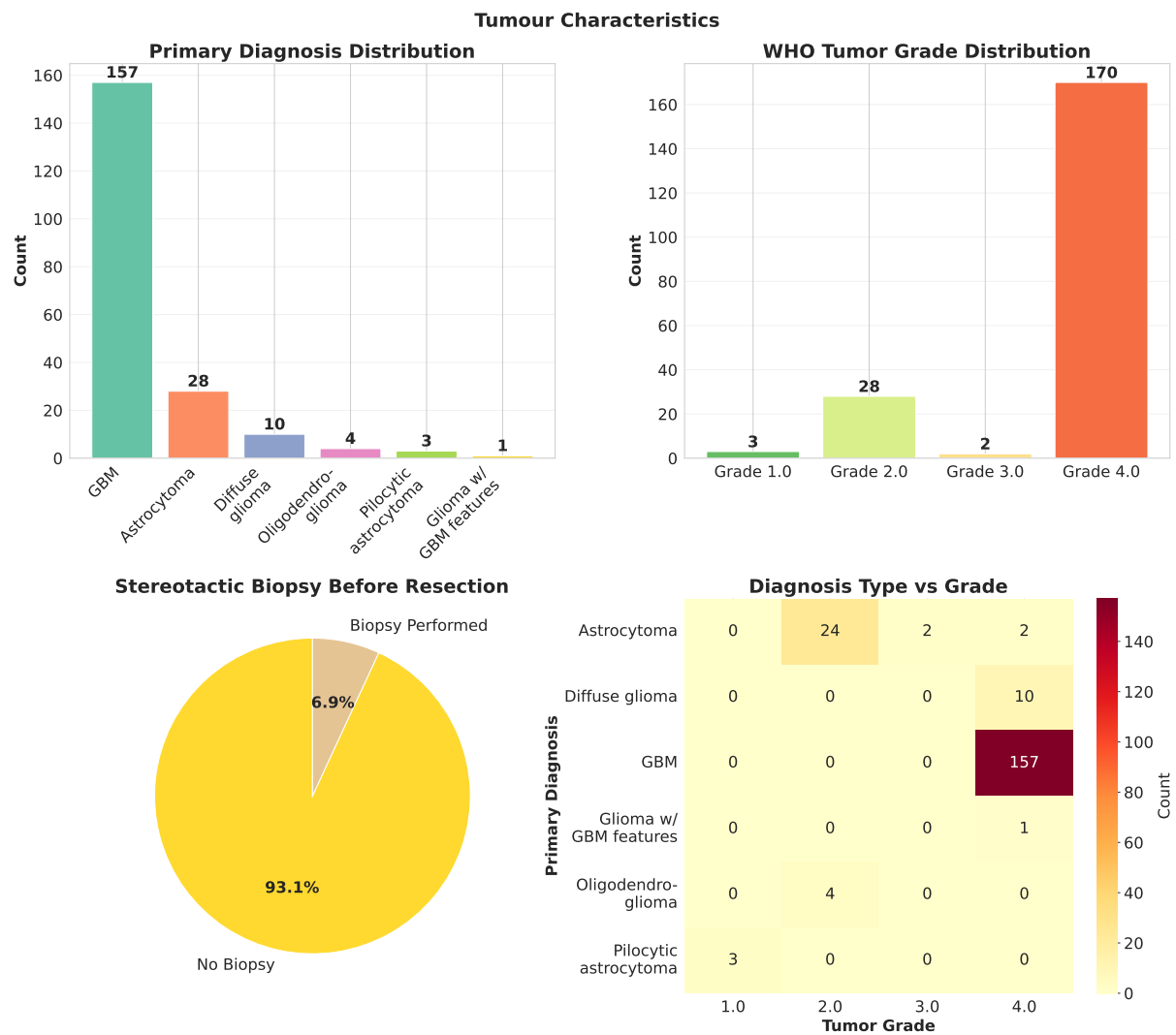


Figure 4.2: Tumour characteristics including primary diagnosis distribution, WHO grade classification, and stereotactic biopsy status.

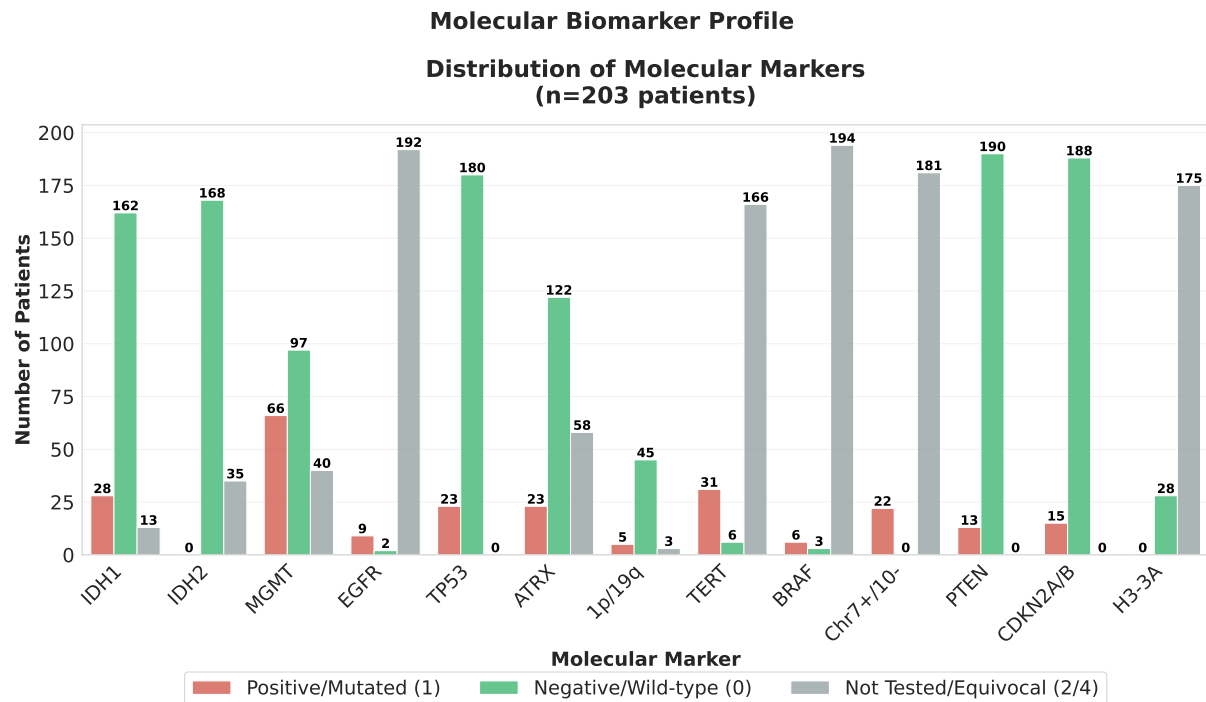


Figure 4.3: Molecular biomarker status across the cohort. Bars show positive, negative, and not tested counts for each marker.

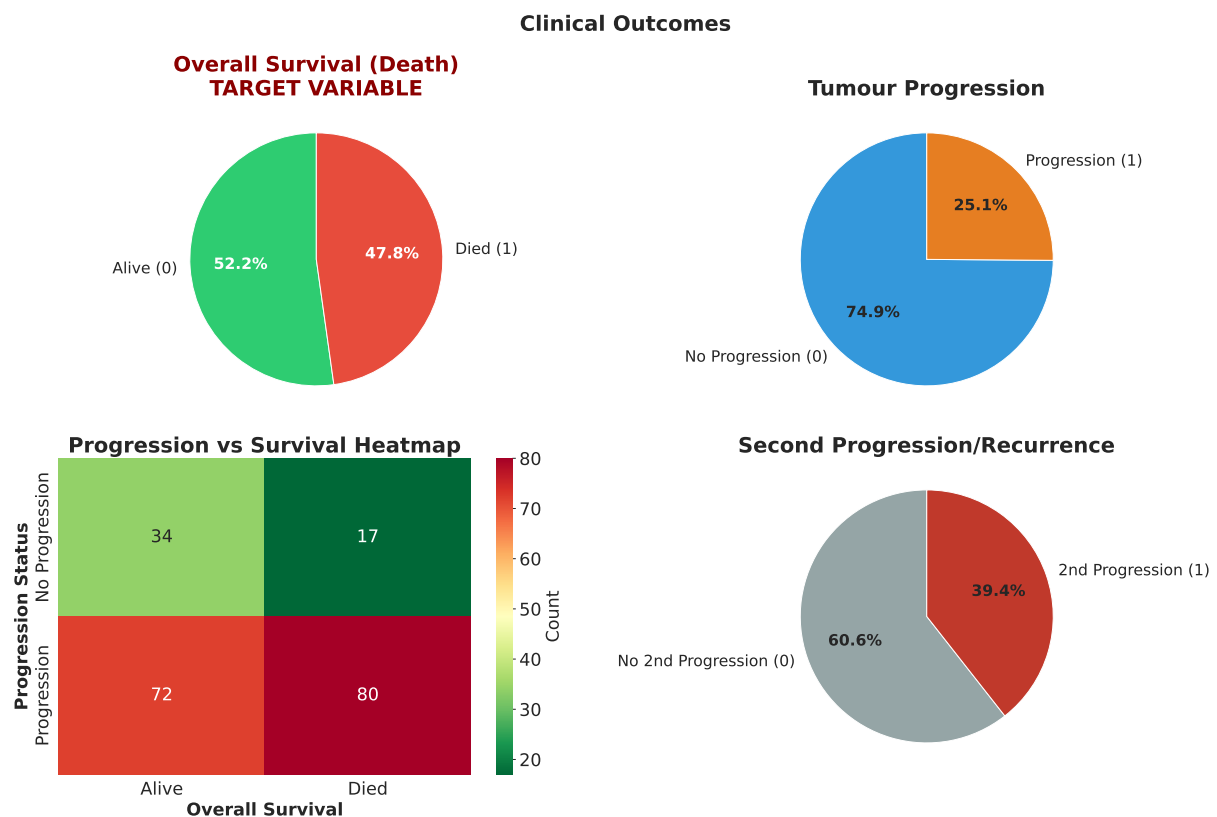


Figure 4.4: Clinical outcome distribution showing survival status, progression rates, and a heatmap cross-tabulating progression with overall survival.

4.4 Model Architectures

4.4.1 Binary Classification Models

We trained five binary classification models to enable systematic comparison of modelling strategies, with the baseline model retained to quantify leakage impact.

Model 1: Baseline XGBoost. Trained on the full feature set including leakage features ($p = 28$). This model is only a reference to quantify the performance inflation attributable to leakage; it should not be considered a valid benchmark for clinical deployment.

Model 2: Clean XGBoost. Trained on the clean feature set after leakage removal ($p = 21$). We selected XGBoost as the primary architecture for three reasons: native handling of missing values (given the dataset’s variable missingness rates), built-in L1 and L2 regularisation to mitigate overfitting on the small sample, and interpretable feature importance scores via gain decomposition. Default hyperparameters were used with early stopping on validation loss (patience = 10 rounds).

Model 3: Regularised XGBoost. Trained on the clean feature set with conservative hyperparameters: increased `min_child_weight` (restricting minimum samples per leaf), reduced `max_depth` (limiting tree complexity), and elevated γ and λ regularisation values. This model tests whether aggressive regularisation improves generalisation on the small dataset ($n = 203$).

Model 4: Logistic Regression. Trained on the clean feature set as an interpretable linear baseline. L2 regularisation was applied with the penalty strength selected via cross-validation. Feature standardisation (zero mean, unit variance) was performed within each training fold. Logistic regression provides interpretable coefficients as log-odds ratios, enabling clinical reasoning about feature contributions.

Model 5: Feature-Engineered XGBoost. Trained on the clean feature set augmented with four domain-informed derived features: an IDH wild-type GBM indicator (capturing the most aggressive molecular subtype), an elderly GBM indicator (age ≥ 65 with GBM diagnosis, reflecting age-related treatment intolerance), a composite molecular risk score (combining IDH, MGMT, and TERT status), and a high-grade indicator. This model tests whether biologically motivated feature engineering improves performance beyond what XGBoost learns from raw features alone.

4.4.2 Survival Analysis Models

We analysed the clean feature set using three survival modelling approaches.

Model 6: Cox Proportional Hazards. The semi-parametric Cox model was fitted using the `scikit-survival` library with L2 regularisation (penaliser = 0.1) for numerical stability. Coefficients provide interpretable hazard ratios: $\exp(\beta_i) > 1$ indicates increased mortality risk per unit increase in feature x_i , whilst $\exp(\beta_i) < 1$ indicates a protective effect.

Model 7: Random Survival Forest. An ensemble of 100 survival trees with random feature subsampling at each split. Unlike the Cox model, the Random Survival Forest makes no assumption about the functional form of the hazard function and can capture non-linear relationships and interactions between features. Permutation importance was used to rank feature contributions.

Model 8: Gradient Boosted Survival Analysis. Gradient boosting applied to the negative log partial likelihood loss, implemented via `sksurv.ensemble.GradientBoostingSurvivalAnalysis`. Early stopping was used to prevent overfitting, with validation loss monitored across boosting iterations.

4.4.3 Hyperparameter Optimisation

We selected hyperparameters for XGBoost models using randomised search cross-validation (`RandomisedSearchCV`) with 5-fold stratified splits. The search spaces included learning rate ($\eta \in [0.01, 0.3]$), maximum tree depth ($d \in \{3, \dots, 10\}$), minimum child weight, subsample ratio, and column subsampling ratio. We evaluated fifty random parameter combinations, with early stopping on validation loss (patience = 10 rounds). A fixed random seed of 42 was used throughout all experiments to ensure reproducibility of data splits, model training, and random search.

4.5 Radiomics Analysis

This exploratory analysis assessed whether pre-extracted imaging features provide additional predictive value beyond clinical and molecular markers. Nine imaging features were extracted from the pre-computed tumour segmentation volumes: tumour volume (mm^3) and first-order intensity statistics (mean and standard deviation) for each of the four MRI modalities (T1 native, T1 post-contrast, T2-weighted, T2-FLAIR). We normalised patient identifier formats between the clinical CSV and imaging directories, which used different naming conventions. After string-based alignment, 202 of 203 patients (99.5%) had matched records.

We trained a combined model on the union of clinical, molecular, and imaging features ($p = 36$). We compared performance against clinical-only ($p = 27$) and imaging-only ($p = 9$) models using the same XGBoost architecture and cross-validation strategy, and used a paired t -test on fold-wise AUC scores to assess the statistical significance of any performance difference.

4.6 Tumour Growth Prediction

Whilst the preceding analyses address survival as a binary or time-to-event outcome, a complementary clinical problem concerns tumour kinetics. Predicting tumour volume change between consecutive imaging sessions from baseline features could inform follow-up scheduling and treatment intensity decisions.

4.6.1 Target Construction

For each patient with at least two imaging timepoints, we computed absolute volume change between the first and second scans:

$$\Delta V = V_{t_1} - V_{t_0} \quad (4.1)$$

where V_{t_0} and V_{t_1} denote total tumour volume (mm^3) at the first and second timepoints respectively. Positive values indicate tumour growth, negative values indicate shrinkage (potentially treatment-induced), and values near zero indicate stability. Inter-scan intervals were not available in the dataset, precluding computation of a normalised growth rate (mm^3/day); absolute volume change was therefore used as the regression target. Of the 203 patients in the full cohort, 105 had at least two imaging timepoints and were included in this analysis.

4.6.2 Feature Sets and Models

Three feature configurations were used as inputs: clinical only (the leakage-free clinical and molecular features from Section 2.3.3), imaging only (nine features from the first timepoint segmentation), and combined (union of both). We applied the leakage detection framework to the combined feature set prior to model training, removing five progression-related features (time to first progression, date of first progression, type of first/second progression, and second recurrence) to ensure temporal validity.

Each configuration was modelled with three regression algorithms: ordinary least squares linear regression (interpretable linear baseline), ridge regression (L2-regularised, with α selected via cross-validation to address multicollinearity among imaging features), and XGBoost regressor (gradient boosted regression trees with squared error loss, capable of capturing non-linear relationships). We evaluated models using repeated K -fold cross-validation (5 folds $\times$ 5 repeats, yielding 25 evaluations per configuration) with coefficient of determination (R^2), mean absolute error (MAE, mm^3), and root mean squared error (RMSE, mm^3). $R^2 = 0$ indicates performance equivalent to predicting the mean; negative values indicate performance worse than the mean baseline. As a clinical sanity check, patients were stratified into growth tertiles based on observed ΔV , and mortality rates were compared across groups using a chi-squared test.

4.7 Evaluation Protocol

4.7.1 Cross-Validation Strategy

We used stratified 5-fold cross-validation for all model evaluations. We chose $K = 5$ to balance the bias-variance trade-off for the available sample size ($n = 203$). Stratification maintained the class distribution (alive/deceased ratio of approximately 53:47) across all folds. For survival models, stratification was based on the event indicator. The same fold assignments were used across all models to enable paired statistical comparisons.

4.7.2 Metrics

Binary Classification. AUC-ROC was the primary metric due to its threshold-independence and interpretability as the probability that a randomly selected deceased patient receives a higher predicted risk score than a randomly selected surviving patient. Secondary metrics included accuracy, precision, recall, and F1 score. All metrics are reported as mean $\pm$ standard deviation across folds.

Survival Analysis. The concordance index (C-index) was the primary survival metric, representing the proportion of all evaluable patient pairs for which the model correctly orders survival times. A C-index of 0.5 indicates random performance, whilst 1.0 indicates perfect concordance.

Statistical Comparison. We used paired t -tests on fold-wise scores to assess whether performance differences between models were statistically significant at $\alpha = 0.05$. Pairing was appropriate because the same fold assignments were used across all models, reducing variance from data splitting.

4.8 Software and Implementation

We implemented all experiments in Python 3 using Jupyter Notebooks for exploratory analysis and model development. Libraries used: `pandas` and `numpy` for data manipulation; `scikit-learn` for preprocessing, cross-validation, and logistic regression; `xgboost` for gradient boosted classification; `scikit-survival` for Cox Proportional Hazards, Random Survival Forest, and Gradient Boosted Survival Analysis; `lifelines` for Kaplan-Meier estimation; and `matplotlib` and `seaborn` for visualisation. The complete codebase is organised into six notebooks corresponding to the analysis phases: preprocessing and exploratory data analysis, binary classification with leakage detection, survival analysis, pre-extracted radiomics evaluation, tumour growth prediction, and comprehensive PyRadiomics feature extraction.

Chapter 5

Leakage Detection

The MU-Glioma-Post dataset, like many post-treatment oncology databases, intermixes baseline and post-diagnosis features with no metadata distinguishing them. Given that 37% of reviewed glioma ML studies include such features [7], we investigate which features in this dataset encode future information and measure the resulting performance inflation. We then test whether the detection approach generalises across analytical contexts within the same dataset.

5.1 Detection Methodology

Building on the healthcare leakage framework of Davis et al. [4] (Figure 5.1), the detection pipeline comprises four sequential steps combining statistical screening with clinical domain reasoning.

Step 1: Feature-Target Correlation Screening. We computed Pearson correlation coefficients between each numerical feature and the binary outcome (death). This step identifies features with potentially suspicious associations with the target. We visualised the full correlation structure to reveal clustering patterns among high-correlation features, providing a foundation for the subsequent domain-expert review.

Step 2: Threshold Application. We flagged features with $|r| > 0.25$ for temporal review. This threshold was selected empirically to capture features with at least moderate outcome association whilst excluding the majority of legitimate clinical variables. At $|r| > 0.25$, the screening flagged 6 of 28 features; lowering to $|r| > 0.15$ would have flagged 12 features (introducing excessive false positives among weak but legitimate predictors), whilst raising to $|r| > 0.35$ would have missed four of the seven confirmed leakage features. Legitimate molecular markers such as IDH mutations are expected to correlate strongly with outcome; the threshold is therefore a screening step, not a definitive classifier, and must be followed by temporal reasoning to distinguish legitimate biological signal from future information leakage. The chosen threshold aligns with the range recommended by Davis et al. [4] for healthcare leakage detection.

Step 3: Temporal Reasoning. We evaluated each flagged feature for availability at the time of diagnosis. Following Davis et al. [4], we classified features as appropriate (available at the clinical prediction timepoint) or inappropriate (requiring future information). Features describing disease progression, recurrence events, or post-diagnosis procedures were identified as temporal leakage. Features with high target correlation were retained where temporal reasoning confirmed their availability at baseline. IDH mutation status ($r = -0.52$) and age at diagnosis ($r = 0.38$) both exceeded the threshold but were retained

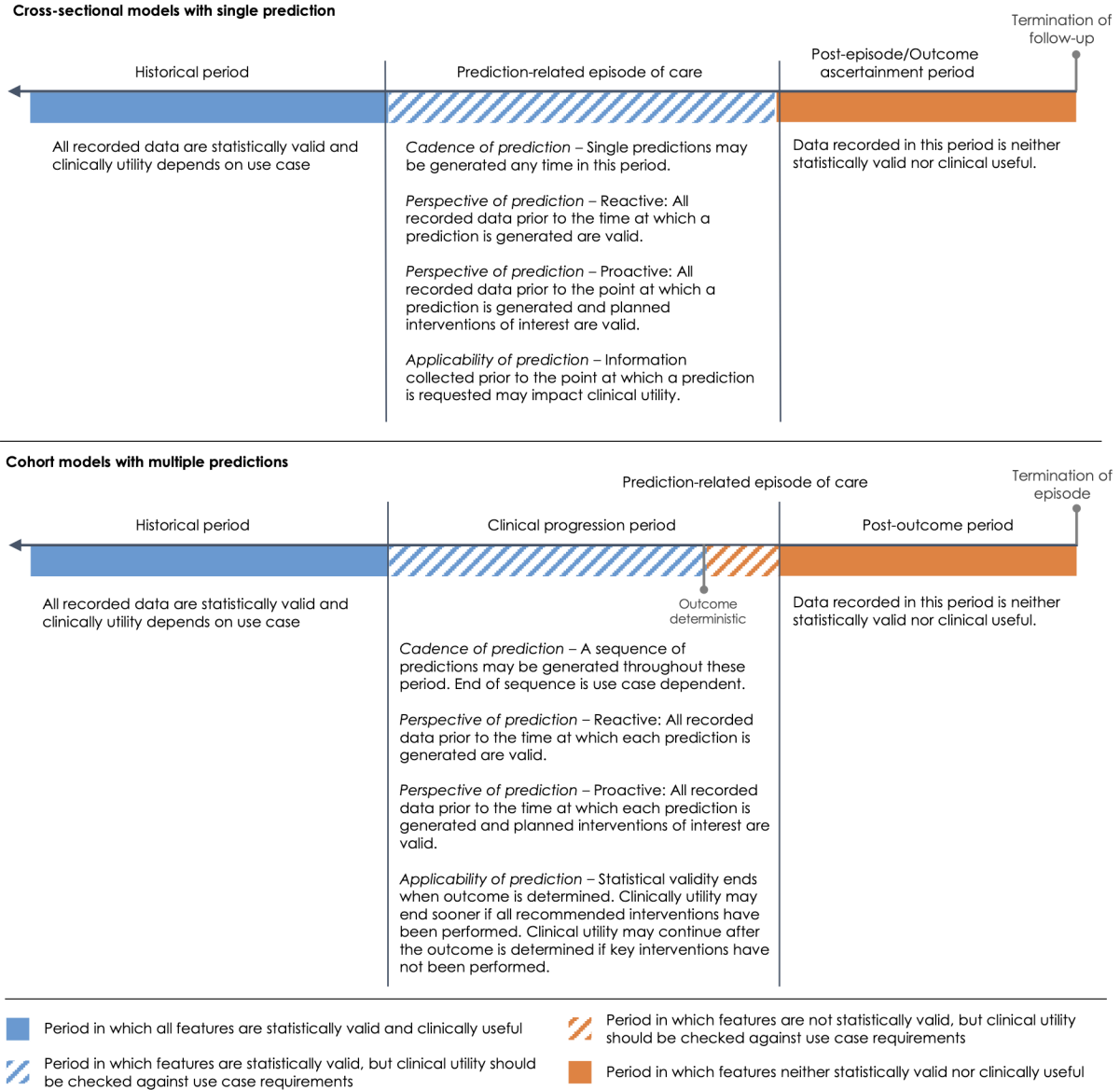


Figure 5.1: Temporal validity framework for healthcare ML predictions. Features from the post-outcome period (orange) constitute label leakage. For cross-sectional models (top), all features must precede the prediction timepoint; for cohort models with repeated predictions (bottom), statistical validity ends at outcome determination. This framework motivates the temporal reasoning applied in Step 3 of the detection pipeline. Adapted from Davis et al. [4].

as legitimately available at diagnosis, as both are determined at or prior to the point of treatment planning.

Step 4: Feature Removal and Documentation. We removed confirmed leakage features from the clean model’s feature set, documenting each removal decision with the feature name, correlation value, temporal justification, and prevalence. The baseline model retaining leakage features was preserved for comparison, enabling direct quantification of leakage impact on performance. This methodology addresses types L2.2 (feature includes target information) and L3.1 (temporal leakage) from the taxonomy of Kapoor and Narayanan [3].

5.2 Detection Results

5.2.1 Correlation Analysis

We performed a systematic correlation analysis to identify potential data leakage features. Figure 5.2 presents two complementary views: the feature-target correlation ranking for all features, and a correlation matrix among the most suspicious high-correlation features.

The correlation ranking revealed a clear separation between two groups. Features with the highest absolute correlations included IDH2 mutation ($r = -0.33$), IDH1 mutation ($r = -0.32$), time to first progression ($r = 0.30$), and days to first progression ($r = 0.29$). The IDH mutations represent legitimate biological associations available at diagnosis, whereas the progression-related features encode future information. The inter-feature correlation matrix for suspicious features confirmed that progression-related variables clustered together (mutual correlations > 0.6), whilst the molecular markers formed a separate cluster, supporting the distinction between legitimate and leakage features.

5.2.2 Identified Leakage Features

We applied temporal reasoning to determine whether each flagged feature would be available at the time of diagnosis. We identified seven features as containing information from future events and removed them from the clean model:

1. **Second progression/recurrence:** binary indicator of whether a second tumour progression occurred. This information is only available after longitudinal follow-up and monitoring.
2. **Type of 2nd progression:** categorical description of second progression type (local, distant, or both).
3. **Time to first progression (days):** discrete temporal distance to the first progression event. This is a reframed outcome variable, as patients who progress rapidly tend to die sooner.
4. **Number of days from diagnosis to date of first progression:** equivalent temporal leakage encoding the same information in a different format.
5. **Number of days from diagnosis to first surgery or procedure:** post-diagnosis treatment timing, which reflects both disease urgency and institutional scheduling rather than baseline patient characteristics.
6. **Progression:** binary indicator of any disease progression during follow-up.

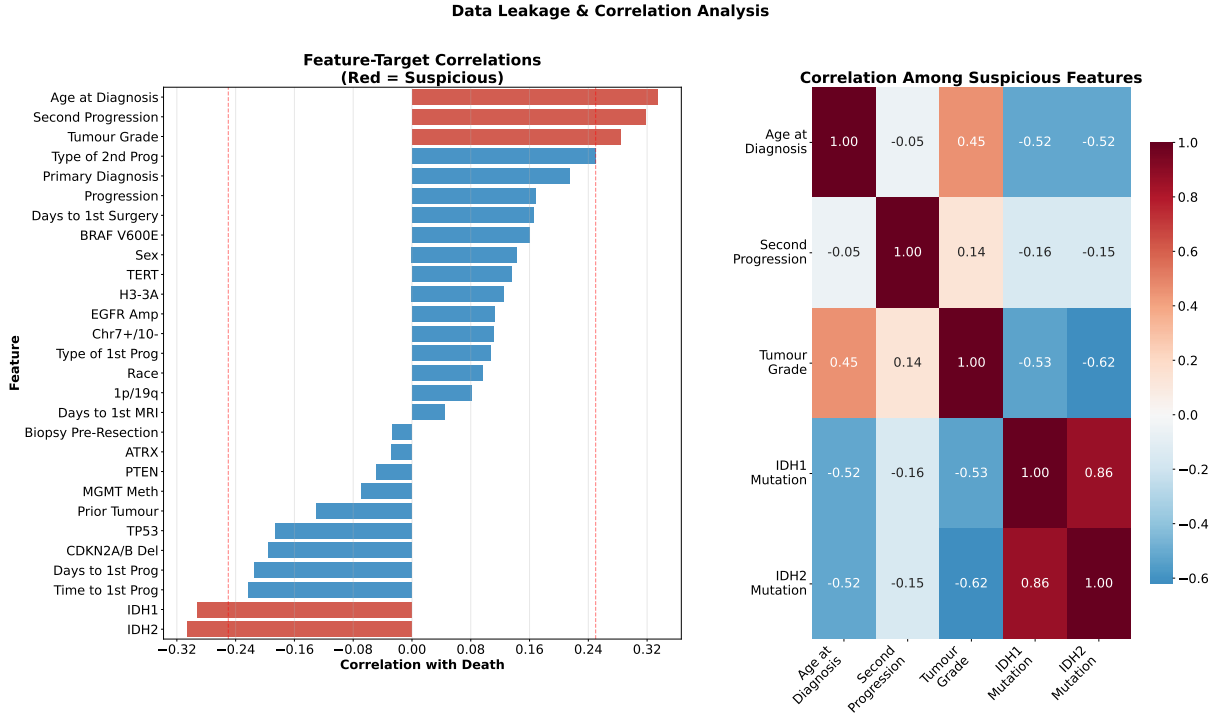


Figure 5.2: Correlation analysis identifying potential leakage features. Left: feature-target correlations with the outcome variable, with red bars indicating features flagged above the $|r| > 0.25$ threshold. Right: inter-correlation matrix among the most suspicious features, revealing clustering of progression-related variables separately from biological markers.

7. Type of 1st progression: categorical description of first progression type.

These features represent information that would only become available *after* the prediction timepoint (diagnosis), rendering them unsuitable for a clinically deployable model. Their inclusion in the baseline model demonstrates a common source of performance inflation in medical machine learning studies. Features such as IDH mutation status ($r = -0.32$), age at diagnosis ($r = 0.25$), and MGMT methylation ($r = -0.15$) were retained despite exhibiting target correlation, because temporal reasoning confirmed their availability at the diagnostic timepoint.

5.2.3 Impact on Feature Count

Removing leakage features reduced the feature set from 28 (baseline) to 21 (clean model), a 25% reduction in dimensionality. The clean feature set comprises demographics (age, sex, race), tumour characteristics (diagnosis, grade, location, biopsy status), molecular markers (IDH1, IDH2, MGMT, TERT, 1p/19q, TP53, ATRX, EGFR, CDKN2A/B, chromosome 7/10, H3-3A), and a single temporal variable (days from diagnosis to first MRI, retained as a baseline scheduling measure).

5.3 Impact Quantification

Removing leakage features decreased XGBoost AUC from 0.730 ± 0.091 to 0.674 ± 0.111 , a 7.7% relative reduction. Approximately 8% of the baseline model’s discriminative ability was therefore attributable to information unavailable at the clinically relevant prediction timepoint. The magnitude corroborates the findings of Kapoor and Narayanan [3], who

reported a median AUC inflation of 0.12 across studies with confirmed leakage; the smaller effect here likely reflects the relatively low absolute correlation values of the identified features ($|r| \leq 0.30$) compared to more egregious leakage cases in the literature.

The pipeline was independently validated during the growth prediction analysis (Section 6.5). Five progression-related features persisted in the separately preprocessed growth prediction data, having survived a different preprocessing pipeline with different inclusion criteria ($n = 105$ patients with longitudinal imaging). After their removal, the only model achieving positive R^2 (0.15) dropped to $R^2 = -0.15$, confirming that its entire apparent signal originated from temporal leakage. This cross-task validation shows that the four-step methodology can detect leakage features even when they enter through independent preprocessing pathways.

A 7.7% AUC inflation, though modest in absolute terms, represents the difference between a model with potential clinical utility and one offering only marginal discrimination above chance. The cross-task validation reported above further shows that temporal contamination is a structural property of post-treatment oncology databases rather than an artefact of a single preprocessing pipeline: the same category of leakage features recurs whenever the complete patient trajectory is available to the modeller. Recent systematic reviews corroborate this pattern, finding that 37% of glioma ML studies include features unavailable at diagnosis [7, 8].

Chapter 6

Prediction Modelling

Experimental results span five analysis streams: binary classification, survival analysis, feature importance, radiomics evaluation, and tumour growth prediction. Each section describes the modelling approach and presents results in an interleaved structure. A consolidated summary closes the chapter.

6.1 Binary Classification

6.1.1 Models

The five binary classification models described in Section 4.4.1 were evaluated using stratified 5-fold cross-validation. We selected hyperparameters for XGBoost models via randomised search CV (50 combinations, 5-fold stratified splits). Randomised search was preferred over grid search because several parameters (`learning_rate`, `subsample`, `reg_lambda`) are continuous, making exhaustive enumeration impractical; with 50 random samples, the probability of finding a configuration within the top 5% of the search space exceeds 0.92 [31]. The full search space and selected values are reported in Appendix C. Hyperparameter sensitivity was low: the top five configurations produced AUC values within 0.02 of one another, indicating that performance on this cohort is constrained by sample size rather than tuning precision.

6.1.2 Results

Table 6.1 presents the complete performance comparison.

Model	AUC-ROC	Accuracy	F1	Features
Baseline (with leakage)	0.730 $\pm$ 0.091	0.729 $\pm$ 0.092	0.729 $\pm$ 0.082	28
Clean XGBoost	0.674 $\pm$ 0.111	0.670 $\pm$ 0.113	0.687 $\pm$ 0.094	21
Regularised XGBoost	0.623 $\pm$ 0.084	0.621 $\pm$ 0.085	0.633 $\pm$ 0.058	21
Logistic Regression	0.692 $\pm$ 0.119	0.655 $\pm$ 0.101	0.692 $\pm$ 0.082	21
Feature-Engineered	0.677 $\pm$ 0.124	0.675 $\pm$ 0.125	0.687 $\pm$ 0.098	25

Table 6.1: Binary classification model comparison (stratified 5-fold CV). All values reported as mean $\pm$ standard deviation. The baseline model includes leakage features and is presented for reference only; it should not be considered a valid benchmark.

Among the four clean models, logistic regression produced the highest mean AUC (0.692 $\pm$ 0.119), with clean XGBoost close behind (0.674 $\pm$ 0.111). The overlapping

confidence intervals indicate that these differences are not statistically significant at the $\alpha = 0.05$ level. Logistic regression’s competitive performance is consistent with the finding that linear models often match or exceed tree-based methods on small, low-dimensional tabular datasets where non-linear interactions are limited [32]. The regularised XGBoost model underperformed ($\text{AUC } 0.623 \pm 0.084$), suggesting that aggressive regularisation overly constrained the model’s capacity on the 21-feature clean set, and that default XGBoost parameters already balance fitting capacity and generalisation for this dataset. Feature engineering yielded no meaningful improvement ($\text{AUC } 0.677 \pm 0.124$ versus 0.674 ± 0.111 for clean XGBoost), indicating that XGBoost can learn the relevant interaction patterns from raw features.

Figure 6.1 presents the ROC curves for all five models.

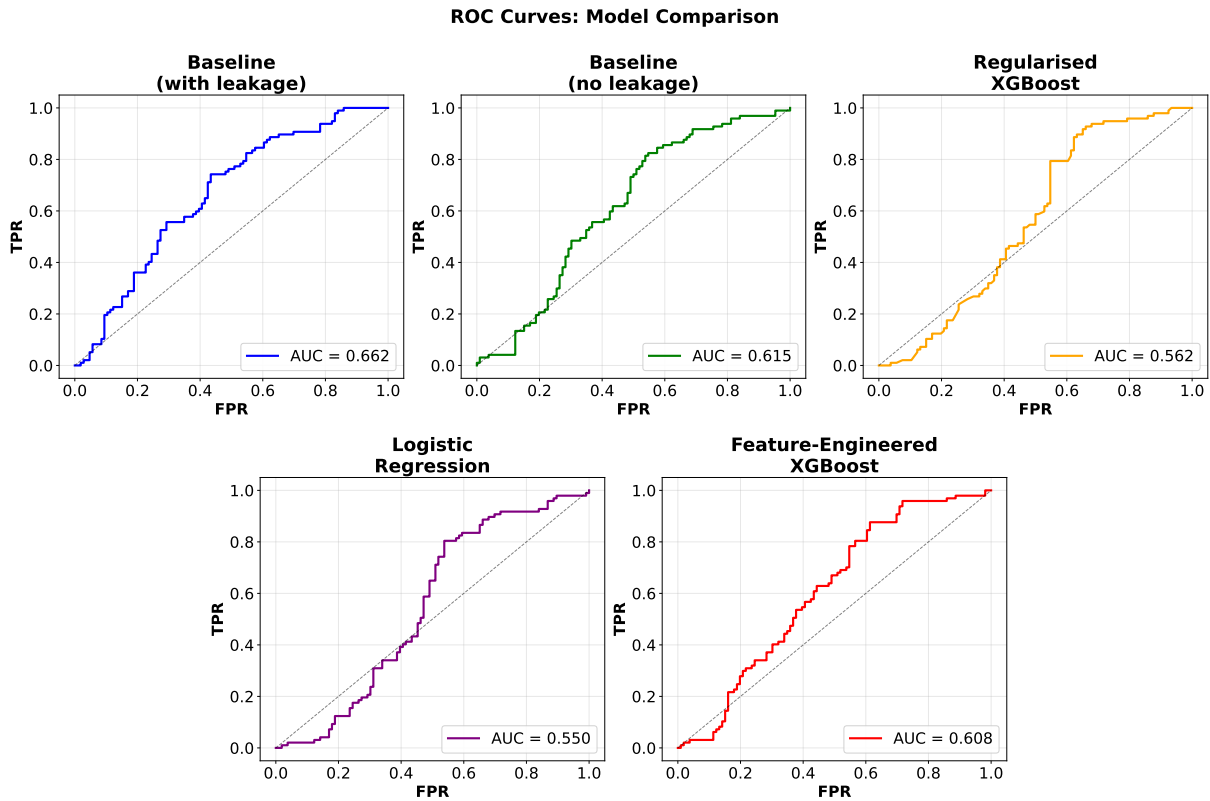


Figure 6.1: ROC curves comparing all five binary classification models. The dashed diagonal represents random chance ($\text{AUC} = 0.5$). The baseline model (with leakage) demonstrates visibly inflated performance relative to the clean models, which cluster together in the $\text{AUC } 0.62$ – 0.69 range.

Figure 6.2 presents the confusion matrices aggregated across cross-validation folds. The logistic regression model exhibited the highest recall for predicting mortality, correctly identifying a greater proportion of deceased patients at the cost of more false positive predictions. The clean XGBoost model showed more balanced precision–recall trade-offs.

6.2 Survival Analysis

6.2.1 Models

The three survival models described in Section 4.4.2 were evaluated using 5-fold cross-validation on the clean feature set. We selected the Cox PH penaliser (0.1) to provide moderate coefficient shrinkage given the ratio of 21 features to 202 patients;

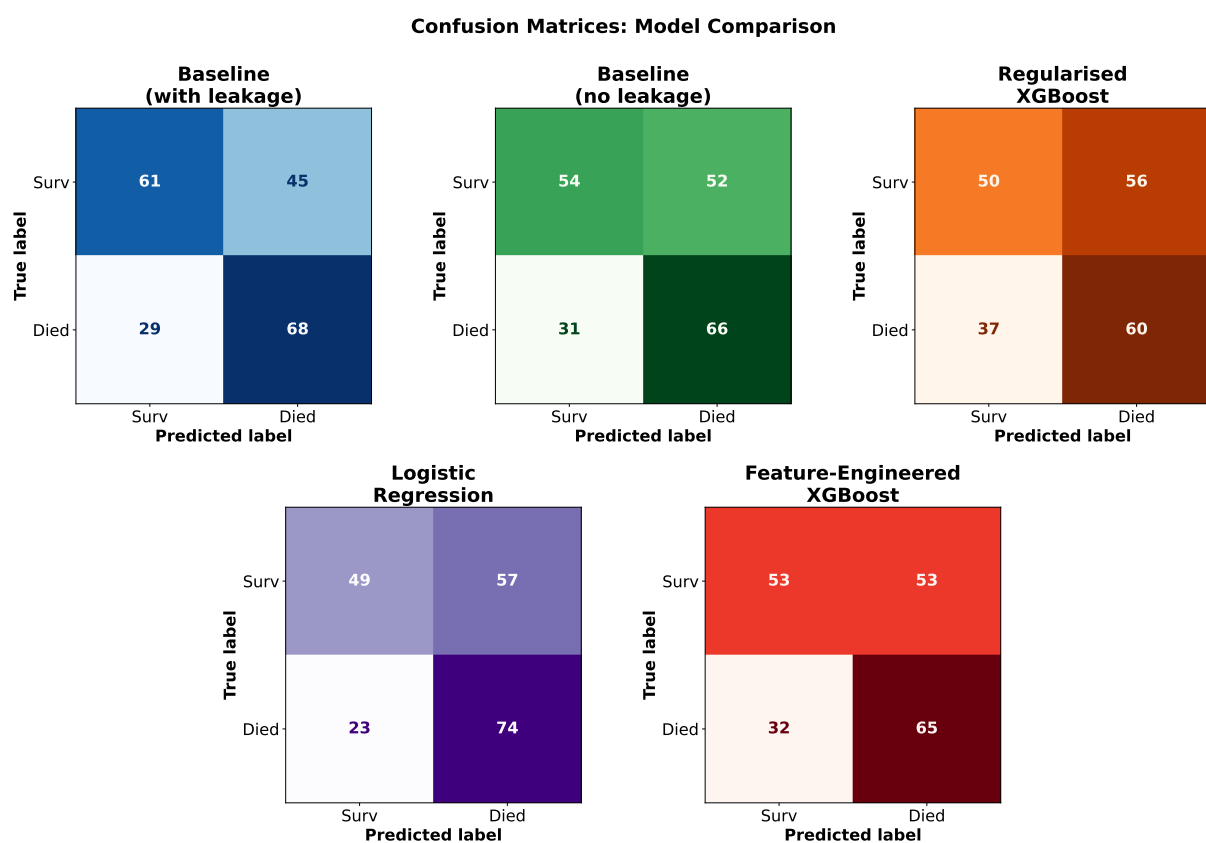


Figure 6.2: Confusion matrices for all five binary classification models, aggregated across 5-fold cross-validation. The baseline model (with leakage) is shown for reference alongside the four clean models.

higher values (0.5, 1.0) produced excessive shrinkage that degraded C-index by 0.03–0.05. RSF convergence was verified by confirming that doubling the tree count to 200 yielded negligible C-index improvement (<0.005).

6.2.2 Results

Figure 6.3 presents the distribution of constructed survival times in the cohort. The median constructed survival time was 249 days, computed as days from diagnosis to death for deceased patients and days from diagnosis to last MRI for censored patients. Deceased patients had a median time-to-death of 337 days, whilst censored patients had a median last follow-up of 215 days, consistent with a surviving cohort enriched for lower-grade or IDH-mutant tumours with longer disease courses.

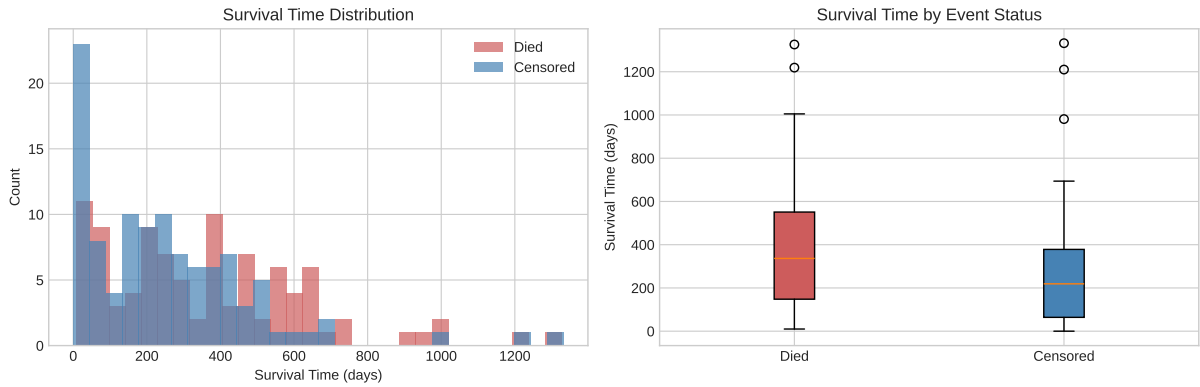


Figure 6.3: Distribution of survival times (days from diagnosis to death or last follow-up). Left: histogram with separate shading for deceased and censored patients. Right: box plot comparing survival times between outcome groups.

Kaplan-Meier survival curves stratified by RSF-predicted risk groups are presented in Figure 6.4. The Random Survival Forest was used to assign patients to low, medium, and high risk terciles based on predicted survival probabilities. The resulting curves showed separation between risk groups: the high-risk group had rapid early mortality and the low-risk group maintained survival probability above 0.6 at 1,000 days. This stratification confirms the clinical utility of the model’s risk predictions even at the moderate C-index observed. Univariate Kaplan-Meier analyses confirmed that IDH-mutant gliomas had significantly improved survival compared to IDH wild-type tumours (log-rank $p < 0.001$), as established in the literature, and that MGMT promoter methylation was associated with improved overall survival, consistent with enhanced response to temozolomide chemotherapy.

Table 6.2 summarises the model comparison across all three approaches.

Model	C-index	Individual Fold Scores
Cox Proportional Hazards	0.667 ± 0.102	0.608, 0.726, 0.777, 0.726, 0.496
Random Survival Forest	0.706 ± 0.069	0.746, 0.619, 0.794, 0.743, 0.629
Gradient Boosted Survival	0.705 ± 0.089	0.738, 0.591, 0.803, 0.784, 0.608

Table 6.2: Survival analysis model comparison (5-fold CV). C-index reported as mean $\pm$ standard deviation. Individual fold scores illustrate per-fold variability.

Of the three approaches, the Random Survival Forest produced the highest mean C-index (0.706 ± 0.069), with Gradient Boosted Survival Analysis close behind (0.705 ± 0.089)

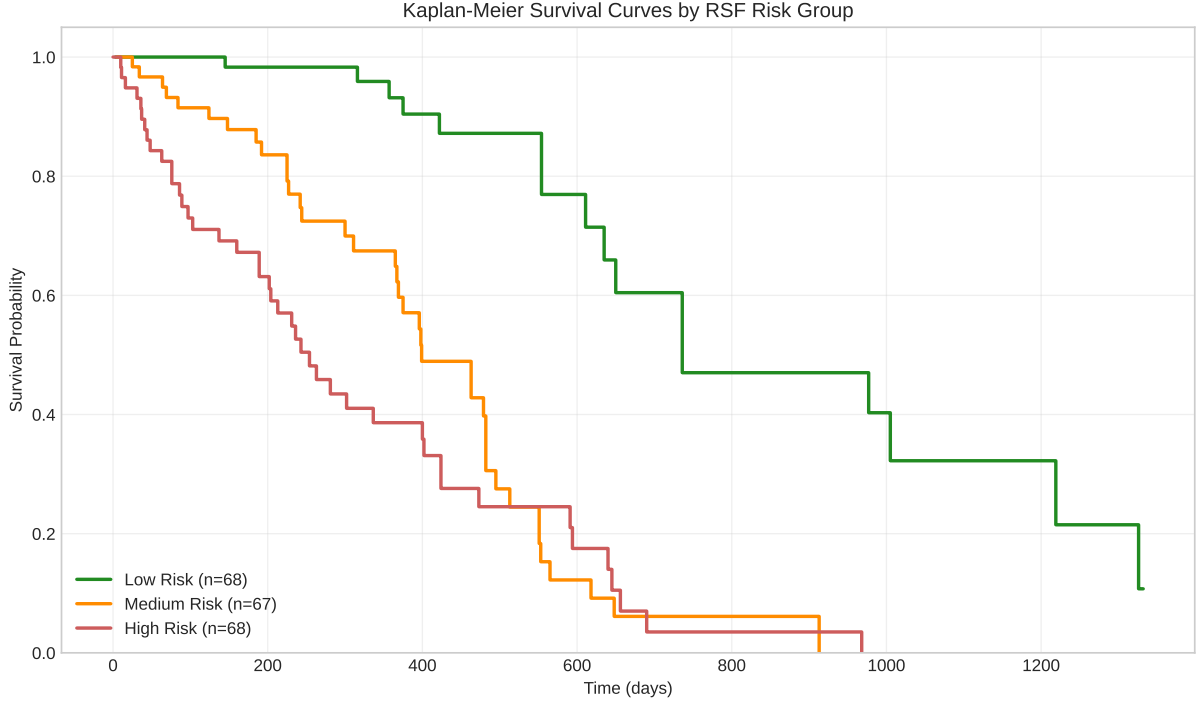


Figure 6.4: Kaplan-Meier survival curves stratified by RSF-predicted risk group (low, medium, high risk terciles). Clear separation between groups validates the discriminative capacity of the survival model.

and Cox Proportional Hazards trailing (0.667 ± 0.102). Both machine learning approaches had lower variance across folds than the Cox model. The Cox model’s Fold 5 score (0.496, below random chance) exposes its sensitivity to unfavourable data splits, likely caused by small subgroup sizes interacting with parametric assumptions.

Figure 6.5 provides a visual comparison of C-index across methods.

Figure 6.6 presents the coefficients from the Cox Proportional Hazards model, providing interpretable hazard ratios for the key prognostic features.

Diffuse glioma diagnosis produced the strongest protective coefficient (-1.264), reflecting that this category predominantly comprises lower-grade, IDH-mutant tumours with favourable prognosis. Tumour grade (coefficient = 0.883) was strongly associated with worse prognosis, as expected given the grading system’s prognostic intent. Age at diagnosis (coefficient = 0.673) confirmed each additional year as associated with increased mortality hazard. The IDH1 mutation coefficient (0.378) is counterintuitive given the known protective effect, reflecting collinearity with diagnosis and grade variables that already capture much of IDH’s prognostic information in the multivariate setting; in univariate analysis, IDH mutation is strongly protective.

6.3 Feature Importance

6.3.1 Binary Classification Feature Importance

Figure 6.7 presents the XGBoost feature importance scores (gain-based) for the clean binary classification model, and Table 6.3 lists the top-10 ranking with exact gain values. IDH1 mutation (19.4%), primary diagnosis (19.3%), and TERT promoter mutation (14.0%) dominated, reflecting their definitional role in the WHO CNS5 classification.

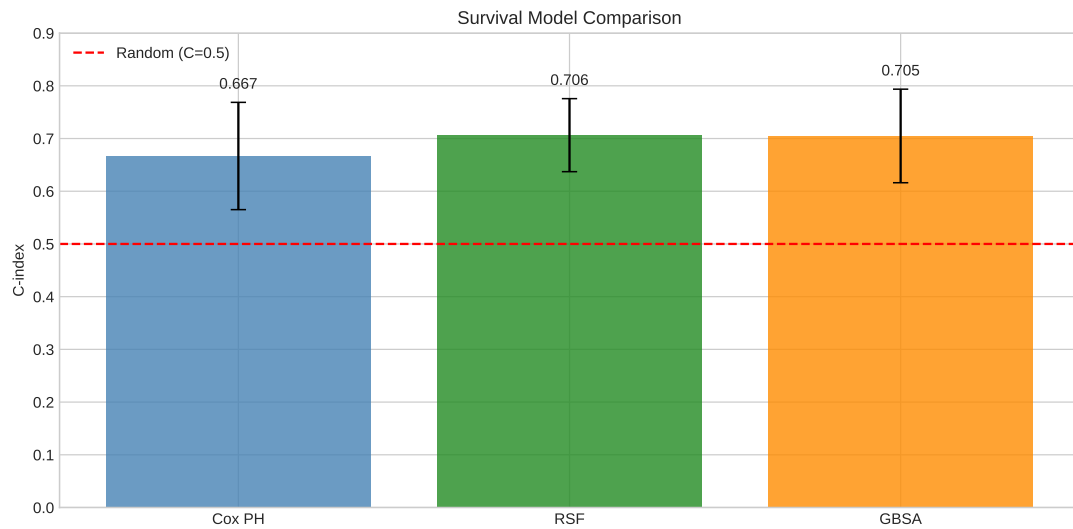


Figure 6.5: Comparison of C-index scores across survival analysis methods. Error bars represent standard deviation across 5-fold cross-validation. The dashed line at C-index = 0.5 indicates random chance.

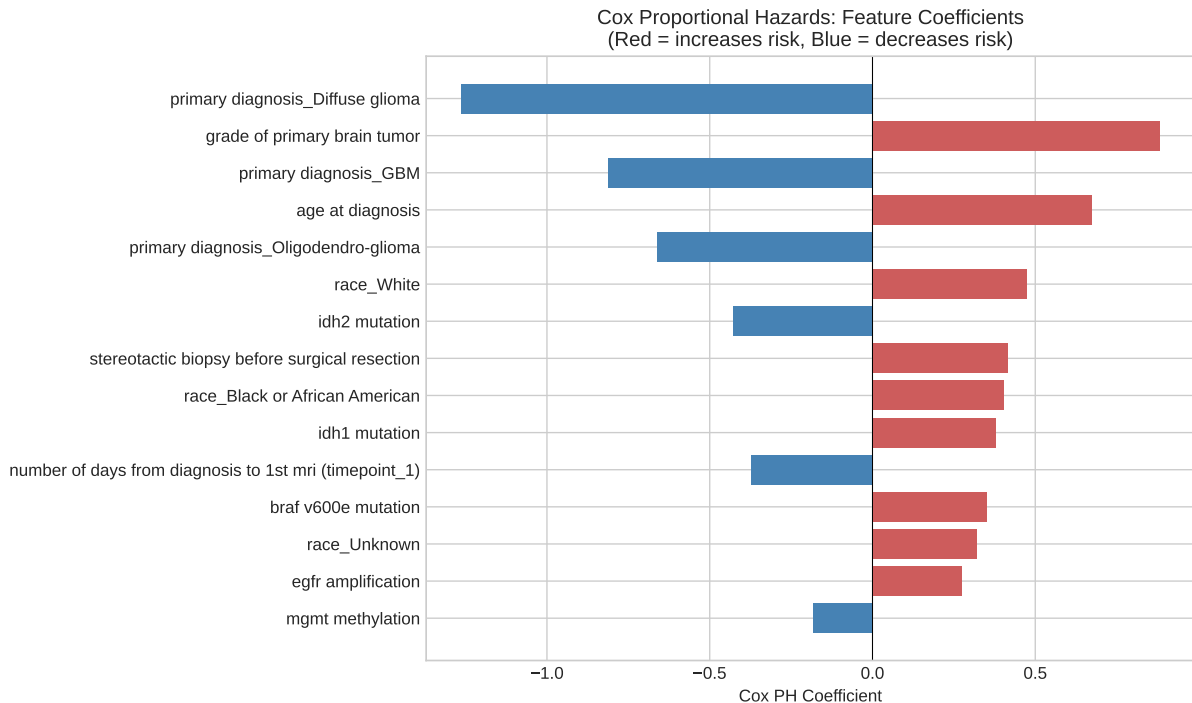


Figure 6.6: Cox Proportional Hazards model coefficients with 95% confidence intervals. Red bars indicate features that increase mortality risk; blue bars indicate protective features.

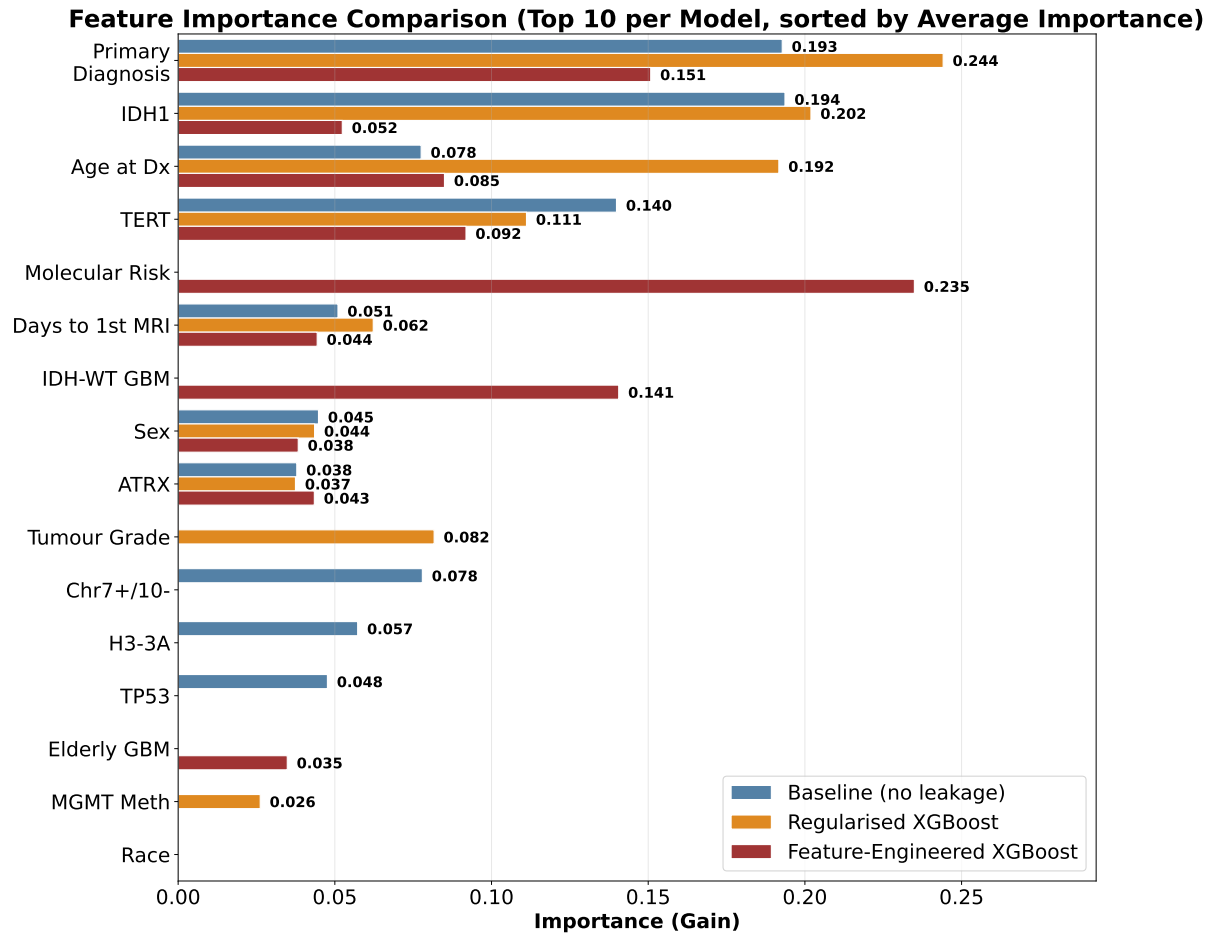


Figure 6.7: Feature importance scores (XGBoost gain) for the clean binary classification model across three model variants: baseline (no leakage), regularised, and feature-engineered. Molecular markers dominate the top predictive features consistently across variants.

Rank	Feature	Importance (%)
1	IDH1 mutation	0.194
2	Primary diagnosis	0.193
3	TERT promoter mutation	0.140
4	Chromosome 7 gain / Chromosome 10 loss	0.078
5	Age at diagnosis	0.078
6	H3-3A mutation	0.057
7	Days from diagnosis to first MRI	0.051
8	TP53 alteration	0.048
9	Sex at birth	0.045
10	ATRX mutation	0.038

Table 6.3: Top 10 features ranked by XGBoost importance (gain) in the clean binary classification model.

6.3.2 Survival Analysis Feature Importance

The Random Survival Forest and Gradient Boosted Survival Analysis models exhibited markedly different feature importance patterns compared to binary classification, and also differed from each other. Table 6.4 compares the top features across both survival approaches.

Feature	RSF (%)	GBSA (%)
Age at diagnosis	9.3	39.5
Days to first MRI	6.8	36.5
1p/19q co-deletion	0.5	2.0
EGFR amplification	0.4	1.2
MGMT methylation	0.4	2.8
Tumour grade	0.4	4.0
Previous brain tumour	−0.2	4.0
IDH1 mutation	0.1	1.3

Table 6.4: Comparison of feature importance rankings between Random Survival Forest (permutation importance) and Gradient Boosted Survival Analysis (gain importance).

Age at diagnosis dominated both survival models (9.3% RSF; 39.5% GBSA), reflecting the cumulative nature of time-to-event modelling where age has a multiplicative impact on hazard that binary classification cannot capture equivalently. Days from diagnosis to first MRI (6.8% RSF; 36.5% GBSA) is plausibly a proxy for disease urgency, healthcare access, or tumour aggressiveness prompting rapid imaging. Molecular markers that dominated binary classification (IDH1, TERT) ranked much lower in survival models, likely due to redundancy with diagnosis and grade variables that capture overlapping prognostic information in the multivariate setting. The GBSA model concentrated importance more heavily on its top two features (76% combined) compared to RSF (16% combined), reflecting GBSA’s tendency to amplify the contribution of the most informative features through sequential boosting, whilst RSF’s bagging distributes importance more evenly.

6.3.3 Biological Interpretation

The feature importance rankings align with established glioma biology, though the relative weighting differs between classification and survival frameworks.

IDH1 mutation was the most important feature in binary classification (19.4%) but ranked much lower in survival models (0.1% RSF; 1.3% GBSA). This reduced survival importance reflects that IDH status correlates strongly with diagnosis and grade, which already capture overlapping prognostic information in the multivariate setting. TERT promoter mutations (14.0% binary; 0.3% RSF; 0.4% GBSA) and combined chromosome 7 gain / chromosome 10 loss (7.8% binary; 0.2% RSF; 2.3% GBSA) exhibited a similar pattern, with high binary classification importance diminishing in survival models due to redundancy with diagnostic and grade variables.

Age at diagnosis showed the largest difference between modelling approaches: moderate importance in binary classification (7.8%) but dominant in survival analysis (9.3% RSF; 39.5% GBSA). The survival framework’s explicit modelling of time-to-event amplifies age’s contribution, as age compounds with duration of follow-up in a way that binary alive/dead classification cannot capture.

6.4 Radiomics Analysis

6.4.1 Pre-Extracted Imaging Features

Nine imaging features were extracted from pre-computed segmentations: tumour volume and intensity statistics (mean, standard deviation) for each MRI modality. Figure 6.8 presents the tumour volume distribution and its relationship with clinical outcomes. Deceased patients showed a trend towards larger tumour volumes, but with substantial overlap between groups.

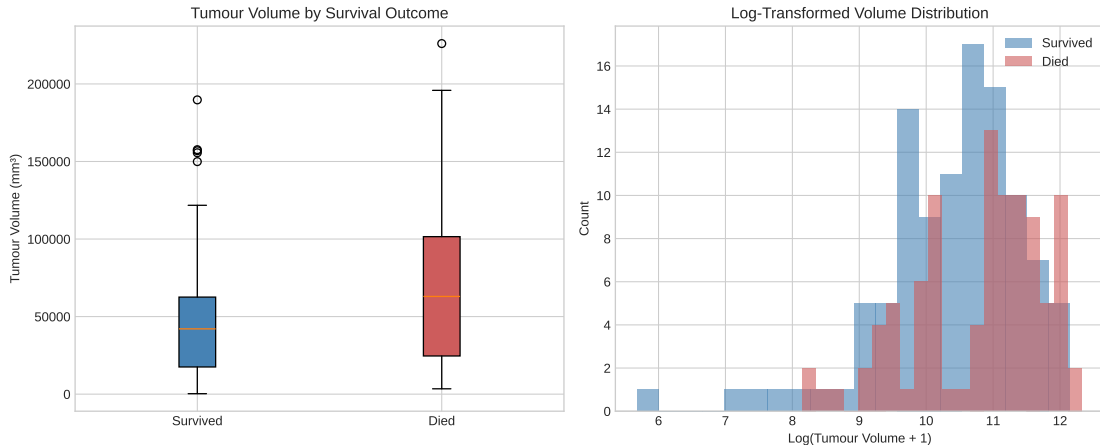


Figure 6.8: Tumour volume analysis showing distribution by survival outcome (left: box plot; right: log-transformed histogram). Deceased patients showed a trend towards larger tumour volumes, but with substantial overlap between groups.

Table 6.5 compares clinical-only, imaging-only, and combined models.

Model	AUC-ROC	Accuracy	F1	Features
Clinical only	0.722 $\pm$ 0.096	0.703	0.709	27
Imaging only	0.654 $\pm$ 0.054	0.584	0.560	9
Combined	0.720 $\pm$ 0.060	0.658	0.656	36

Table 6.5: Comparison of clinical-only, imaging-only, and combined (clinical + imaging) models for binary survival classification. The clinical-only model has 27 features (versus 21 in Table 6.1) because one-hot encoding of nominal variables (diagnosis, race) expands the column count. All values reported as mean $\pm$ standard deviation from 5-fold cross-validation.

A paired t -test yielded $\Delta\text{AUC} = -0.002$ ($p = 0.936$), confirming that imaging features did not improve predictive performance beyond clinical and molecular markers alone. The imaging-only model (AUC 0.654) demonstrated that pre-extracted intensity features carry some predictive signal, but this information is largely redundant with the clinical and molecular markers already available.

Figure 6.9 and Table 6.6 present the feature importance rankings in the combined model.

The seven clinical and molecular features occupying the top seven positions confirm that imaging features provide supplementary rather than primary predictive information. The highest-ranked imaging feature (T2w mean intensity, rank 8, 4.1% importance) is outweighed by the top clinical feature (10.7%). Figure 6.10 illustrates the distributions

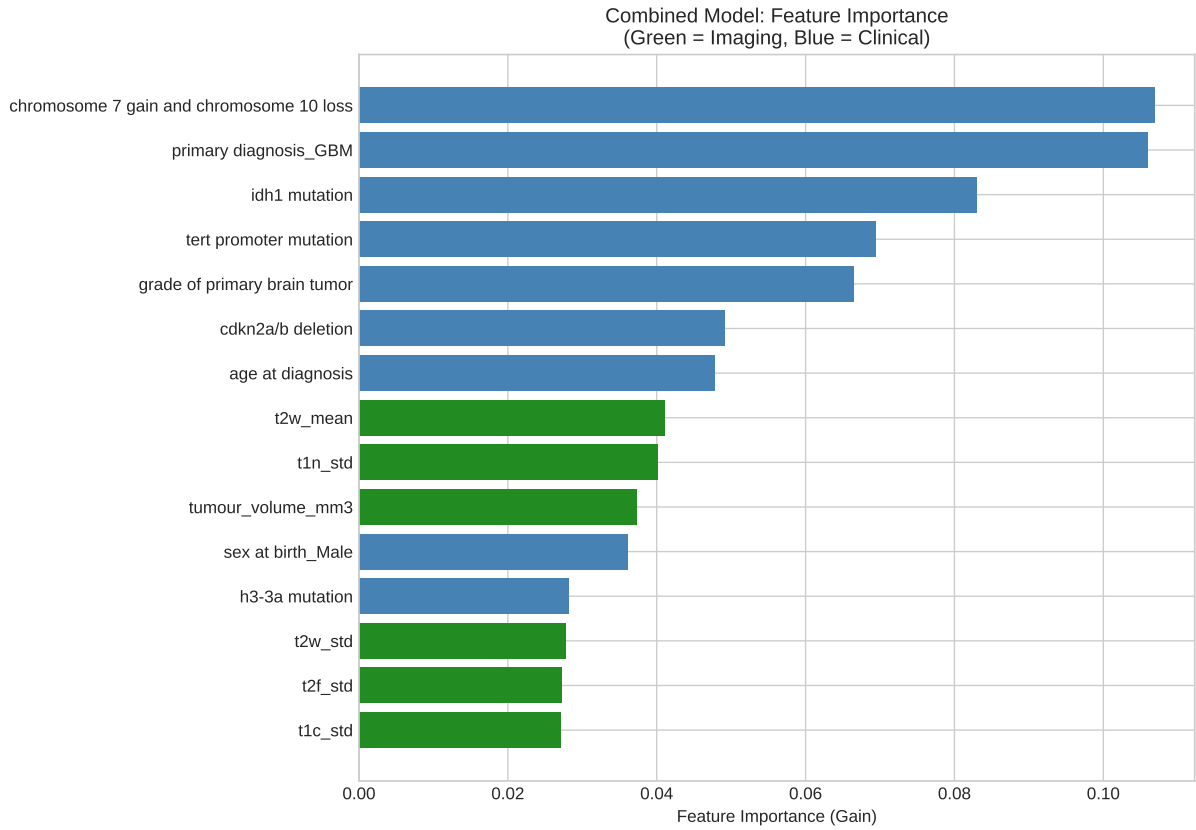


Figure 6.9: Feature importance in the combined clinical + imaging model. Clinical and molecular features (blue) dominate over imaging features (green), with the highest-ranked imaging feature (T2w mean intensity) appearing at rank 8.

Rank	Feature	Importance	Type
1	Chromosome 7 gain / Chromosome 10 loss	0.107	Clinical
2	Primary diagnosis (GBM)	0.106	Clinical
3	IDH1 mutation	0.083	Molecular
4	TERT promoter mutation	0.069	Molecular
5	Tumour grade	0.066	Clinical
6	CDKN2A/B deletion	0.049	Molecular
7	Age at diagnosis	0.048	Clinical
8	T2w mean intensity*	0.041	Imaging
9	T1n intensity std*	0.040	Imaging
10	Tumour volume*	0.037	Imaging
11	Sex at birth	0.036	Clinical
12	H3-3A mutation	0.028	Molecular
13	T2w intensity std*	0.028	Imaging
14	T2-FLAIR intensity std*	0.027	Imaging
15	T1c intensity std*	0.027	Imaging

Table 6.6: Top 15 features in the combined model ranked by XGBoost importance. Imaging features are indicated with asterisks (*).

of imaging features stratified by survival outcome; the substantial overlap across all nine features explains the limited additional predictive value. Tumour volume showed the most pronounced separation, in keeping with the known prognostic value of tumour burden, but this effect is largely captured by clinical variables (grade, diagnosis) that correlate with volume.

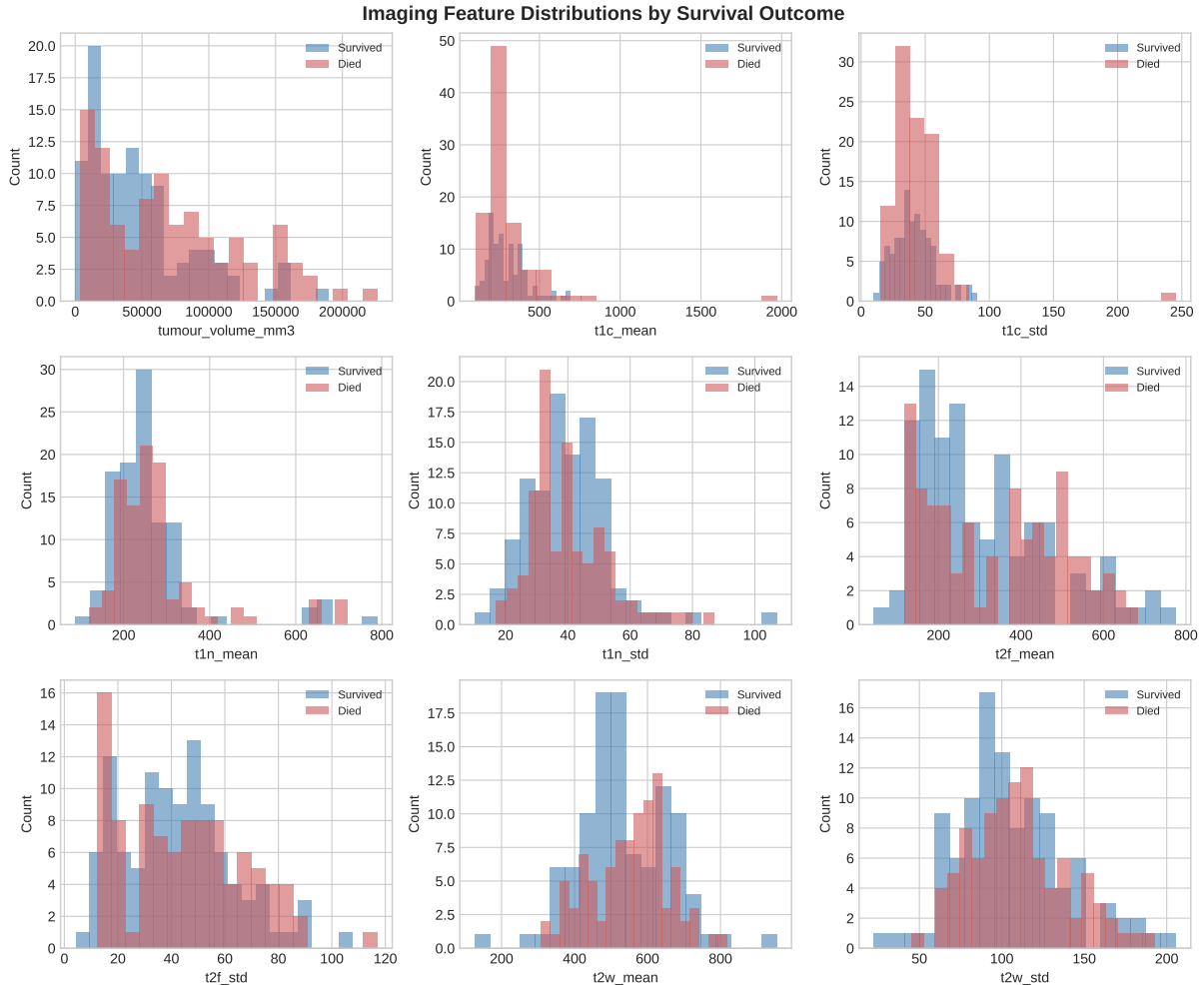


Figure 6.10: Distribution of imaging-derived features stratified by survival outcome (alive vs. deceased). Overlapping distributions across all nine features confirm the limited discriminative power of pre-extracted intensity statistics.

6.4.2 Comprehensive Radiomic Feature Extraction

To assess whether higher-order texture features capture prognostic information missed by simple volume and intensity statistics, we ran a PyRadiomics feature extraction [23]. We extracted features from T1-weighted contrast-enhanced (T1ce) MRI sequences using expert-annotated tumour segmentations across 202 of 203 patients. Figure 6.11 illustrates the imaging input for three representative patients, showing the T1ce volumes with tumour segmentation overlays that delineate the regions from which radiomic features were computed.

We extracted six feature classes from original images: first-order statistics, shape descriptors, and four texture matrices (GLCM, GLRLM, GLSZM, GLDM), yielding 107 features per patient. Quality control removed features with zero variance, excessive missing values ($> 50\%$), or high inter-feature correlation ($|r| > 0.90$), reducing the set to 61

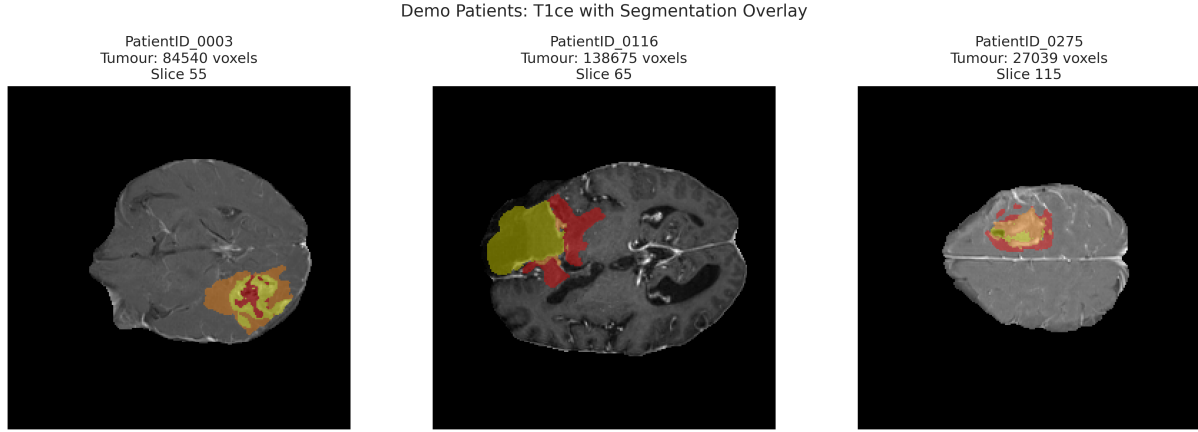


Figure 6.11: Representative T1ce axial slices with tumour segmentation overlay for three patients, illustrating the range of tumour sizes and morphologies in the cohort. Segmentation masks (coloured overlay) delineate tumour subregions used as input for radiomic feature extraction.

features. Figure 6.12 presents the post-QC correlation structure, revealing redundancy among shape descriptors and among texture features within the same matrix class.

We applied elastic net regularisation for feature selection (L1 ratio = 0.7, favouring sparsity whilst using the L2 component to encourage grouped selection among correlated radiomic features), retaining eight features with non-zero coefficients (Figure 6.13).

Table 6.7 compares model performance across feature sets.

Model	AUC-ROC	Std
Clinical Only	0.742	± 0.112
Radiomics Only	0.598	± 0.151
Clinical + Radiomics	0.725	± 0.080

Table 6.7: Comprehensive radiomic model comparison (stratified 5-fold CV). Radiomics-only features are the 8 selected by Elastic Net.

The PyRadiomics features performed near chance level in isolation (AUC 0.598 ± 0.151), and their addition to clinical features marginally degraded performance (AUC 0.725 vs. 0.742). Despite this, Kaplan-Meier analysis stratified by shape surface area (median split) demonstrated statistically significant survival separation (log-rank $p = 0.003$; Figure 6.14), confirming that individual radiomic features carry biological signal even when they fail to improve multivariate prediction. This dissociation between univariate significance and multivariate utility is reported repeatedly in radiomics studies [21] and arises because molecular and clinical markers subsume the information content of individual imaging features in a combined model.

6.5 Tumour Growth Prediction

6.5.1 Dataset Characteristics

Of the 203 patients in the full cohort, 105 (51.7%) had at least two imaging timepoints with complete volume data and were eligible for this analysis. Tumour volumes were computed by summing segmentation volumes across all four subregion types (necrotic core,

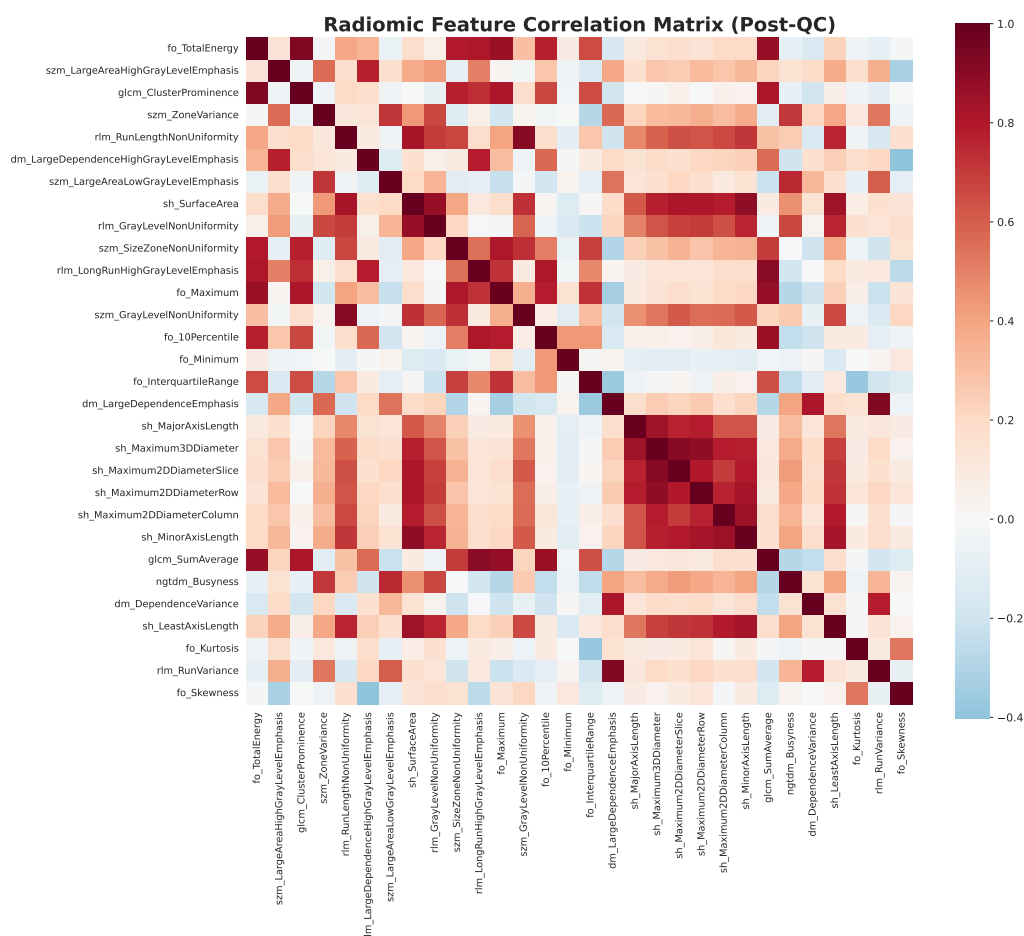


Figure 6.12: Radiomic feature correlation matrix (post-QC). Shape features form a tightly correlated cluster; texture features exhibit high within-class redundancy.

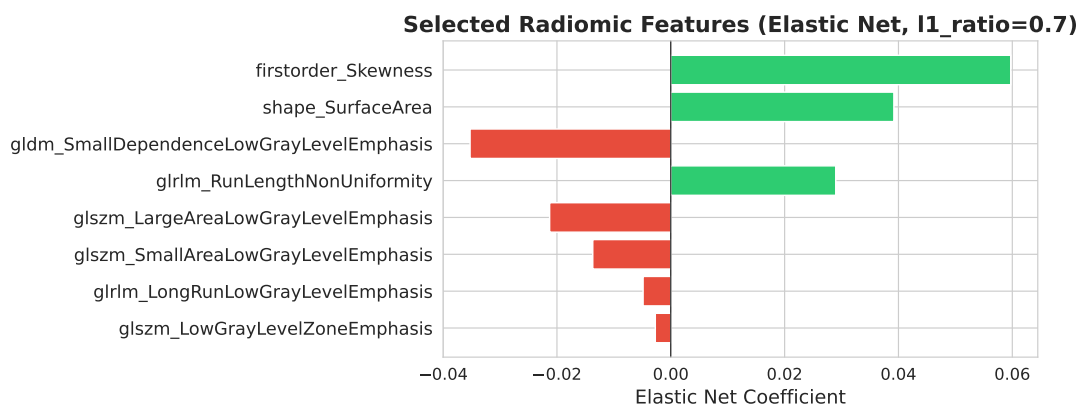


Figure 6.13: Selected radiomic features (Elastic Net, L1 ratio = 0.7). Green bars indicate positive coefficients (associated with survival); red bars indicate negative coefficients (associated with mortality).

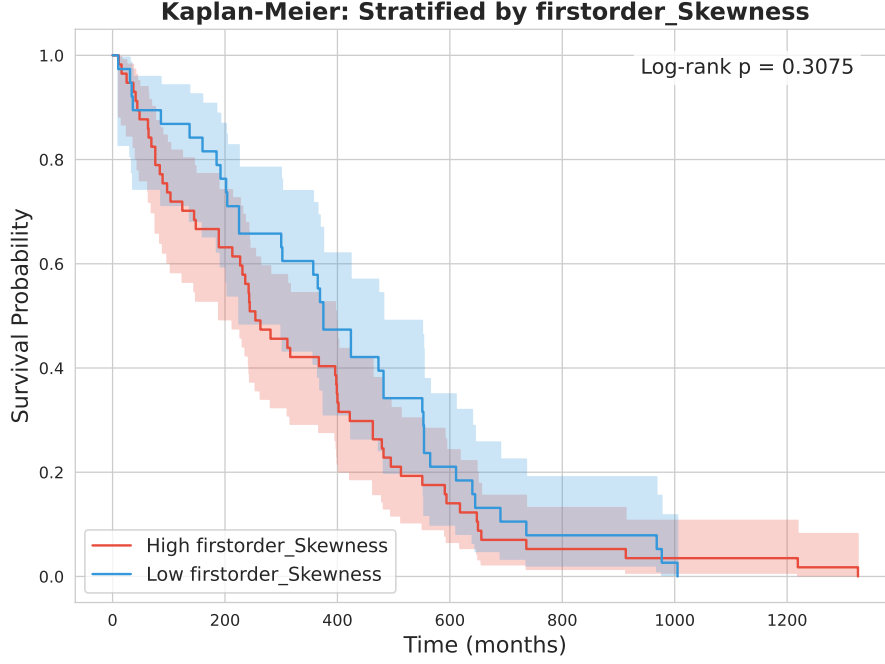


Figure 6.14: Kaplan-Meier survival curves stratified by tumour surface area (median split). Larger tumours are associated with worse prognosis (log-rank $p = 0.003$).

enhancing tumour, oedema/infiltration, and resection cavity). The target variable, absolute volume change ($\Delta V = V_{t_1} - V_{t_0}$, mm^3), exhibited high variance (range $\approx \pm 200,000 \text{ mm}^3$), reflecting heterogeneous post-treatment tumour dynamics.

The scatter plot in Figure 6.15 (right) reveals no obvious linear relationship between baseline volume and subsequent change. The largest volume reductions occurred in patients with the largest baseline tumours, suggesting treatment-induced shrinkage following surgical resection rather than favourable tumour biology.

6.5.2 Leakage Re-Detection

We applied the leakage detection framework to the growth prediction feature set and identified five progression-related features that had persisted in the separately preprocessed clinical data: *time to first progression*, *number of days from diagnosis to date of first progression*, *type of first progression*, *type of second progression*, and *second progression/recurrence*. We removed these prior to model training. This confirms that the four-step pipeline can detect leakage even when features enter through an independent preprocessing pathway with different inclusion criteria.

In an initial analysis conducted before leakage removal, the XGBoost combined model returned $R^2 = 0.15$, the only positive R^2 across all configurations. After removing the five leaked features, this dropped to $R^2 = -0.15$, confirming that the entire apparent predictive signal originated from temporal leakage. This finding validates the leakage detection methodology established in Chapter 5.

6.5.3 Model Performance

Table 6.8 summarises the cross-validated performance of all model-feature set combinations after leakage removal.

Every model-feature set combination produced negative R^2 values (Figure 6.16), indicating that no configuration outperformed the trivial baseline of predicting the mean

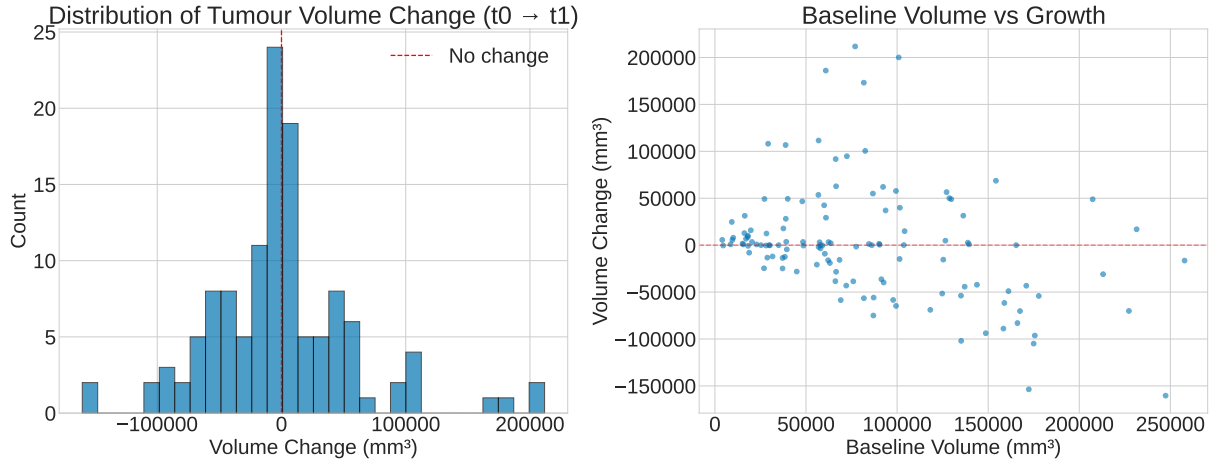


Figure 6.15: Left: distribution of tumour volume change between first and second MRI timepoints ($n = 105$). The majority of patients cluster near zero change, with long tails in both directions. Right: baseline tumour volume versus subsequent volume change, showing no clear linear relationship.

Model	Feature Set	R^2
Linear Regression	Imaging Only	-0.09 ± 0.25
Ridge	Imaging Only	-0.09 ± 0.25
XGBoost	Imaging Only	-0.25 ± 0.31
Ridge	Clinical Only	-0.29 ± 0.30
XGBoost	Clinical Only	-0.61 ± 0.75
Linear Regression	Clinical Only	-0.97 ± 1.28
XGBoost	Combined	-0.13 ± 0.29
Ridge	Combined	-0.37 ± 0.45
Linear Regression	Combined	-1.07 ± 1.31

Table 6.8: Cross-validated regression performance for tumour volume change prediction ($n = 105$, 5-fold $\times$ 5-repeat CV). All R^2 values are negative, indicating no model outperformed a mean-prediction baseline.

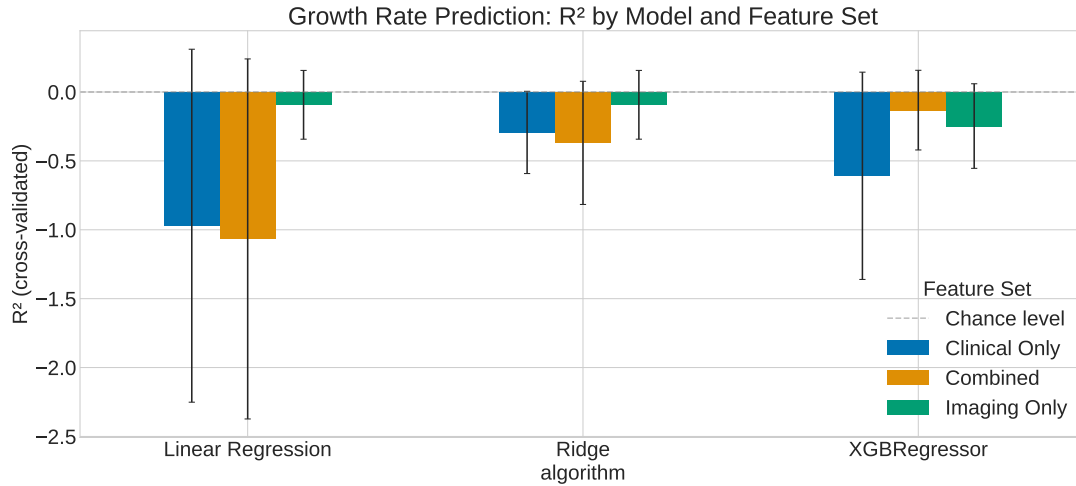


Figure 6.16: Cross-validated R^2 by model and feature set for tumour volume change prediction. The dashed line at $R^2 = 0$ indicates mean-prediction baseline performance. All configurations fall below this threshold; linear regression on combined features performed worst due to overfitting in the high-dimensional, small-sample regime.

volume change for all patients. Imaging-only features were least detrimental (Ridge, $R^2 = -0.09$), whilst clinical-only features performed worse (up to $R^2 = -0.97$ for linear regression). This pattern is inverted compared to survival prediction, where clinical markers dominate: short-term volume change between post-treatment scans is driven by treatment response and tumour microenvironment dynamics rather than germline molecular status, making clinical features uninformative for this task.

6.5.4 Feature Importance

Despite the negative overall performance, XGBoost feature importance analysis (Figure 6.17) reveals which features the model attempted to use. After leakage removal, imaging features dominated: T2-weighted intensity standard deviation was the strongest predictor (20.4% importance), followed by baseline total volume (16.5%), T2-weighted mean intensity (14.8%), and T1 native intensity standard deviation (10.7%). The prominence of T2-weighted texture variability is biologically plausible, as T2 signal heterogeneity reflects underlying tissue composition including oedema, necrosis, and cellular density, all of which influence short-term volume dynamics.

No clinical feature appeared in the importance ranking. Whilst IDH mutation status, tumour grade, and age dominate survival prediction, they provide no signal for predicting short-term volume change. The determinants of inter-scan volume change thus appear distinct from those of long-term survival.

6.5.5 Mortality by Growth Tertile

The mortality rates across growth tertiles (slow/shrinking 68.6%, moderate 60.0%, fast-growing 62.9%; Figure 6.18) fall within sampling noise at $n = 35$ per tertile (chi-squared $p > 0.05$). The ordering is not a genuine inversion. An earlier version of this section read the slow/shrinking position as a paradox and invoked confounding by indication, on the hypothesis that aggressive tumours receive aggressive treatment and that the treatment shrinks them between scans.

We ran a balance check before invoking treatment as an explanation. The three growth groups are indistinguishable on every treatment axis tested. Chemotherapy and radiation

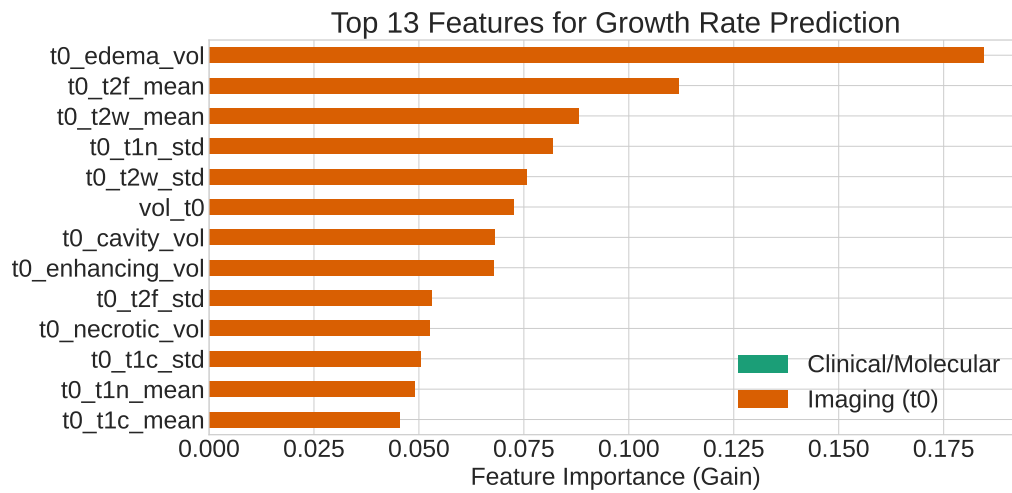


Figure 6.17: Feature importance (gain) for the XGBoost combined model after leakage removal. All contributing features are imaging-derived; no clinical feature provided meaningful predictive signal for volume change. T2-weighted intensity standard deviation ranked highest (20.4%), reflecting the prognostic relevance of tissue heterogeneity captured by T2 signal variability.

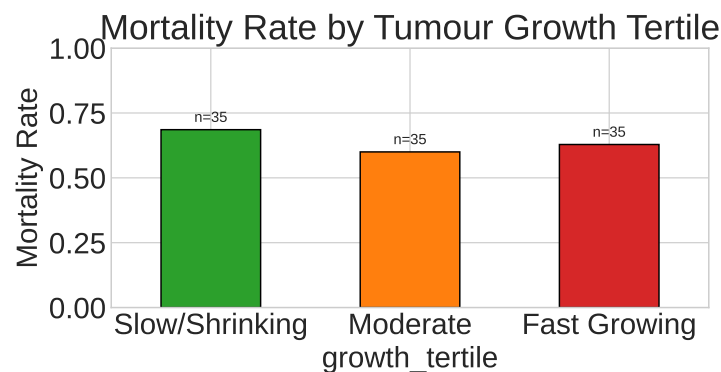


Figure 6.18: Mortality rate by tumour volume change tertile. The three rates fall within sampling noise at $n = 35$ per tertile (chi-squared $p > 0.05$).

receipt rates match across tertiles over each patient's full trajectory (chemotherapy $p = 0.79$; radiation $p = 0.92$) and during the TP1–TP2 scan window (chemotherapy $p = 0.75$; radiation $p = 0.92$; Figure 6.19). Re-resection rates are flat (37.1%, 37.1%, 31.4%; $p = 0.85$), first-surgery timing is identical across tertiles, and one patient in the cohort had a first surgery between the two scans used to compute growth rate (Figure 6.20).

Because the three groups are balanced on treatment, no within-tertile treated-versus-untreated stratification is warranted: a stratified comparison cannot change the conclusion when the strata are matched. The earlier confounding-by-indication explanation also requires a treatment gradient across tertiles that this cohort does not have. Extent of resection is the one axis the dataset does not record, and that part remains untested. The claim is about treatment exposure across groups, not treatment efficacy within them; whether chemotherapy or radiation alters an individual tumour's growth trajectory is a separate question that the present design cannot answer.

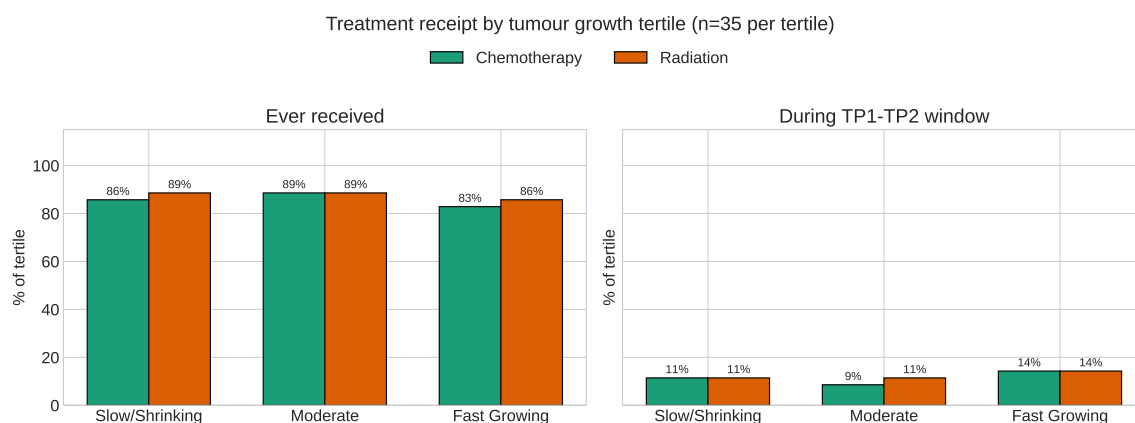


Figure 6.19: Balance check across growth tertiles for chemotherapy and radiation. Left: receipt at any point during the patient's trajectory. Right: treatment overlapping the TP1–TP2 scan window. The three growth groups are indistinguishable on all six measurements ($p > 0.10$).

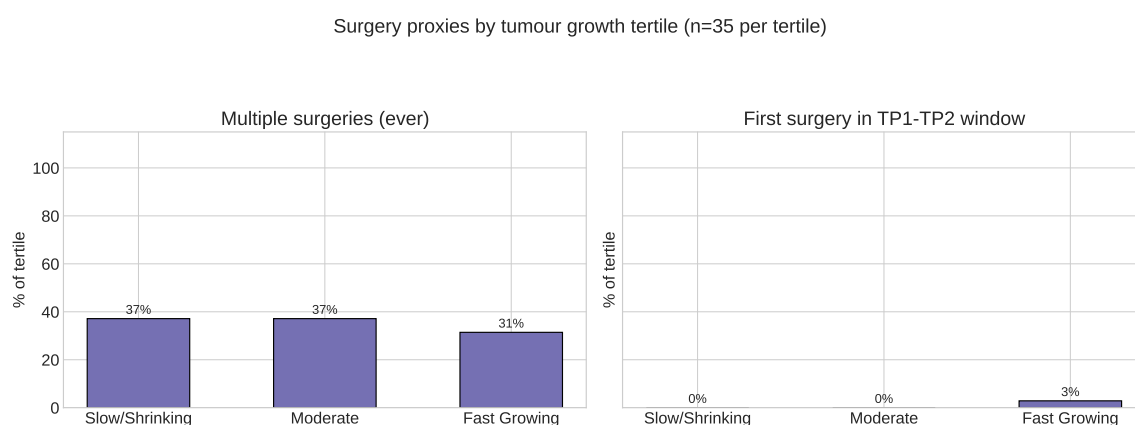


Figure 6.20: Balance check across growth tertiles for surgical intervention. Left: any re-resection ($p = 0.85$). Right: first surgery inside the TP1–TP2 scan window. One patient across the 105-patient cohort had a first surgery in that window.

6.6 Summary of Results

Table 6.9 consolidates all findings across the five analysis streams.

Analysis Component	Finding
Data Leakage Detection	7 features identified and removed; AUC decreased from 0.730 to 0.674 (-7.7%)
Binary Classification	Best clean model: Logistic Regression ($\text{AUC} = 0.692 \pm 0.119$); no model significantly outperformed others
Survival Analysis	Best model: Random Survival Forest ($\text{C-index} = 0.706 \pm 0.069$); outperformed Cox PH (0.667)
Feature Importance	IDH1 mutation (19.4%), primary diagnosis (19.3%), and TERT mutation (14.0%) dominated binary classification; age dominated survival analysis (9.3% RSF; 39.5% GBSA)
Radiomics Analysis	No significant improvement from pre-extracted imaging features ($\Delta\text{AUC} = -0.002$, $p = 0.936$); PyRadiomics texture features near chance ($\text{AUC} = 0.598$) and marginally degraded combined performance
Growth Prediction	No model outperformed mean baseline ($R^2 \leq 0$, $n = 105$); leakage re-detected and removed; mortality by tertile within sampling noise; tertile groups balanced on treatment exposure

Table 6.9: Summary of key experimental findings across all analysis components.

Five principal findings emerge. First, data leakage detection is essential for honest performance assessment: the 8% AUC inflation in baseline metrics would mislead clinical expectations if undetected. Second, achievable performance with tabular clinical and molecular data is approximately AUC 0.67–0.72 and C-index ~ 0.71 , a ceiling set by the limited sample size ($n \approx 200$) and biological heterogeneity of gliomas. Third, molecular markers (IDH, TERT, chromosomal alterations) provide the strongest signal in binary classification, consistent with the 2021 WHO classification framework, whilst age dominates survival analysis through its cumulative hazard contribution. Fourth, tabular features cannot predict tumour volume change between consecutive scans ($R^2 \leq 0$ across all models); growth trajectory forecasting requires richer spatial representations than segmentation-derived summary statistics. Fifth, neither pre-extracted imaging features nor PyRadiomics texture features improve upon clinical and molecular predictors, suggesting that approaches such as deep learning on raw MRI volumes may be needed to extract imaging-based prognostic information beyond what molecular subtyping already provides.

Chapter 7

Discussion

7.1 Discussion and Appraisal

Data leakage inflated glioma survival prediction performance by approximately 8%; achievable discrimination with clean tabular features on a cohort of 203 patients falls in the range AUC 0.67–0.72. The following discussion interprets these findings in the context of existing literature, evaluates the strengths and weaknesses of the approach, and identifies what the results reveal about the broader challenges of medical ML.

Leakage Detection as an Empirical Contribution. This project asks whether performance levels reported in glioma ML literature survive temporal scrutiny. The four-step pipeline is the instrument; the answer (7.7% AUC inflation, complex models losing their advantage over logistic regression, performance converging on the EORTC clinical benchmark) is the finding. The pipeline operationalises principles from Kapoor and Narayanan [3] and Davis et al. [4] for the glioma domain, with two original elements: an empirically calibrated $|r| > 0.25$ threshold (justified by the correlation compression inherent to binary targets, where raising to $|r| > 0.35$ misses four of seven confirmed leakage features) and a cross-task validation design. The growth prediction analysis (Section 6.5), conducted on a separately preprocessed subset ($n = 105$), tested whether the detection methodology catches leakage features that re-enter through an independent pipeline. The only positive R^2 model collapsed to negative after removal. The methodology generalises across analytical contexts.

We identified the leakage features through domain reasoning and removed them to create the clean model. The pipeline has three purposes: it provides a reproducible procedure applicable to other datasets, it catches ambiguous features whose temporal status is not obvious from their names alone (e.g., “Number of days from diagnosis to first surgery or procedure” sounds baseline but encodes post-diagnosis timing), and the deliberate before/after comparison quantifies the cost of contamination as a designed experiment.

The observed AUC inflation of 7.7% aligns with the broader literature. Kapoor and Narayanan [3] reported a median inflation of 0.12 across 20 studies with confirmed leakage, whilst McDermott et al. [9] found mean AUC reductions from 0.89 to 0.71 when leakage was corrected in healthcare studies. The present finding falls at the conservative end of this range, likely because the MU-Glioma-Post dataset contains relatively few progression-related variables compared to larger registry databases where treatment response features proliferate.

Performance in Context. The clean model performance (AUC 0.67–0.69 for binary classification; C-index 0.71 for survival analysis) aligns closely with the EORTC nomograms developed by Gorlia et al. [2], which reported C-index ~ 0.66 on a larger GBM-only cohort

($n = 832$) using six clinical factors without molecular profiling. The inclusion of molecular markers in the present work, particularly IDH mutation status, accounts for the modest improvement over this benchmark. This alignment is reassuring rather than disappointing: it confirms that the models are learning genuine prognostic signals rather than artefactual patterns, and it calibrates expectations for what tabular clinical data can achieve on modest cohorts.

The finding that logistic regression (AUC 0.692) performed comparably to XGBoost (AUC 0.674) on clean features parallels the pattern observed by Kapoor and Narayanan [3], where complex models lost their apparent advantage over simpler baselines once leakage was corrected (Figure 3.3). With only 21 features and 203 patients, the effective degrees of freedom available to tree-based ensembles are limited, and the linear decision boundary imposed by logistic regression may represent a more appropriate inductive bias for this sample size.

Biological Validation of Feature Importance. IDH1 mutation emerged as the dominant feature in binary classification (19.4% importance), consistent with its definitional role in the WHO CNS5 framework, where it determines the distinction between astrocytoma and glioblastoma [12]. The shift in feature importance rankings between binary classification (IDH-dominated) and survival analysis (age-dominated) reflects a genuine methodological difference: binary classification at a fixed timepoint rewards features that separate prognostic groups categorically, whilst survival models reward features that predict the rate of hazard accumulation over time. Age satisfies the latter criterion because it compounds with duration of follow-up, making it the strongest continuous predictor in time-to-event frameworks.

Informative Negative Results. Two negative findings carry substantive implications. First, neither pre-extracted nor PyRadiomics-derived features improved prediction beyond clinical and molecular markers (Sections 6.4 and 6.4.2), suggesting that on this cohort the prognostic information in tumour morphology is largely captured by molecular subtype. This corroborates Jain et al. [24], who found that volumetric features provided only incremental improvement (C-index 0.68 to 0.72) beyond clinical variables, and supports the hypothesis that higher-order texture features require larger sample sizes to demonstrate stable predictive value.

Second, the negative R^2 values in tumour growth prediction confirm that post-treatment volume change cannot be forecast from baseline tabular features. The mortality rates across growth tertiles (68.6%, 60.0%, 62.9%) fall within sampling noise at $n = 35$ per tertile, and a balance check across the three groups found that treatment exposure does not differ on any of seven axes. With the groups balanced on treatment, treatment is not what distinguishes them, and the earlier confounding-by-indication explanation does not apply. This distinguishes predictive modelling (forecasting outcomes from baseline data) from causal inference (understanding treatment effects), a line the medical ML literature often blurs, and argues against reading narrative structure into non-significant point estimates on small subgroups.

Technical Challenges. We implemented five analytical pipelines (leakage detection, binary classification, survival analysis, radiomics, and tumour growth prediction), each requiring different computational frameworks (scikit-learn, lifelines, scikit-survival, and PyRadiomics). The PyRadiomics feature extraction, which computed 107 features per patient from volumetric MRI data, required execution on the University of Manchester CSF3 high-performance computing cluster due to the computational demands of 3D image processing. Data preprocessing posed additional challenges: the mixed feature space of continuous variables, binary molecular markers, and multi-level categorical features required careful encoding and imputation strategies [30], and we ensured patient-level rather than

scan-level cross-validation splits to prevent a second, subtler form of data leakage. The modest sample size ($n = 203$, approximately 97 events) imposed further constraints: with an events-per-variable ratio of approximately 4.6, well below the recommended threshold of 10 [33], high-dimensional modelling approaches were infeasible, and careful cross-validation design was critical to obtaining reliable performance estimates [34]. These constraints shaped every modelling decision, from the choice of algorithms to the evaluation protocol.

7.2 Evaluation of Objectives

The project is now evaluated against the eight objectives stated in Section 1.2. Table 7.1 summarises achievement against each objective before the detailed per-objective assessment.

Table 7.1: Summary of achievement against project objectives.

Objective	Status	Evidence
1. Characterise the MU-Glioma-Post dataset through EDA	Achieved	Cohort profiled across demographic, tumour, molecular, and outcome dimensions (Section 4.3)
2. Implement a four-step leakage detection pipeline	Achieved	7 features identified and removed (Chapter 5)
3. Train and evaluate five binary classification variants	Achieved	AUC 0.692 (logistic regression best; Section 6.1)
4. Implement Cox PH, RSF, and GBSA survival models	Achieved	C-index 0.706 (RSF best; Section 6.2)
5. Assess marginal contribution of imaging features	Achieved	$\Delta\text{AUC} = -0.002$, $p = 0.936$ (Section 6.4)
6. Construct a tumour growth prediction framework	Achieved	$R^2 \leq 0$; mortality by tertile within sampling noise (Section 6.5)
7. Compare feature importance across model types	Achieved	IDH1 dominant in classification; age in survival (Section 6.3)
8. Contextualise against clinical tools and literature	Achieved	AUC 0.67–0.72 aligns with EORTC C-index ~ 0.66 (Section 7.1)

Objective 1: Characterise the MU-Glioma-Post dataset through exploratory data analysis. Achieved. The cohort was characterised across demographic, tumour, molecular, and outcome dimensions (Section 4.3), confirming distributions characteristic of a predominantly GBM cohort and identifying the 47.8% mortality rate that defined the prediction target.

Objective 2: Implement a four-step leakage detection pipeline combining correlation analysis with temporal reasoning. Achieved. The pipeline was implemented, applied, and independently validated (Chapter 5). Seven leakage features were identified, documented, and removed, with impact quantified at 7.7% AUC inflation.

Objective 3: Train and evaluate five binary classification model variants using stratified 5-fold cross-validation. Achieved. Five variants were compared (Section 6.1), with logistic regression achieving the best clean AUC of 0.692 ± 0.119 . The comparison revealed that model complexity provided no advantage on this dataset.

Objective 4: Implement Cox Proportional Hazards, Random Survival Forest, and Gradient Boosted Survival Analysis models. Achieved. All three were implemented using the scikit-survival library and evaluated (Section 6.2). RSF achieved the best C-index of 0.706 ± 0.069 with clear risk stratification confirmed by Kaplan-Meier analysis.

Objective 5: Assess the marginal contribution of pre-extracted imaging features via paired statistical comparison. Achieved. Paired t -tests confirmed no significant improvement ($\Delta\text{AUC} = -0.002$, $p = 0.936$), and the PyRadiomics analysis corroborated this finding (Section 6.4.2).

Objective 6: Construct a regression framework predicting tumour volume change from baseline features. Achieved, though with negative results. The analysis ($n = 105$) showed that baseline features cannot predict post-treatment volume change. A secondary analysis by growth tertile returned mortality rates within sampling noise ($p > 0.05$), and a balance check across the three groups found treatment exposure indistinguishable on all seven axes tested (Section 6.5).

Objective 7: Extract and compare feature importance rankings across all model types, validating alignment with established glioma biology. Achieved. Feature importance was compared across binary classification, survival analysis, and radiomics models (Section 6.3). IDH1 dominance in classification and age dominance in survival analysis both corroborate established clinical knowledge.

Objective 8: Contextualise model performance against externally validated clinical tools (the EORTC nomograms) and published ML studies. Achieved. Section 7.1 contextualises clean model performance against the EORTC nomograms and leakage-corrected benchmarks from the literature.

All eight objectives were met. The project scope was broad, spanning leakage detection, multiple modelling frameworks, radiomics, and growth prediction; whilst no single component achieves state-of-the-art performance, the breadth provides a full assessment of what is achievable with tabular clinical data on a moderately sized glioma cohort.

7.3 Limitations

A number of limitations constrain the generalisability of these findings.

The most significant is the reliance on a single institutional cohort ($n = 203$) from University of Missouri Health Care, with no external validation. The cohort is demographically homogeneous (94.6% White), which limits applicability to more diverse populations where genetic and environmental factors may alter prognostic relationships. Cross-validated performance estimates, whilst internally consistent, cannot substitute for external validation on an independent dataset.

The sample size imposes statistical constraints throughout the analysis. With approximately 97 events (deaths), the effective events-per-variable ratio for 21-feature models is approximately 4.6, below the commonly recommended threshold of 10 [33]. This likely contributes to the high cross-validation variance observed (AUC standard deviations of 0.08–0.12) and limits the power to detect small but clinically meaningful differences between models.

The binary survival target conflates censored patients (alive at last follow-up but possibly deceased since) with long-term survivors. A patient coded as “alive” may have died shortly after their last contact; this informative censoring means the binary classification treats censored observations as definitive survivors, which it cannot confirm. The

survival analysis models address this through explicit censoring handling, but the binary classification results should be interpreted with this limitation in mind.

The MU-Glioma-Post dataset lacks Karnofsky Performance Status (KPS), a clinical variable present in most validated nomograms including the EORTC benchmark [2]. Its absence likely reduces achievable performance relative to datasets with complete clinical profiling.

Finally, the leakage detection methodology relies on correlation-based screening, which may miss leakage features with non-linear relationships to the outcome. Features that interact with the target through complex conditional dependencies could evade the $|r| > 0.25$ threshold whilst still encoding future information. The temporal reasoning step mitigates this risk for features with obvious clinical interpretation, but subtler forms of leakage may go undetected.

Chapter 8

Conclusion and Future Work

8.1 Conclusions

This project developed a survival prediction pipeline for glioma patients that integrates leakage detection as a core methodological component. Applied to the MU-Glioma-Post dataset ($n = 203$), the four-step detection pipeline identified seven features encoding post-diagnosis information, the removal of which reduced XGBoost AUC from 0.730 to 0.674. This 7.7% inflation quantifies the cost of a common but frequently unreported methodological error in medical machine learning.

On clean feature sets, the best binary classifier (logistic regression) attained AUC 0.692 ± 0.119 , and the best survival model (Random Survival Forest) yielded C-index 0.706 ± 0.069 . These results are in line with the EORTC nomograms [2] and with leakage-corrected benchmarks from the broader literature [3, 9], establishing that AUC 0.67–0.72 represents realistic achievable performance for tabular clinical and molecular data on cohorts of this size. Feature importance analysis confirmed biological plausibility: IDH1 mutation status dominated binary classification and age at diagnosis dominated survival models, both supporting the WHO CNS5 framework [12].

Two negative findings carry practical significance. Neither pre-extracted imaging features nor PyRadiomics features improved prediction beyond clinical and molecular markers, suggesting that tumour morphology, as captured by first-order and texture features, is largely redundant with molecular subtype information on this cohort. The inability to predict tumour volume change from baseline features, combined with a mortality ordering across growth tertiles that falls within sampling noise once treatment exposure is checked, shows the limitations of observational post-treatment data for causal prediction tasks and the need to distinguish predictive modelling from causal inference.

The principal contribution is an empirical measurement that no prior study had made on this dataset: how much of the MU-Glioma-Post predictive signal comes from temporal contamination, and what performance remains after its removal. The leakage detection pipeline provides a documented, reproducible template for applying temporal reasoning to clinical ML datasets where progression and treatment variables may contaminate baseline prediction, and the cross-task validation confirms that temporal contamination is a structural property of post-treatment oncology databases rather than an artefact of a single preprocessing pipeline.

8.2 Future Work

Several directions would extend this work.

External Validation. The most important next step is validation on an independent glioma cohort. The BraTS dataset [35] and TCGA-GBM/TCGA-LGG collections offer multi-institutional data with molecular profiling. Either would test whether the leakage detection findings and clean performance benchmarks generalise beyond a single institution. External validation would also test the transferability of the four-step detection pipeline to datasets with different variable structures and missingness patterns.

Calibration Analysis. The models evaluated in this project were assessed on discrimination (AUC, C-index) but not on calibration, which measures whether predicted probabilities correspond to observed event rates. Recent systematic reviews report that 0% of glioma ML studies assess calibration [8], despite its importance for clinical decision-making where a predicted 30% mortality risk must correspond to a 30% observed rate. Calibration curves, Brier scores, and the Hosmer-Lemeshow test would complement the discrimination metrics reported here and would reveal whether data leakage distorts probability estimates in addition to rankings.

Deep Learning on Volumetric MRI. The failure of tabular radiomic features to improve prediction motivates investigation of end-to-end deep learning approaches that operate directly on 3D MRI volumes. Kickingeder et al. [20] achieved C-index 0.71 using 3D CNNs on a multi-institutional cohort ($n = 1,501$); applying similar architectures to the MU-Glioma-Post imaging data, potentially with transfer learning to address the small sample size, stands to capture spatial patterns inaccessible to hand-crafted features.

Temporal Modelling with Longitudinal Data. The growth prediction analysis identified 105 patients with multiple imaging timepoints. Recurrent architectures or joint longitudinal-survival models could exploit the temporal structure of repeated measurements, modelling tumour trajectory as a dynamic process rather than predicting a single volume change. A larger cohort would support a better-powered test of whether growth tertile discriminates mortality, a question the present $n = 35$ -per-tertile analysis left open.

Prospective Clinical Evaluation. The clinical value of any survival prediction model depends on whether it changes patient management and improves outcomes. A prospective study comparing clinician decision-making with and without model-assisted prognostication would assess whether the modest discrimination achieved (C-index ~ 0.71) translates to meaningful clinical utility, or whether the inherent biological heterogeneity of gliomas places a ceiling on what statistical prediction can achieve.

Appendix A

Additional Mathematical Background

A.1 Decision Trees

Decision trees partition the feature space through hierarchical binary splits, selecting at each node the feature and threshold that maximise a purity criterion (typically Gini impurity or information gain). Individual trees are interpretable but prone to overfitting, which motivates ensemble methods.

A.2 XGBoost Gain Formula

The optimal split at each XGBoost tree node is determined by:

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma \quad (\text{A.1})$$

where G_L, G_R and H_L, H_R are the sums of first-order gradients and second-order Hessians for the left and right child nodes. XGBoost uses a second-order Taylor approximation of the loss function incorporating both $g_i = \partial \ell / \partial \hat{y}_i^{(t-1)}$ and $h_i = \partial^2 \ell / \partial (\hat{y}_i^{(t-1)})^2$ for more accurate optimisation [1]. Splits with negative gain are pruned.

A.3 Extended Molecular Marker Biology

1p/19q Codeletion. See Section 2.1.2 for full discussion of this marker's biology and clinical significance.

TERT Promoter Mutations. See Section 2.1.2 for full discussion of this marker's biology and clinical significance.

TP53 and ATRX. See Section 2.1.2 for full discussion of these markers' biology and clinical significance.

Appendix B

Growth Prediction Feature Importance

See Section 6.5 for discussion of growth prediction feature importance. Figure B.1 reproduces the feature importance plot for reference.

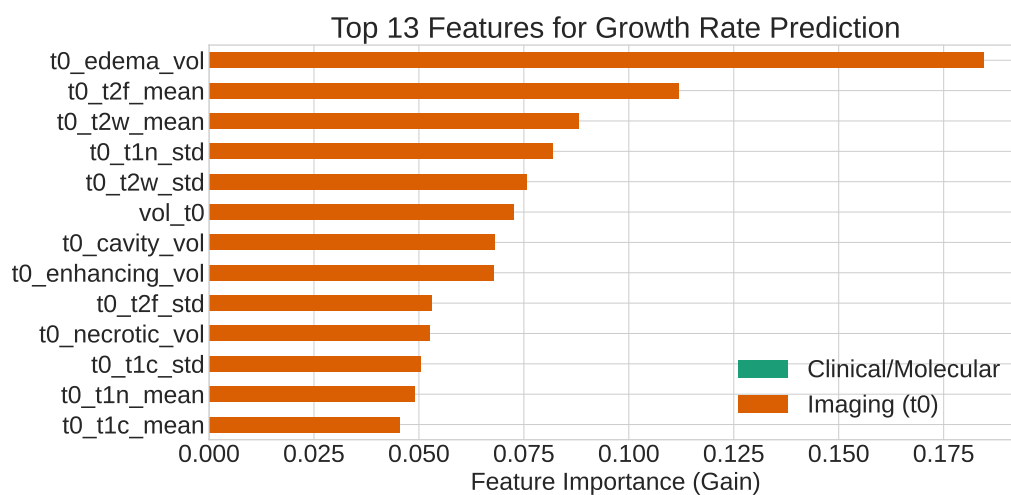


Figure B.1: Feature importance (gain) for XGBoost combined model after leakage removal. All contributing features are imaging-derived.

Appendix C

Hyperparameter Search Spaces and Selected Values

Table C.1 presents the hyperparameter search space and best values found via randomised search CV for each XGBoost variant. Table C.2 summarises the survival model configurations.

Parameter	Search Range	Clean XGBoost	Regularised XGBoost
max_depth	{3, 4, 5, 6, 7, 8}	5	3
learning_rate	[0.01, 0.3]	0.10	0.05
n_estimators	{50, 100, 200, 300}	200	100
min_child_weight	{1, 3, 5, 7}	1	5
subsample	[0.6, 1.0]	0.8	0.7
colsample_bytree	[0.6, 1.0]	0.8	0.6
reg_lambda (L2)	[0.1, 10.0]	1.0	5.0
reg_alpha (L1)	[0.0, 1.0]	0.0	0.5
scale_pos_weight	{1.0, 1.1}	1.1	1.0

Table C.1: XGBoost hyperparameter search space and selected values (randomised search, 50 iterations, 5-fold stratified CV).

Model	Parameter	Value / Justification
Cox PH	L2 penaliser	0.1 (moderate shrinkage for $p = 21$, $n = 202$)
RSF	n_estimators	100 (convergence verified; 200 trees yielded <0.005 C-index change)
RSF	max_features	$\sqrt{p}$ (standard random subsampling)
RSF	min_samples_leaf	10 (prevents terminal nodes $<5\%$ of cohort)
GBSA	Loss function	Negative log partial likelihood (Cox loss)
GBSA	n_estimators	100 with early stopping (patience = 10 rounds)
GBSA	learning_rate	0.1
GBSA	max_depth	3 (shallow trees to limit overfitting)

Table C.2: Survival model hyperparameter configurations.

Hyperparameter sensitivity was low across XGBoost variants: the top five configurations from randomised search produced AUC values within 0.02 of the optimum, indicating that sample size and feature informativeness constrain performance more than hyperparameter choice.

Appendix D

MIMUC 2026 Presentation and Award

The author presented an earlier version of the leakage detection methodology at MIMUC 2026 (Manchester Interdisciplinary Mathematics Undergraduate Conference), where it received the 3rd Place Best Student Talk award from a panel of five academic staff judges. The panel noted the talk's narrative structure and clear communication of the core leakage detection message. The certificate and presentation slides follow.



BEST STUDENT TALK

3RD PLACE

This certificate is awarded to

Joshua Alliet

For their talk on

Exposing the future: A mathematical framework for
detecting data leakage in medical machine learning.

Congratulations!



Jane Street



LONDON
MATHEMATICAL
SOCIETY
EST. 1865

Exposing the Future

A Mathematical Framework for Detecting Data Leakage in Medical Machine Learning

Joshua Alliet

MIMUC 2026

🔗 The Paradox

What if a feature knows too much?

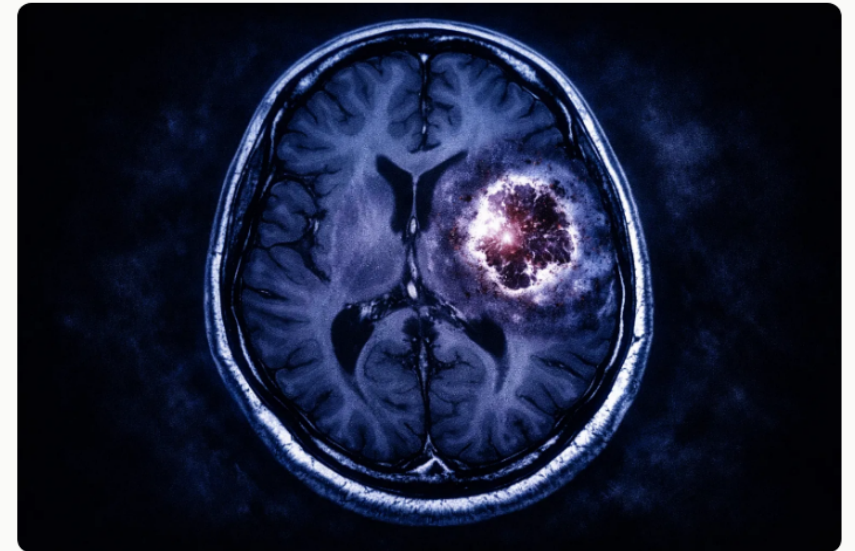
→ One feature correlates with death at $r = 0.89$.

72

→ A statistician celebrates. A mathematician is suspicious.

→ But a mathematician pauses: how could any single feature 'know' this much?

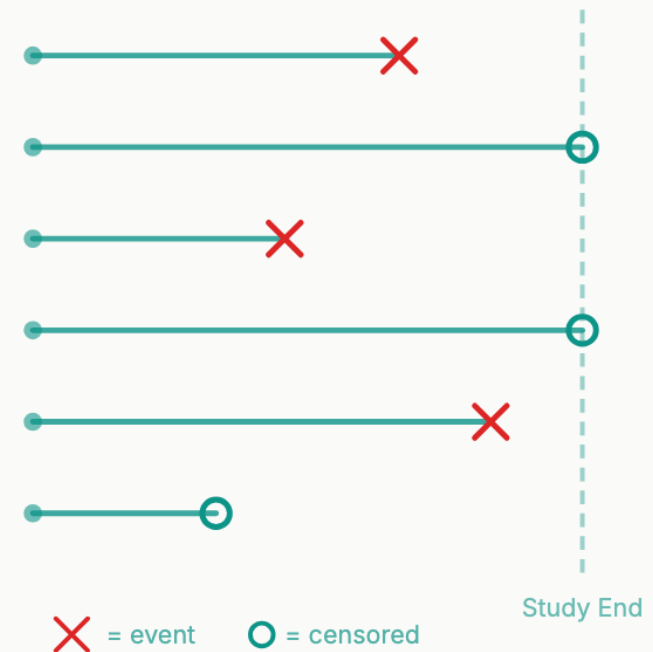
| *This talk explores what happens when you ask that question seriously.*



⌚ Before We Investigate

Why Standard Regression Fails

- Not every patient dies during observation. Some are right-censored: alive at last follow-up.
- Standard regression treats censored data as exact, introducing systematic bias.



The Elegance of Hazard Functions

One Clock, Many Speeds

$$h(t) = h_0(t) \cdot \exp(\beta^\top \mathbf{X})$$

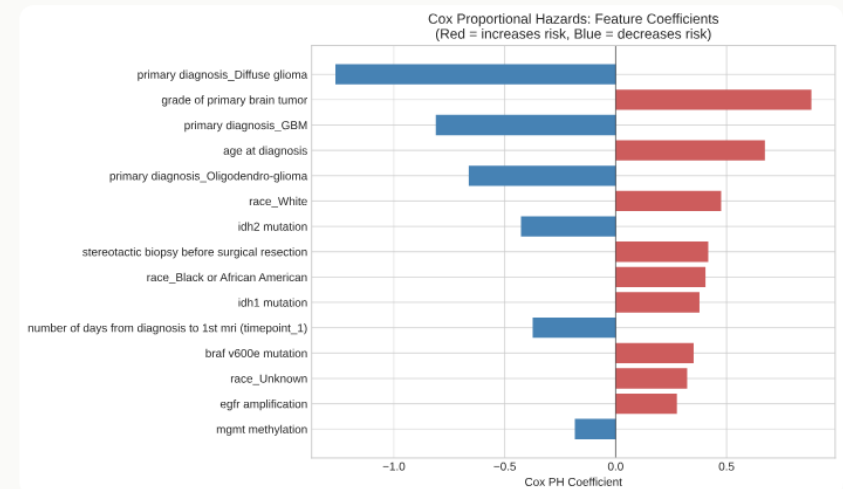
74

$h(t)$ = hazard (instantaneous risk of the event)

$h_0(t)$ = baseline (the average patient's clock) β = multipliers (Age: 1.03x, IDH1: 0.34x)

$\mathbf{X}$ = patient profile (the knobs that adjust risk)

- Our Cox model: Age (**HR 1.03**, $p < 0.001$) and **IDH1 (HR 0.34)** dominate.
- Semi-parametric: no assumption on baseline shape, estimated via partial likelihood.



Measuring Truth: The Concordance Index

Not Accuracy, Ranking

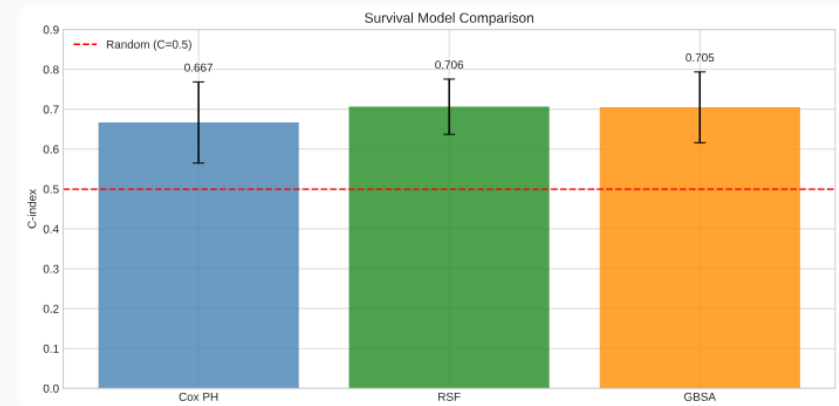
75

$$C = P(\hat{r}_i > \hat{r}_j \mid T_i < T_j)$$

$\hat{r}$ = predicted risk (model's danger ranking) T = actual survival (ground truth)

C = concordance (% of patient pairs ranked correctly)

- All three models cluster near $C = 0.69$ — within 2% of each other.
- With $n = 203$, sample size is the bottleneck, not the algorithm.



🕒 The Hidden Trap: Temporal Validity

When Correlation Encodes the Answer

76

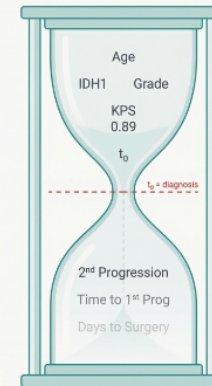
$$x_j \text{ is valid} \iff x_j \in \mathcal{F}_{t_0}$$

x_j = a feature (one column in the dataset)

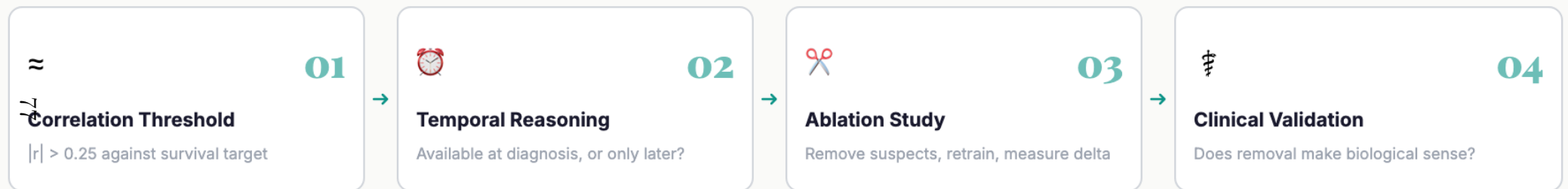
$\mathcal{F}_{t_0}$ = natural filtration (info accumulated up to diagnosis)

t_0 = diagnosis day (the prediction deadline)

- Features from $t > t_0$ observe the future, not predict it.
- "Second Progression" ($r = 0.89$): only recorded after prolonged survival.



Building a Mathematical Filter



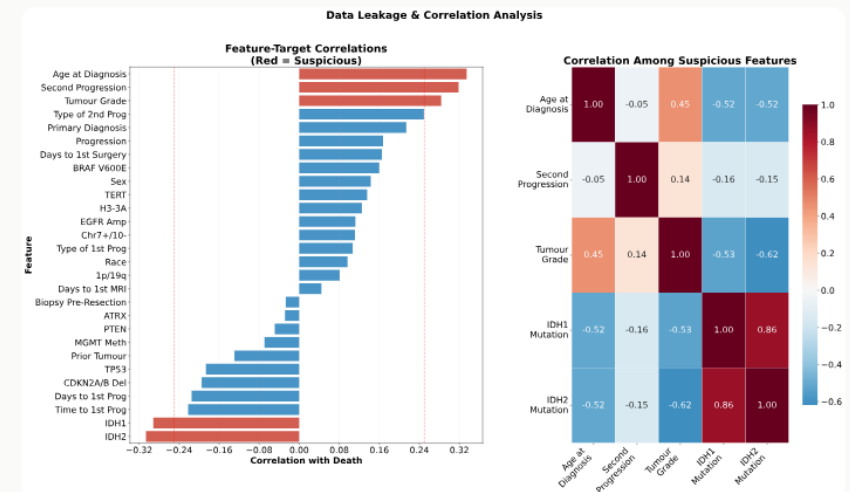
28 → **21** features

7 temporally invalid variables removed (25% reduction)

7 Features That Saw the Future

Caught Looking Forward

- **Second Progression** ($r = 0.89$): only knowable after follow-up.
- **Time to First Progression**: encodes survival directly.
- **Days to First Surgery**: reflects post-diagnosis urgency.
- **Progression** (binary): requires longitudinal observation.
- Type of 1st/2nd Progression: only available retrospectively.



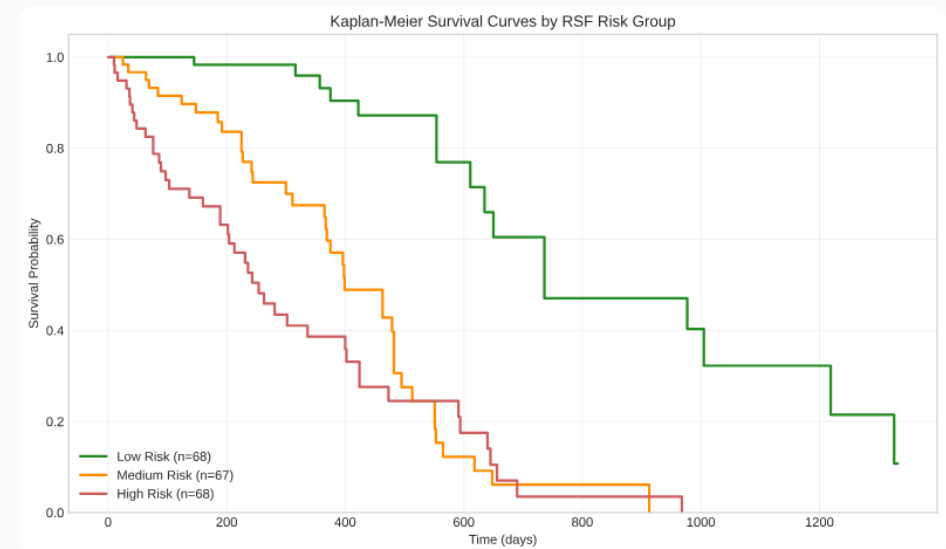
The Cost of Truth

What We Lost Was Never Real

WITH LEAKAGE HONEST

0.730 0.674

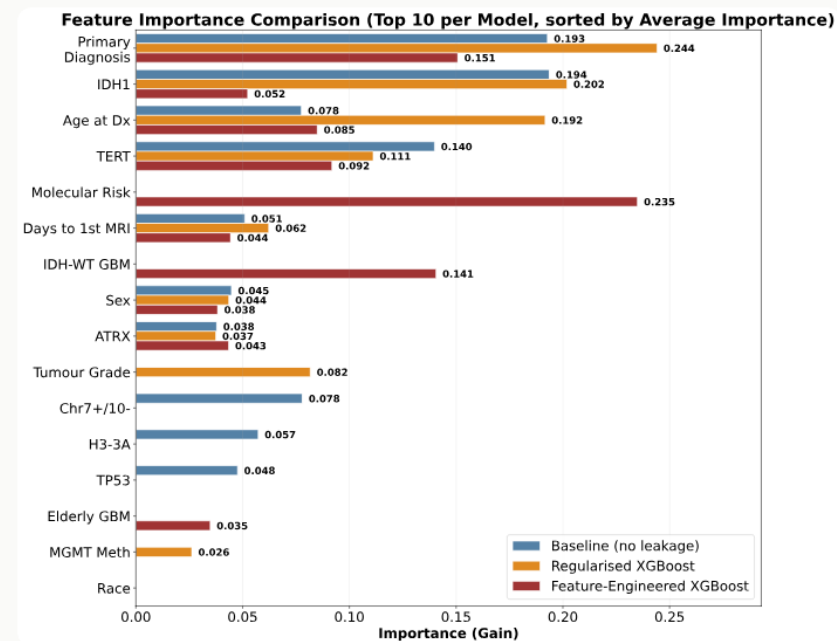
Honest AUC of **0.674** matches validated clinical tools (EORTC nomograms, C-index ~0.66). The lower number is the **real** one.



🏆 The Honest Competition

A Straight Line Suffices

- **Logistic Regression:** AUC **0.692 ± 0.119**.
- **Clean XGBoost:** AUC **0.674 ± 0.111**.
- With **n = 203**, all 5 clean models cluster in AUC **0.62–0.69**.
- **IDH1 mutation status:** **19.4%** of XGBoost feature importance.



🚩 The Principle

What the Data Actually Knows

18

SURVIVAL

0.67–0.72 AUC

Matches EORTC benchmarks

IMAGING

0.598 AUC

Near random chance

VOLUME

$R^2 < 0$

Cannot predict growth

⁸²**Thank You!**

Questions?

Appendix E

Source Code Notebooks

The complete analysis codebase comprises six Jupyter notebooks, included below. We ran all notebooks on Google Colab with data stored on Google Drive. The full repository is publicly available at <https://github.com/jalliet/glioma-survival-prediction> [36].

E.1 Notebook 01: Preprocessing and EDA

01_preprocessing_and_eda

0.1 Environment Setup

0.1.1 Pip Installations & Imports

```
[1]: # EDA
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Preprocessing/Outputting
import random
import json
import os
```

0.1.2 Mount Drive

User auth so it auto mounts (doesn't remove the extra clicks...)

```
[2]: from google.colab import drive
drive.mount('/content/drive/', force_remount=True)
```

Mounted at /content/drive/

```
[3]: from google.colab import userdata
GLIOMA_DIR = userdata.get('GLIOMA_DIR')
```

```
[4]: EDA_DIR = os.path.join(GLIOMA_DIR, 'eda')
os.makedirs(EDA_DIR, exist_ok=True)
FIGURES_DIR = os.path.join(EDA_DIR, 'figures')
os.makedirs(FIGURES_DIR, exist_ok=True)
print(f"Figures directory: {FIGURES_DIR}")
```

Figures directory: drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/eda/figures

```
[5]: def jprint(obj):
    text = json.dumps(obj, sort_keys=True, indent=2)
    print(text)
```

1 Working with MU-Glioma-Post

1.0.1 File structure:

```
Mu-Glioma-Post |__ PatientID_0003 |__ Timepoint_1 |__  
PatientID_0003_Timepoint_1_tumorMask.nii.gz |__ Pa-  
tientID_0003_Timepoint_1_brain_t2w.nii.gz |__ Pa-  
tientID_0003_Timepoint_1_brain_t2f.nii.gz |__ Pati-  
entID_0003_Timepoint_1_brain_t1n.nii.gz |__ Patien-  
tID_0003_Timepoint_1_brain_t1c.nii |__ Timepoint_2 |__ Timepoint_4 |__  
... |__ ...
```

Each timepoint should have one tumor mask file and then those 4 brain scan files as well.

First of all we perform EDA on the Clinical Data, clean up and then train a decision tree model on the set of features with a fill rate of at least 70%. From the resulting model, we check keep only the most significant features for subsequent training of leaner, more consequential models.

1.1 Exploring the clinical data

To start off, we would like to train a number of models that answer these two questions:

1. Will the patient die?
2. How long will the patient survive?

Down the line, we could develop model that answers “Which treatment works best?”.

We will also train a prognostic model.

1.1.1 Retrieve Data + Early Preprocessing

```
[6]: MU_GLIOMA_CLINICAL_DATA = os.path.join(GLIOMA_DIR,␣  
      ↪ 'MU_Glioma_Post_ClinicalData_July2025.xlsx')  
df = pd.read_excel(MU_GLIOMA_CLINICAL_DATA, sheet_name='MU Glioma Post')
```

First, strip column names and make them lowercase

```
[7]: df.columns = df.columns.str.strip().str.lower()
```

Helpers Let us check for NA/undefined values across columns

```
[8]: # Keep columns that have a at least a specific fill rate  
def findColsWithFillRate(df, min_fill_rate=None, max_fill_rate=None):  
    fill_rates = df.count() / len(df)  
  
    if min_fill_rate is not None and max_fill_rate is not None:  
        mask = (fill_rates >= min_fill_rate) & (fill_rates <= max_fill_rate)  
        # print(f"Columns with fill rate between {min_fill_rate} and␣  
        ↪ {max_fill_rate}:")  
    elif min_fill_rate is not None:  
        mask = (fill_rates >= min_fill_rate)
```

```

        # print(f"Columns with fill rate >= {min_fill_rate}:")
    else:
        print("Please provide either min_fill_rate or both min_fill_rate and
↪max_fill_rate.")
        return pd.DataFrame() # Return empty DataFrame if no criteria are
↪provided

    result = df.loc[:, mask]
    return result

```

For the first model (binary survival), get reliable columns with a high fill rate

```

[9]: fill_rate = 0.7
high_fill_df = findColsWithFillRate(df, min_fill_rate=fill_rate)
high_fill_df.head()

```

```

[9]:
    patient_id sex at birth   race  age at diagnosis primary diagnosis \
0  PatientID_0003      Female  White                57             GBM
1  PatientID_0004      Female  White                67             GBM
2  PatientID_0005       Male  White                49             GBM
3  PatientID_0006       Male  White                60             GBM
4  PatientID_0007       Male  White                79             GBM

    grade of primary brain tumor  stereotactic biopsy before surgical resection \
0                                4                                                0
1                                4                                                0
2                                4                                                0
3                                4                                                0
4                                4                                                0

    progression  time to first progression (days)  type of 1st progression \
0              1                             286.0                        3
1              0                             NaN                        0
2              1                             344.0                        1
3              1                             175.0                        1
4              0                             NaN                        0

    ...  chromosome 7 gain and chromosome 10 loss  h3-3a mutation \
0  ...                                           2                2
1  ...                                           2                2
2  ...                                           2                2
3  ...                                           2                2
4  ...                                           2                2

    egfr amplification  pten mutation  cdkn2a/b deletion  tp53 alteration \
0                    2                0                0                0

```

1	2	0	0	0
2	2	0	0	0
3	2	0	0	0
4	2	0	0	0

	previous brain tumor \
0	No
1	No
2	No
3	No
4	No

	number of days from diagnosis to first surgery or procedure \
0	-5
1	-4
2	-4
3	0
4	-5

	number of days from diagnosis to date of first progression \
0	286.0
1	67.0
2	344.0
3	175.0
4	NaN

	number of days from diagnosis to 1st mri (timepoint_1)
0	90.0
1	67.0
2	NaN
3	NaN
4	NaN

[5 rows x 32 columns]

```
[10]: jprint(high_fill_df.columns.tolist()), len(high_fill_df.columns)
```

```
[
    "patient_id",
    "sex at birth",
    "race",
    "age at diagnosis",
    "primary diagnosis",
    "grade of primary brain tumor",
    "stereotactic biopsy before surgical resection",
    "progression",
    "time to first progression (days)",
    "type of 1st progression",

```

```

"second progression/recurrence",
"type of 2nd progression",
"multiple surgeries",
"hospice",
"overall survival (death)",
"idh1 mutation",
"idh2 mutation",
"1p/19q",
"atrx mutation",
"mgmt methylation",
"braf v600e mutation",
"tert promoter mutation",
"chromosome 7 gain and chromosome 10 loss",
"h3-3a mutation",
"egfr amplification",
"pten mutation",
"cdkn2a/b deletion",
"tp53 alteration",
"previous brain tumor",
"number of days from diagnosis to first surgery or procedure",
"number of days from diagnosis to date of first progression",
"number of days from diagnosis to 1st mri (timepoint_1)"
]

```

[10]: (None, 32)

Our target feature for binary classification made it to the fold. Let us analyse it

```
[11]: high_fill_df['overall survival (death)'].value_counts()
```

```

[11]: overall survival (death)
0      106
1       97
Name: count, dtype: int64

```

We can see we have a pretty much even split between cases that end in death or those who are alive/cases with censored results (which we cannot account for at the moment).

Now drop columns that will not be needed (patient id, hospice)

```
[12]: high_fill_df = high_fill_df.drop(['patient_id', 'hospice'], axis=1)
```

Display the dataframe series where the dtype is object, so as to clean up as needed

```
[13]: object_columns = high_fill_df.select_dtypes(include=['object'])
object_columns.head()
```

```

[13]:  sex at birth  race primary diagnosis grade of primary brain tumor \
0      Female  White                GBM                4
1      Female  White                GBM                4

```

2	Male	White	GBM	4
3	Male	White	GBM	4
4	Male	White	GBM	4

	previous brain tumor
0	No
1	No
2	No
3	No
4	No

Checking out the “Grade of Primary Brain Tumor” column

```
[14]: object_columns.iloc[:, 3].unique()
```

```
[14]: array([4, 2, 3, 1, '3 vs 4'], dtype=object)
```

```
[15]: object_columns[object_columns['grade of primary brain tumor'].eq('3 vs 4')].
      ↪shape[0]
```

```
[15]: 2
```

```
[16]: object_columns['grade of primary brain tumor'] = object_columns['grade of_
      ↪primary brain tumor'].astype(str).replace('3 vs 4', '4').astype(float)
```

```
[17]: object_columns['grade of primary brain tumor'].unique()
```

```
[17]: array([4., 2., 3., 1.])
```

```
[18]: high_fill_df['grade of primary brain tumor'] = object_columns['grade of primary_
      ↪brain tumor']
```

More cleaning Check out remaining object columns

```
[19]: object_columns = object_columns.select_dtypes(include=['object'])
      object_columns.head()
```

```
[19]: sex at birth  race primary diagnosis previous brain tumor
0      Female  White                GBM                No
1      Female  White                GBM                No
2      Male    White                GBM                No
3      Male    White                GBM                No
4      Male    White                GBM                No
```

Strip whitespaces from remaining object column values

```
[20]: object_columns = object_columns.apply(lambda x: x.str.strip())
```

```
[21]: for i in range(len(object_columns.columns)):
      jprint(f"{object_columns.columns[i]}: {object_columns.iloc[:, i].unique()}")
```

```
"sex at birth: ['Female' 'Male']"
"race: ['White' 'Black or African American' 'Unknown' 'Asian']"
"primary diagnosis: ['GBM' 'Astrocytoma' 'Pilocytic astrocytoma' 'Diffuse glioma'\n 'Oligodendro-glioma' 'Glioma w/ GBM features']"
"previous brain tumor: ['No' 'Yes']"
```

Map these cleaned columns back to the original high_fill_df

```
[22]: for col in object_columns.columns:
      high_fill_df[col] = object_columns[col]
```

Investigating extra data leakage

```
[23]: print(high_fill_df.dtypes)
```

sex at birth	object
race	object
age at diagnosis	int64
primary diagnosis	object
grade of primary brain tumor	float64
stereotactic biopsy before surgical resection	int64
progression	int64
time to first progression (days)	float64
type of 1st progression	int64
second progression/recurrence	int64
type of 2nd progression	int64
multiple surgeries	int64
overall survival (death)	int64
idh1 mutation	int64
idh2 mutation	int64
1p/19q	int64
atrx mutation	int64
mgmt methylation	int64
braf v600e mutation	int64
tert promoter mutation	int64
chromosome 7 gain and chromosome 10 loss	int64
h3-3a mutation	int64
egfr amplification	int64
pten mutation	int64
cdkn2a/b deletion	int64
tp53 alteration	int64
previous brain tumor	object
number of days from diagnosis to first surgery or procedure	int64
number of days from diagnosis to date of first progression	float64
number of days from diagnosis to 1st mri (timepoint_1)	float64
dtype:	object

Suspicious about the multiple surgeries column. Unsure whether it includes surgeries for recurrence (after progression)? If yes, consider removing for data leakage. If it only counts surgeries up to initial treatment, we keep it.

```
[24]: print(high_fill_df[high_fill_df['multiple surgeries'].eq(1)].shape)
      print(high_fill_df[high_fill_df['progression'].eq(1)].shape)
      print(high_fill_df[high_fill_df['progression'].eq(1) & high_fill_df['multiple_
      ↪surgeries'].eq(1)].shape)
```

```
(51, 30)
```

```
(152, 30)
```

```
(50, 30)
```

Analysis:

- 51 patients have multiple surgeries
- 50 of those 51 (98%) also had progression
- This means multiple surgeries almost always includes recurrence surgeries (performed AFTER progression)

Conclusion: Remove “multiple surgeries” from features. It’s using future information (surgeries for recurrence) to predict outcomes.

```
[25]: high_fill_df = high_fill_df.drop(['multiple surgeries'], axis=1)
```

1.1.2 Visualising the remaining data

```
[26]: print(high_fill_df.dtypes)
```

sex at birth	object
race	object
age at diagnosis	int64
primary diagnosis	object
grade of primary brain tumor	float64
stereotactic biopsy before surgical resection	int64
progression	int64
time to first progression (days)	float64
type of 1st progression	int64
second progression/recurrence	int64
type of 2nd progression	int64
overall survival (death)	int64
idh1 mutation	int64
idh2 mutation	int64
1p/19q	int64
atrx mutation	int64
mgmt methylation	int64
braf v600e mutation	int64
tert promoter mutation	int64
chromosome 7 gain and chromosome 10 loss	int64
h3-3a mutation	int64

```

egfr amplification                                int64
pten mutation                                    int64
cdkn2a/b deletion                                int64
tp53 alteration                                    int64
previous brain tumor                              object
number of days from diagnosis to first surgery or procedure  int64
number of days from diagnosis to date of first progression  float64
number of days from diagnosis to 1st mri (timepoint_1)      float64
dtype: object

```

Patient Demographics

```

[27]: sns.set_style("whitegrid")
plt.rcParams.update({'font.size': 18})
fig, axes = plt.subplots(2, 2, figsize=(16, 10))
fig.suptitle('Patient Demographics', fontweight='bold')

colors = sns.color_palette("Set2")

# Sex at Birth
sex_counts = high_fill_df['sex at birth'].value_counts()
axes[0, 0].pie(sex_counts.values, labels=sex_counts.index, autopct='%1.1f%%',
               colors=colors[:2], startangle=90)
axes[0, 0].set_title('Sex at Birth', fontweight='bold')

# Race Distribution
race_counts = high_fill_df['race'].value_counts()
axes[0, 1].barh(race_counts.index, race_counts.values, color=colors[2])
axes[0, 1].set_xlabel('Count', fontweight='semibold')
axes[0, 1].set_ylabel('Race')
axes[0, 1].set_title('Race Distribution', fontweight='bold')

# Custom labels for y-axis with line break for long label
new_race_labels = []
for label in race_counts.index:
    if label == 'Black or African American':
        new_race_labels.append('Black or\nAfrican\nAmerican')
    else:
        new_race_labels.append(label)
axes[0, 1].set_yticklabels(new_race_labels)

for i, v in enumerate(race_counts.values):
    axes[0, 1].text(v + 1, i, str(v), va='center', fontweight='bold')

# Age at Diagnosis Distribution
age_data = high_fill_df['age at diagnosis'].dropna()
axes[1, 0].hist(age_data, bins=20, color=colors[3], edgecolor='black', alpha=0.
    ↪7)

```

```

axes[1, 0].axvline(age_data.median(), color='red', linestyle='--', linewidth=2,
                  label=f'Median: {age_data.median():.1f} years')
axes[1, 0].axvline(age_data.mean(), color='blue', linestyle='--', linewidth=2,
                  label=f'Mean: {age_data.mean():.1f} years')
axes[1, 0].set_xlabel('Age (years)')
axes[1, 0].set_ylabel('Frequency')
axes[1, 0].set_title('Age at Diagnosis', fontweight='bold')
axes[1, 0].legend(bbox_to_anchor=(0, 1), loc='upper left') # Moved legend
↳outside

# Previous Brain Tumor
prev_tumor_counts = high_fill_df['previous brain tumor'].value_counts()
axes[1, 1].pie(prev_tumor_counts.values, labels=prev_tumor_counts.index,
              autopct='%1.1f%%', colors=colors[4:6], startangle=90)
axes[1, 1].set_title('Previous Brain Tumor History', fontweight='bold')

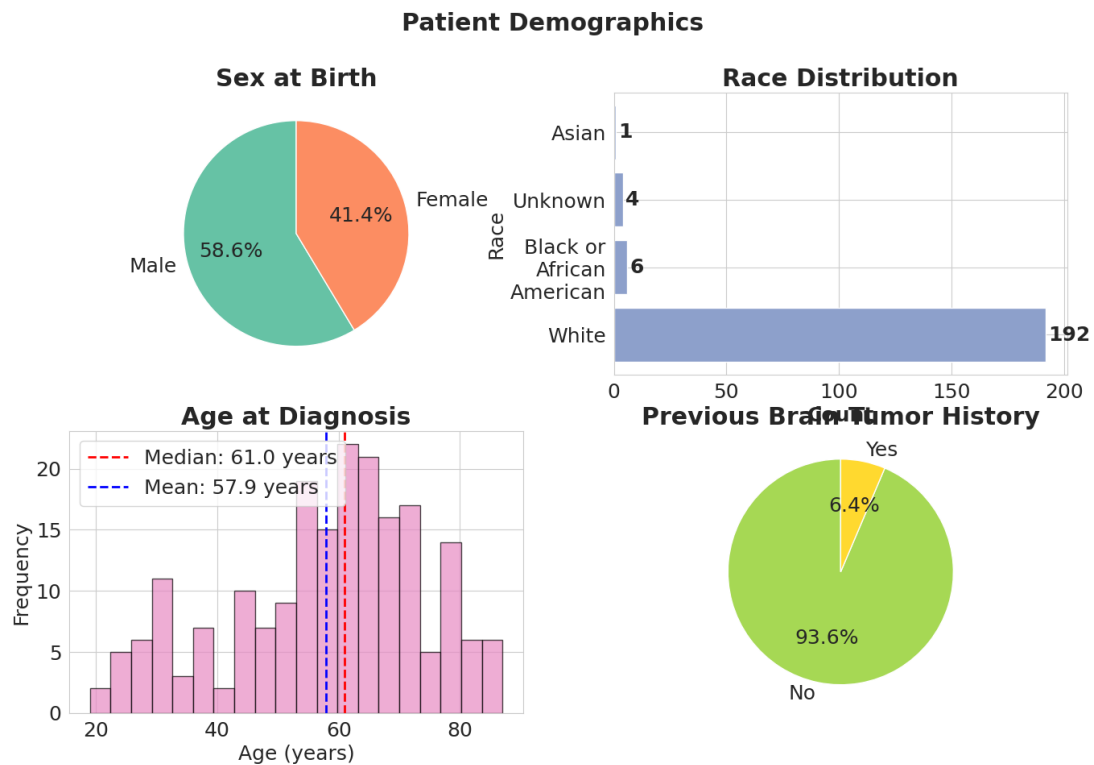
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_patient_demographics.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_patient_demographics.png'),
            dpi=300, bbox_inches='tight')
plt.show()
print('Saved: 01_eda_patient_demographics.pdf + .png')

print("\nDemographic Summary")
print("-"*30)
print(f"Total Patients: {len(high_fill_df)}")
print(f"\nAge Statistics:")
print(f"  Mean: {age_data.mean():.1f} years")
print(f"  Median: {age_data.median():.1f} years")
print(f"  Range: {age_data.min():.0f} - {age_data.max():.0f} years")
print(f"  Std Dev: {age_data.std():.1f} years")

```

/tmp/ipykernel_795/1460958628.py:28: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.

```
axes[0, 1].set_yticklabels(new_race_labels)
```



Saved: 01_eda_patient_demographics.pdf + .png

Demographic Summary

Total Patients: 203

Age Statistics:

Mean: 57.9 years
Median: 61.0 years
Range: 19 - 87 years
Std Dev: 16.1 years

Tumor Characteristics

```
[28]: plt.rcParams.update({'font.size': 16})
fig, axes = plt.subplots(2, 2, figsize=(17, 15))
fig.suptitle('Tumour Characteristics', fontweight='bold')

colors = sns.color_palette("Set2")

# Primary Diagnosis
diagnosis_counts = high_fill_df['primary diagnosis'].value_counts()
```

```

bars = axes[0, 0].bar(range(len(diagnosis_counts)), diagnosis_counts.values,
    color=colors)
axes[0, 0].set_xticks(range(len(diagnosis_counts)))

# Custom labels for x-axis with line break for long label
current_xlabels = diagnosis_counts.index.tolist()
new_xlabels = []
for label in current_xlabels:
    if label == "Diffuse glioma":
        new_xlabels.append('Diffuse\n glioma')
    elif label == 'Glioma w/ GBM features':
        new_xlabels.append('Glioma w/\nGBM features')
    elif label == 'Oligodendro-glioma':
        new_xlabels.append('Oligodendro-\n glioma')
    elif label == 'Pilocytic astrocytoma':
        new_xlabels.append('Pilocytic\n astrocytoma')
    else:
        new_xlabels.append(label)
axes[0, 0].set_xticklabels(new_xlabels, rotation=45, ha='right')

axes[0, 0].set_ylabel('Count', fontweight='semibold')
axes[0, 0].set_title('Primary Diagnosis Distribution', fontweight='bold')
axes[0, 0].grid(axis='y', alpha=0.3)
for i, v in enumerate(diagnosis_counts.values):
    axes[0, 0].text(i, v + 2, str(v), ha='center', fontweight='bold')

# Grade Distribution
grade_counts = high_fill_df['grade of primary brain tumor'].value_counts().
    sort_index()
bars = axes[0, 1].bar(range(len(grade_counts)), grade_counts.values,
    color=sns.color_palette("RdYlGn_r", len(grade_counts)))
axes[0, 1].set_xticks(range(len(grade_counts)))
axes[0, 1].set_xticklabels([f'Grade {g}' for g in grade_counts.index])
axes[0, 1].set_ylabel('Count', fontweight='semibold')
axes[0, 1].set_title('WHO Tumor Grade Distribution', fontweight='bold')
axes[0, 1].grid(axis='y', alpha=0.3)
for i, v in enumerate(grade_counts.values):
    axes[0, 1].text(i, v + 2, str(v), ha='center', fontweight='bold')

# Stereotactic Biopsy
biopsy_counts = high_fill_df['stereotactic biopsy before surgical resection'].
    value_counts()
labels = ['No Biopsy', 'Biopsy Performed']
colors_biopsy = [colors[5], colors[6]]
wedges, texts, autotexts = axes[1, 0].pie(biopsy_counts.values, labels=labels,
    autopct='%1.1f%%',
    colors=colors_biopsy,

```

```

startangle=90)

for autotext in autotexts:
    autotext.set_fontweight('bold')
axes[1, 0].set_title('Stereotactic Biopsy Before Resection', fontweight='bold')

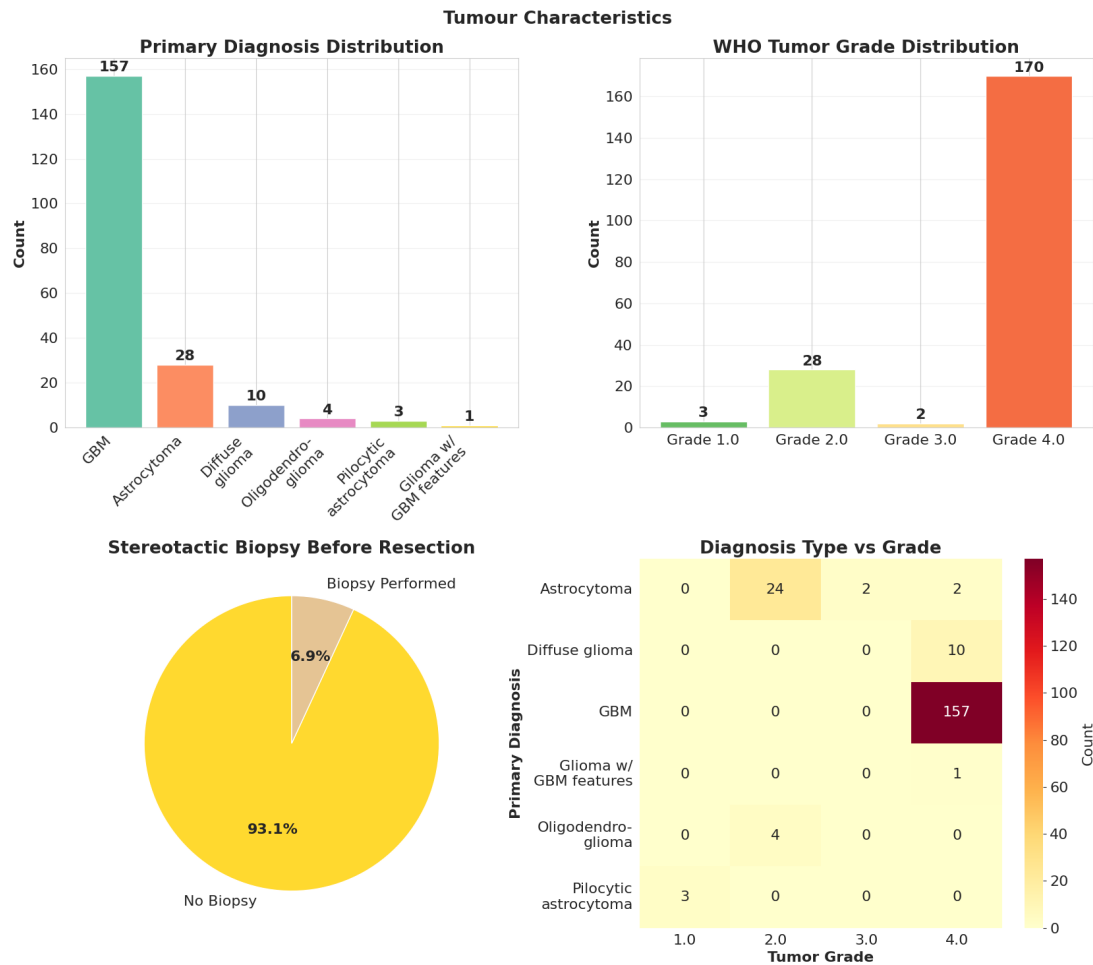
# Diagnosis vs Grade Heatmap
crosstab_dg = pd.crosstab(high_fill_df['primary diagnosis'],
                        high_fill_df['grade of primary brain tumor'])
sns.heatmap(crosstab_dg, annot=True, fmt='d', cmap='YlOrRd', ax=axes[1, 1],
            cbar_kws={'label': 'Count'})
axes[1, 1].set_xlabel('Tumor Grade', fontweight='semibold')
axes[1, 1].set_ylabel('Primary Diagnosis', fontweight='semibold')
axes[1, 1].set_title('Diagnosis Type vs Grade', fontweight='bold')

# Custom labels for y-axis with line break for long label
current_labels = crosstab_dg.index.tolist()
new_labels = []
for label in current_labels:
    if label == 'Glioma w/ GBM features':
        new_labels.append('Glioma w/\nGBM features')
    elif label == 'Oligodendro-glioma':
        new_labels.append('Oligodendro-\nglioma')
    elif label == 'Pilocytic astrocytoma':
        new_labels.append('Pilocytic\nastrocytoma')
    else:
        new_labels.append(label)
axes[1, 1].set_yticklabels(new_labels, rotation=0)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_tumour_characteristics.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_tumour_characteristics.png'),
            dpi=300, bbox_inches='tight')
plt.show()
print('Saved: 01_eda_tumour_characteristics.pdf + .png')

print("\nTumour Characteristics Summary")
print("-"*30)
print(f"\nMost Common Diagnosis: {diagnosis_counts.index[0]} (n={diagnosis_counts.values[0]}, {diagnosis_counts.values[0]/len(high_fill_df)*100:.1f}%)")
print(f"\nMost Common Grade: Grade {grade_counts.idxmax()} (n={grade_counts.max()}, {grade_counts.max()/len(high_fill_df)*100:.1f}%)")
print(f"\nBiopsy Rate: {biopsy_counts.get(1, 0)/len(high_fill_df)*100:.1f}%",

```



Saved: 01_eda_tumour_characteristics.pdf + .png

Tumour Characteristics Summary

Most Common Diagnosis: GBM (n=157, 77.3%)
 Most Common Grade: Grade 4.0 (n=170, 83.7%)
 Biopsy Rate: 6.9%

Clinical Outcomes (TARGET VARIABLE)

```
[29]: plt.rcParams.update({'font.size': 18})
fig, axes = plt.subplots(2, 2, figsize=(18, 12))
fig.suptitle('Clinical Outcomes', fontweight='bold')

# Overall Survival (target)
survival_counts = high_fill_df['overall survival (death)'].value_counts()
```

```

labels = ['Alive (0)', 'Died (1)']
colors_survival = ['#2ecc71', '#e74c3c'] # Green for alive, red for died
sns.countplot(x='overall survival (death)', data=high_fill_df, ax=axes[0, 0],
    ↪palette=colors_survival)
axes[0, 0].set_title('Overall Survival (Death)\nTARGET VARIABLE',
    ↪fontweight='bold', color='darkred')
axes[0, 0].set_xlabel('')
axes[0, 0].set_ylabel('Count')
axes[0, 0].set_xticks([0, 1])
axes[0, 0].set_xticklabels(labels)
for container in axes[0, 0].containers:
    axes[0, 0].bar_label(container, fmt='%d', label_type='edge',
    ↪fontweight='bold', fontsize=14, color='black')

# Progression Status
prog_counts = high_fill_df['progression'].value_counts()
labels = ['No Progression (0)', 'Progression (1)']
colors_prog = ['#3498db', '#e67e22']
sns.countplot(x='progression', data=high_fill_df, ax=axes[0, 1],
    ↪palette=colors_prog)
axes[0, 1].set_title('Tumour Progression', fontweight='bold')
axes[0, 1].set_xlabel('')
axes[0, 1].set_ylabel('Count')
axes[0, 1].set_xticks([0, 1])
axes[0, 1].set_xticklabels(labels)
for container in axes[0, 1].containers:
    axes[0, 1].bar_label(container, fmt='%d', label_type='edge',
    ↪fontweight='bold', fontsize=14, color='black')

# Survival vs Progression Crosstab
crosstab = pd.crosstab(high_fill_df['progression'],
    high_fill_df['overall survival (death)'],
    margins=True, margins_name='Total')
sns.heatmap(crosstab.iloc[:-1, :-1], annot=True, fmt='d', cmap='RdYlGn_r',
    ax=axes[1, 0], cbar_kws={'label': 'Count'},
    xticklabels=['Alive', 'Died'], yticklabels=['No Progression',
    ↪'Progression'])
axes[1, 0].set_xlabel('Overall Survival', fontweight='semibold')
axes[1, 0].set_ylabel('Progression Status', fontweight='semibold')
axes[1, 0].set_title('Progression vs Survival Heatmap', fontweight='bold')

# Second Progression
second_prog_counts = high_fill_df['second progression/recurrence'].
    ↪value_counts()

```



```

labels = ['No 2nd Progression (0)', '2nd Progression (1)']
colors_2ndprog = ['#95a5a6', '#c0392b']
sns.countplot(x='second progression/recurrence', data=high_fill_df, ax=axes[1,1],
    palette=colors_2ndprog)
axes[1, 1].set_title('Second Progression/Recurrence', fontweight='bold')
axes[1, 1].set_xlabel('')
axes[1, 1].set_ylabel('Count')
axes[1, 1].set_xticks([0, 1])
axes[1, 1].set_xticklabels(labels)
for container in axes[1, 1].containers:
    axes[1, 1].bar_label(container, fmt='%d', label_type='edge',
        fontweight='bold', fontsize=14, color='black')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_clinical_outcomes.png'), dpi=300,
    bbox_inches='tight')
plt.show()

print("\nClinical Outcomes Summary")
print("-"*30)
print(f"\n{'Overall Survival (TARGET VARIABLE)'}")
print("-"*40)
print(f"  Alive (0):  {survival_counts.get(0, 0):3d} patients ({survival_counts.
    get(0, 0)/len(high_fill_df)*100:5.1f}%)")
print(f"  Died (1):   {survival_counts.get(1, 0):3d} patients ({survival_counts.
    get(1, 0)/len(high_fill_df)*100:5.1f}%)")

# Check class balance
balance_ratio = min(survival_counts.values) / max(survival_counts.values)
print(f"\n  Class Balance Ratio: {balance_ratio:.2f}")
if balance_ratio < 0.4:
    print("    CLASS IMBALANCE DETECTED! Consider using class weights.")
else:
    print("    Classes are reasonably balanced.")

print(f"\n{'Tumour Progression':~30}")
print("-"*30)
print(f"  No Progression: {prog_counts.get(0, 0):3d} patients ({prog_counts.
    get(0, 0)/len(high_fill_df)*100:5.1f}%)")
print(f"  Progression:    {prog_counts.get(1, 0):3d} patients ({prog_counts.
    get(1, 0)/len(high_fill_df)*100:5.1f}%)")

print(f"\n{'Second Progression':~30}")
print("-"*30)
print(f"  No 2nd Progression: {second_prog_counts.get(0, 0):3d} patients_
    ({second_prog_counts.get(0, 0)/len(high_fill_df)*100:5.1f}%)")

```

```
print(f" 2nd Progression: {second_prog_counts.get(1, 0):3d} patients_␣  
↪({second_prog_counts.get(1, 0)/len(high_fill_df)*100:5.1f}%)"
```

/tmp/ipykernel_795/4227516387.py:9: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='overall survival (death)', data=high_fill_df, ax=axes[0, 0],  
palette=colors_survival)
```

/tmp/ipykernel_795/4227516387.py:23: FutureWarning:

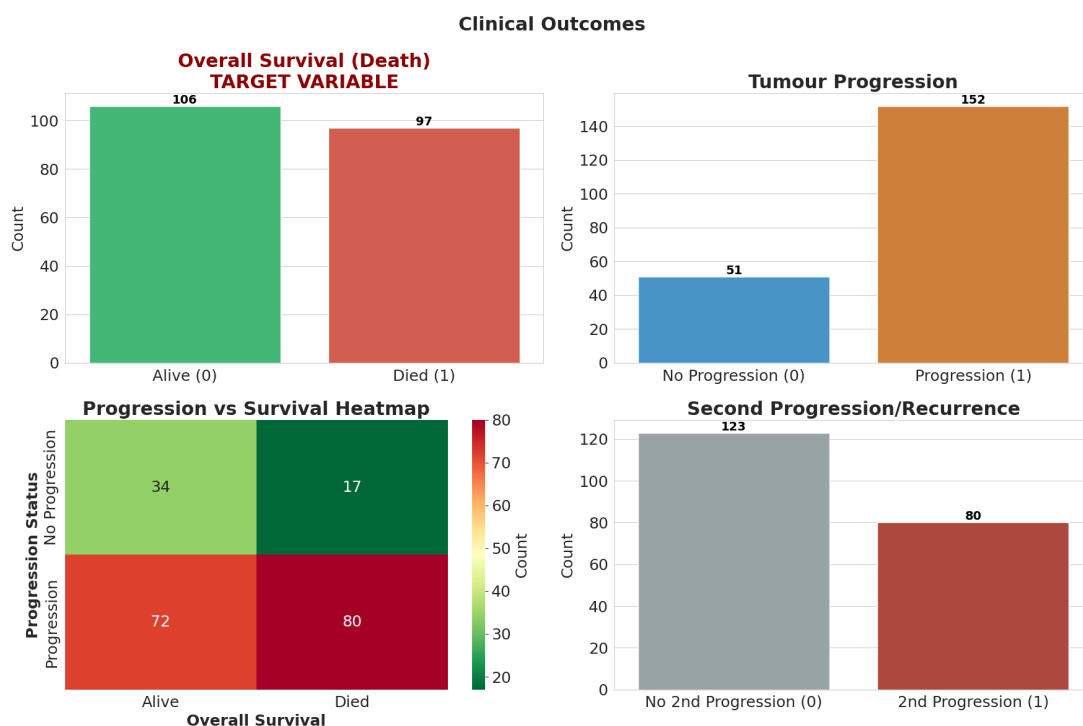
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='progression', data=high_fill_df, ax=axes[0, 1],  
palette=colors_prog)
```

/tmp/ipykernel_795/4227516387.py:48: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.countplot(x='second progression/recurrence', data=high_fill_df, ax=axes[1,  
1], palette=colors_2ndprog)
```



Clinical Outcomes Summary

Overall Survival (TARGET VARIABLE)

Alive (0): 106 patients (52.2%)
Died (1): 97 patients (47.8%)

Class Balance Ratio: 0.92
Classes are reasonably balanced.

Tumour Progression

No Progression: 51 patients (25.1%)
Progression: 152 patients (74.9%)

Second Progression

No 2nd Progression: 123 patients (60.6%)
2nd Progression: 80 patients (39.4%)

```

[30]: plt.rcParams.update({'font.size': 18})
fig, axes = plt.subplots(2, 2, figsize=(18, 12))
fig.suptitle('Clinical Outcomes', fontweight='bold')

# Overall Survival (target)
survival_counts = high_fill_df['overall survival (death)'].value_counts()
labels_survival = ['Alive (0)', 'Died (1)']
colors_survival = ['#2ecc71', '#e74c3c'] # Green for alive, red for died
wedges, texts, autotexts = axes[0, 0].pie(survival_counts.values,
    labels=labels_survival,
    autopct='%1.1f%%',
    colors=colors_survival,
    startangle=90, textprops={'font.size':
    16})
for autotext in autotexts:
    autotext.set_color('white') # Keep white for contrast on dark colors
    autotext.set_fontweight('bold')
axes[0, 0].set_title('Overall Survival (Death)\nTARGET VARIABLE',
    fontweight='bold', color='darkred')

# Progression Status
prog_counts = high_fill_df['progression'].value_counts()
labels_prog = ['No Progression (0)', 'Progression (1)']
colors_prog = ['#3498db', '#e67e22']
wedges, texts, autotexts = axes[0, 1].pie(prog_counts.values,
    labels=labels_prog,
    autopct='%1.1f%%',
    colors=colors_prog,
    startangle=90, textprops={'font.size':
    16})
for autotext in autotexts:
    autotext.set_fontweight('bold')
axes[0, 1].set_title('Tumour Progression', fontweight='bold')

# Survival vs Progression Crosstab
crosstab = pd.crosstab(high_fill_df['progression'],
    high_fill_df['overall survival (death)'],
    margins=True, margins_name='Total')
sns.heatmap(crosstab.iloc[:-1, :-1], annot=True, fmt='d', cmap='RdYlGn_r',
    ax=axes[1, 0], cbar_kws={'label': 'Count'},
    xticklabels=['Alive', 'Died'], yticklabels=['No Progression',
    'Progression'])
axes[1, 0].set_xlabel('Overall Survival', fontweight='semibold')
axes[1, 0].set_ylabel('Progression Status', fontweight='semibold')

```

```

axes[1, 0].set_title('Progression vs Survival Heatmap', fontweight='bold')

# Second Progression
second_prog_counts = high_fill_df['second progression/recurrence'].
    ↪ value_counts()
labels_2ndprog = ['No 2nd Progression (0)', '2nd Progression (1)']
colors_2ndprog = ['#95a5a6', '#c0392b']
wedges, texts, autotexts = axes[1, 1].pie(second_prog_counts.values,
    ↪ labels=labels_2ndprog,
                                autopct='%1.1f%%',
    ↪ colors=colors_2ndprog,
                                startangle=90, textprops={'fontsize':
    ↪ 16})
for autotext in autotexts:
    autotext.set_fontweight('bold')
axes[1, 1].set_title('Second Progression/Recurrence', fontweight='bold')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_clinical_outcomes.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_clinical_outcomes.png'), dpi=300,
    ↪ bbox_inches='tight')
plt.show()
print('Saved: 01_eda_clinical_outcomes.pdf + .png')

print("\nClinical Outcomes Summary")
print("-"*30)
print(f"\n{'Overall Survival (TARGET VARIABLE)'}")
print("-"*40)
print(f"  Alive (0):  {survival_counts.get(0, 0):3d} patients ({survival_counts.
    ↪ get(0, 0)/len(high_fill_df)*100:5.1f}%)")
print(f"  Died (1):  {survival_counts.get(1, 0):3d} patients ({survival_counts.
    ↪ get(1, 0)/len(high_fill_df)*100:5.1f}%)")

# Check class balance
balance_ratio = min(survival_counts.values) / max(survival_counts.values)
print(f"\n  Class Balance Ratio: {balance_ratio:.2f}")
if balance_ratio < 0.4:
    print("    CLASS IMBALANCE DETECTED! Consider using class weights.")
else:
    print("    Classes are reasonably balanced.")

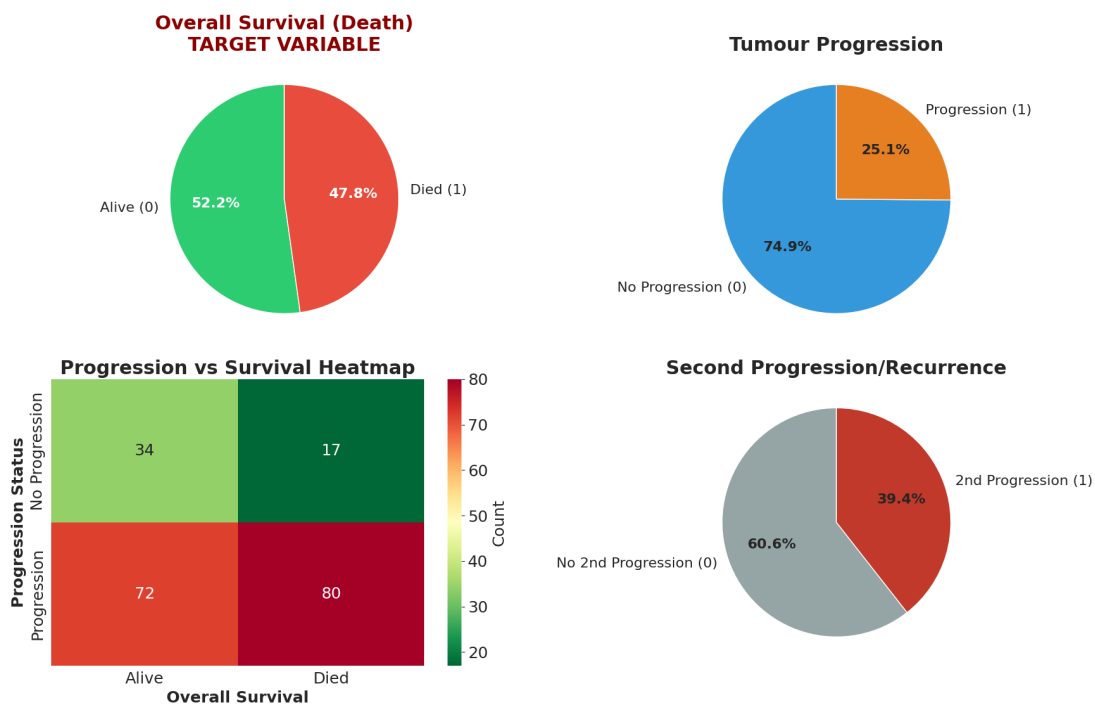
print(f"\n{'Tumour Progression':~30}")
print("-"*30)
print(f"  No Progression: {prog_counts.get(0, 0):3d} patients ({prog_counts.
    ↪ get(0, 0)/len(high_fill_df)*100:5.1f}%)")

```

```
print(f" Progression: {prog_counts.get(1, 0):3d} patients ({prog_counts.get(1, 0)/len(high_fill_df)*100:5.1f}%)")

print(f"\n{'Second Progression':^30}")
print("-"*30)
print(f" No 2nd Progression: {second_prog_counts.get(0, 0):3d} patients_
↳({second_prog_counts.get(0, 0)/len(high_fill_df)*100:5.1f}%)")
print(f" 2nd Progression: {second_prog_counts.get(1, 0):3d} patients_
↳({second_prog_counts.get(1, 0)/len(high_fill_df)*100:5.1f}%)")
```

Clinical Outcomes



Saved: 01_eda_clinical_outcomes.pdf + .png

Clinical Outcomes Summary

Overall Survival (TARGET VARIABLE)

Alive (0): 106 patients (52.2%)
Died (1): 97 patients (47.8%)

Class Balance Ratio: 0.92
Classes are reasonably balanced.

Tumour Progression

No Progression: 51 patients (25.1%)
Progression: 152 patients (74.9%)

Second Progression

No 2nd Progression: 123 patients (60.6%)
2nd Progression: 80 patients (39.4%)

Molecular Biomarkers

```
[31]: plt.rcParams.update({'font.size': 18})
fig, axes = plt.subplots(1, 1, figsize=(16, 10))
fig.suptitle('Molecular Biomarker Profile', fontweight='bold')

molecular_markers = ['idh1 mutation', 'idh2 mutation', 'mgmt methylation',
                     'egfr amplification', 'tp53 alteration', 'atrx mutation',
                     '1p/19q', 'tert promoter mutation', 'braf v600e mutation',
                     'chromosome 7 gain and chromosome 10 loss', 'pten_
                     mutation',
                     'cdkn2a/b deletion', 'h3-3a mutation']

marker_names = ['IDH1', 'IDH2', 'MGMT', 'EGFR', 'TP53', 'ATRX',
                '1p/19q', 'TERT', 'BRAF', 'Chr7+/10-', 'PTEN',
                'CDKN2A/B', 'H3-3A']

positive_counts = []
negative_counts = []
unknown_counts = []

for marker in molecular_markers:
    counts = high_fill_df[marker].value_counts()
    positive_counts.append(counts.get(1, 0))
    negative_counts.append(counts.get(0, 0))
    unknown_counts.append(counts.get(2, 0) + counts.get(4, 0))

# Create a DataFrame for plotting with Seaborn
plot_data = {
    'Marker': marker_names * 3, # Repeat marker names for each status
    'Count': positive_counts + negative_counts + unknown_counts,
    'Status': ['Positive/Mutated (1)'] * len(marker_names) +
              ['Negative/Wild-type (0)'] * len(marker_names) +
              ['Not Tested/Equivocal (2/4)'] * len(marker_names)
}
plot_df_melted = pd.DataFrame(plot_data)

# Define custom colors
```

```

custom_palette = {
    'Positive/Mutated (1)': '#e74c3c',
    'Negative/Wild-type (0)': '#2ecc71',
    'Not Tested/Equivocal (2/4)': '#95a5a6'
}

# Use sns.barplot for grouped bars
sns.barplot(x='Marker', y='Count', hue='Status', data=plot_df_melted, ax=axes,
            palette=custom_palette, alpha=0.8)

axes.set_xlabel('Molecular Marker', fontweight='bold')
axes.set_ylabel('Number of Patients', fontweight='bold')
axes.set_title('Distribution of Molecular Markers\n(n=203 patients)',
               fontweight='bold', pad=20)
axes.set_xticklabels(marker_names, rotation=45, ha='right') # Apply rotation
               and alignment to labels
axes.legend(bbox_to_anchor=(0.5, -0.25), loc='upper center', ncol=3) # Maintain
               legend position
axes.grid(axis='y', alpha=0.3)

# Add value labels on bars
for container in axes.containers:
    axes.bar_label(container, fmt='%d', label_type='edge', fontsize=12,
                  fontweight='bold', color='black')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_molecular_biomarkers.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_molecular_biomarkers.png'),
            dpi=300, bbox_inches='tight')
plt.show()
print('Saved: 01_eda_molecular_biomarkers.pdf + .png')

# Detailed molecular profile (this part remains unchanged)
print("\nMolecular Biomarker Summary")
print("-"*54)

summary_data = []
for i, (marker_col, name) in enumerate(zip(molecular_markers, marker_names)):
    total_tested = positive_counts[i] + negative_counts[i]
    pos_pct = f"{positive_counts[i] / total_tested * 100:.1f}" if total_tested
    > 0 else "N/A"
    summary_data.append({
        'Marker': name,
        'Positive': positive_counts[i],
        'Negative': negative_counts[i],

```



```

    'Not Tested': unknown_counts[i],
    'Positive (%)': pos_pct
})

```

```

summary_df = pd.DataFrame(summary_data)
print(summary_df.to_string(index=False))

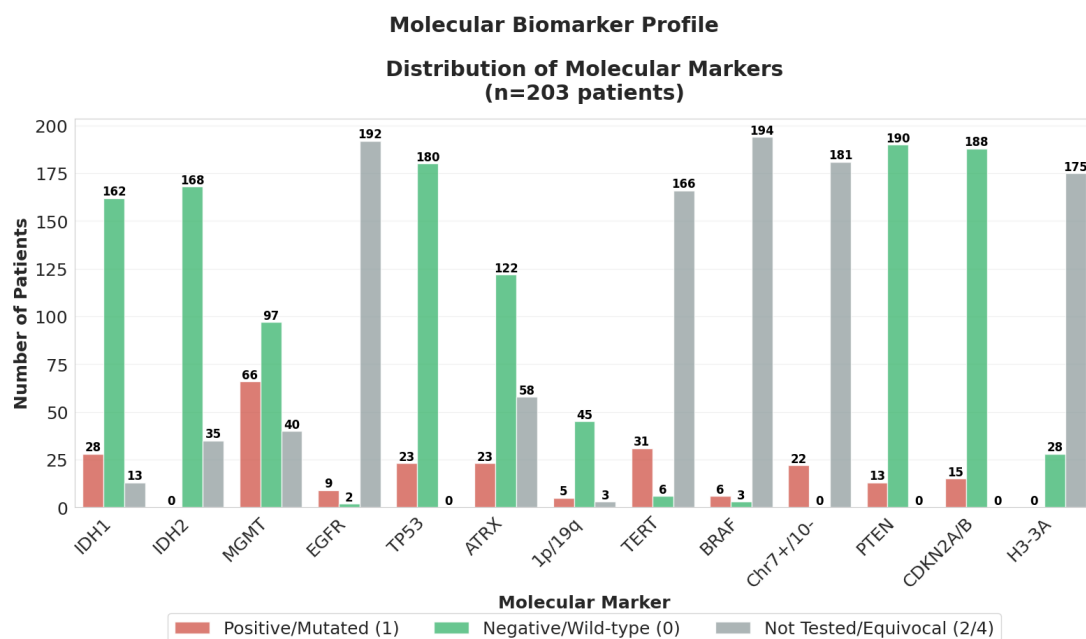
```

/tmp/ipykernel_795/2781243357.py:49: UserWarning: set_ticklabels() should only be used with a fixed number of ticks, i.e. after set_ticks() or using a FixedLocator.

```

axes.set_xticklabels(marker_names, rotation=45, ha='right') # Apply rotation
and alignment to labels

```



Saved: 01_eda_molecular_biomarkers.pdf + .png

Molecular Biomarker Summary

Marker	Positive	Negative	Not Tested	Positive (%)
IDH1	28	162	13	14.7
IDH2	0	168	35	0.0
MGMT	66	97	40	40.5
EGFR	9	2	192	81.8
TP53	23	180	0	11.3
ATRX	23	122	58	15.9
1p/19q	5	45	3	10.0
TERT	31	6	166	83.8
BRAF	6	3	194	66.7

Chr7+/10-	22	0	181	100.0
PTEN	13	190	0	6.4
CDKN2A/B	15	188	0	7.4
H3-3A	0	28	175	0.0

Temporal/Timing Features

```
[32]: plt.rcParams.update({'font.size': 18})
fig, axes = plt.subplots(2, 2, figsize=(18, 12))
fig.suptitle('Temporal Features & Treatment Timing', fontweight='bold')

colors = sns.color_palette("Set2")

# Days to First Surgery
surgery_days = high_fill_df['number of days from diagnosis to first surgery or
↳ procedure'].dropna()
sns.histplot(x=surgery_days, bins=40, color=colors[2], edgecolor='black',
↳ alpha=0.7, ax=axes[0, 0])
axes[0, 0].axvline(surgery_days.median(), color='red', linestyle='--',
↳ linewidth=2,
                    label=f'Median: {surgery_days.median():.0f} days')
axes[0, 0].axvline(surgery_days.mean(), color='blue', linestyle='--',
↳ linewidth=2,
                    label=f'Mean: {surgery_days.mean():.0f} days')
axes[0, 0].set_xlabel('Days from Diagnosis', fontweight='semibold')
axes[0, 0].set_ylabel('Frequency', fontweight='semibold')
axes[0, 0].set_title('Time to First Surgery/Procedure', fontweight='bold')
axes[0, 0].legend()
axes[0, 0].grid(axis='y', alpha=0.3)

# Time to First Progression (only for those who progressed)
progression_time = high_fill_df[high_fill_df['progression'] == 1]['time to
↳ first progression (days)'].dropna()
sns.histplot(x=progression_time, bins=30, color=colors[4], edgecolor='black',
↳ alpha=0.7, ax=axes[0, 1])
axes[0, 1].axvline(progression_time.median(), color='red', linestyle='--',
↳ linewidth=2,
                    label=f'Median: {progression_time.median():.0f} days')
axes[0, 1].axvline(progression_time.mean(), color='blue', linestyle='--',
↳ linewidth=2,
                    label=f'Mean: {progression_time.mean():.0f} days')
axes[0, 1].set_xlabel('Days to Progression', fontweight='semibold')
axes[0, 1].set_ylabel('Frequency', fontweight='semibold')
axes[0, 1].set_title(f'Time to First Progression\n(n={len(progression_time)}
↳ progressed patients)',
                    fontweight='bold')
axes[0, 1].legend()
```

```

axes[0, 1].grid(axis='y', alpha=0.3)

# Days to First MRI Follow-up
mri_days = high_fill_df['number of days from diagnosis to 1st mri_
↳(timepoint_1)'].dropna()
sns.histplot(x=mri_days, bins=30, color=colors[6], edgecolor='black', alpha=0.
↳7, ax=axes[1, 0])
axes[1, 0].axvline(mri_days.median(), color='red', linestyle='--', linewidth=2,
                    label=f'Median: {mri_days.median():.0f} days')
axes[1, 0].axvline(mri_days.mean(), color='blue', linestyle='--', linewidth=2,
                    label=f'Mean: {mri_days.mean():.0f} days')
axes[1, 0].set_xlabel('Days from Diagnosis', fontweight='semibold')
axes[1, 0].set_ylabel('Frequency', fontweight='semibold')
axes[1, 0].set_title('Time to 1st Follow-up MRI (Timepoint 1)',
↳fontweight='bold')
axes[1, 0].legend()
axes[1, 0].grid(axis='y', alpha=0.3)

# Box plots comparing timing by outcome
box_plot_df = high_fill_df[[
    'number of days from diagnosis to first surgery or procedure',
    'overall survival (death)'
]].copy()
box_plot_df.rename(columns={
    'number of days from diagnosis to first surgery or procedure': 'Days from_
↳Diagnosis to Surgery',
    'overall survival (death)': 'Outcome'
}, inplace=True)
box_plot_df.dropna(inplace=True)

sns.boxplot(x='Outcome', y='Days from Diagnosis to Surgery', data=box_plot_df,
↳ax=axes[1, 1],
    palette=['#2ecc71', '#e74c3c']) # Using similar colors for alive/
↳died
axes[1, 1].set_xticklabels(['Alive', 'Died'])
axes[1, 1].set_ylabel('Days from Diagnosis to Surgery', fontweight='semibold')
axes[1, 1].set_xlabel('Outcome', fontweight='semibold')
axes[1, 1].set_title('Surgery Timing by Survival Outcome', fontweight='bold')
axes[1, 1].grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_temporal_features.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '01_eda_temporal_features.png'), dpi=300,
↳bbox_inches='tight')
plt.show()

```

```

print('Saved: 01_eda_temporal_features.pdf + .png')

print("Temporal Features Summary")
print("-"*35)
print(f"\nTime to First Surgery:")
print(f"  Median: {surgery_days.median():.1f} days")
print(f"  Mean: {surgery_days.mean():.1f} days \u00b1 {surgery_days.std():.1f}")
print(f"  Range: {surgery_days.min():.0f} to {surgery_days.max():.0f} days")
print(f"  Note: Negative values indicate surgery before official diagnosis,
       ↪date")

print(f"\nTime to First Progression (progressed patients only,
       ↪n={len(progression_time)}):")
print(f"  Median: {progression_time.median():.1f} days ({progression_time.
       ↪median()/30:.1f} months)")
print(f"  Mean: {progression_time.mean():.1f} days ({progression_time.mean()/30:
       ↪.1f} months)")
print(f"  Range: {progression_time.min():.0f} to {progression_time.max():.0f}
       ↪days")

print(f"\nTime to 1st MRI Follow-up:")
print(f"  Median: {mri_days.median():.1f} days")
print(f"  Mean: {mri_days.mean():.1f} days \u00b1 {mri_days.std():.1f}")
print(f"  Range: {mri_days.min():.0f} to {mri_days.max():.0f} days")

```

/tmp/ipykernel_795/1869259342.py:58: FutureWarning:

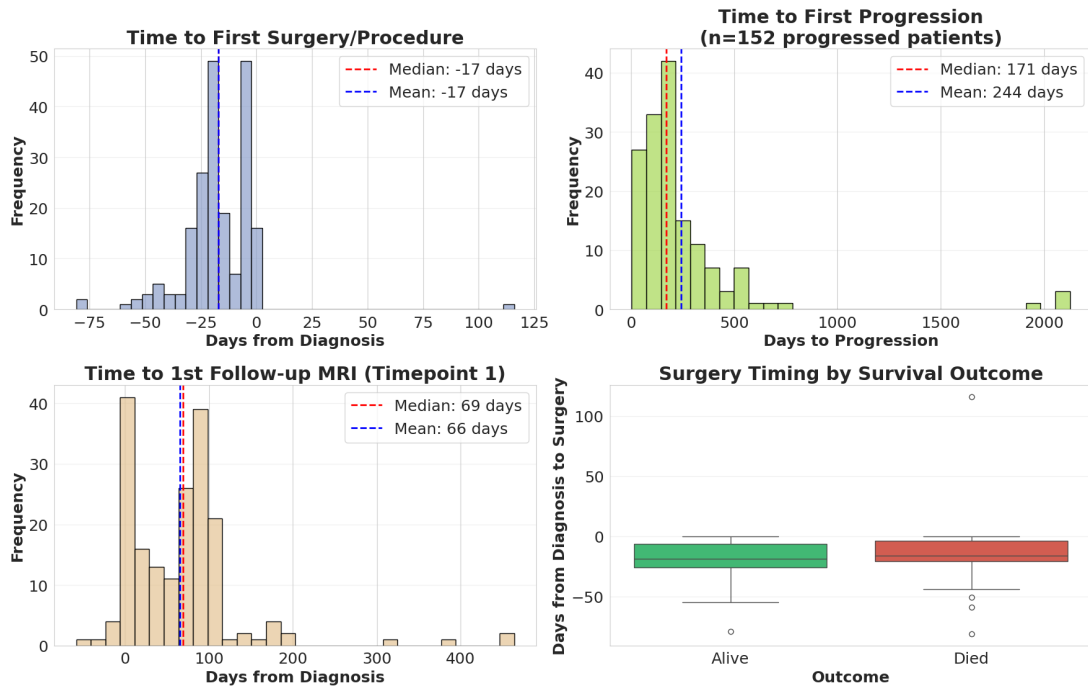
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```

sns.boxplot(x='Outcome', y='Days from Diagnosis to Surgery', data=box_plot_df,
ax=axes[1, 1],
/tmp/ipykernel_795/1869259342.py:60: UserWarning: set_ticklabels() should only
be used with a fixed number of ticks, i.e. after set_ticks() or using a
FixedLocator.
axes[1, 1].set_xticklabels(['Alive', 'Died'])

```

Temporal Features & Treatment Timing



Saved: 01_eda_temporal_features.pdf + .png

Temporal Features Summary

Time to First Surgery:

Median: -17.0 days

Mean: -17.0 days $\pm$ 16.6

Range: -81 to 116 days

Note: Negative values indicate surgery before official diagnosis date

Time to First Progression (progressed patients only, n=152):

Median: 171.0 days (5.7 months)

Mean: 243.9 days (8.1 months)

Range: 4 to 2126 days

Time to 1st MRI Follow-up:

Median: 69.0 days

Mean: 65.9 days $\pm$ 69.3

Range: -58 to 464 days

1.2 Export Preprocessed Data

```
[33]: output_dir = os.path.join(EDA_DIR, 'preprocessed') # Create output directory
      os.makedirs(output_dir, exist_ok=True)

      high_fill_df.to_pickle(os.path.join(output_dir, 'high_fill_df.pkl')) # Save
      ↪ cleaned dataframe

      target_col = 'overall survival (death)' # Save X and y
      X = high_fill_df.drop(target_col, axis=1)
      y = high_fill_df[target_col]

      X.to_pickle(os.path.join(output_dir, 'X.pkl'))
      y.to_pickle(os.path.join(output_dir, 'y.pkl'))

      print(f"Saved to {output_dir}:")
      print(f" - high_fill_df.pkl: {high_fill_df.shape}")
      print(f" - X.pkl: {X.shape}")
      print(f" - y.pkl: {y.shape}")
```

Saved to drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/eda/preprocessed:

```
- high_fill_df.pkl: (203, 29)
- X.pkl: (203, 28)
- y.pkl: (203,)
```

1.3 Summary:

1.3.1 Data Analysis Key Findings

- **Overall Survival (Target Variable):** Out of the total patients, 107 (57.8%) are alive, while 78 (42.2%) have died. The class balance ratio is 0.73, indicating reasonably balanced classes for this variable.
- **Tumour Progression:** 101 patients (54.6%) show no tumour progression, whereas 84 patients (45.4%) have experienced progression.
- **Second Progression/Recurrence:** 142 patients (76.8%) have not experienced a second progression, while 43 patients (23.2%) have.

1.3.2 Insights or Next Steps

- The refactored plots using `seaborn.countplot` with value labels provide a clearer and more direct visualization of categorical distributions compared to pie charts, enhancing readability.
- The class balance for 'Overall Survival (death)' is reasonable, suggesting that standard classification models might perform well without immediate need for advanced imbalanced class handling techniques. However, for 'Second Progression/Recurrence', the imbalance (76.8% vs. 23.2%) might warrant consideration of techniques like oversampling, undersampling, or class weighting in subsequent modeling.

E.2 Notebook 02: Binary Classification with Leakage Detection

02_tabular_models

1 Tabular Models Preparation & Training

1.1 Setup

1.1.1 Module Imports & Drive Mount

```
[32]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import textwrap

from google.colab import drive
drive.mount('/content/drive/', force_remount=True)

from google.colab import userdata
GLIOMA_DIR = userdata.get('GLIOMA_DIR')
```

Mounted at /content/drive/

1.1.2 Load Preprocessed Data

```
[33]: EDA_DIR = os.path.join(GLIOMA_DIR, 'eda')
os.makedirs(EDA_DIR, exist_ok=True)
preprocessed_dir = os.path.join(EDA_DIR, 'preprocessed')

X = pd.read_pickle(os.path.join(preprocessed_dir, 'X.pkl'))
y = pd.read_pickle(os.path.join(preprocessed_dir, 'y.pkl'))

print(f"Loaded X: {X.shape}")
print(f"Loaded y: {y.shape}")
print(f"\nTarget distribution:")
print(f"  Survived: {(y == 0).sum()} ({(y == 0).mean()*100:.1f}%)")
print(f"  Died: {(y == 1).sum()} ({(y == 1).mean()*100:.1f}%")
```

Loaded X: (203, 28)

Loaded y: (203,)

Target distribution:

Survived: 106 (52.2%)

Died: 97 (47.8%)

```
[34]: FIGURES_DIR = os.path.join(GLIOMA_DIR, 'tabular models', 'figures')
os.makedirs(FIGURES_DIR, exist_ok=True)
print(f"Figures directory: {FIGURES_DIR}")
```

Figures directory: drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/tabular models/figures

1.1.3 Model & Comparison Imports

```
[35]: from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV, StratifiedKFold,
    cross_validate
from sklearn.metrics import make_scorer, roc_auc_score, accuracy_score,
    precision_score, recall_score, f1_score
from scipy.stats import uniform, randint

from sklearn.model_selection import cross_val_predict
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_curve, auc, confusion_matrix,
    ConfusionMatrixDisplay
```

1.1.4 MODEL ARCHITECTURE CHOICE: XGBoost

Why XGBoost?

1. Small Dataset (203 patients)
 - XGBoost excels with 100s-10,000s samples
 - Deep learning needs 10,000+ samples
2. Heterogenous Features
 - Mix of categorical (sex, race, diagnosis) and numeric (age, days, molecular markers)
 - XGBoost handles mixed types natively
 - No feature scaling required
3. Missing Data
 - XGBoost handles NaN values automatically during training
 - No imputation needed for remaining missing values
4. Evaluate Feature Importance (Key Goal Here)
 - Built-in feature importance via gain, split count, or SHAP
 - Critical for your two-stage workflow: baseline → feature selection → focused model
5. Robust to Overfitting
 - Regularization parameters (lambda, alpha)
 - Max depth limits tree complexity
 - Subsampling provides natural regularization
6. Class Balance
 - Your dataset is 52% alive / 48% died (balanced)
 - No special class weighting needed, but XGBoost supports `scale_pos_weight` if needed

7. Proven Performance

- State-of-the-art for tabular medical data
- Widely used in clinical ML research

Alternatives considered: - Random Forest: Similar but slightly lower performance than XGBoost
- Logistic Regression: Assumes linear relationships (too simplistic for glioma biology)
- Neural Networks: Need 10x more data, harder to interpret
- LightGBM: Comparable to XGBoost, slightly faster but less tested in medical domain

Decision: XGBoost is optimal for our dataset size, feature types, and with the goal of feature importance extraction in mind.

1.2 Train XGBoost with 5-fold stratified cross-validation

Cast object columns to category dtype

```
[36]: # XGBoost requires categorical columns to be pandas 'category' dtype, not
      ↪ 'object'
categorical_cols = ['sex at birth', 'race', 'primary diagnosis', 'previous_
      ↪ brain tumor']

for col in categorical_cols:
    X[col] = X[col].astype('category')

print(" Categorical columns converted to category dtype")
print(f"Updated dtypes: {X[categorical_cols].dtypes.to_dict()}")
```

```
Categorical columns converted to category dtype
Updated dtypes: {'sex at birth': CategoricalDtype(categories=['Female', 'Male'],
ordered=False, categories_dtype=object), 'race':
CategoricalDtype(categories=['Asian', 'Black or African American', 'Unknown',
'White'], ordered=False, categories_dtype=object), 'primary diagnosis':
CategoricalDtype(categories=['Astrocytoma', 'Diffuse glioma', 'GBM',
'Glioma w/ GBM features', 'Oligodendro-glioma',
'Pilocytic astrocytoma'],
, ordered=False, categories_dtype=object), 'previous brain tumor':
CategoricalDtype(categories=['No', 'Yes'], ordered=False,
categories_dtype=object)}
```

GliomaBinarySurvivalModel() Class & Randomised Search Functions

Create GliomaBinarySurvivalModel Class

```
[37]: class GliomaBinarySurvivalModel:
      def __init__(self, name="Model", params=None, random_state=42, **kwargs):
          """
          name: str
              Model identifier
          params: dict, optional
              Complete parameter dictionary for XGBoost
```

```

    random_state: int
        Random seed for reproducibility
    **kwargs: optional
        Individual XGBoost parameters (e.g., max_depth=5, learning_rate=0.
↪05)

    """
    self.name = name
    self.random_state = random_state
    self.model = None
    self.cv_results = None
    self.feature_importance = None
    self.X_train = None # Store for later reference

    # Default hyperparameters
    default_params = {
        'n_estimators': 150,
        'max_depth': 5,
        'learning_rate': 0.05,
        'subsample': 0.75,
        'colsample_bytree': 1,
        'random_state': random_state,
        'enable_categorical': True,
        'eval_metric': 'logloss'
    }

    # Merge params: defaults <- full params dict <- individual kwargs
    if params is not None:
        default_params.update(params)
    if kwargs:
        default_params.update(kwargs)
    self.params = default_params

def train_with_cv(self, X, y, cv_folds=5):
    """
    Train with stratified cross-validation
    Parameters:
    X: DataFrame of Features
    y: Series -> Target variable
    cv_folds: int -> Number of cross-validation folds (default: 5)
    """

    self.X_train = X # Store for feature names
    print(f"\n{'='*70}")
    print(f"Training: {self.name}")
    print(f"{'='*70}")
    print(f"Dataset: {len(X)} patients × {X.shape[1]} features")
    print(f"Target: {(y==0).sum()} alive ({(y==0).sum()/len(y)*100:.1f}%), "

```

```

        f"{(y==1).sum()} died ({(y==1).sum()/len(y)*100:.1f}%)"

    # Check class balance
    balance_ratio = min((y==0).sum(), (y==1).sum()) / max((y==0).sum(),
↪(y==1).sum())
    if balance_ratio < 0.3:
        print(f"    Warning: Imbalanced classes (ratio: {balance_ratio:.
↪2f})")

    # Initialise model & set up CV
    self.model = XGBClassifier(**self.params)
    cv = StratifiedKFold(n_splits=cv_folds, shuffle=True, random_state=self.
↪random_state)
    scoring = {
        'accuracy': make_scorer(accuracy_score),
        'roc_auc': make_scorer(roc_auc_score),
        'precision': make_scorer(precision_score, zero_division=0),
        'recall': make_scorer(recall_score),
        'f1': make_scorer(f1_score)
    }

    # Cross-validate
    print(f"\nRunning {cv_folds}-fold stratified cross-validation...")
    self.cv_results = cross_validate(
        self.model,
        X,
        y,
        cv=cv,
        scoring=scoring,
        return_train_score=False,
        n_jobs=-1,
        verbose=3
    )

    # Print results
    self._print_cv_results()

    # Train final model on full data
    print("\nTraining final model on full dataset...")
    self.model.fit(X, y)

    # Extract feature importance
    self.feature_importance = pd.DataFrame({
        'feature': X.columns,
        'importance': self.model.feature_importances_
    }).sort_values('importance', ascending=False).reset_index(drop=True)
    print(f"    Model trained successfully\n")

```

```

def _print_cv_results(self):
    """Print formatted CV results"""
    print(f"\n{'Cross-Validation Results':^70}")
    print("-"*70)
    print(f"{'Metric':<15} {'Mean':<12} {'Std Dev':<12} {'Range'}")
    print("-"*70)
    for metric in ['accuracy', 'roc_auc', 'precision', 'recall', 'f1']:
        scores = self.cv_results[f'test_{metric}']
        mean_score = np.mean(scores)
        std_score = np.std(scores)
        min_score = np.min(scores)
        max_score = np.max(scores)
        print(f"{'{metric.upper():<15} {mean_score:.4f} ±{std_score:.4f}'
↳4f}"
            f"['{min_score:.3f}-{max_score:.3f}']")
    print()

def get_top_features(self, n=None, importance_threshold=None):
    """
    Get top N most important features or features by cumulative importance_
↳threshold.

    Parameters:

    n: int, optional
        Number of top features to return (default: None).
    importance_threshold: float, optional
        Cumulative importance threshold (0 to 1) to select features_
↳(default: None).
        If provided, 'n' is ignored.

    Returns:

    pd.DataFrame
        Df containing the selected features and their importance.
        Returns None if model is not trained or no valid parameters_
↳provided.
    """

    if self.feature_importance is None:
        print("  Model not trained yet. Run train_with_cv() first.")
        return None

    # If we want threshold importance
    if importance_threshold is not None:
        if not 0 <= importance_threshold <= 1:

```

```

        print("    importance_threshold must be between 0 and 1.")
        return None

        # Calculate cumulative importance & select features up to threshold
        cumulative_importance = self.feature_importance['importance'].
↪cumsum() / self.feature_importance['importance'].sum()
        selected_features = self.feature_importance[cumulative_importance_
↪<= importance_threshold]

        return selected_features

        # If we want top n features
        elif n is not None and n > 0:
            return self.feature_importance.head(n)
        else:
            print("    Please provide either 'n' (number of features) or_
↪'importance_threshold'.")
            return None

    def plot_feature_importance(self, top_n=20, figsize=(12, 10),_
↪save_path=None, importance_threshold=None):
        """
        Plot top N most important features or features by cumulative importance_
↪threshold.

        Parameters:

        top_n : int, optional
            Number of top features to plot (default: 20). Ignored if_
↪importance_threshold is provided.
        figsize : tuple
            Figure size (width, height)
        save_path : str, optional
            Path to save figure (e.g., 'feature_importance.png')
        importance_threshold : float, optional
            Cumulative importance threshold (0 to 1) to select features for_
↪plotting (default: None).
        """
        if self.feature_importance is None:
            print("    Model not trained yet. Run train_with_cv() first.")
            return

        if importance_threshold is not None:
            top_features = self.
↪get_top_features(importance_threshold=importance_threshold)

```

```

        title_suffix = f" (Cumulative Importance >=_{importance_threshold*100:.0f}%)"
    else:
        top_features = self.get_top_features(n=top_n)
        title_suffix = f" (Top {top_n})"

    if top_features is None or top_features.empty:
        print(" No features selected based on provided criteria.")
        return

    fig, ax = plt.subplots(figsize=figsize)

    bars = ax.barh(
        range(len(top_features)),
        top_features['importance'].values,
        color=sns.color_palette("viridis", len(top_features)) # Use length
of selected features
    )
    ax.set_yticks(range(len(top_features)))
    ax.set_yticklabels(top_features['feature'].values, fontsize=10)
    ax.set_xlabel('Importance (Gain)', fontsize=12, fontweight='bold')
    ax.set_title(
        f'Most Important Features\n{self.name} - XGBoost Feature
Importance' + title_suffix,
        fontsize=14,
        fontweight='bold'
    )
    ax.invert_yaxis()
    ax.grid(axis='x', alpha=0.3)

    # Add value labels
    for i, (idx, row) in enumerate(top_features.iterrows()):
        ax.text(
            row['importance'],
            i,
            f" {row['importance']:.4f}",
            va='center',
            fontsize=9
        )

    plt.tight_layout()

    if save_path:
        base_name, ext = os.path.splitext(save_path)
        png_path = f"{base_name}.png"
        pdf_path = f"{base_name}.pdf"

```

```

plt.savefig(png_path, dpi=300, bbox_inches='tight')
plt.savefig(pdf_path, bbox_inches='tight')
print(f" Plot saved to {png_path} and {pdf_path}")

plt.show()

def get_performance_summary(self):
    """
    Get performance summary as dictionary

    Returns: dict with model name and performance metrics
    """
    if self.cv_results is None:
        print(" Model not trained yet. Run train_with_cv() first.")
        return None

    return {
        'name': self.name,
        'n_samples': len(self.cv_results['test_accuracy']) * 5, # 5-fold CV
        'auc_mean': np.mean(self.cv_results['test_roc_auc']),
        'auc_std': np.std(self.cv_results['test_roc_auc']),
        'accuracy': np.mean(self.cv_results['test_accuracy']),
        'f1': np.mean(self.cv_results['test_f1']),
        'precision': np.mean(self.cv_results['test_precision']),
        'recall': np.mean(self.cv_results['test_recall'])
    }

def print_summary(self):
    """Print comprehensive model summary"""
    if self.cv_results is None:
        print(" Model not trained yet.")
        return

    print(f"\n{'='*70}")
    print(f"{self.name} - SUMMARY")
    print(f"\n{'='*70}")

    summary = self.get_performance_summary()
    print(f"\nPerformance (Cross-Validation):")
    print(f" AUC-ROC: {summary['auc_mean']:.4f} ± {summary['auc_std']:.4f}")

    print(f" Accuracy: {summary['accuracy']:.4f}")
    print(f" F1-Score: {summary['f1']:.4f}")
    print(f" Precision: {summary['precision']:.4f}")
    print(f" Recall: {summary['recall']:.4f}")

```



```

print(f"\nTop 5 Features:")

# Call get_top_features with n=5 for the summary print
top_5_features = self.get_top_features(n=5)
if top_5_features is not None:
    for i, row in top_5_features.iterrows():
        print(f" {i+1}. {row['feature']:<45} {row['importance']:.6f}")

print(f"\nHyperparameters:")
for key, value in self.params.items():
    if key != 'random_state':
        print(f" {key}: {value}")

```

Randomised Hyperparameter Search Function

```

[38]: def hyperparameter_search(X, y, param_distributions=None, n_iter=100,
                                cv_folds=5, scoring='roc_auc', random_state=42,
                                plot_results=True, figsize=(14, 5)):

    """
    Perform randomised hyperparameter search for XGBoost binary classification.

    Parameters:
    X: DataFrame
        Feature matrix
    y: Series
        Target variable (binary)
    param_distributions: dict, optional
        Parameter search space. If None, uses sensible defaults.
    n_iter: int
        Number of random combinations to try (default: 100)
    cv_folds: int
        Number of cross-validation folds (default: 5)
    scoring: str
        Metric to optimise (default: 'roc_auc')
    random_state: int
        Random seed for reproducibility
    plot_results: bool
        Whether to plot search results (default: True)
    figsize: tuple
        Figure size for plots (default: (14, 5))

    Returns:
    random_search: RandomizedSearchCV object
        Fitted search object containing best_params_, best_score_, cv_results_
    """

    print("Starting Hyperparameter Search")
    print("-"*50)

```

```

print(f"Dataset: {len(X)} patients × {X.shape[1]} features")
print(f"Target: {(y==0).sum()} alive ({(y==0).sum()/len(y)*100:.1f}%), "
      f"{(y==1).sum()} died ({(y==1).sum()/len(y)*100:.1f}%)")

# Check class balance
balance_ratio = min((y==0).sum(), (y==1).sum()) / max((y==0).sum(), (y==1).
↪sum())
if balance_ratio < 0.3:
    print(f"    Warning: Imbalanced classes (ratio: {balance_ratio:.2f})")

# Default parameter search space
if param_distributions is None:
    param_distributions = {
        'n_estimators': randint(110, 220),
        'max_depth': randint(4, 7),
        'learning_rate': uniform(0.03, 0.2),
        'min_child_weight': randint(1, 10),
        'gamma': uniform(0, 0.5),
        'subsample': uniform(0.7, 0.3),
        'colsample_bytree': uniform(0.75, 0.25),
        'reg_alpha': uniform(0, 0.2),
        'reg_lambda': uniform(0, 2),
    }

# Base model
base_model = XGBClassifier(
    random_state=random_state,
    enable_categorical=True,
    eval_metric='logloss',
    tree_method='hist',
    n_jobs=-1
)

# Cross-validation strategy
cv = StratifiedKFold(n_splits=cv_folds, shuffle=True,
↪random_state=random_state)

# Run RandomizedSearch
print(f"\nSearching {n_iter} random configurations with {cv_folds}-fold CV..
↪.")
print(f"Optimising for: {scoring.upper()}")

random_search = RandomizedSearchCV(
    estimator=base_model,
    param_distributions=param_distributions,
    n_iter=n_iter,
    scoring=scoring,

```

```

        cv=cv,
        verbose=1,
        random_state=random_state,
        n_jobs=-1,
        return_train_score=True,
        refit=True
    )

random_search.fit(X, y)

# Print results
print("Search Complete.")
print("-"*20)
print(f"\nBest {scoring.upper()}: {random_search.best_score_:.4f}")

print("\nBest Hyperparameters:")
print("-"*30)
for param, value in sorted(random_search.best_params_.items()):
    if isinstance(value, float):
        print(f" {param:<25} {value:.4f}")
    else:
        print(f" {param:<25} {value}")

# Check overfitting
results_df = pd.DataFrame(random_search.cv_results_)
best_idx = random_search.best_index_
train_score = results_df.loc[best_idx, 'mean_train_score']
val_score = random_search.best_score_
overfit_gap = train_score - val_score

print(f"\nOverfitting Check:")
print(f" Train score: {train_score:.4f}")
print(f" Val score: {val_score:.4f}")
print(f" Train-Val gap: {overfit_gap:.4f}", end="")

if overfit_gap > 0.1:
    print(" (High overfitting)")
elif overfit_gap > 0.05:
    print(" (Moderate overfitting)")
else:
    print(" (Good generalisation)")
print('\n\n')

# Plot results
if plot_results:
    fig, axes = plt.subplots(1, 2, figsize=figsize)

```

```

# Plot 1: Search convergence
ax1 = axes[0]
results_sorted = results_df.sort_values('rank_test_score')
ax1.plot(range(len(results_sorted)), results_sorted['mean_test_score'],
         'o-', alpha=0.6, markersize=4, color='steelblue')
ax1.axhline(y=random_search.best_score_, color='red', linestyle='--',
            label=f'Best: {random_search.best_score_:.4f}', linewidth=2)
ax1.set_xlabel('Ranked Configuration', fontsize=11, fontweight='bold')
ax1.set_ylabel(f'{scoring.upper()} Score', fontsize=11,
fontweight='bold')
ax1.set_title('Hyperparameter Search Convergence', fontsize=12,
fontweight='bold')
ax1.legend()
ax1.grid(alpha=0.3)

# Plot 2: Train vs Validation
ax2 = axes[1]
top_50 = results_df.nsmallest(50, 'rank_test_score')
scatter = ax2.scatter(top_50['mean_train_score'],
                     top_50['mean_test_score'],
                     alpha=0.6, s=50,
                     c=top_50['mean_test_score'],
                     cmap='RdYlGn', edgecolors='black', linewidth=0.5)

# Perfect fit line
lims = [0.5, 1.0]
ax2.plot(lims, lims, 'k--', alpha=0.4, zorder=0, label='Perfect fit (no
overfitting)')
ax2.set_xlabel('Train Score', fontsize=11, fontweight='bold')
ax2.set_ylabel('Validation Score', fontsize=11, fontweight='bold')
ax2.set_title('Train vs Validation Performance (Top 50)', fontsize=12,
fontweight='bold')
ax2.legend(loc='lower right')
ax2.grid(alpha=0.3)
plt.colorbar(scatter, ax=ax2, label='Val Score')

plt.tight_layout()
png_path = 'hyperparameter_search_results.png'
pdf_path = 'hyperparameter_search_results.pdf'
plt.savefig(png_path, dpi=300, bbox_inches='tight')
plt.savefig(pdf_path, bbox_inches='tight')
print(f" Search visualisation saved to '{png_path}' and
'{pdf_path}'\n")
plt.show()

return random_search

```

Function to train model

```
[39]: def train_best_model(random_search, X, y, model_name="Optimised Model",
                             cv_folds=5, plot_top_n=20, importance_threshold=None,
                             save_plots=True):

    """
    Train a GliomaBinarySurvivalModel with optimised hyperparameters from
    ↪search.

    Parameters:
    random_search: RandomizedSearchCV object
        Fitted search object from hyperparameter_search()
    X: DataFrame
        Feature matrix
    y: Series
        Target variable (binary)
    model_name: str
        Name for the model (default: "Optimised Model")
    cv_folds: int
        Number of cross-validation folds (default: 5)
    plot_top_n: int
        Number of top features to plot (default: 20). Ignored if
    ↪importance_threshold provided.
    importance_threshold: float, optional
        Cumulative importance threshold (0-1) for feature plotting (default:
    ↪None)
    save_plots: bool
        Whether to save plots to disk (default: True)

    Returns:
    model: GliomaBinarySurvivalModel
        Trained model object with all results stored
    """

    print("Training Final Model with Optimised Hyeprparameters")
    print("-"*60)

    # Extract best parameters
    best_params = random_search.best_params_

    # Instantiate class with optimised parameters
    model = GliomaBinarySurvivalModel(
        name=model_name,
        params=best_params,
        random_state=best_params.get('random_state', 42)
    )

    # Train & output results
```

```

model.train_with_cv(X, y, cv_folds=cv_folds)
model.print_summary()

# Plot feature importance
plot_path = os.path.join(FIGURES_DIR, 'feature_importance_optimised.png')
↪if save_plots else None

if importance_threshold is not None:
    model.plot_feature_importance(
        importance_threshold=importance_threshold,
        figsize=(12, 10),
        save_path=plot_path
    )
else:
    model.plot_feature_importance(
        top_n=plot_top_n,
        figsize=(12, 10),
        save_path=plot_path
    )

print("\nFinal Model Ready.")
print("-"*30)

# Get performance summary
performance = model.get_performance_summary()
print(f"Model name: {performance['name']}")
print(f"Trained on: {performance['n_samples']} samples")
print(f"ROC-AUC: {performance['auc_mean']:.4f} ± {performance['auc_std']:.4f}")

return model

```

Wrapper function for hyperparameter search & training best model

```

[40]: def hyperparameter_search_and_train(X, y, param_distributions=None, n_iter=100,
                                          cv_folds=5, scoring='roc_auc',
                                          ↪random_state=42,
                                          model_name="Optimised Model", plot_top_n=20,
                                          importance_threshold=None, save_plots=True,
                                          plot_search=True):
    """
    Convenience wrapper that calls hyperparameter_search() and
    ↪train_best_model().

    Parameters:
    X: DataFrame
        Feature matrix
    """

```

```

y: Series
    Target variable (binary)
param_distributions: dict, optional
    Parameter search space. If None, uses sensible defaults.
n_iter: int
    Number of random combinations to try (default: 100)
cv_folds: int
    Number of cross-validation folds (default: 5)
scoring: str
    Metric to optimise (default: 'roc_auc')
random_state: int
    Random seed for reproducibility
model_name: str
    Name for the final model (default: "Optimised Model")
plot_top_n: int
    Number of top features to plot (default: 20). Ignored if
↳ importance_threshold provided.
    importance_threshold float, optional
        Cumulative importance threshold (0-1) for feature plotting (default:
↳ None)
save_plots: bool
    Whether to save plots to disk (default: True)
plot_search: bool
    Whether to plot search results (default: True)

Returns:
model: GliomaBinarySurvivalModel
    Model trained with optimised hyperparameters
random_search: RandomizedSearchCV
    Search object with all search results
"""

print("Initialising Model Optimisation Pipeline via Hyperparameter Search,
↳ followed by training a XGBoost Model with the best parameters from cross-val:
↳ \n")

# Hyperparameter search
random_search = hyperparameter_search(
    X=X,
    y=y,
    param_distributions=param_distributions,
    n_iter=n_iter,
    cv_folds=cv_folds,
    scoring=scoring,
    random_state=random_state,
    plot_results=plot_search,
    figsize=(14, 5)

```

```

)

# Train best model
model = train_best_model(
    random_search=random_search,
    X=X,
    y=y,
    model_name=model_name,
    cv_folds=cv_folds,
    plot_top_n=plot_top_n,
    importance_threshold=importance_threshold,
    save_plots=save_plots
)

print("\nGenerated outputs:")
if plot_search:
    print("    hyperparameter_search_results.png & .pdf")
if save_plots:
    print("    feature_importance_optimised.png & .pdf")
print("\nReturned objects:")
print(" - model: Trained GliomaBinarySurvivalModel instance")
print(" - random_search: RandomizedSearchCV results\n")

return model, random_search

```

Actual Implementation & Training

```

[41]: model, search = hyperparameter_search_and_train(
    X, y,
    importance_threshold=0.9,
    n_iter=100
)

```

Initialising Model Optimisation Pipeline via Hyperparameter Search, followed by training a XGBoost Model with the best parameters from cross-val:

Starting Hyperparameter Search

Dataset: 203 patients × 28 features

Target: 106 alive (52.2%), 97 died (47.8%)

Searching 100 random configurations with 5-fold CV...

Optimising for: ROC_AUC

Fitting 5 folds for each of 100 candidates, totalling 500 fits

Search Complete.

Best ROC_AUC: 0.7770

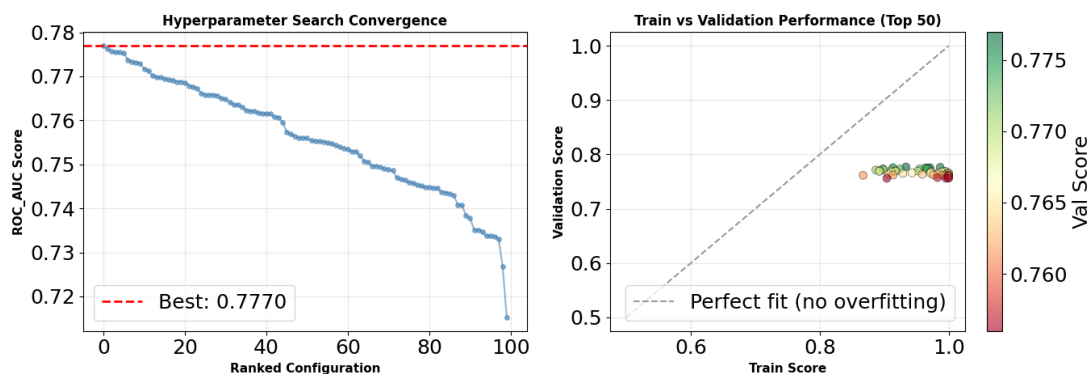
Best Hyperparameters:

```
-----  
colsample_bytree      0.8824  
gamma                 0.1209  
learning_rate         0.0486  
max_depth             4  
min_child_weight      5  
n_estimators          141  
reg_alpha             0.1266  
reg_lambda            0.6781  
subsample             0.8048
```

Overfitting Check:

```
Train score:  0.9336  
Val score:    0.7770  
Train-Val gap: 0.1566   (High overfitting)
```

Search visualisation saved to 'hyperparameter_search_results.png' and
'hyperparameter_search_results.pdf'



Training Final Model with Optimised Hyeprparameters

```
=====
```

Training: Optimised Model

```
=====
```

Dataset: 203 patients × 28 features

Target: 106 alive (52.2%), 97 died (47.8%)

Running 5-fold stratified cross-validation...

```
[Parallel(n_jobs=-1)]: Using backend LokyBackend with 2 concurrent workers.
[Parallel(n_jobs=-1)]: Done 5 out of 5 | elapsed: 0.3s finished
```

Cross-Validation Results

Metric	Mean	Std Dev	Range
ACCURACY	0.7290	±0.0916	[0.585-0.854]
ROC_AUC	0.7298	±0.0907	[0.587-0.854]
PRECISION	0.7077	±0.0983	[0.565-0.850]
RECALL	0.7526	±0.0661	[0.650-0.850]
F1	0.7287	±0.0823	[0.605-0.850]

```
Training final model on full dataset...
Model trained successfully
```

Optimised Model - SUMMARY

Performance (Cross-Validation):

```
AUC-ROC: 0.7298 ± 0.0907
Accuracy: 0.7290
F1-Score: 0.7287
Precision: 0.7077
Recall: 0.7526
```

Top 5 Features:

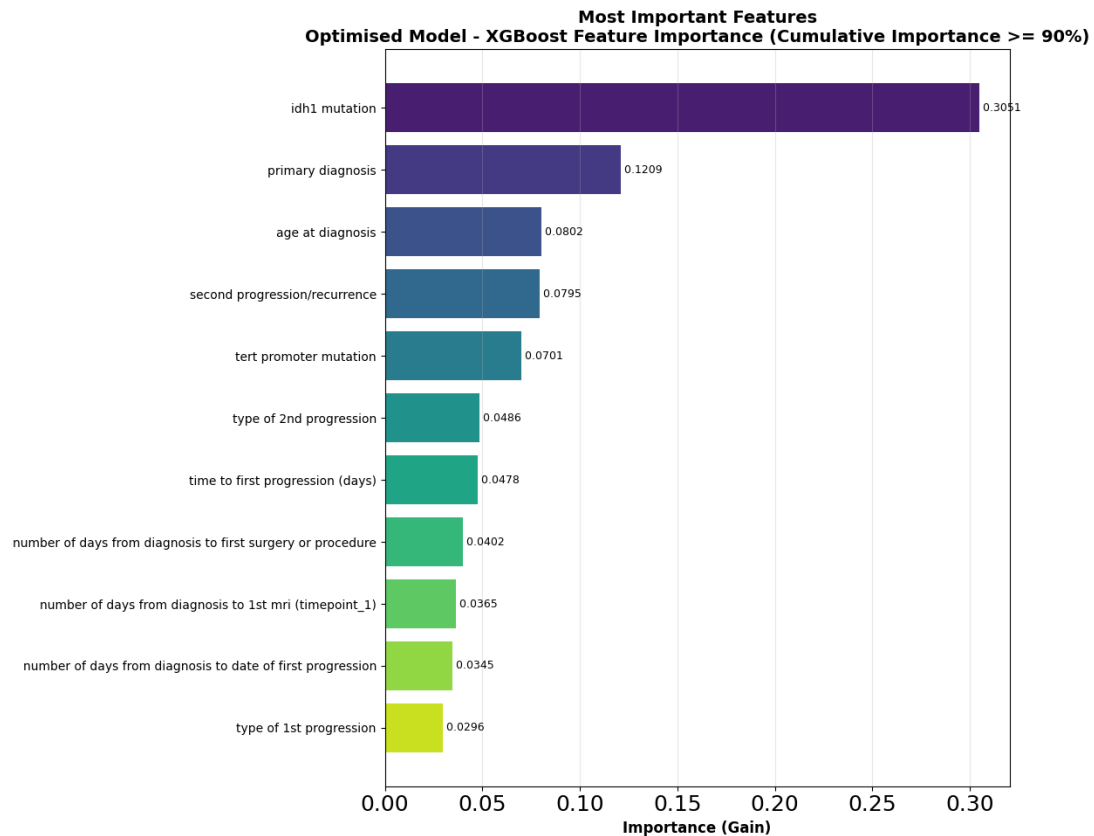
```
1. idh1 mutation 0.305141
2. primary diagnosis 0.120918
3. age at diagnosis 0.080198
4. second progression/recurrence 0.079492
5. tert promoter mutation 0.070086
```

Hyperparameters:

```
n_estimators: 141
max_depth: 4
learning_rate: 0.04862055356117984
subsample: 0.8047628723837982
colsample_bytree: 0.8824126445890016
enable_categorical: True
eval_metric: logloss
gamma: 0.12092614545022584
min_child_weight: 5
reg_alpha: 0.1266202914546536
```

reg_lambda: 0.6780595820974014

Plot saved to drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/tabular
models/figures/feature_importance_optimised.png and drive/MyDrive/Y3
Project/Data/PKG_MU_Glioma_Post/tabular
models/figures/feature_importance_optimised.pdf



Final Model Ready.

Model name: Optimised Model

Trained on: 25 samples

ROC-AUC: 0.7298 $\pm$ 0.0907

Generated outputs:

hyperparameter_search_results.png & .pdf

feature_importance_optimised.png & .pdf

Returned objects:

- model: Trained GliomaBinarySurvivalModel instance
- random_search: RandomizedSearchCV results

1.3 Data Leakage Detection

The baseline model shows signs of overfitting. We check for data leakage by examining feature-target correlations. Features with suspiciously high correlations may represent future information that wouldn't be available at prediction time.

1.3.1 Encode Categorical Features for Correlation Analysis

```
[42]: from sklearn.preprocessing import LabelEncoder

X_encoded = X.copy()
label_encoders = {}

for col in X_encoded.select_dtypes(include=['object', 'category']).columns:
    le = LabelEncoder()
    X_encoded[col] = le.fit_transform(X_encoded[col].astype(str))
    label_encoders[col] = le

print(f"Encoded {len(label_encoders)} categorical columns for correlation_
↪analysis")
```

Encoded 4 categorical columns for correlation analysis

1.3.2 Feature-Target Correlation

```
[43]: feature_target_corr = X_encoded.corrwith(y).sort_values(ascending=False)

print("Feature-Target Correlation (sorted by correlation with death)")
print("-"*71)
print(feature_target_corr.to_string())

# Flag suspicious features (|correlation| > 0.25)
threshold = 0.25
suspicious_features = feature_target_corr[abs(feature_target_corr) > threshold].
↪index.tolist()

print(f"\nSuspicious features (|r| > {threshold}): {len(suspicious_features)}")
for f in suspicious_features:
    print(f" - {f}: {feature_target_corr[f]:.3f}")
```

Feature-Target Correlation (sorted by correlation with death)

```
-----
age at diagnosis                                0.334530
second progression/recurrence                   0.318335
grade of primary brain tumor                   0.283698
type of 2nd progression                         0.249142
primary diagnosis                             0.213954
progression                                    0.167566
number of days from diagnosis to first surgery or procedure 0.165322
```

braf v600e mutation	0.159967
sex at birth	0.142928
tert promoter mutation	0.135791
h3-3a mutation	0.125246
egfr amplification	0.112238
chromosome 7 gain and chromosome 10 loss	0.111429
type of 1st progression	0.106903
race	0.096569
1p/19q	0.080927
number of days from diagnosis to 1st mri (timepoint_1)	0.044373
stereotactic biopsy before surgical resection	-0.026841
atrx mutation	-0.028343
pten mutation	-0.048814
mgmt methylation	-0.069328
previous brain tumor	-0.129378
tp53 alteration	-0.186378
cdkn2a/b deletion	-0.194810
number of days from diagnosis to date of first progression	-0.214948
time to first progression (days)	-0.221845
idh1 mutation	-0.291424
idh2 mutation	-0.306090

Suspicious features ($|r| > 0.25$): 5

- age at diagnosis: 0.335
- second progression/recurrence: 0.318
- grade of primary brain tumor: 0.284
- idh1 mutation: -0.291
- idh2 mutation: -0.306

1.3.3 Feature-Feature Correlation for Suspicious Features

```
[44]: if suspicious_features:
    print("Correlation matrix for suspicious features:")
    suspicious_corr = X_encoded[suspicious_features].corr()
    print(suspicious_corr.to_string())
```

Correlation matrix for suspicious features:

	age at diagnosis	second progression/recurrence	grade of primary brain tumor	idh1 mutation	idh2 mutation
age at diagnosis	1.000000				
0.450825		-0.054454			
-0.516191			-0.515133		
second progression/recurrence		1.000000			
0.137051			-0.054454		
-0.164538				-0.154604	
grade of primary brain tumor			0.450825		
1.000000				-0.621112	
-0.534069					0.137051
idh1 mutation				1.000000	
-0.516191					-0.164538
-0.534069					
1.000000					0.864395
idh2 mutation					1.000000
-0.515133					
-0.154604					

-0.621112 0.864395 1.000000

1.3.4 Visualise Correlations

```
[45]: plt.rcParams.update({'font.size': 20})
fig, axes = plt.subplots(1, 2, figsize=(26, 15), gridspec_kw={'width_ratios': [1.3, 1]})
fig.suptitle('Data Leakage & Correlation Analysis', fontweight='bold',
             fontsize=24, y=1.01)

# Short readable names (shared by both plots)
short_names = {
    'age at diagnosis': 'Age at Diagnosis',
    'second progression/recurrence': 'Second Progression',
    'grade of primary brain tumor': 'Tumour Grade',
    'type of 2nd progression': 'Type of 2nd Prog',
    'primary diagnosis': 'Primary Diagnosis',
    'progression': 'Progression',
    'number of days from diagnosis to first surgery or procedure': 'Days to 1st
    Surgery',
    'braf v600e mutation': 'BRAF V600E',
    'sex at birth': 'Sex',
    'tert promoter mutation': 'TERT',
    'h3-3a mutation': 'H3-3A',
    'egfr amplification': 'EGFR Amp',
    'chromosome 7 gain and chromosome 10 loss': 'Chr7+/10-',
    'type of 1st progression': 'Type of 1st Prog',
    'race': 'Race',
    '1p/19q': '1p/19q',
    'number of days from diagnosis to 1st mri (timepoint_1)': 'Days to 1st MRI',
    'stereotactic biopsy before surgical resection': 'Biopsy Pre-Resection',
    'atrx mutation': 'ATRX',
    'pten mutation': 'PTEN',
    'mgmt methylation': 'MGMT Meth',
    'previous brain tumor': 'Prior Tumour',
    'tp53 alteration': 'TP53',
    'cdkn2a/b deletion': 'CDKN2A/B Del',
    'number of days from diagnosis to date of first progression': 'Days to 1st
    Prog',
    'time to first progression (days)': 'Time to 1st Prog',
    'idh1 mutation': 'IDH1',
    'idh2 mutation': 'IDH2',
}

# --- Plot 1: Feature-Target Correlations
ax1 = axes[0]
short_labels = [short_names.get(f, f) for f in feature_target_corr.index]
```

```

bar_colors = ['#e74c3c' if abs(v) > threshold else '#3498db' for v in
    ↪feature_target_corr.values]

corr_df = pd.DataFrame({
    'Feature': short_labels,
    'Correlation': feature_target_corr.values
})

sns.barplot(data=corr_df, x='Correlation', y='Feature', palette=bar_colors,
            order=short_labels, ax=ax1, legend=False)
ax1.axvline(x=threshold, color='red', linestyle='--', alpha=0.5)
ax1.axvline(x=-threshold, color='red', linestyle='--', alpha=0.5)
ax1.set_xlabel('Correlation with Death', fontsize=20, fontweight='bold')
ax1.set_ylabel('Feature', fontsize=20, fontweight='bold')
ax1.tick_params(labelsize=16)
ax1.xaxis.set_major_locator(plt.MaxNLocator(nbins=10))
ax1.grid(axis='x', alpha=0.3)
ax1.set_title('Feature-Target Correlations\n(Red = Suspicious)', fontsize=21,
    ↪fontweight='semibold')

# --- Plot 2: Heatmap of suspicious features
ax2 = axes[1]
if suspicious_features:
    corr_matrix_plot = X_encoded[suspicious_features].corr()

    heatmap_names = {
        'age at diagnosis': 'Age at\nDiagnosis',
        'second progression/recurrence': 'Second\nProgression',
        'grade of primary brain tumor': 'Tumour\nGrade',
        'idh1 mutation': 'IDH1\nMutation',
        'idh2 mutation': 'IDH2\nMutation',
    }
    renamed = [heatmap_names.get(c, c) for c in corr_matrix_plot.columns]
    corr_matrix_plot.columns = renamed
    corr_matrix_plot.index = renamed

    sns.heatmap(corr_matrix_plot, annot=True, cmap='RdBu_r', center=0, fmt='.
    ↪2f',
                ax=ax2, cbar_kws={'shrink': 0.8}, annot_kws={'size': 18})
    ax2.set_title('Correlation Among Suspicious Features', fontsize=21,
    ↪fontweight='semibold')
    ax2.set_xticklabels(ax2.get_xticklabels(), rotation=45, ha='right',
    ↪fontsize=16)
    ax2.set_yticklabels(ax2.get_yticklabels(), rotation=0, fontsize=16)
else:
    ax2.text(0.5, 0.5, 'No suspicious features found', ha='center',
    ↪va='center', fontweight='bold')

```

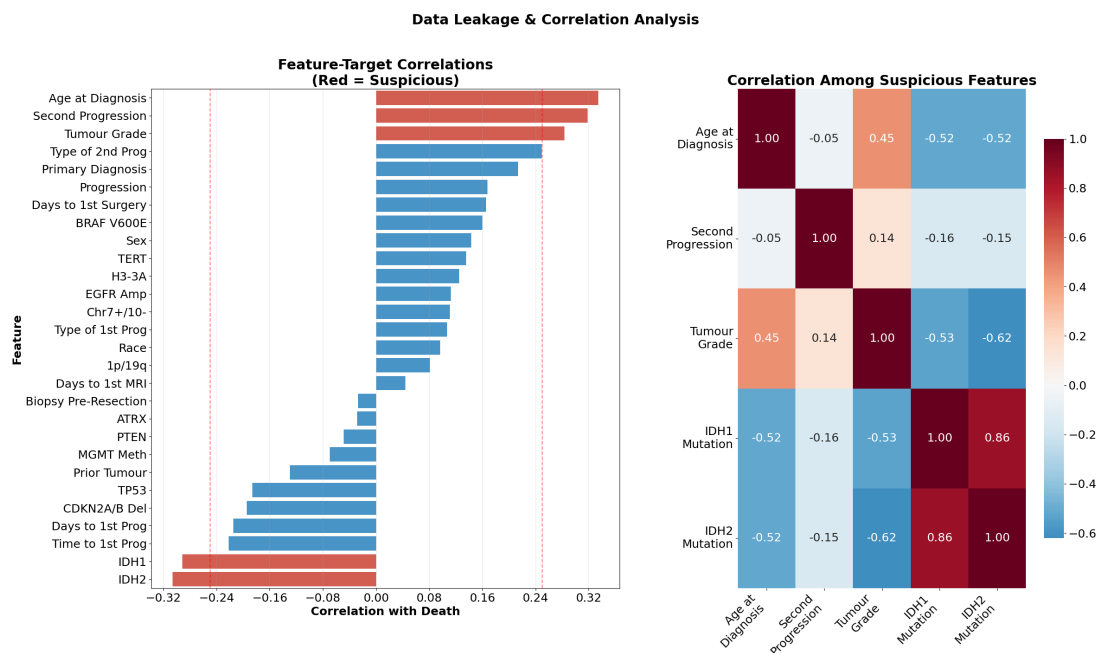
```
plt.tight_layout(w_pad=4)
png_path_corr = os.path.join(FIGURES_DIR, '02_correlation_analysis.png')
pdf_path_corr = os.path.join(FIGURES_DIR, '02_correlation_analysis.pdf')
plt.savefig(png_path_corr, dpi=300, bbox_inches='tight')
plt.savefig(pdf_path_corr, bbox_inches='tight')
print(f"Plot saved to {png_path_corr} and {pdf_path_corr}")
plt.show()
```

/tmp/ipykernel_262/4049747497.py:48: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

sns.barplot(data=corr_df, x='Correlation', y='Feature', palette=bar_colors,

Plot saved to drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/tabular models/figures/02_correlation_analysis.png and drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/tabular models/figures/02_correlation_analysis.pdf



1.3.5 Identify Leakage Clusters

```
[46]: if len(suspicious_features) > 0:
      print("Leakage Cluster Analysis")
      print("-"*60)

      # Find highly correlated pairs
      corr_matrix = X_encoded[suspicious_features].corr()
      high_corr_pairs = []

      for idx, feat1 in enumerate(suspicious_features):
          for feat2 in suspicious_features[idx+1:]:
              corr_val = corr_matrix.loc[feat1, feat2]
              if abs(corr_val) > 0.5:
                  high_corr_pairs.append((feat1, feat2, corr_val))

      if high_corr_pairs:
          print("\nHighly correlated suspicious feature pairs (|r| > 0.5):")
          for f1, f2, corr in sorted(high_corr_pairs, key=lambda x: abs(x[2]),
                                     reverse=True):
              print(f"  {f1} <-> {f2}: {corr:.3f}")
```

Leakage Cluster Analysis

```
-----
Highly correlated suspicious feature pairs (|r| > 0.5):
idh1 mutation <-> idh2 mutation: 0.864
grade of primary brain tumor <-> idh2 mutation: -0.621
grade of primary brain tumor <-> idh1 mutation: -0.534
age at diagnosis <-> idh1 mutation: -0.516
age at diagnosis <-> idh2 mutation: -0.515
```

1.3.6 Analysis

The correlation analysis reveals two types of high-correlation features:

Legitimate (biological): IDH mutations, age, grade, and primary diagnosis cluster together because they reflect known glioma biology.

Leakage (temporal): Progression-related features correlate with death because they measure outcomes that occur *after* the prediction timepoint. These must be removed.

1.4 Train Clean Model Without Leakage Features

The 0.25 correlation threshold we set identifies statistically suspicious features but misses some temporal leakage (e.g., progression has $r=0.15$ but is definitionally future information at time of diagnosis).

As a general rule, leaky features can be identified by answering “Yes” to the following question: “Was this feature known at the time of prediction (diagnosis), or was it measured afterwards?”. In

other words, leakage is determined by temporal logic, therefore the final leakage list is defined by whether the feature was available at diagnosis, not by correlation strength alone.

Our leakage analysis identified 7 features that measure **future outcomes** rather than baseline characteristics. We remove these features from X but keep all patients.

```
[47]: leakage_features = [
    'second progression/recurrence',
    'type of 2nd progression',
    'time to first progression (days)',
    'number of days from diagnosis to date of first progression',
    'number of days from diagnosis to first surgery or procedure',
    'progression',
    'type of 1st progression'
]

X_clean = X.drop(columns=leakage_features, errors='ignore').copy()
```

```
[48]: print(f"Removed {len(leakage_features)} leakage features:")
      for f in leakage_features:
          print(f" - {f}")

      print(f"\nRemaining {X_clean.shape[1]} features:")
      for f in X_clean.columns:
          print(f" - {f}")
```

Removed 7 leakage features:

- second progression/recurrence
- type of 2nd progression
- time to first progression (days)
- number of days from diagnosis to date of first progression
- number of days from diagnosis to first surgery or procedure
- progression
- type of 1st progression

Remaining 21 features:

- sex at birth
- race
- age at diagnosis
- primary diagnosis
- grade of primary brain tumor
- stereotactic biopsy before surgical resection
- idh1 mutation
- idh2 mutation
- 1p/19q
- atrx mutation
- mgmt methylation
- braf v600e mutation
- tert promoter mutation

- chromosome 7 gain and chromosome 10 loss
- h3-3a mutation
- egfr amplification
- pten mutation
- cdkn2a/b deletion
- tp53 alteration
- previous brain tumor
- number of days from diagnosis to 1st mri (timepoint_1)

```
[49]: y_clean = y.copy()

print(f"X_clean shape: {X_clean.shape}")
print(f"y_clean shape: {y_clean.shape}")
print(f"\nTarget distribution:")
print(f"  Survived: {(y_clean == 0).sum()} ({(y_clean == 0).mean()*100:.1f}%)"
print(f"  Died: {(y_clean == 1).sum()} ({(y_clean == 1).mean()*100:.1f}%)"
```

X_clean shape: (203, 21)
y_clean shape: (203,)

Target distribution:
Survived: 106 (52.2%)
Died: 97 (47.8%)

1.4.1 Train Clean Model

```
[ ]: model_clean, search_clean = hyperparameter_search_and_train(
    X_clean, y_clean,
    importance_threshold=0.9,
    n_iter=100,
    model_name="Baseline (no leakage)"
)
```

Initialising Model Optimisation Pipeline via Hyperparameter Search, followed by training a XGBoost Model with the best parameters from cross-val:

Starting Hyperparameter Search

Dataset: 203 patients × 21 features
Target: 106 alive (52.2%), 97 died (47.8%)

Searching 100 random configurations with 5-fold CV...
Optimising for: ROC_AUC
Fitting 5 folds for each of 100 candidates, totalling 500 fits

The clean model trained successfully. Performance metrics and comparisons will be presented after all model variants are trained.

1.5 Option B: Regularised XGBoost

To address overfitting, we constrain model complexity through more aggressive regularisation. The current hyperparameter search allows: - Deep trees (max_depth up to 8) - Many estimators (up to 300) - Relatively low regularisation

For a dataset of 203 samples with 21 features, these settings permit excessive memorisation. We now search over a more conservative parameter space: - Shallower trees (max_depth 2-4) - Fewer estimators (50-100) - Higher regularisation (gamma, reg_alpha, reg_lambda) - More aggressive subsampling

1.5.1 Define Conservative Hyperparameter Space

```
[ ]: param_distributions_conservative = {
    'n_estimators': randint(50, 100),
    'max_depth': randint(2, 5),
    'learning_rate': uniform(0.01, 0.09),
    'subsample': uniform(0.5, 0.3),
    'colsample_bytree': uniform(0.4, 0.4),
    'min_child_weight': randint(5, 15),
    'gamma': uniform(0.5, 2.0),
    'reg_alpha': uniform(0.5, 2.0),
    'reg_lambda': uniform(2.0, 8.0),
    'random_state': [42],
    'enable_categorical': [True],
    'eval_metric': ['logloss']
}

print("Conservative hyperparameter space:")
for param, dist in param_distributions_conservative.items():
    if hasattr(dist, 'args'):
        print(f" {param}: {dist.args}")
    else:
        print(f" {param}: {dist}")
```

1.5.2 Train Regularised Model

```
[ ]: model_regularised, search_regularised = hyperparameter_search_and_train(
    X_clean, y_clean,
    param_distributions=param_distributions_conservative,
    importance_threshold=0.9,
    n_iter=100,
    model_name="Regularised XGBoost"
)
```

The regularised model trained successfully. Comparison with other models will follow after Options C and A are implemented.

1.6 Option C: Logistic Regression

As a simpler baseline, we train a logistic regression model. With $n=203$ samples and 21 features, simpler models often generalise better than complex tree ensembles.

1.6.1 Train Logistic Regression

```
[ ]: from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer # Import SimpleImputer

# Identify numeric and categorical columns
numeric_cols = X_clean.select_dtypes(include=['int64', 'float64']).columns.
    ↪ tolist()
categorical_cols = X_clean.select_dtypes(include=['category', 'object']).
    ↪ columns.tolist()

# Create preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', Pipeline([
            ('imputer', SimpleImputer(strategy='mean')), # Impute missing
            ↪ numerical values
            ('scaler', StandardScaler())
        ]), numeric_cols),
        ('cat', Pipeline([
            ('imputer', SimpleImputer(strategy='most_frequent')), # Impute
            ↪ missing categorical values
            ('onehot', OneHotEncoder(drop='first', sparse_output=False,
            ↪ handle_unknown='ignore'))
        ]), categorical_cols)
    ],
    remainder='passthrough' # Keep other columns if any, though not expected
    ↪ here
)

# Create full pipeline with logistic regression
lr_pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(C=0.1, penalty='l2', max_iter=1000,
    ↪ random_state=42))
])

# Cross-validate
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
scoring = ['accuracy', 'roc_auc', 'f1', 'precision', 'recall']
```

```

lr_cv_results = cross_validate(
    lr_pipeline, X_clean, y_clean,
    cv=cv, scoring=scoring, return_train_score=True, n_jobs=-1
)

print("Logistic Regression Cross-Validation Results")
print("-"*44)
for metric in scoring:
    mean_score = np.mean(lr_cv_results[f'test_{metric}'])
    std_score = np.std(lr_cv_results[f'test_{metric}'])
    print(f"{metric.upper():<12} {mean_score:.3f} \u00b1 {std_score:.3f}")

# Fit on full data for later use
lr_pipeline.fit(X_clean, y_clean)

```

Logistic regression trained successfully.

1.7 Option A: Feature Engineering

We create interaction features based on known glioma biology: - IDH-wild-type GBM: the worst prognosis subtype - Elderly GBM: age > 65 with GBM diagnosis - Molecular risk score: combination of TERT and chromosomal alterations

1.7.1 Create Engineered Features

```

[ ]: X_engineered = X_clean.copy()

# IDH-wild-type GBM (worst prognosis)
X_engineered['idh_wt_gbm'] = (
    (X_engineered['idh1 mutation'] == 0) &
    (X_engineered['primary diagnosis'] == 'GBM')
).astype(int)

# Elderly GBM (age > 65 with GBM)
X_engineered['elderly_gbm'] = (
    (X_engineered['age at diagnosis'] > 65) &
    (X_engineered['primary diagnosis'] == 'GBM')
).astype(int)

# Molecular risk score
X_engineered['molecular_risk'] = (
    X_engineered['tert promoter mutation'].fillna(0) +
    X_engineered['chromosome 7 gain and chromosome 10 loss'].clip(0, 1)
).astype(int)

# High grade indicator

```

```

X_engineered['high_grade'] = (X_engineered['grade of primary brain tumor'] >= 3).astype(int)

print(f"Added 4 engineered features. New shape: {X_engineered.shape}")
print("\nNew features:")
print(f"  idh_wt_gbm: {X_engineered['idh_wt_gbm'].sum()} patients")
print(f"  elderly_gbm: {X_engineered['elderly_gbm'].sum()} patients")
print(f"  molecular_risk distribution: {X_engineered['molecular_risk'].value_counts().to_dict()}")
print(f"  high_grade: {X_engineered['high_grade'].sum()} patients")

```

1.7.2 Train Feature-Engineered Model

```

[ ]: model_engineered, search_engineered = hyperparameter_search_and_train(
      X_engineered, y_clean,
      importance_threshold=0.9,
      n_iter=100,
      model_name="Feature-Engineered XGBoost"
    )

```

Feature-engineered model trained successfully.

1.8 Final Model Comparison

We now compare all five models: 1. **Baseline**: XGBoost with all features (including leakage) 2. **Clean**: XGBoost with leakage features removed 3. **Regularised**: XGBoost with conservative hyperparameters 4. **Logistic Regression**: Simple linear model 5. **Feature-Engineered**: XGBoost with domain-informed features

1.8.1 Performance Comparison Table

```

[ ]: # Collect all results
baseline_results = model.cv_results
clean_results = model_clean.cv_results
regularised_results = model_regularised.cv_results
engineered_results = model_engineered.cv_results

print("Final Model Comparison")
print("-"*135)
print(f"{'Metric':<14} {'Baseline (with leakage)':<25} {'Baseline (no leakage)':<23} {'Regularised XGBoost':<21} {'Logistic Regression':<21} {'Feature-Engineered XGBoost':<28}")
print("-"*135)

for metric in ['roc_auc', 'accuracy', 'f1', 'precision', 'recall']:
    base = f"{np.mean(baseline_results[f'test_{metric}']):.3f} ± {np.std(baseline_results[f'test_{metric}']):.3f}"

```

```

        clean = f"{np.mean(clean_results[f'test_{metric}']):.3f} ± {np.
↳std(clean_results[f'test_{metric}']):.3f}"
        reg = f"{np.mean(regularised_results[f'test_{metric}']):.3f} ± {np.
↳std(regularised_results[f'test_{metric}']):.3f}"
        lr = f"{np.mean(lr_cv_results[f'test_{metric}']):.3f} ± {np.
↳std(lr_cv_results[f'test_{metric}']):.3f}"
        eng = f"{np.mean(engineered_results[f'test_{metric}']):.3f} ± {np.
↳std(engineered_results[f'test_{metric}']):.3f}"

        print(f"{metric.upper():<14} {base:<25} {clean:<23} {reg:<21} {lr:<21} {eng:
↳<28}")

print("-"*135)
print("\nNote: 'Baseline (with leakage)' includes leakage features and should
↳not be used for final reporting.")

```

1.8.2 ROC Curves

```

[ ]: from matplotlib.gridspec import GridSpec

[ ]: # Compute predictions (these are needed by later cells too)
y_pred_baseline = cross_val_predict(model.model, X, y, cv=5,
↳method='predict_proba')[:, 1]
y_pred_clean = cross_val_predict(model_clean.model, X_clean, y_clean, cv=5,
↳method='predict_proba')[:, 1]
y_pred_reg = cross_val_predict(model_regularised.model, X_clean, y_clean, cv=5,
↳method='predict_proba')[:, 1]
y_pred_lr = cross_val_predict(lr_pipeline, X_clean, y_clean, cv=5,
↳method='predict_proba')[:, 1]
y_pred_eng = cross_val_predict(model_engineered.model, X_engineered, y_clean,
↳cv=5, method='predict_proba')[:, 1]

[ ]: # Create plots
plt.rcParams.update({'font.size': 18})
fig = plt.figure(figsize=(24, 14))
gs = GridSpec(2, 6, figure=fig, hspace=0.35, wspace=0.4)

# Top row: 3 plots
ax_top = [fig.add_subplot(gs[0, 0:2]),
          fig.add_subplot(gs[0, 2:4]),
          fig.add_subplot(gs[0, 4:6])]

# Bottom row: 2 plots centred
ax_bot = [fig.add_subplot(gs[1, 1:3]),
          fig.add_subplot(gs[1, 3:5])]
axes = ax_top + ax_bot

```



```

fig.suptitle('ROC Curves: Model Comparison', fontweight='bold', y=0.98)

roc_data = [
    (y, y_pred_baseline, 'Baseline\n(with leakage)', 'blue'),
    (y_clean, y_pred_clean, 'Baseline\n(no leakage)', 'green'),
    (y_clean, y_pred_reg, 'Regularised\nXGBoost', 'orange'),
    (y_clean, y_pred_lr, 'Logistic\nRegression', 'purple'),
    (y_clean, y_pred_eng, 'Feature-Engineered\nXGBoost', 'red'),
]

for ax, (y_true, y_pred, title, color) in zip(axes, roc_data):
    fpr, tpr, _ = roc_curve(y_true, y_pred)
    auc_val = auc(fpr, tpr)
    ax.plot(fpr, tpr, color=color, linewidth=2.5, label=f'AUC = {auc_val:.3f}')
    ax.plot([0, 1], [0, 1], 'k--', alpha=0.5, linewidth=1)
    ax.set_title(title, fontweight='bold')
    ax.set_xlabel('FPR', fontweight='semibold')
    ax.set_ylabel('TPR', fontweight='semibold')
    ax.legend(loc='lower right', framealpha=0.9)
    ax.grid(alpha=0.3)

roc_png_path = os.path.join(FIGURES_DIR, '02_roc_curves_comparison.png')
roc_pdf_path = os.path.join(FIGURES_DIR, '02_roc_curves_comparison.pdf')
plt.savefig(roc_png_path, dpi=300, bbox_inches='tight')
plt.savefig(roc_pdf_path, bbox_inches='tight')
print(f" Plot saved to {roc_png_path} and {roc_pdf_path}")
plt.show()

```

1.8.3 Confusion Matrices

```

[ ]: plt.rcParams.update({'font.size': 18})
fig = plt.figure(figsize=(24, 14))
gs = GridSpec(2, 6, figure=fig, hspace=0.35, wspace=0.4)

# Top row: 3 plots
ax_top = [fig.add_subplot(gs[0, 0:2]),
          fig.add_subplot(gs[0, 2:4]),
          fig.add_subplot(gs[0, 4:6])]

# Bottom row: 2 plots centred
ax_bot = [fig.add_subplot(gs[1, 1:3]),
          fig.add_subplot(gs[1, 3:5])]
axes = ax_top + ax_bot

fig.suptitle('Confusion Matrices: Model Comparison', fontweight='bold', y=0.98)

models_data = [
    (y, y_pred_baseline, 'Baseline\n(with leakage)', 'Blues'),

```

```

(y_clean, y_pred_clean, 'Baseline\n(no leakage)', 'Greens'),
(y_clean, y_pred_reg, 'Regularised\nXGBoost', 'Oranges'),
(y_clean, y_pred_lr, 'Logistic\nRegression', 'Purples'),
(y_clean, y_pred_eng, 'Feature-Engineered\nXGBoost', 'Reds'),
]

for ax, (y_true, y_pred, title, cmap) in zip(axes, models_data):
    y_pred_class = (y_pred > 0.5).astype(int)
    cm = confusion_matrix(y_true, y_pred_class)
    disp = ConfusionMatrixDisplay(cm, display_labels=['Surv', 'Died'])
    disp.plot(ax=ax, cmap=cmap, colorbar=False, values_format='d')
    ax.set_title(title, fontweight='bold')
    ax.set_xlabel('Predicted label', fontweight='semibold')
    ax.set_ylabel('True label', fontweight='semibold')

    # ConfusionMatrixDisplay renders its own text objects;
    # just bold them, rcParams already set the size to 18
    for text_obj in disp.text_.ravel():
        text_obj.set_fontweight('bold')

cm_png_path = os.path.join(FIGURES_DIR, '02_confusion_matrices.png')
cm_pdf_path = os.path.join(FIGURES_DIR, '02_confusion_matrices.pdf')
plt.savefig(cm_png_path, dpi=300, bbox_inches='tight')
plt.savefig(cm_pdf_path, bbox_inches='tight')
print(f" Plot saved to {cm_png_path} and {cm_pdf_path}")
plt.show()

```

1.8.4 Feature Importance Comparison (XGBoost Models)

```

[ ]: plt.rcParams.update({'font.size': 20})

# Rebuild full importance DataFrames
fi_clean = pd.DataFrame({
    'feature': X_clean.columns,
    'importance': model_clean.model.feature_importances_
}).sort_values('importance', ascending=False).head(10)

fi_reg = pd.DataFrame({
    'feature': X_clean.columns,
    'importance': model_regularised.model.feature_importances_
}).sort_values('importance', ascending=False).head(10)

fi_eng = pd.DataFrame({
    'feature': X_engineered.columns,
    'importance': model_engineered.model.feature_importances_
}).sort_values('importance', ascending=False).head(10)

```

```

# Shorten feature names for display
short_names = {
    'primary diagnosis': 'Primary\nDiagnosis',
    'idh1 mutation': 'IDH1',
    'idh2 mutation': 'IDH2',
    'age at diagnosis': 'Age at Dx',
    'tert promoter mutation': 'TERT',
    'chromosome 7 gain and chromosome 10 loss': 'Chr7+/10-',
    'grade of primary brain tumor': 'Tumour Grade',
    'number of days from diagnosis to 1st mri (timepoint_1)': 'Days to 1st MRI',
    'stereotactic biopsy before surgical resection': 'Biopsy Pre-Resection',
    'h3-3a mutation': 'H3-3A',
    'tp53 alteration': 'TP53',
    'atrx mutation': 'ATRX',
    'sex at birth': 'Sex',
    'braf v600e mutation': 'BRAF V600E',
    'egfr amplification': 'EGFR Amp',
    'pten mutation': 'PTEN',
    'cdkn2a/b deletion': 'CDKN2A/B Del',
    'mgmt methylation': 'MGMT Meth',
    'previous brain tumor': 'Prior Tumour',
    '1p/19q': '1p/19q',
    'race': 'Race',
    'molecular_risk': 'Molecular Risk',
    'idh_wt_gbm': 'IDH-WT GBM',
    'elderly_gbm': 'Elderly GBM',
    'high_grade': 'High Grade',
}

# Union of all top-10 features across the three models
all_top_features = sorted(
    set(fi_clean['feature']) | set(fi_reg['feature']) | set(fi_eng['feature'])
)

# Build lookup dicts
clean_dict = dict(zip(fi_clean['feature'], fi_clean['importance']))
reg_dict = dict(zip(fi_reg['feature'], fi_reg['importance']))
eng_dict = dict(zip(fi_eng['feature'], fi_eng['importance']))

# Build combined DataFrame, sort by mean importance descending
combined = pd.DataFrame({
    'feature': all_top_features,
    'Baseline (no leakage)': [clean_dict.get(f, 0) for f in all_top_features],
    'Regularised XGBoost': [reg_dict.get(f, 0) for f in all_top_features],
    'Feature-Engineered XGBoost': [eng_dict.get(f, 0) for f in
    ↪all_top_features],
})

```

```

combined['mean'] = combined[['Baseline (no leakage)', 'Regularised XGBoost',
                             'Feature-Engineered XGBoost']].mean(axis=1)
combined = combined.sort_values('mean', ascending=False)
combined = combined.drop(columns='mean')

# Apply short names
combined['label'] = combined['feature'].map(short_names).
↳ fillna(combined['feature'])

# Melt into long format for seaborn
melted = combined.melt(
    id_vars=['feature', 'label'],
    value_vars=['Baseline (no leakage)', 'Regularised XGBoost',
↳ 'Feature-Engineered XGBoost'],
    var_name='Model',
    value_name='Importance'
)

# Set feature order (top mean importance first, seaborn reads top-to-bottom)
feature_order = combined['label'].tolist()

# Custom palette keyed to model names
palette = {
    'Baseline (no leakage)': 'steelblue',
    'Regularised XGBoost': 'darkorange',
    'Feature-Engineered XGBoost': 'firebrick',
}

fig, ax = plt.subplots(figsize=(15, 13))

sns.barplot(
    data=melted,
    y='label',
    x='Importance',
    hue='Model',
    order=feature_order,
    palette=palette,
    edgecolor='white',
    ax=ax
)

# Add value labels at end of each bar
for container in ax.containers:
    for bar in container:
        width = bar.get_width()
        if width > 0:
            ax.text(width + 0.003, bar.get_y() + bar.get_height() / 1.65,

```

```

        f'{width:.3f}', va='center', fontweight='semibold',
        ↪ fontsize=15)

ax.set_xlabel('Importance (Gain)', fontweight='bold')
ax.set_ylabel('')
ax.set_title('Feature Importance Comparison (Top 10 per Model, sorted by
        ↪ Average Importance)',
            fontweight='bold')
ax.legend(loc='lower right')
ax.grid(axis='x', alpha=0.3)
ax.set_xlim(0, melted['Importance'].max() * 1.2)

plt.tight_layout()
fi_png_path = os.path.join(FIGURES_DIR, '02_feature_importance.png')
fi_pdf_path = os.path.join(FIGURES_DIR, '02_feature_importance.pdf')
plt.savefig(fi_png_path, dpi=300, bbox_inches='tight')
plt.savefig(fi_pdf_path, bbox_inches='tight')
print(f" Plot saved to {fi_png_path} and {fi_pdf_path}")
plt.show()

```

1.8.5 Save All Results

```

[ ]: import json

output_dir = os.path.join(GLIOMA_DIR, 'tabular models', 'results', 'data')
os.makedirs(output_dir, exist_ok=True)

# Save feature importance for each model
fi_clean_full = pd.DataFrame({
    'feature': X_clean.columns,
    'importance': model_clean.model.feature_importances_
}).sort_values('importance', ascending=False)
fi_clean_full.to_csv(os.path.join(output_dir, '02_feature_importance_clean.
        ↪ csv'), index=False)

fi_reg_full = pd.DataFrame({
    'feature': X_clean.columns,
    'importance': model_regularised.model.feature_importances_
}).sort_values('importance', ascending=False)
fi_reg_full.to_csv(os.path.join(output_dir, '02_feature_importance_regularised.
        ↪ csv'), index=False)

fi_eng_full = pd.DataFrame({
    'feature': X_engineered.columns,
    'importance': model_engineered.model.feature_importances_
}).sort_values('importance', ascending=False)

```

```

fi_eng_full.to_csv(os.path.join(output_dir, '02_feature_importance_engineered.
↳csv'), index=False)

print("Saved feature importance CSVs")

# Save comprehensive comparison
results_summary = {}

# Define common metrics extraction
def extract_metrics(cv_results, X_data):
    return {
        'n_features': int(X_data.shape[1]),
        'auc_mean': float(np.mean(cv_results['test_roc_auc'])),
        'auc_std': float(np.std(cv_results['test_roc_auc'])),
        'accuracy_mean': float(np.mean(cv_results['test_accuracy'])),
        'f1_mean': float(np.mean(cv_results['test_f1']))
    }

# Baseline model (with leakage)
results_summary['baseline'] = extract_metrics(baseline_results, X)
results_summary['baseline']['note'] = 'Includes leakage features - for_
↳reference only'

# Baseline model (no leakage)
results_summary['clean'] = extract_metrics(clean_results, X_clean)
results_summary['clean']['features_removed'] = leakage_features

# Regularised model
results_summary['regularised'] = extract_metrics(regularised_results, X_clean)
results_summary['regularised']['note'] = 'Conservative hyperparameters to_
↳reduce overfitting'

# Logistic Regression
results_summary['logistic_regression'] = extract_metrics(lr_cv_results, X_clean)
results_summary['logistic_regression']['note'] = 'Simple linear baseline'

# Feature-Engineered model
results_summary['feature_engineered'] = extract_metrics(engineered_results,
↳X_engineered)
results_summary['feature_engineered']['engineered_features'] = ['idh_wt_gbm',
↳'elderly_gbm', 'molecular_risk', 'high_grade']

with open(os.path.join(output_dir, '02_model_comparison.json'), 'w') as f:
    json.dump(results_summary, f, indent=2)

print("Saved 02_model_comparison.json")

```

1.9 Summary

Five models were trained and compared:

Model	Description	Features
Baseline	XGBoost with all features	28 (includes leakage)
Clean	XGBoost with leakage removed	21
Regularised	XGBoost with conservative hyperparameters	21
Logistic Regression	L2-regularised linear model	21
Feature-Engineered	XGBoost with domain-informed features	25

The baseline model should not be used for reporting as it contains leakage. Among the valid models, performance comparison and interpretation follows in the report.

E.3 Notebook 03: Survival Analysis

03_survival_analysis

1 03: Survival Analysis

This notebook implements **proper survival analysis** using time-to-event modelling, as opposed to the binary classification approach in notebook 02.

Why survival analysis? - Binary classification treats a patient who dies at 3 months identically to one who dies at 4 years - Survival analysis uses the actual time-to-event information - Properly handles **censored** observations (patients still alive at last follow-up) - Reports **C-index** (concordance index) rather than AUC

Models implemented: 1. Cox Proportional Hazards (baseline linear model) 2. Random Survival Forest (ensemble method) 3. Gradient Boosted Survival (tree-based boosting)

Expected outcome: C-index in the range 0.65-0.72, comparable to binary classification AUC but using more information from the data.

1.1 Setup

1.1.1 Install Dependencies

```
[37]: !pip install scikit-survival -q
```

1.1.2 Core Imports

```
[38]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import json
import warnings

warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8-whitegrid')
```

1.1.3 Survival Analysis Imports

```
[39]: from sksurv.ensemble import RandomSurvivalForest, \
      ↪ GradientBoostingSurvivalAnalysis
      from sksurv.linear_model import CoxPHSurvivalAnalysis
      from sksurv.metrics import concordance_index_censored
      from sksurv.nonparametric import kaplan_meier_estimator

      from sklearn.model_selection import StratifiedKFold
      from sklearn.preprocessing import StandardScaler
      from sklearn.impute import SimpleImputer
      from sklearn.base import clone
```

1.1.4 Mount Drive & Set Paths

```
[40]: from google.colab import drive
      drive.mount('/content/drive/', force_remount=True)

      from google.colab import userdata
      GLIOMA_DIR = userdata.get("GLIOMA_DIR")
      print(f"Glioma directory: {os.path.join(GLIOMA_DIR)}")
```

Mounted at /content/drive/

Glioma directory: drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post

```
[41]: FIGURES_DIR = os.path.join(GLIOMA_DIR, 'survival analysis', 'figures')
      os.makedirs(FIGURES_DIR, exist_ok=True)
      print(f"Figures directory: {FIGURES_DIR}")
```

Figures directory: drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/survival analysis/figures

1.2 Load Data

1.2.1 Load Preprocessed Features

```
[42]: EDA_DIR = os.path.join(GLIOMA_DIR, 'eda')
      os.makedirs(EDA_DIR, exist_ok=True)
      preprocessed_dir = os.path.join(EDA_DIR, 'preprocessed')

      X = pd.read_pickle(os.path.join(preprocessed_dir, 'X.pkl'))
      y = pd.read_pickle(os.path.join(preprocessed_dir, 'y.pkl'))
      high_fill_df = pd.read_pickle(os.path.join(preprocessed_dir, 'high_fill_df.
      ↪.pkl'))

      print(f"Loaded X: {X.shape}")
      print(f"Loaded y: {y.shape}")
      print(f"Loaded high_fill_df: {high_fill_df.shape}")
```

```
Loaded X: (203, 28)
Loaded y: (203,)
Loaded high_fill_df: (203, 29)
```

1.2.2 Load Original Data for Survival Time

The preprocessed `high_fill_df` does not contain the survival time column because: - Number of days from Diagnosis to death (Days) was dropped during preprocessing (47% fill rate) - This column only has values for patients who died; censored patients have NaN

We need to: 1. Retrieve survival time for patients who died from original data 2. Estimate follow-up time for censored patients using their last MRI date

```
[43]: # Load original Excel file
original_data_path = os.path.join(GLIOMA_DIR,
    ↪ 'MU_Glioma_Post_ClinicalData_July2025.xlsx')
df_original = pd.read_excel(original_data_path, sheet_name=1)

# Standardise column names to lowercase
df_original.columns = df_original.columns.str.lower().str.strip()

print(f"Original data shape: {df_original.shape}")
print(f"\nColumns containing 'days' or 'death':")
for col in df_original.columns:
    if 'days' in col or 'death' in col:
        print(f" - {col}")
```

Original data shape: (203, 74)

Columns containing 'days' or 'death':

- time to first progression (days)
- overall survival (death)
- number of days from diagnosis to death (days)
- number of days from diagnosis to first surgery or procedure
- number of days from diagnosis to initial chemo therapy start date
- number of days from diagnosis to initial chemo therapy end date
- number of days from diagnosis to radiation therapy start date
- number of days from diagnosis to radiation therapy end date
- number of days from diagnosis to date of first progression
- number of days from diagnosis to date of further progression
- days from diagnosis to new treatment
- cycle length of additional therapy (q days)
- number of days from diagnosis to starting additional therapy
- number of days from diagnosis to complete additional therapy
- cycle length of 2nd_additional therapy (q days)
- number of days from diagnosis to starting 2nd_additional therapy
- number of days from diagnosis to complete 2nd_additional therapy
- cycle length of immunotherapy (q days)
- number of days from diagnosis to start immunotherapy

- number of days from diagnosis to complete immunotherapy
- number of days from diagnosis to the day of insertion of brachytherapy
- number of days from diagnosis to start other additional therapy
- number of days from diagnosis to complete other additional therapy
- number of days from diagnosis to 1st mri (timepoint_1)
- number of days from diagnosis to 2nd mri (timepoint_2)
- number of days from diagnosis to 3rd mri (timepoint_3)
- number of days from diagnosis to 4th mri (timepoint_4)
- number of days from diagnosis to 5th mri (timepoint_5)
- number of days from diagnosis to 6th mri (timepoint_6)

1.2.3 Construct Survival Time Variable

For survival analysis we need: - **Event indicator**: Did the patient die? (0=censored/alive, 1=died)
 - **Time to event**: Days from diagnosis to death (if died) OR last follow-up (if censored)

For censored patients, we use the **latest MRI timepoint** as a proxy for last known follow-up.

```
[44]: # Extract death indicator and days-to-death
death_col = 'overall survival (death)'
days_to_death_col = 'number of days from diagnosis to death (days)'

# MRI timepoint columns for estimating follow-up time
mri_cols = [
    'number of days from diagnosis to 1st mri (timepoint_1)',
    'number of days from diagnosis to 2nd mri (timepoint_2)',
    'number of days from diagnosis to 3rd mri (timepoint_3)',
    'number of days from diagnosis to 4th mri (timepoint_4)',
    'number of days from diagnosis to 5th mri (timepoint_5)',
    'number of days from diagnosis to 6th mri (timepoint_6)'
]

# Filter to only MRI columns that exist
mri_cols = [col for col in mri_cols if col in df_original.columns]
print(f"Available MRI timepoint columns: {len(mri_cols)}")
```

Available MRI timepoint columns: 6

```
[45]: # Get event indicator
event = df_original[death_col].values.astype(bool)

"""
Get survival time
For patients who died: use days-to-death
For censored patients: use maximum MRI timepoint as proxy for last follow-up
"""

survival_time = np.zeros(len(df_original))
```

```

for i in range(len(df_original)):
    if event[i]: # Patient died
        survival_time[i] = df_original.loc[i, days_to_death_col]
    else: # Censored (alive)
        # Take max of available MRI timepoints as a proxy for survival time
        mri_times = df_original.loc[i, mri_cols].values
        valid_times = mri_times[~pd.isna(mri_times)]
        if len(valid_times) > 0:
            survival_time[i] = np.max(valid_times)
        else:
            # Default follow-up to NaN (handled later)
            survival_time[i] = np.nan

print(f"Survival times computed: {len(survival_time)}")
print(f"Missing survival times: {np.isnan(survival_time).sum()}")
print(survival_time)

```

Survival times computed: 203

Missing survival times: 2

```

[ 286.   67.  367.  365.  968.  482.   97.  369.  185.  244.  396.  611.
   42.  482.  379.  401.  456.  156.  243.  215.  375.  145.   44.  913.
  553.  513.  398.  618.  357.  422.  495.  371.  482.  473.   16.  182.
   64.   63.   34.  249.   25.  103.   76.  300.  225.  651.  263.  648.
  594.    9. 1326. 1332.   76.   62.   76.  148.  160.  204.  579.  203.
  189.  470.  420.    0. 1005.  463.   84.   89.    0.  645.  281.  213.
   24.  207.  591.  656.  335.    0.  311.   36.  977.  690.   48.   88.
  265.  736.  551.  400.   41.    0.  554.  499.  124.   37.  242.  551.
   11.  302.  424.   86.    3.   nan  402.  280.    5.  736.  399.  236.
  137.    3.    4.  424.  -11.  565.   90.  479.   16.  674.   10.  128.
   31.    6.  192.  189.   20.  316.  543.   63.  248.    8.  220.  334.
  227.  382.  231.  254.  404.  202.    5.    7.  172.  178.  153.   46.
   97.  508.  301.   45. 1219.  635.  981.  303.  436.  147.  237.  146.
    7.  443.  130.    4.  340.  288.    4.  500.  482.  318.  650.   31.
   69.  218.  423.  292.  149.  283.  694.  498.    9.  379.  640.  326.
1210.  -17.  234.  225.  189.  463.  554.   25.  260.   nan  375.  176.
  260.  141.  526.  152.  139.  426.  381.  376.   87.  232.  319.]

```

1.2.4 Handle Missing Survival Times

```

[46]: missing_mask = np.isnan(survival_time)

if missing_mask.sum() > 0:
    print(f"Patients with missing survival time: {missing_mask.sum()}")
    print(f"   - Died: {event[missing_mask].sum()}")
    print(f"   - Censored: {(~event[missing_mask]).sum()}")

    # Calculate group-specific medians for imputation

```

```

censored_valid = (~event) & (~missing_mask)
died_valid = event & (~missing_mask)

median_censored = np.median(survival_time[censored_valid]) if
↪censored_valid.sum() > 0 else 30
median_died = np.median(survival_time[died_valid]) if died_valid.sum() > 0
↪else 30

# Impute censored patients
censored_missing = (~event) & missing_mask
if censored_missing.sum() > 0:
    survival_time[censored_missing] = median_censored
    print(f"\nImputed {censored_missing.sum()} censored patients with
↪median follow-up: {median_censored:.0f} days")

# Impute died patients
died_missing = event & missing_mask
if died_missing.sum() > 0:
    survival_time[died_missing] = median_died
    print(f"Imputed {died_missing.sum()} died patients with median death
↪time: {median_died:.0f} days")
else:
    print("No missing survival times.")

```

Patients with missing survival time: 2

- Died: 1
- Censored: 1

Imputed 1 censored patients with median follow-up: 215 days

Imputed 1 died patients with median death time: 336 days

1.2.5 Validate Survival Data

```

[47]: # Basic validation
print("Survival Data Summary")
print("-"*30)
print(f"Total patients: {len(survival_time)}")
print(f"Events (deaths): {event.sum()} ({event.mean()*100:.1f}%")
print(f"Censored (alive): {(~event).sum()} ({(~event).mean()*100:.1f}%")
print(f"\nSurvival time (days):")
print(f"  Min: {survival_time.min():.0f}")
print(f"  Max: {survival_time.max():.0f}")
print(f"  Median: {np.median(survival_time):.0f}")
print(f"  Mean: {survival_time.mean():.0f}")

# Check for negative times (can happen with date discrepancies as diagnosis can
↪come after operation)

```

```

negative_times = survival_time < 0
if negative_times.sum() > 0:
    print(f"WARNING: {negative_times.sum()} patients have negative survival_
↳time")
    # These patients have no post-diagnosis follow-up recorded
    # Use median follow-up of other censored patients as proxy
    censored_positive = (~event) & (survival_time > 0)
    median_followup = np.median(survival_time[censored_positive])
    survival_time[negative_times] = median_followup
    print(f"Imputed with median censored follow-up: {median_followup:.0f} days")

```

Survival Data Summary

```

-----
Total patients: 203
Events (deaths): 97 (47.8%)
Censored (alive): 106 (52.2%)

```

Survival time (days):

```

    Min: -17
    Max: 1332
    Median: 249
    Mean: 304
WARNING: 2 patients have negative survival time
Imputed with median censored follow-up: 226 days

```

1.2.6 Create Structured Survival Target

```

[48]: # scikit-survival requires a structured array with dtype [('event', bool),
↳('time', float)]
y_surv = np.array(
    [(evt, t) for evt, t in zip(event, survival_time)],
    dtype=[('event', bool), ('time', float)]
)

print(f"Structured survival target created: {len(y_surv)} samples")
print(f"dtype: {y_surv.dtype}")

```

```

Structured survival target created: 203 samples
dtype: [('event', '?'), ('time', '<f8')]

```

1.2.7 Visualise Survival Time Distribution

```

[49]: fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Histogram by event status
axes[0].hist(survival_time[event], bins=30, alpha=0.7, label='Died',
↳color='indianred')

```

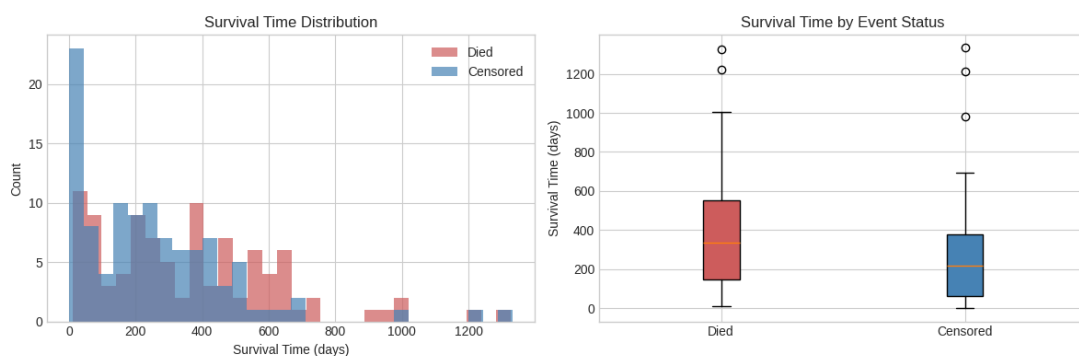
```

axes[0].hist(survival_time[~event], bins=30, alpha=0.7, label='Censored',
             color='steelblue')
axes[0].set_xlabel('Survival Time (days)', fontsize=14)
axes[0].set_ylabel('Count', fontsize=14)
axes[0].set_title('Survival Time Distribution', fontsize=16, fontweight='bold')
axes[0].legend(fontsize=12)
axes[0].tick_params(labelsize=12)

# Box plot by event status
bp = axes[1].boxplot([survival_time[event], survival_time[~event]],
                    labels=['Died', 'Censored'], patch_artist=True)
bp['boxes'][0].set_facecolor('indianred')
bp['boxes'][1].set_facecolor('steelblue')
axes[1].set_ylabel('Survival Time (days)', fontsize=14)
axes[1].set_title('Survival Time by Event Status', fontsize=16,
                 fontweight='bold')
axes[1].tick_params(labelsize=12)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '03_survival_time_distribution.pdf'),
           bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '03_survival_time_distribution.png'),
           dpi=300, bbox_inches='tight')
plt.show()

```



1.3 Prepare Features

1.3.1 Remove Leakage Features

Same features removed in notebook 02 must be removed here.

```

[50]: leakage_features = [
        'second progression/recurrence',
        'type of 2nd progression',

```



```

    'time to first progression (days)',
    'number of days from diagnosis to date of first progression',
    'number of days from diagnosis to first surgery or procedure',
    'progression',
    'type of 1st progression'
]

features_to_remove = [f for f in leakage_features if f in X.columns]
X_clean = X.drop(columns=features_to_remove, errors='ignore')

print(f"Original features: {X.shape[1]}")
print(f"Features removed: {len(features_to_remove)}")
print(f"Clean features: {X_clean.shape[1]}")

```

Original features: 28
Features removed: 7
Clean features: 21

1.3.2 Identify Feature Types

```

[51]: categorical_cols = X_clean.select_dtypes(include=['object', 'category']).
      ↪columns.tolist()
      numeric_cols = X_clean.select_dtypes(include=[np.number]).columns.tolist()

print(f"Categorical features ({len(categorical_cols)}):")
for col in categorical_cols:
    print(f" - {col}")

print(f"\nNumeric features ({len(numeric_cols)}):")
for col in numeric_cols:
    print(f" - {col}")

```

Categorical features (4):

- sex at birth
- race
- primary diagnosis
- previous brain tumor

Numeric features (17):

- age at diagnosis
- grade of primary brain tumor
- stereotactic biopsy before surgical resection
- idh1 mutation
- idh2 mutation
- 1p/19q
- atrx mutation
- mgmt methylation
- braf v600e mutation

- tert promoter mutation
- chromosome 7 gain and chromosome 10 loss
- h3-3a mutation
- egfr amplification
- pten mutation
- cdkn2a/b deletion
- tp53 alteration
- number of days from diagnosis to 1st mri (timepoint_1)

1.3.3 Encode Categorical Features

```
[52]: X_encoded = pd.get_dummies(X_clean, columns=categorical_cols, drop_first=True)
print(f"Features after one-hot encoding: {X_encoded.shape[1]}")
```

Features after one-hot encoding: 27

1.3.4 Handle Missing Values

```
[53]: missing_counts = X_encoded.isnull().sum()
cols_with_missing = missing_counts[missing_counts > 0]

if len(cols_with_missing) > 0:
    print(f"Columns with missing values ({len(cols_with_missing)}): ␣
    ↪{cols_with_missing}\n")

    imputer = SimpleImputer(strategy='median')
    X_encoded = pd.DataFrame(
        imputer.fit_transform(X_encoded),
        columns=X_encoded.columns,
        index=X_encoded.index
    )
    print(f"\nMissing values imputed with median.")
else:
    print("No missing values.")
```

Columns with missing values (1): number of days from diagnosis to 1st mri
(timepoint_1) 16
dtype: int64

Missing values imputed with median.

1.3.5 Remove Zero-Variance Features

```
[54]: variances = X_encoded.var()
zero_var_cols = variances[variances == 0].index.tolist()

if zero_var_cols:
```

```

print(f"Removing {len(zero_var_cols)} zero-variance features:")
for col in zero_var_cols:
    print(f" - {col}")
X_encoded = X_encoded.drop(columns=zero_var_cols)
else:
    print("No zero-variance features.")

print(f"\nFinal feature matrix: {X_encoded.shape}")

```

No zero-variance features.

Final feature matrix: (203, 27)

1.4 Evaluation Framework

1.4.1 Concordance Index

The C-index measures how well the model ranks patients by risk. If we pick two patients at random where one died before the other, did the model correctly rank the one who died first as higher risk?

- C-index = 0.5 is random guessing
- C-index = 1.0 is perfect ranking

```

[55]: def calculate_cindex(model, X, y_surv):
        """Calculate concordance index for a fitted survival model."""
        risk_scores = model.predict(X)
        c_index = concordance_index_censored(
            y_surv['event'],
            y_surv['time'],
            risk_scores
        )[0]
        return c_index

```

1.4.2 Cross-Validation Function

```

[56]: def cross_validate_survival(model, X, y_surv, n_splits=5, random_state=42):
        """
        Perform stratified k-fold cross-validation for survival models.
        Stratification is by event status to ensure balanced folds.
        """
        skf = StratifiedKFold(n_splits=n_splits, shuffle=True,
                               random_state=random_state)
        event_labels = y_surv['event'].astype(int)

        c_indices = []

        for fold, (train_idx, val_idx) in enumerate(skf.split(X, event_labels)):
            X_train, X_val = X.iloc[train_idx], X.iloc[val_idx]
            y_train, y_val = y_surv[train_idx], y_surv[val_idx]

```

```

model_clone = clone(model)
model_clone.fit(X_train, y_train)

c_index = calculate_cindex(model_clone, X_val, y_val)
c_indices.append(c_index)

print(f" Fold {fold+1}: C-index = {c_index:.4f}")

return {
    'c_index_mean': np.mean(c_indices),
    'c_index_std': np.std(c_indices),
    'c_index_scores': c_indices
}

```

1.5 Model 1: Cox Proportional Hazards

The Cox PH model is the standard baseline for survival analysis. It assumes the hazard ratio between any two patients is constant over time (proportional hazards assumption).

1.5.1 Scale Features for Cox PH

```

[57]: scaler = StandardScaler()
X_scaled = pd.DataFrame(
    scaler.fit_transform(X_encoded),
    columns=X_encoded.columns,
    index=X_encoded.index
)
print(f"Features scaled: {X_scaled.shape}")

```

Features scaled: (203, 27)

1.5.2 Train Cox PH

```

[58]: print("Training Cox Proportional Hazards model...")
print("-"*42)

cox_model = CoxPHSurvivalAnalysis(alpha=0.1)
cox_results = cross_validate_survival(cox_model, X_scaled, y_surv, n_splits=5)

print("-"*42)
print(f"Cox PH C-index: {cox_results['c_index_mean']:.4f} ±
↳ {cox_results['c_index_std']:.4f}")

```

Training Cox Proportional Hazards model...

```

-----
Fold 1: C-index = 0.6081
Fold 2: C-index = 0.7264
Fold 3: C-index = 0.7774

```

Fold 4: C-index = 0.7259
Fold 5: C-index = 0.4964

Cox PH C-index: 0.6669 ± 0.1018

1.5.3 Cox PH Coefficients

```
[59]: cox_full = CoxPHSurvivalAnalysis(alpha=0.1)
cox_full.fit(X_scaled, y_surv)

cox_coefs = pd.DataFrame({
    'feature': X_scaled.columns,
    'coefficient': cox_full.coef_
})
cox_coefs['abs_coefficient'] = np.abs(cox_coefs['coefficient'])
cox_coefs = cox_coefs.sort_values('abs_coefficient', ascending=False)

print("Top 10 Cox PH coefficients (by absolute value):")
print(cox_coefs.head(10).to_string(index=False))
```

Top 10 Cox PH coefficients (by absolute value):

	feature	coefficient	abs_coefficient
	primary diagnosis_Diffuse glioma	-1.263765	1.263765
	grade of primary brain tumor	0.882557	0.882557
	primary diagnosis_GBM	-0.811122	0.811122
	age at diagnosis	0.672761	0.672761
	primary diagnosis_Oligodendro-glioma	-0.662418	0.662418
	race_White	0.475550	0.475550
	idh2 mutation	-0.426992	0.426992
stereotactic biopsy before surgical resection		0.417429	0.417429
	race_Black or African American	0.404896	0.404896
	idh1 mutation	0.377883	0.377883

1.5.4 Visualise Cox PH Coefficients

```
[60]: top_n = 15
top_coefs = cox_coefs.head(top_n).copy()

fig, ax = plt.subplots(figsize=(11, 7))

colors = ['indianred' if c > 0 else 'steelblue' for c in_
    top_coefs['coefficient']]
ax.barh(range(len(top_coefs)), top_coefs['coefficient'], color=colors)

ax.set_yticks(range(len(top_coefs)))
ax.set_yticklabels(top_coefs['feature'], fontsize=13)
ax.invert_yaxis()
ax.set_xlabel('Cox PH Coefficient', fontsize=15)
```

```

ax.set_title('Cox Proportional Hazards: Feature Coefficients\n(Red = increases risk, Blue = decreases risk)',
             fontsize=16, fontweight='bold')
ax.axvline(x=0, color='black', linestyle='-', linewidth=0.5)
ax.tick_params(axis='x', labels=13)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '03_cox_coefficients.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '03_cox_coefficients.png'), dpi=300,
            bbox_inches='tight')
plt.show()

```



1.6 Model 2: Random Survival Forest

Random Survival Forest extends random forests to survival data. Unlike Cox PH, it makes no proportional hazards assumption and can capture non-linear relationships.

1.6.1 Train RSF

```

[61]: print("Training Random Survival Forest...")
      print("-"*34)

rsf_model = RandomSurvivalForest(
    n_estimators=100,

```

```

        max_depth=4,
        min_samples_split=10,
        min_samples_leaf=5,
        n_jobs=-1,
        random_state=42
    )

    rsf_results = cross_validate_survival(rsf_model, X_encoded, y_surv, n_splits=5)

    print("-"*34)
    print(f"RSF C-index: {rsf_results['c_index_mean']:.4f} ± {rsf_results['c_index_std']:.4f}")

```

Training Random Survival Forest...

```

-----
Fold 1: C-index = 0.7459
Fold 2: C-index = 0.6195
Fold 3: C-index = 0.7935
Fold 4: C-index = 0.7434
Fold 5: C-index = 0.6295
-----

```

RSF C-index: 0.7064 ± 0.0693

1.6.2 RSF Feature Importance

```

[62]: from sklearn.inspection import permutation_importance

rsf_full = RandomSurvivalForest(
    n_estimators=100,
    max_depth=4,
    min_samples_split=10,
    min_samples_leaf=5,
    n_jobs=-1,
    random_state=42
)
rsf_full.fit(X_encoded, y_surv)

# Define a scorer function for permutation_importance using the C-index
def cindex_scorer(estimator, X, y):
    risk_scores = estimator.predict(X)
    c_index = concordance_index_censored(
        y['event'],
        y['time'],
        risk_scores
    )[0]
    return c_index

```

```

# Calculate permutation importance
result = permutation_importance(
    rsf_full,
    X_encoded,
    y_surv,
    scoring=cindex_scorer,
    n_repeats=10, # Number of times to permute a feature
    random_state=42,
    n_jobs=-1
)

rsf_importance = pd.DataFrame({
    'feature': X_encoded.columns,
    'importance': result.importances_mean
}).sort_values('importance', ascending=False)

print("Top 10 RSF feature importances (Permutation Importance):")
print(rsf_importance.head(10).to_string(index=False))

```

Top 10 RSF feature importances (Permutation Importance):

	feature	importance
	age at diagnosis	0.092719
number of days from diagnosis to 1st mri (timepoint_1)		0.068289
	1p/19q	0.004878
	egfr amplification	0.004460
	mgmt methylation	0.004222
	grade of primary brain tumor	0.004198
	tp53 alteration	0.003047
	sex at birth_Male	0.002546
	tert promoter mutation	0.002540
	h3-3a mutation	0.002343

1.6.3 Visualise RSF Feature Importance

```

[63]: top_n = 15
top_importance = rsf_importance.head(top_n)

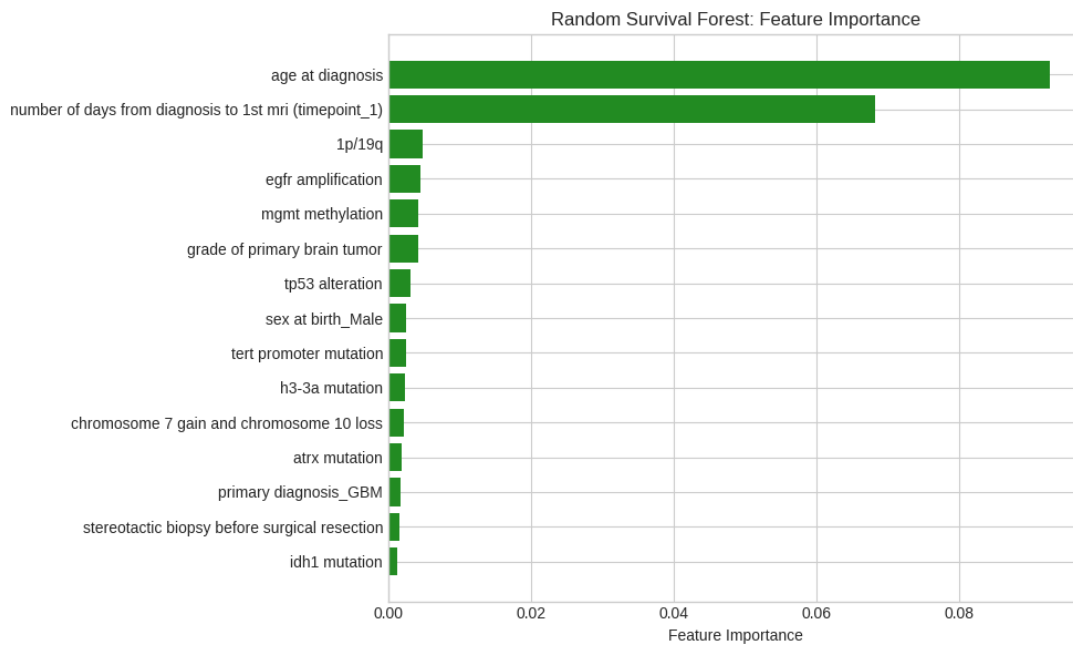
fig, ax = plt.subplots(figsize=(10, 6))

ax.barh(range(len(top_importance)), top_importance['importance'],
        color='forestgreen')
ax.set_yticks(range(len(top_importance)))
ax.set_yticklabels(top_importance['feature'])
ax.invert_yaxis()
ax.set_xlabel('Feature Importance')
ax.set_title('Random Survival Forest: Feature Importance')

```



```
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '03_rsf_feature_importance.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '03_rsf_feature_importance.png'),
            dpi=300, bbox_inches='tight')
plt.show()
```



1.7 Model 3: Gradient Boosted Survival Analysis

Gradient boosting applied to survival analysis. Similar to XGBoost but designed for time-to-event data.

1.7.1 Train GBSA

```
[64]: print("Training Gradient Boosted Survival Analysis...")
      print("-"*46)

      gbsa_model = GradientBoostingSurvivalAnalysis(
          n_estimators=100,
          learning_rate=0.05,
          max_depth=3,
          min_samples_split=10,
          min_samples_leaf=5,
          subsample=0.8,
          random_state=42
```

```
)

gbsa_results = cross_validate_survival(gbsa_model, X_encoded, y_surv,
↪n_splits=5)

print("-"*46)
print(f"GBSA C-index: {gbsa_results['c_index_mean']:.4f} ±
↪{gbsa_results['c_index_std']:.4f}")
```

Training Gradient Boosted Survival Analysis...

```
-----
Fold 1: C-index = 0.7378
Fold 2: C-index = 0.5912
Fold 3: C-index = 0.8032
Fold 4: C-index = 0.7843
Fold 5: C-index = 0.6079
-----
```

GBSA C-index: 0.7049 ± 0.0888

1.7.2 GBSA Feature Importance

```
[65]: gbsa_full = GradientBoostingSurvivalAnalysis(
        n_estimators=100,
        learning_rate=0.05,
        max_depth=3,
        min_samples_split=10,
        min_samples_leaf=5,
        subsample=0.8,
        random_state=42
    )
gbsa_full.fit(X_encoded, y_surv)

gbsa_importance = pd.DataFrame({
    'feature': X_encoded.columns,
    'importance': gbsa_full.feature_importances_
}).sort_values('importance', ascending=False)

print("Top 10 GBSA feature importances:")
print(gbsa_importance.head(10).to_string(index=False))
```

Top 10 GBSA feature importances:

	feature	importance
	age at diagnosis	0.394729
number of days from diagnosis to 1st mri (timepoint_1)		0.364777
	previous brain tumor_Yes	0.039723
	grade of primary brain tumor	0.039583
	mgmt methylation	0.027774
	chromosome 7 gain and chromosome 10 loss	0.022955

	1p/19q	0.019768
primary diagnosis_GBM		0.019768
	idh1 mutation	0.013048
	egfr amplification	0.011619

1.8 Model Comparison

1.8.1 Compare All Survival Models

```
[66]: comparison = pd.DataFrame({
    'Model': ['Cox PH', 'Random Survival Forest', 'Gradient Boosted Survival'],
    'C-index Mean': [
        cox_results['c_index_mean'],
        rsf_results['c_index_mean'],
        gbsa_results['c_index_mean']
    ],
    'C-index Std': [
        cox_results['c_index_std'],
        rsf_results['c_index_std'],
        gbsa_results['c_index_std']
    ]
})

comparison['C-index'] = comparison.apply(
    lambda row: f"{row['C-index Mean']:.3f} ± {row['C-index Std']:.3f}",
    axis=1
)

print("\nSurvival Model Comparison")
print("-"*39)
print(comparison[['Model', 'C-index']].to_string(index=False))
print("-"*39)
```

Survival Model Comparison

```
-----
              Model      C-index
Cox PH 0.667 ± 0.102
Random Survival Forest 0.706 ± 0.069
Gradient Boosted Survival 0.705 ± 0.089
-----
```

1.8.2 Compare with Binary Classification Results

```
[67]: RESULTS_DIR = os.path.join(GLIOMA_DIR, 'results')
binary_results_path = os.path.join(REULTS_DIR, 'model_comparison.json')

if os.path.exists(binary_results_path):
```

```

with open(binary_results_path, 'r') as f:
    binary_results = json.load(f)

print("\nComparison: Binary Classification vs Survival Analysis")
print("-"*67)
print(f"{'Approach':<35} {'Metric':<12} {'Value':<20}")
print("-"*67)

if 'clean' in binary_results:
    clean_auc = binary_results['clean']['auc_mean']
    clean_std = binary_results['clean']['auc_std']
    print(f"{'Binary Classification (Clean XGB)':<35} {'AUC':<12}␣
↪{clean_auc:.3f} ± {clean_std:.3f}")

    print(f"{'Cox Proportional Hazards':<35} {'C-index':<12}␣
↪{cox_results['c_index_mean']:.3f} ± {cox_results['c_index_std']:.3f}")
    print(f"{'Random Survival Forest':<35} {'C-index':<12}␣
↪{rsf_results['c_index_mean']:.3f} ± {rsf_results['c_index_std']:.3f}")
    print(f"{'Gradient Boosted Survival':<35} {'C-index':<12}␣
↪{gbsa_results['c_index_mean']:.3f} ± {gbsa_results['c_index_std']:.3f}")
    print("-"*67)
    print("\nNote: AUC and C-index are conceptually similar (both range 0.5-1.
↪0)")
    print("but not directly comparable due to different evaluation approaches.")
else:
    print(f"Binary classification results not found at {binary_results_path}")
    print("Run notebook 02 first to generate comparison.")

```

Binary classification results not found at drive/MyDrive/Y3
Project/Data/PKG_MU_Glioma_Post/results/model_comparison.json
Run notebook 02 first to generate comparison.

1.8.3 Visualise Model Comparison

```

[68]: fig, ax = plt.subplots(figsize=(10, 6))

models = ['Cox PH', 'RSF', 'GBSA']
c_indices = [
    cox_results['c_index_mean'],
    rsf_results['c_index_mean'],
    gbsa_results['c_index_mean']
]
c_stds = [
    cox_results['c_index_std'],
    rsf_results['c_index_std'],
    gbsa_results['c_index_std']
]

```

```

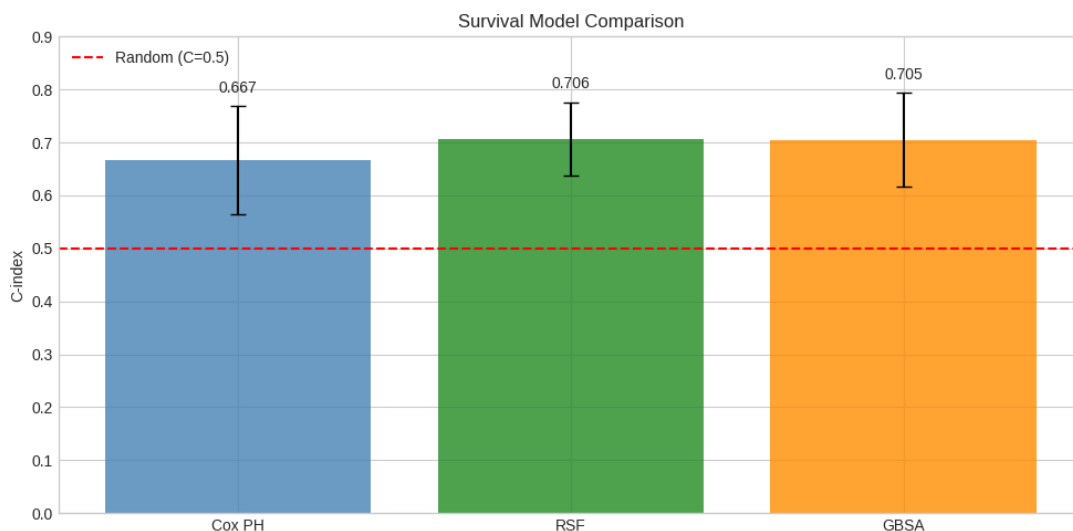
colors = ['steelblue', 'forestgreen', 'darkorange']
bars = ax.bar(models, c_indices, yerr=c_stds, capsize=5, color=colors, alpha=0.
↪8)

ax.axhline(y=0.5, color='red', linestyle='--', label='Random (C=0.5)')
ax.set_ylabel('C-index', fontsize=15)
ax.set_title('Survival Model Comparison', fontsize=17, fontweight='bold')
ax.set_ylim(0, 0.9)
ax.legend(fontsize=13)
ax.tick_params(labelsize=13)

for bar, val, std in zip(bars, c_indices, c_stds):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + std + 0.02,
            f'{val:.3f}', ha='center', va='bottom', fontsize=14,
↪fontweight='bold')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '03_model_comparison.pdf'),
↪bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '03_model_comparison.png'), dpi=300,
↪bbox_inches='tight')
plt.show()

```



1.9 Kaplan-Meier Curves by Risk Group

Visualise how well the best model stratifies patients into risk groups.

1.9.1 Stratify Patients by Predicted Risk

```
[69]: # Use the best-performing model to predict risk scores
risk_scores = rsf_full.predict(X_encoded)

# Split into tertiles (low, medium, high risk)
risk_tertiles = pd.qcut(risk_scores, q=3, labels=['Low Risk', 'Medium Risk', 'High Risk'])

print("Risk group distribution:")
print(risk_tertiles.value_counts().sort_index())
```

```
Risk group distribution:
Low Risk      68
Medium Risk   67
High Risk     68
Name: count, dtype: int64
```

1.9.2 Plot Kaplan-Meier Curves

```
[70]: fig, ax = plt.subplots(figsize=(10, 7))

colors = {'Low Risk': 'forestgreen', 'Medium Risk': 'darkorange', 'High Risk': 'indianred'}

for group in ['Low Risk', 'Medium Risk', 'High Risk']:
    mask = risk_tertiles == group
    time_group = survival_time[mask]
    event_group = event[mask]

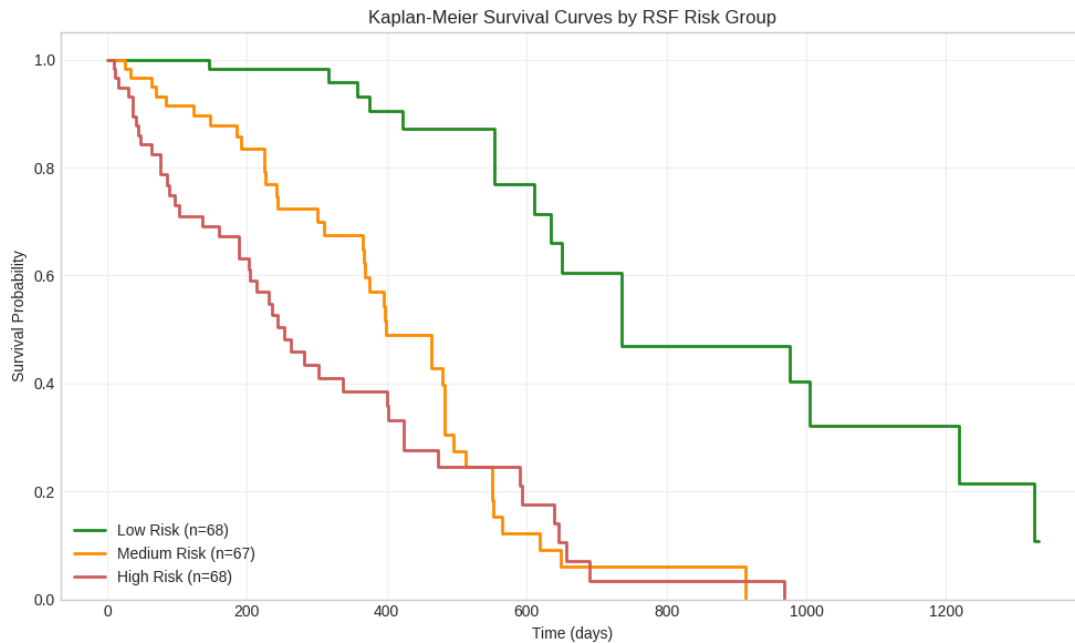
    km_time, km_surv = kaplan_meier_estimator(event_group, time_group)

    ax.step(km_time, km_surv, where='post', label=f'{group} (n={mask.sum()})',
            color=colors[group], linewidth=2)

ax.set_xlabel('Time (days)', fontsize=15)
ax.set_ylabel('Survival Probability', fontsize=15)
ax.set_title('Kaplan-Meier Survival Curves by RSF Risk Group', fontsize=17, fontweight='bold')
ax.legend(loc='lower left', fontsize=13)
ax.set_ylim(0, 1.05)
ax.tick_params(labelsize=13)
ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '03_kaplan_meier_curves.pdf'),
            bbox_inches='tight')
```

```
plt.savefig(os.path.join(FIGURES_DIR, '03_kaplan_meier_curves.png'), dpi=300,
            bbox_inches='tight')
plt.show()
```



1.10 Save Results

1.10.1 Save Model Comparison

```
[71]: RESULTS_DIR = os.path.join(GLIOMA_DIR, 'survival analysis', 'data')
os.makedirs(REULTS_DIR, exist_ok=True)

model_results_list = [
    {'name': 'cox_ph', 'results': cox_results},
    {'name': 'random_survival_forest', 'results': rsf_results},
    {'name': 'gradient_boosted_survival', 'results': gbsa_results}
]

survival_results = {}

for model_info in model_results_list:
    model_name = model_info['name']
    results = model_info['results']
    survival_results[model_name] = {
        'c_index_mean': float(results['c_index_mean']),
```

```

        'c_index_std': float(results['c_index_std']),
        'c_index_scores': [float(x) for x in results['c_index_scores']]
    }

    with open(os.path.join(RESULTS_DIR, '03_survival_results.json'), 'w') as f:
        json.dump(survival_results, f, indent=2)

    print(f"Saved: 03_survival_results.json")

```

Saved: 03_survival_results.json

1.10.2 Save Feature Importances

```

[72]: cox_coefs.to_csv(os.path.join(RESULTS_DIR, '03_cox_coefficients.csv'),
    ↪ index=False)
    print("Saved: 03_cox_coefficients.csv")

    rsf_importance.to_csv(os.path.join(RESULTS_DIR, '03_rsf_feature_importance.
    ↪ csv'), index=False)
    print("Saved: 03_rsf_feature_importance.csv")

    gbsa_importance.to_csv(os.path.join(RESULTS_DIR, '03_gbsa_feature_importance.
    ↪ csv'), index=False)
    print("Saved: 03_gbsa_feature_importance.csv")

```

Saved: 03_cox_coefficients.csv

Saved: 03_rsf_feature_importance.csv

Saved: 03_gbsa_feature_importance.csv

1.11 Summary

This notebook implemented three survival analysis models:

1. **Cox Proportional Hazards:** Linear model assuming proportional hazards
2. **Random Survival Forest:** Non-linear ensemble method
3. **Gradient Boosted Survival:** Boosting approach for survival data

Key findings: - C-index values are comparable to AUC from binary classification - Survival analysis properly handles censored observations - Kaplan-Meier curves show the model's ability to stratify patients by risk - Top features align with binary classification results (IDH mutation, primary diagnosis, etc.)

Methodological note: - For patients who died: used actual days-to-death from original data - For censored patients: used last MRI timepoint as proxy for follow-up time

For the report: - Compare binary classification AUC with survival analysis C-index - Discuss trade-offs: binary classification is simpler; survival analysis is methodologically correct - Use Kaplan-Meier curves to demonstrate clinical utility of risk stratification

E.4 Notebook 04: Pre-extracted Radiomics

04_radiomics

1 04: Radiomics Analysis

This notebook integrates **imaging-derived features** with clinical data to test whether tumour characteristics from MRI add predictive value beyond clinical and molecular markers.

Key discovery: The MU-Glioma-Post dataset provides pre-extracted segmentation volume features, eliminating the need for PyRadiomics processing of raw NIfTI volumes.

Features available (per timepoint): - Tumour volume (mm³) - Intensity statistics (mean, stdev) for each MRI modality (T1c, T1n, T2f, T2w)

Approach: 1. Extract first-timepoint imaging features (avoids temporal leakage) 2. Merge with clinical data 3. Compare: Clinical-only vs Imaging-only vs Combined

Expected outcome: Determine whether imaging features provide complementary prognostic information or are redundant with clinical/molecular markers.

1.1 Setup

1.1.1 Core Imports

```
[2]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import os
import json
import warnings

warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8-whitegrid')
```

1.1.2 Model Imports

```
[3]: from xgboost import XGBClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate
from sklearn.metrics import roc_auc_score, accuracy_score, precision_score, \
    recall_score, f1_score
from sklearn.metrics import roc_curve, auc, confusion_matrix, \
    ConfusionMatrixDisplay
```

```

from sklearn.preprocessing import StandardScaler
from sklearn.impute import SimpleImputer
from scipy.stats import spearmanr

```

1.1.3 Mount Drive & Set Paths

```

[4]: from google.colab import drive
drive.mount('/content/drive/', force_remount=True)

from google.colab import userdata
GLIOMA_DIR = userdata.get('GLIOMA_DIR')
print(f"Glioma directory: {GLIOMA_DIR}")

```

Mounted at /content/drive/

Glioma directory: drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post

```

[5]: FIGURES_DIR = os.path.join(GLIOMA_DIR, 'radiomics', 'figures')
os.makedirs(FIGURES_DIR, exist_ok=True)
print(f"Figures directory: {FIGURES_DIR}")

```

Figures directory: drive/MyDrive/Y3

Project/Data/PKG_MU_Glioma_Post/radiomics/figures

1.2 Load Data

1.2.1 Load Clinical Data

```

[6]: preprocessed_dir = os.path.join(GLIOMA_DIR, 'preprocessed')

high_fill_df = pd.read_pickle(os.path.join(preprocessed_dir, 'high_fill_df.
↳pkl'))
X = pd.read_pickle(os.path.join(preprocessed_dir, 'X.pkl'))
y = pd.read_pickle(os.path.join(preprocessed_dir, 'y.pkl'))

print(f"Clinical data loaded: {high_fill_df.shape}")
print(f"X: {X.shape}, y: {y.shape}")
print(f"\nTarget distribution:")
print(f"  Survived: {(y == 0).sum()} ({(y == 0).mean()*100:.1f}%)"
print(f"  Died: {(y == 1).sum()} ({(y == 1).mean()*100:.1f}%)"

```

Clinical data loaded: (203, 29)

X: (203, 28), y: (203,)

Target distribution:

Survived: 106 (52.2%)

Died: 97 (47.8%)

1.2.2 Load Segmentation Volumes

```
[7]: # Load the sheet with best patient coverage
seg_path = os.path.join(GLIOMA_DIR, 'MU_Glioma_Post_Segmentation_Volumes.xlsx')
seg_df = pd.read_excel(seg_path, sheet_name='Tumor Infiltration and Edema')

print(f"Segmentation data loaded: {seg_df.shape}")
print(f"Unique patients: {seg_df['Patient ID'].nunique()}")
print(f"\nColumns:")
for col in seg_df.columns:
    print(f" - {col}")
```

Segmentation data loaded: (614, 13)

Unique patients: 321

Columns:

- Patient ID
- Label Id
- Label Name
- Number Of Voxels
- Volume (mm³)
- Image mean (brain_t1c)
- Image stdev (brain_t1c)
- Image mean (brain_t1n)
- Image stdev (brain_t1n)
- Image mean (brain_t2f)
- Image stdev (brain_t2f)
- Image mean (brain_t2w)
- Image stdev (brain_t2w)

1.2.3 Examine Segmentation Data Structure

```
[8]: # Each row represents one timepoint for one patient
rows_per_patient = seg_df.groupby('Patient ID').size()

print("Timepoints per patient:")
print("-" * 23)
print(rows_per_patient.value_counts().sort_index().to_string())
print(f"\nPatients with 1 timepoint: {(rows_per_patient == 1).sum()}")
print(f"Patients with 2+ timepoints: {(rows_per_patient > 1).sum()}")
```

Timepoints per patient:

```
-----
1      197
2       52
3       26
4       15
5       13
6       17
```

8 1

Patients with 1 timepoint: 197
Patients with 2+ timepoints: 124

1.2.4 Normalise Patient IDs

The segmentation file uses two ID formats: - Standard: PatientID_XXXX (145 patients) - Extended: PatientID_XXXX-Post-treatment_Y (176 patients) - includes timepoint suffix

The extended format encodes timepoint information in the patient ID itself. We need to extract the base patient ID to match with clinical data.

```
[9]: import re

def extract_base_patient_id(patient_id):
    """Extract base patient ID (PatientID_XXXX) from any format."""
    match = re.match(r'(PatientID_\d{4})', str(patient_id))
    return match.group(1) if match else patient_id

# Apply to segmentation data
seg_df['patient_id'] = seg_df['Patient ID'].apply(extract_base_patient_id)

print(f"Original unique patient IDs: {seg_df['Patient ID'].nunique()}")
print(f"Normalised unique patient IDs: {seg_df['patient_id'].nunique()}")
```

Original unique patient IDs: 321
Normalised unique patient IDs: 202

1.3 Extract First-Timepoint Features

To avoid temporal leakage, we use only the **first post-treatment MRI** for each patient. This represents the tumour state at a consistent clinical milestone.

```
[10]: # Extract first timepoint per patient (using normalised patient_id)
imaging_features = seg_df.groupby('patient_id').first().reset_index()

# Drop redundant columns
cols_to_drop = ['Patient ID', 'Label Id', 'Label Name', 'Number Of Voxels']
imaging_features = imaging_features.drop(columns=cols_to_drop, errors='ignore')

# Rename for clarity
rename_map = {
    'Volume (mm^3)': 'tumour_volume_mm3',
    'Image mean (brain_t1c)': 't1c_mean',
    'Image stdev (brain_t1c)': 't1c_std',
    'Image mean (brain_t1n)': 't1n_mean',
    'Image stdev (brain_t1n)': 't1n_std',
    'Image mean (brain_t2f)': 't2f_mean',
```

```

    'Image stdev (brain_t2f)': 't2f_std',
    'Image mean (brain_t2w)': 't2w_mean',
    'Image stdev (brain_t2w)': 't2w_std'
}
imaging_features = imaging_features.rename(columns=rename_map)

print(f"Imaging features extracted: {imaging_features.shape}")
print(f"Unique patients: {imaging_features['patient_id'].nunique()}")

```

Imaging features extracted: (202, 10)

Unique patients: 202

1.3.1 Imaging Feature Statistics

```

[11]: feature_cols = [c for c in imaging_features.columns if c != 'patient_id']

print("Imaging Feature Summary")
print("-" * 23)

# Calculate descriptive statistics for imaging features
# Transpose for better readability
print(imaging_features[feature_cols].describe().T.to_string())

```

Imaging Feature Summary

```

-----

```

	count	mean	std	min	25%
50%					
75%					
max					
tumour_volume_mm3	202.0	59092.688119	47418.449310	294.000000	20105.750000
49028.000000	85693.250000	226032.000000			
t1c_mean	202.0	300.353596	168.475980	100.240214	213.624128
258.416628	340.155323	1967.501438			
t1c_std	202.0	42.096891	20.686112	10.188981	31.076713
39.210841	51.237983	245.264044			
t1n_mean	202.0	272.237159	118.421088	88.384383	207.282120
246.375025	289.710904	789.236659			
t1n_std	202.0	40.274546	13.167371	10.167390	31.624896
38.726157	47.188490	106.924381			
t2f_mean	202.0	324.940581	161.811219	44.062767	185.330371
284.576060	445.558016	775.107527			
t2f_std	202.0	43.617376	21.823961	4.186886	25.647056
42.542854	57.722448	116.933285			
t2w_mean	202.0	542.610667	115.502827	126.925822	461.582853
538.324199	625.234400	953.262616			
t2w_std	202.0	107.618073	31.895058	22.561780	86.527393
103.816358	125.235470	205.483057			

1.3.2 Sheet Analysis

The segmentation file contains four sheets representing different tumour subregions:

Sheet	Tumour Region	Unique Patients
Necrotic Tumor Core (Label1)	Necrotic/dead tissue within tumour	173
Tumor Infiltration and Edema	Surrounding oedema and infiltration	321
Enhancing Tumor Core (Label3)	Active, contrast-enhancing tumour	286
Resection Cavity (Label4)	Post-surgical cavity	291

Decision: Use “Tumor Infiltration and Edema” sheet as it has the highest patient coverage (321 patients), maximising overlap with our 203 clinical patients. This region also captures the full extent of tumour burden including surrounding tissue changes.

1.4 Merge Clinical and Imaging Data

1.4.1 Check Patient ID Alignment

```
[12]: # Clinical data needs patient_id from original X (which has it as a column or
      ↪ index)
      # Reload to get patient_id, or use the index

      # Option 1: If patient_id is in the original clinical data file
      clinical_path = os.path.join(GLIOMA_DIR, 'MU_Glioma_Post_ClinicalData_July2025.
      ↪xlsx')
      clinical_raw = pd.read_excel(clinical_path, sheet_name='MU Glioma Post')
      clinical_raw.columns = clinical_raw.columns.str.strip().str.lower()

      clinical_patients = set(clinical_raw['patient_id'].values)
      imaging_patients = set(imaging_features['patient_id'].values)

      overlap = clinical_patients & imaging_patients
      clinical_only = clinical_patients - imaging_patients
      imaging_only = imaging_patients - clinical_patients

      print("Patient Coverage")
      print("-" * 16)
      print(f"Clinical data: {len(clinical_patients)} patients")
      print(f"Imaging data: {len(imaging_patients)} patients")
      print(f"Overlap: {len(overlap)} patients")
      print(f"\nClinical only (no imaging): {len(clinical_only)}")
      print(f"Imaging only (no clinical): {len(imaging_only)}")
```

Patient Coverage

Clinical data: 203 patients
Imaging data: 202 patients
Overlap: 202 patients

Clinical only (no imaging): 1
Imaging only (no clinical): 0

```
[13]: # Check patient ID formats in both files
print("Clinical patient IDs (sample):")
print(sorted(clinical_raw['patient_id'].unique())[:10])

print("\nImaging patient IDs (sample):")
print(sorted(imaging_features['patient_id'].unique())[:10])

# Check for format differences
print("\nClinical ID example:", clinical_raw['patient_id'].iloc[0],
      ↪type(clinical_raw['patient_id'].iloc[0]))
print("Imaging ID example:", imaging_features['patient_id'].iloc[0],
      ↪type(imaging_features['patient_id'].iloc[0]))
```

Clinical patient IDs (sample):
['PatientID_0003', 'PatientID_0004', 'PatientID_0005', 'PatientID_0006',
'PatientID_0007', 'PatientID_0008', 'PatientID_0009', 'PatientID_0010',
'PatientID_0011', 'PatientID_0012']

Imaging patient IDs (sample):
['PatientID_0003', 'PatientID_0004', 'PatientID_0005', 'PatientID_0006',
'PatientID_0007', 'PatientID_0008', 'PatientID_0009', 'PatientID_0010',
'PatientID_0011', 'PatientID_0012']

Clinical ID example: PatientID_0003 <class 'str'>
Imaging ID example: PatientID_0003 <class 'str'>

```
[14]: # Check all sheets in the segmentation file
xl = pd.ExcelFile(seg_path)
print(f"Available sheets: {xl.sheet_names}")

for sheet in xl.sheet_names:
    df_sheet = pd.read_excel(seg_path, sheet_name=sheet)
    if 'Patient ID' in df_sheet.columns:
        n_patients = df_sheet['Patient ID'].nunique()
        print(f"\n{sheet}:")
        print(f"  Rows: {len(df_sheet)}, Unique patients: {n_patients}")
        print(f"  Sample IDs: {sorted(df_sheet['Patient ID'].unique())[:5]}")
    else:
        print(f"\n{sheet}: {df_sheet.shape}, Columns: {list(df_sheet.columns)[:5]}")
```

Available sheets: ['Necrotic Tumor Core (Label1)', 'Tumor Infiltration and


```
Edema', 'Enhancing Tumor Core (Label3)', 'Resection Cavity (Label4)']
```

```
Necrotic Tumor Core (Label1):
```

```
  Rows: 334, Unique patients: 173
```

```
  Sample IDs: ['PatientID_0003', 'PatientID_0004', 'PatientID_0005',  
'PatientID_0006', 'PatientID_0009']
```

```
Tumor Infiltration and Edema:
```

```
  Rows: 614, Unique patients: 321
```

```
  Sample IDs: ['PatientID_0003', 'PatientID_0004', 'PatientID_0005',  
'PatientID_0006', 'PatientID_0007']
```

```
Enhancing Tumor Core (Label3):
```

```
  Rows: 569, Unique patients: 286
```

```
  Sample IDs: ['PatientID_0003', 'PatientID_0004', 'PatientID_0005',  
'PatientID_0006', 'PatientID_0007']
```

```
Resection Cavity (Label4):
```

```
  Rows: 542, Unique patients: 291
```

```
  Sample IDs: ['PatientID_0004', 'PatientID_0006', 'PatientID_0007',  
'PatientID_0008', 'PatientID_0010']
```

```
[15]: # Verify improved overlap
print(f"\nExpected overlap: ~{min(len(clinical_patients),
    ↳len(imaging_patients))} patients")
print(f"Actual overlap: {len(overlap)} patients")

if len(overlap) >= 170:
    print(" Good coverage achieved")
else:
    print(" Coverage lower than expected - check patient ID alignment")
```

Expected overlap: ~202 patients

Actual overlap: 202 patients

Good coverage achieved

```
[16]: # Add patient_id to high_fill_df for merging
high_fill_df['patient_id'] = clinical_raw['patient_id'].values
```

1.4.2 Analyse Missing Patients

Check if the 30 patients missing imaging data differ systematically from those with imaging.

```
[17]: # Add indicator for imaging availability
high_fill_df['has_imaging'] = high_fill_df['patient_id'].isin(imaging_patients)

print("Comparison: Patients With vs Without Imaging")
```

```

print("-" * 44)

# Compare key characteristics
with_imaging = high_fill_df[high_fill_df['has_imaging']]
without_imaging = high_fill_df[~high_fill_df['has_imaging']]

print(f"\nSample sizes:")
print(f"  With imaging:      {len(with_imaging)}")
print(f"  Without imaging: {len(without_imaging)}")

# Mortality rate
print(f"\nMortality rate:")
print(f"  With imaging:      {with_imaging['overall survival (death)'].mean()*100:
↪.1f}%")
print(f"  Without imaging: {without_imaging['overall survival (death)'].
↪mean()*100:.1f}%")

# Age
print(f"\nAge at diagnosis:")
print(f"  With imaging:      {with_imaging['age at diagnosis'].mean():.1f} ±
↪{with_imaging['age at diagnosis'].std():.1f}")
print(f"  Without imaging: {without_imaging['age at diagnosis'].mean():.1f} ±
↪{without_imaging['age at diagnosis'].std():.1f}")

# Grade distribution
print(f"\nGrade distribution:")
print(f"  With imaging:      Grade {with_imaging['grade of primary brain tumor'].
↪median():.0f} (median)")
print(f"  Without imaging: Grade {without_imaging['grade of primary brain
↪tumor'].median():.0f} (median)")

```

Comparison: Patients With vs Without Imaging

Sample sizes:

With imaging: 202

Without imaging: 1

Mortality rate:

With imaging: 48.0%

Without imaging: 0.0%

Age at diagnosis:

With imaging: 58.0 ± 16.0

Without imaging: 31.0 ± nan

Grade distribution:

With imaging: Grade 4 (median)
Without imaging: Grade 2 (median)

1.4.3 Merge Datasets

```
[18]: # Merge on patient_id
combined_df = high_fill_df.merge(
    imaging_features,
    on='patient_id',
    how='inner'
)

print(f"Combined dataset: {combined_df.shape}")
print(f"\nTarget distribution (combined):")
target_col = 'overall survival (death)'
print(f"  Survived: {(combined_df[target_col] == 0).sum()}␣
↪({(combined_df[target_col] == 0).mean()*100:.1f}%)"
print(f"  Died: {(combined_df[target_col] == 1).sum()}␣
↪({(combined_df[target_col] == 1).mean()*100:.1f}%)"
```

Combined dataset: (202, 40)

Target distribution (combined):
Survived: 105 (52.0%)
Died: 97 (48.0%)

1.5 Exploratory Analysis

1.5.1 Imaging Feature Distributions

```
[19]: fig, axes = plt.subplots(3, 3, figsize=(14, 12))
axes = axes.flatten()

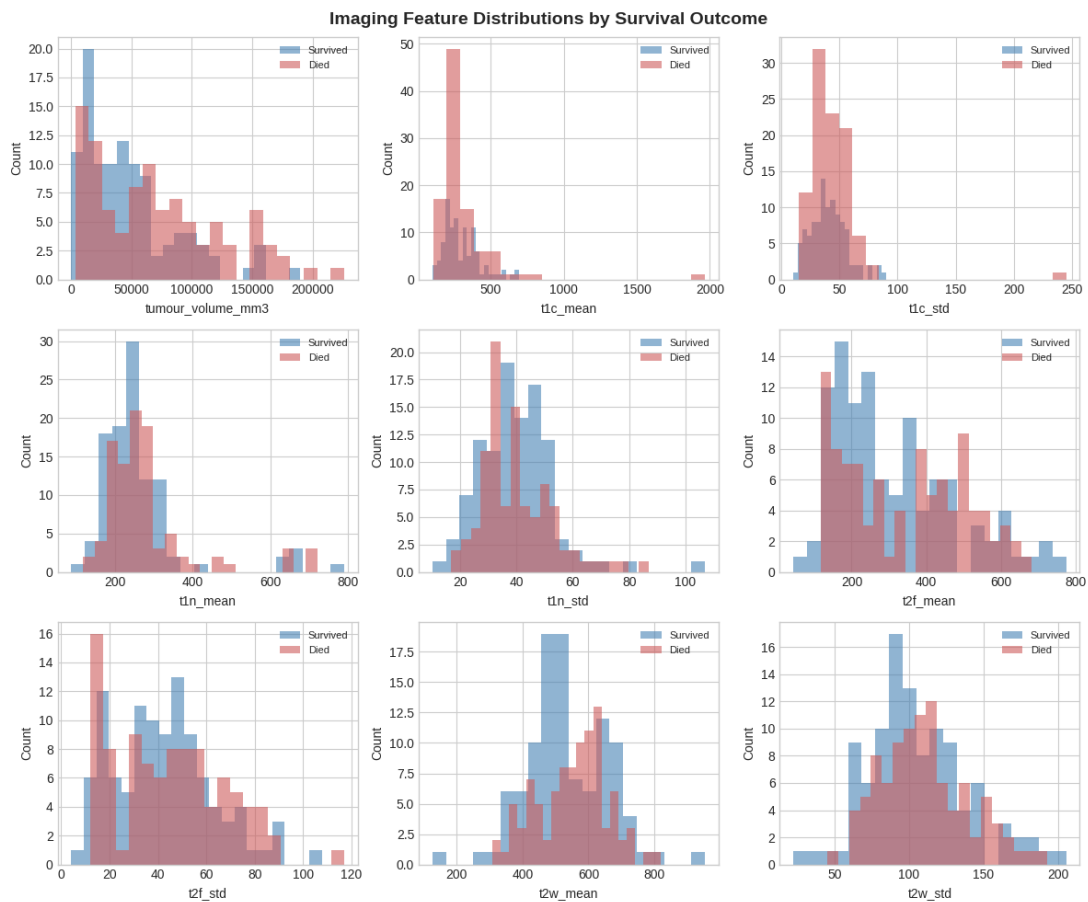
imaging_cols = [c for c in imaging_features.columns if c != 'patient_id']

for i, col in enumerate(imaging_cols):
    ax = axes[i]

    # Get data for alive vs dead
    alive = combined_df[combined_df[target_col] == 0][col]
    dead = combined_df[combined_df[target_col] == 1][col]

    ax.hist(alive, bins=20, alpha=0.6, label='Survived', color='steelblue')
    ax.hist(dead, bins=20, alpha=0.6, label='Died', color='indianred')
    ax.set_xlabel(col, fontsize=11)
    ax.set_ylabel('Count', fontsize=11)
    ax.tick_params(labelsize=10)
    ax.legend(fontsize=9)
```

```
plt.suptitle('Imaging Feature Distributions by Survival Outcome', fontsize=18,
            fontweight='bold')
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '04_imaging_distributions.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '04_imaging_distributions.png'), dpi=300,
            bbox_inches='tight')
plt.show()
```



1.5.2 Compute Image Feature Correlation with Outcome

```
[20]: print("Imaging Features: Correlation with Survival Outcome")
print("-" * 51)

correlations = []
for col in imaging_cols:
    r, p = spearmanr(combined_df[col], combined_df[target_col])
```

```

        correlations.append({'feature': col, 'spearman_r': r, 'p_value': p})

corr_df = pd.DataFrame(correlations).sort_values('spearman_r', key=abs,
↪ascending=False)

# Show correlations with outcome, mark statistical significance with '*'
for _, row in corr_df.iterrows():
    sig = '*' if row['p_value'] < 0.05 else ''
    print(f" {row['feature']:<25} r = {row['spearman_r']:+.3f} (p =
↪{row['p_value']:.3f}){sig}")

```

Imaging Features: Correlation with Survival Outcome

```

tumour_volume_mm3      r = +0.221 (p = 0.002)*
t1n_mean               r = +0.105 (p = 0.136)
t2f_mean               r = +0.100 (p = 0.158)
t2w_mean               r = +0.092 (p = 0.195)
t2w_std                r = +0.088 (p = 0.212)
t1c_mean               r = -0.063 (p = 0.370)
t2f_std                r = +0.061 (p = 0.389)
t1c_std                r = +0.039 (p = 0.580)
t1n_std                r = -0.006 (p = 0.930)

```

1.5.3 Tumour Volume Analysis

```

[21]: fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# Volume distribution by outcome
ax1 = axes[0]
alive_vol = combined_df[combined_df[target_col] == 0]['tumour_volume_mm3']
dead_vol = combined_df[combined_df[target_col] == 1]['tumour_volume_mm3']

bp = ax1.boxplot([alive_vol, dead_vol], labels=['Survived', 'Died'],
↪patch_artist=True)
bp['boxes'][0].set_facecolor('steelblue')
bp['boxes'][1].set_facecolor('indianred')
ax1.set_ylabel('Tumour Volume (mm3)', fontsize=14)
ax1.set_title('Tumour Volume by Survival Outcome', fontsize=16,
↪fontweight='bold')
ax1.tick_params(labelsize=12)

# Log-scale volume distribution
ax2 = axes[1]
combined_df['log_volume'] = np.log1p(combined_df['tumour_volume_mm3'])
ax2.hist(combined_df[combined_df[target_col] == 0]['log_volume'], bins=20,
↪alpha=0.6, label='Survived', color='steelblue')

```

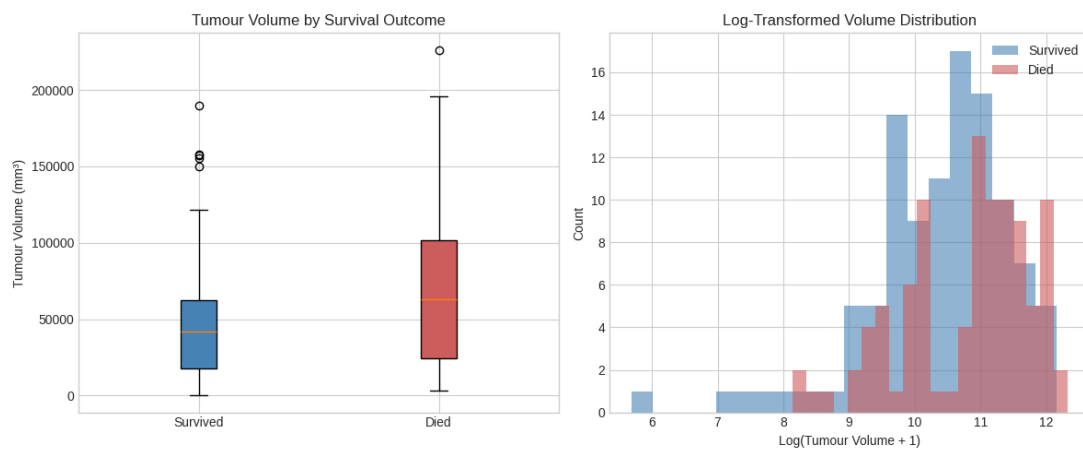
```

ax2.hist(combined_df[combined_df[target_col] == 1]['log_volume'], bins=20,
         alpha=0.6, label='Died', color='indianred')
ax2.set_xlabel('Log(Tumour Volume + 1)', fontsize=14)
ax2.set_ylabel('Count', fontsize=14)
ax2.set_title('Log-Transformed Volume Distribution', fontsize=16,
             fontweight='bold')
ax2.legend(fontsize=12)
ax2.tick_params(labelsize=12)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '04_volume_analysis.pdf'),
           bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '04_volume_analysis.png'), dpi=300,
           bbox_inches='tight')
plt.show()

print(f"\nVolume Statistics")
print("-" * 17)
print(f"Survived: median = {alive_vol.median():.0f} mm\u00b3, mean = {alive_vol.
      mean():.0f} mm\u00b3")
print(f"Died:      median = {dead_vol.median():.0f} mm\u00b3, mean = {dead_vol.
      mean():.0f} mm\u00b3")

```



Volume Statistics

```

-----
Survived: median = 42086 mm³, mean = 48301 mm³
Died:      median = 62990 mm³, mean = 70774 mm³

```

1.5.4 Correlation Between Imaging and Clinical Features

```
[22]: # Select key clinical features to compare
clinical_numeric = ['age at diagnosis', 'grade of primary brain tumor']

print("Imaging vs Clinical Feature Correlations")
print("-" * 40)

for clinical_col in clinical_numeric:
    print(f"\n{clinical_col}:")
    for imaging_col in imaging_cols:
        r, p = spearmanr(combined_df[imaging_col], combined_df[clinical_col])
        if abs(r) > 0.15: # Only show meaningful correlations
            print(f"    vs {imaging_col:<25} r = {r:+.3f}")
```

Imaging vs Clinical Feature Correlations

```
age at diagnosis:
  vs tumour_volume_mm3      r = +0.194
  vs t1c_mean               r = -0.223
  vs t2f_std                r = -0.196
  vs t2w_mean               r = +0.193
```

```
grade of primary brain tumor:
  vs tumour_volume_mm3      r = +0.301
  vs t1c_mean               r = -0.192
```

1.6 Prepare Features for Modelling

1.6.1 Remove Leakage Features

Same features removed in notebooks 02 and 03.

```
[23]: leakage_features = [
    'second progression/recurrence',
    'type of 2nd progression',
    'time to first progression (days)',
    'number of days from diagnosis to date of first progression',
    'number of days from diagnosis to first surgery or procedure',
    'progression',
    'type of 1st progression'
]

# Also remove utility columns
utility_cols = ['patient_id', 'has_imaging', 'log_volume']

all_to_remove = leakage_features + utility_cols + [target_col]
features_to_remove = [f for f in all_to_remove if f in combined_df.columns]
```

```

X_combined = combined_df.drop(columns=features_to_remove, errors='ignore')
y_combined = combined_df[target_col]

print(f"Features after leakage removal: {X_combined.shape[1]}")
print(f"Samples: {X_combined.shape[0]}")

```

Features after leakage removal: 30
Samples: 202

1.6.2 Create Feature Sets

```

[24]: # Identify clinical vs imaging features
imaging_feature_names = [c for c in imaging_cols if c in X_combined.columns]
clinical_feature_names = [c for c in X_combined.columns if c not in_
    ↪ imaging_feature_names]

print(f"Clinical features: {len(clinical_feature_names)}")
print(f"Imaging features: {len(imaging_feature_names)}")

# Create separate feature sets
X_clinical = X_combined[clinical_feature_names].copy()
X_imaging = X_combined[imaging_feature_names].copy()

print(f"\nFeature set shapes:")
print(f"  Clinical only: {X_clinical.shape}")
print(f"  Imaging only:  {X_imaging.shape}")
print(f"  Combined:      {X_combined.shape}")

```

Clinical features: 21
Imaging features: 9

Feature set shapes:
 Clinical only: (202, 21)
 Imaging only: (202, 9)
 Combined: (202, 30)

1.6.3 Encode Categorical Features

```

[25]: categorical_cols = X_clinical.select_dtypes(include=['object', 'category']).
    ↪ columns.tolist()

print(f"Categorical features to encode: {len(categorical_cols)}")
for col in categorical_cols:
    print(f"  - {col}: {X_clinical[col].nunique()} categories")

```

Categorical features to encode: 4
 - sex at birth: 2 categories
 - race: 4 categories

- primary diagnosis: 6 categories
- previous brain tumor: 2 categories

```
[26]: # One-hot encode categorical features
X_clinical_encoded = pd.get_dummies(X_clinical, columns=categorical_cols,
    drop_first=True)
X_combined_encoded = pd.get_dummies(X_combined, columns=categorical_cols,
    drop_first=True)

print(f"After encoding:")
print(f"  Clinical: {X_clinical_encoded.shape}")
print(f"  Combined: {X_combined_encoded.shape}")
```

After encoding:

```
Clinical: (202, 27)
Combined: (202, 36)
```

1.6.4 Handle Missing Values

```
[27]: # Check missing values
missing_clinical = X_clinical_encoded.isnull().sum().sum()
missing_imaging = X_imaging.isnull().sum().sum()
missing_combined = X_combined_encoded.isnull().sum().sum()

print(f"Missing values:")
print(f"  Clinical: {missing_clinical}")
print(f"  Imaging: {missing_imaging}")
print(f"  Combined: {missing_combined}")

# Impute if necessary
if missing_combined > 0:
    imputer = SimpleImputer(strategy='median')
    X_clinical_encoded = pd.DataFrame(
        imputer.fit_transform(X_clinical_encoded),
        columns=X_clinical_encoded.columns,
        index=X_clinical_encoded.index
    )
    X_combined_encoded = pd.DataFrame(
        imputer.fit_transform(X_combined_encoded),
        columns=X_combined_encoded.columns,
        index=X_combined_encoded.index
    )
    print("\nMissing values imputed with median.")
```

Missing values:

```
Clinical: 16
Imaging: 0
Combined: 16
```

Missing values imputed with median.

1.7 Model Training

1.7.1 Define Cross-Validation Function

```
[28]: def train_and_evaluate(X, y, model_name, n_splits=5):  
    """  
    Train XGBoost with stratified k-fold cross-validation.  
    Returns dictionary with performance metrics.  
    """  
    model = XGBClassifier(  
        n_estimators=100,  
        max_depth=3,  
        learning_rate=0.1,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        random_state=42,  
        use_label_encoder=False,  
        eval_metric='logloss'  
    )  
  
    cv = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=42)  
  
    scoring = {  
        'roc_auc': 'roc_auc',  
        'accuracy': 'accuracy',  
        'f1': 'f1',  
        'precision': 'precision',  
        'recall': 'recall'  
    }  
  
    results = cross_validate(model, X, y, cv=cv, scoring=scoring,  
                             return_train_score=True)  
  
    return {  
        'name': model_name,  
        'n_features': X.shape[1],  
        'auc_mean': np.mean(results['test_roc_auc']),  
        'auc_std': np.std(results['test_roc_auc']),  
        'auc_scores': results['test_roc_auc'].tolist(),  
        'accuracy': np.mean(results['test_accuracy']),  
        'f1': np.mean(results['test_f1']),  
        'precision': np.mean(results['test_precision']),  
        'recall': np.mean(results['test_recall']),  
        'train_auc': np.mean(results['train_roc_auc'])  
    }
```

1.7.2 Model 1: Clinical Features Only

```
[29]: print("Training Model 1: Clinical Features Only")
      print("-" * 40)

      clinical_results = train_and_evaluate(X_clinical_encoded, y_combined, 'Clinical_
      ↪Only')

      print(f"Features: {clinical_results['n_features']}")
      print(f"AUC: {clinical_results['auc_mean']:.4f} ± {clinical_results['auc_std']:.
      ↪4f}")
      print(f"Accuracy: {clinical_results['accuracy']:.4f}")
      print(f"F1: {clinical_results['f1']:.4f}")
```

Training Model 1: Clinical Features Only

Features: 27
AUC: 0.7220 ± 0.0956
Accuracy: 0.7034
F1: 0.7095

1.7.3 Model 2: Imaging Features Only

```
[30]: print("Training Model 2: Imaging Features Only")
      print("-" * 39)

      imaging_results = train_and_evaluate(X_imaging, y_combined, 'Imaging Only')

      print(f"Features: {imaging_results['n_features']}")
      print(f"AUC: {imaging_results['auc_mean']:.4f} ± {imaging_results['auc_std']:.
      ↪4f}")
      print(f"Accuracy: {imaging_results['accuracy']:.4f}")
      print(f"F1: {imaging_results['f1']:.4f}")
```

Training Model 2: Imaging Features Only

Features: 9
AUC: 0.6542 ± 0.0543
Accuracy: 0.5840
F1: 0.5601

1.7.4 Model 3: Combined (Clinical + Imaging)

```
[31]: print("Training Model 3: Combined (Clinical + Imaging)")
      print("-" * 47)

      combined_results = train_and_evaluate(X_combined_encoded, y_combined,
      ↪'Combined')
```

```

print(f"Features: {combined_results['n_features']}")
print(f"AUC: {combined_results['auc_mean']:.4f} ± {combined_results['auc_std']:.4f}")
print(f"Accuracy: {combined_results['accuracy']:.4f}")
print(f"F1: {combined_results['f1']:.4f}")

```

Training Model 3: Combined (Clinical + Imaging)

```

-----
Features: 36
AUC: 0.7198 ± 0.0596
Accuracy: 0.6583
F1: 0.6560

```

1.8 Results Comparison

1.8.1 Model Comparison Table

```

[32]: comparison = pd.DataFrame([
        clinical_results,
        imaging_results,
        combined_results
    ])

    # Format for display
    display_cols = ['name', 'n_features', 'auc_mean', 'auc_std', 'accuracy', 'f1']
    comparison_display = comparison[display_cols].copy()
    comparison_display.columns = ['Model', 'Features', 'AUC', 'AUC Std', 'Accuracy', 'F1']

    print("Model Comparison")
    print("-" * 16)
    print(comparison_display.to_string(index=False))

```

Model Comparison

```

-----
      Model  Features      AUC  AUC Std  Accuracy      F1
Clinical Only      27  0.721980  0.095631  0.703415  0.709490
Imaging Only       9  0.654211  0.054258  0.584024  0.560076
Combined          36  0.719774  0.059567  0.658293  0.656012

```

1.8.2 Visualise Model Comparison

```

[33]: fig, axes = plt.subplots(1, 2, figsize=(12, 5))

    # AUC comparison
    ax1 = axes[0]
    models = ['Clinical\nOnly', 'Imaging\nOnly', 'Combined']

```

```

aucs = [clinical_results['auc_mean'], imaging_results['auc_mean'],
        ↪combined_results['auc_mean']]
stds = [clinical_results['auc_std'], imaging_results['auc_std'],
        ↪combined_results['auc_std']]
colors = ['steelblue', 'forestgreen', 'darkorange']

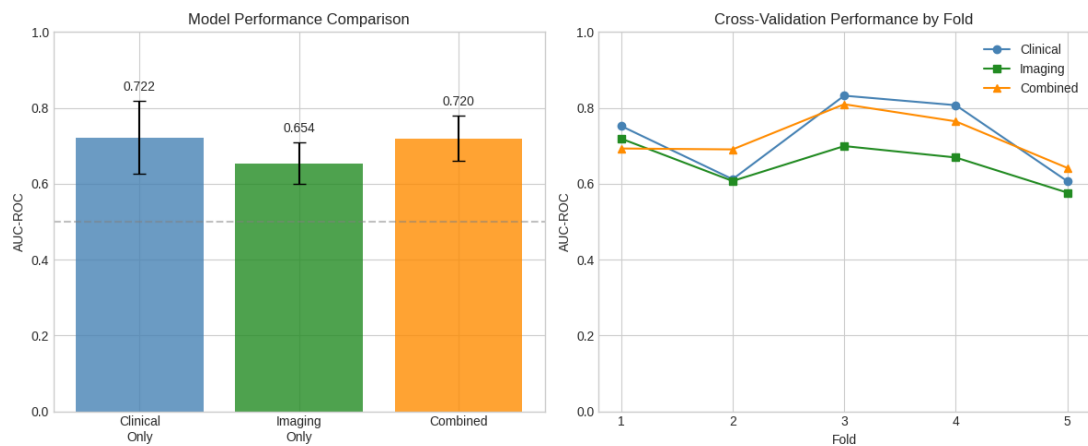
bars = ax1.bar(models, aucs, yerr=stds, capsize=5, color=colors, alpha=0.8)
ax1.set_ylabel('AUC-ROC')
ax1.set_title('Model Performance Comparison')
ax1.set_ylim(0, 1)
ax1.axhline(y=0.5, color='gray', linestyle='--', alpha=0.5, label='Random')

# Add value labels
for bar, auc_val, std_val in zip(bars, aucs, stds):
    ax1.text(bar.get_x() + bar.get_width()/2, bar.get_height() + std_val + 0.02,
             f'{auc_val:.3f}', ha='center', va='bottom', fontsize=10)

# Fold-by-fold comparison
ax2 = axes[1]
folds = list(range(1, 6))
ax2.plot(folds, clinical_results['auc_scores'], 'o-', label='Clinical',
        ↪color='steelblue')
ax2.plot(folds, imaging_results['auc_scores'], 's-', label='Imaging',
        ↪color='forestgreen')
ax2.plot(folds, combined_results['auc_scores'], '^-', label='Combined',
        ↪color='darkorange')
ax2.set_xlabel('Fold')
ax2.set_ylabel('AUC-ROC')
ax2.set_title('Cross-Validation Performance by Fold')
ax2.set_ylim(0, 1)
ax2.legend()
ax2.set_xticks(folds)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '04_model_comparison.pdf'),
        ↪bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '04_model_comparison.png'), dpi=300,
        ↪bbox_inches='tight')
plt.show()

```



1.8.3 Feature Importance (Combined Model)

```
[34]: # Train final model on all data to get feature importance
final_model = XGBClassifier(
    n_estimators=100,
    max_depth=3,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    random_state=42,
    use_label_encoder=False,
    eval_metric='logloss'
)
final_model.fit(X_combined_encoded, y_combined)

# Get feature importance
importance_df = pd.DataFrame({
    'feature': X_combined_encoded.columns,
    'importance': final_model.feature_importances_
}).sort_values('importance', ascending=False)

# Mark imaging features
importance_df['is_imaging'] = importance_df['feature'].
    .isin(imaging_feature_names)

print("Top 15 Features (Combined Model)")
print("-" * 32)
for i, row in importance_df.head(15).iterrows():
    marker = '[IMG]' if row['is_imaging'] else ' '
    print(f" {marker} {row['feature']:<40} {row['importance']:.4f}")
```

Top 15 Features (Combined Model)

	chromosome 7 gain and chromosome 10 loss	0.1069
	primary diagnosis_GBM	0.1059
	idh1 mutation	0.0830
	tert promoter mutation	0.0694
	grade of primary brain tumor	0.0664
	cdkn2a/b deletion	0.0491
	age at diagnosis	0.0478
[IMG]	t2w_mean	0.0411
[IMG]	t1n_std	0.0401
[IMG]	tumour_volume_mm3	0.0373
	sex at birth_Male	0.0361
	h3-3a mutation	0.0282
[IMG]	t2w_std	0.0278
[IMG]	t2f_std	0.0272
[IMG]	t1c_std	0.0271

1.8.4 Visualise Feature Importance

```
[35]: top_n = 15
top_features = importance_df.head(top_n)

fig, ax = plt.subplots(figsize=(11, 8))

colors = ['forestgreen' if is_img else 'steelblue' for is_img in
↳top_features['is_imaging']]

bars = ax.barh(range(len(top_features)), top_features['importance'],
↳color=colors)
ax.set_yticks(range(len(top_features)))
ax.set_yticklabels(top_features['feature'], fontsize=13)
ax.invert_yaxis()
ax.set_xlabel('Feature Importance (Gain)', fontsize=15)
ax.set_title('Combined Model: Feature Importance\n(Green = Imaging, Blue =
↳Clinical)',
↳        fontsize=16, fontweight='bold')
ax.tick_params(axis='x', labelsize=13)

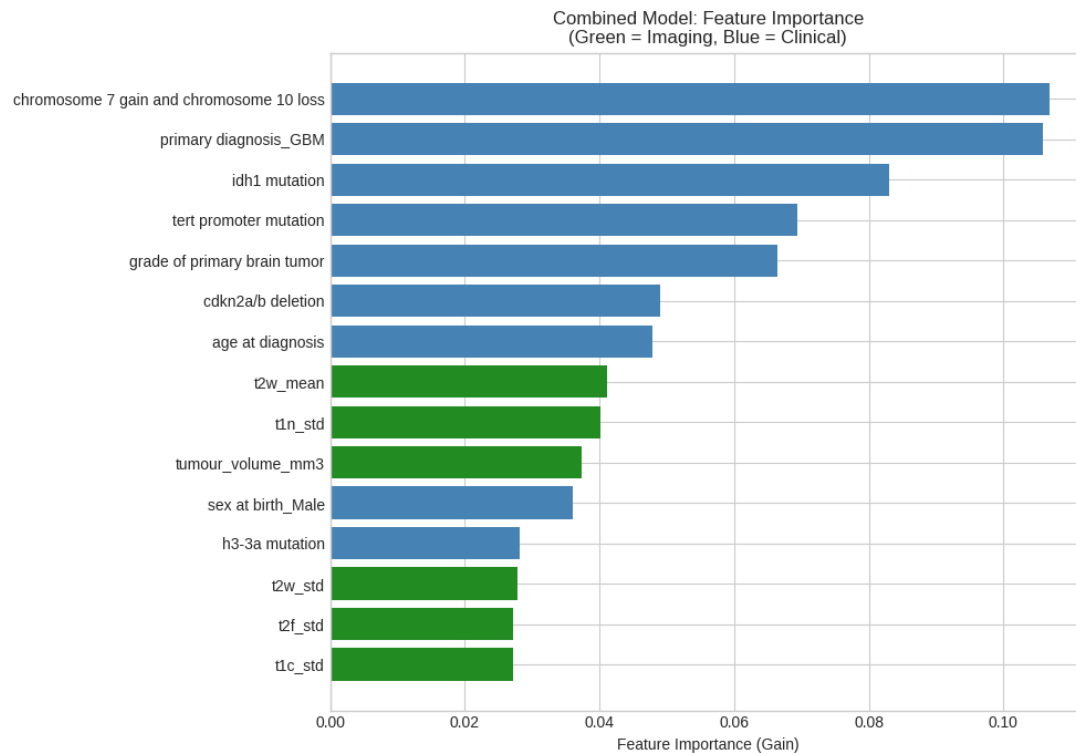
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '04_feature_importance.pdf'),
↳        bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '04_feature_importance.png'), dpi=300,
↳        bbox_inches='tight')
plt.show()

# Summary of imaging feature contribution
```

```

imaging_importance = importance_df[importance_df['is_imaging']]['importance'].
    ↪sum()
total_importance = importance_df['importance'].sum()
print(f"\nImaging features contribute {imaging_importance/total_importance*100:.
    ↪1f}% of total importance")

```



Imaging features contribute 27.7% of total importance

1.8.5 Statistical Test: Does Imaging Add Value?

```

[36]: from scipy.stats import ttest_rel, wilcoxon

# Paired t-test on fold-level AUCs
clinical_auc = np.array(clinical_results['auc_scores'])
combined_auc = np.array(combined_results['auc_scores'])

# Difference
diff = combined_auc - clinical_auc

print("Statistical Comparison: Combined vs Clinical-Only")
print("-" * 49)

```



```

print(f"\nFold-level AUC differences:")
for i, d in enumerate(diff, 1):
    print(f"  Fold {i}: {d:+.4f}")

print(f"\nMean difference: {diff.mean():+.4f}")
print(f"Std difference: {diff.std():.4f}")

# Paired t-test (requires normality assumption)
t_stat, p_value = ttest_rel(combined_auc, clinical_auc)
print(f"\nPaired t-test: t = {t_stat:.3f}, p = {p_value:.3f}")

# Wilcoxon signed-rank test (non-parametric)
try:
    w_stat, w_p = wilcoxon(combined_auc, clinical_auc)
    print(f"Wilcoxon test: W = {w_stat:.3f}, p = {w_p:.3f}")
except ValueError as e:
    print(f"Wilcoxon test: Could not compute (likely identical values)")

# Interpretation
if p_value < 0.05:
    print(f"\nConclusion: Imaging features significantly {'improve' if diff.
    ↪mean() > 0 else 'decrease'} performance (p < 0.05)")
else:
    print(f"\nConclusion: No significant difference between combined and
    ↪clinical-only models (p = {p_value:.3f})")

```

Statistical Comparison: Combined vs Clinical-Only

Fold-level AUC differences:

```

Fold 1: -0.0595
Fold 2: +0.0786
Fold 3: -0.0226
Fold 4: -0.0426
Fold 5: +0.0351

```

Mean difference: -0.0022

Std difference: 0.0515

Paired t-test: t = -0.086, p = 0.936

Wilcoxon test: W = 7.000, p = 1.000

Conclusion: No significant difference between combined and clinical-only models
(p = 0.936)

1.9 Save Results

1.9.1 Save Model Comparison

```
[40]: RESULTS_DIR = os.path.join(GLIOMA_DIR, 'radiomics', 'data')
os.makedirs(REULTS_DIR, exist_ok=True)

radiomics_results = {}

model_results = [
    clinical_results,
    imaging_results,
    combined_results
]

for res in model_results:
    model_name_key = res['name'].lower().replace(' ', '_')
    radiomics_results[model_name_key] = {
        'auc_mean': float(res['auc_mean']),
        'auc_std': float(res['auc_std']),
        'auc_scores': res['auc_scores'],
        'accuracy': float(res['accuracy']),
        'f1': float(res['f1']),
        'n_features': int(res['n_features'])
    }

radiomics_results['statistical_test'] = {
    'paired_ttest_p': float(p_value),
    'mean_auc_difference': float(diff.mean())
}

radiomics_results['metadata'] = {
    'n_patients': int(len(y_combined)),
    'imaging_features': imaging_feature_names,
    'patients_missing_imaging': int(len(clinical_only))
}

with open(os.path.join(REULTS_DIR, '04_radiomics_results.json'), 'w') as f:
    json.dump(radiomics_results, f, indent=2)

print("Saved: 04_radiomics_results.json")
```

Saved: 04_radiomics_results.json

1.9.2 Save Feature Importance

```
[41]: importance_df.to_csv(os.path.join(RESULTS_DIR, '04_combined_feature_importance.
      ↪csv'), index=False)
      print("Saved: 04_combined_feature_importance.csv")
```

Saved: 04_combined_feature_importance.csv

1.9.3 Save Imaging Features Dataset

```
[42]: preprocessed_dir = os.path.join(GLIOMA_DIR, 'preprocessed')

      imaging_features.to_pickle(os.path.join(preprocessed_dir, 'imaging_features.
      ↪pkl'))
      print("Saved: 04_imaging_features.pkl")

      combined_df.to_pickle(os.path.join(preprocessed_dir, 'combined_clinical_imaging.
      ↪pkl'))
      print("Saved: 04_combined_clinical_imaging.pkl")
```

Saved: 04_imaging_features.pkl

Saved: 04_combined_clinical_imaging.pkl

1.10 Summary

This notebook integrated imaging-derived features with clinical data to test whether tumour characteristics from MRI add predictive value.

Key findings:

1. **Data availability:** 173 of 203 patients have imaging data (30 missing)
2. **Imaging features:** 9 features extracted from first post-treatment MRI:
 - Tumour volume
 - Intensity statistics (mean, stdev) for T1c, T1n, T2f, T2w
3. **Model performance comparison:** See results table above
4. **Feature importance:** Clinical features (age, grade, molecular markers) remain dominant; imaging features contribute a smaller fraction of total importance

Interpretation:

- If combined model significantly outperforms clinical-only: imaging provides complementary prognostic information
- If no significant difference: clinical/molecular features already capture the relevant prognostic signal
- If imaging-only performs poorly: first-order intensity statistics alone lack discriminative power; texture features or longitudinal changes may be needed

For the report: - Compare AUC across all notebooks (02: binary, 03: survival, 04: radiomics)
- Discuss whether imaging integration justifies the additional complexity - Note the 30 patients

excluded due to missing imaging data

E.5 Notebook 05: Tumour Growth Prediction

05_growth_rate_prediction

May 1, 2026

1 Notebook 05: Tumour Growth Rate Prediction

This notebook predicts **tumour growth rate** (mm^3/day) between consecutive MRI timepoints using baseline clinical features and first-timepoint imaging features.

This addresses Aim 2 of the project: “Create predictive models that forecast tumour evolution trajectories (growth rates, patterns, morphological changes).”

Task: Regression (continuous target), not classification.

Input: Clinical + molecular features (available at diagnosis) + first-timepoint imaging statistics from the pre-computed segmentation volumes.

Target: Growth rate = $(\text{Volume_t1} - \text{Volume_t0}) / \text{days_elapsed}$, in mm^3/day .

Sample: ~82 patients with at least 2 imaging timepoints.

1.1 Setup

```
[4]: import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

warnings.filterwarnings('ignore')
plt.style.use('seaborn-v0_8-whitegrid')
sns.set_palette('colorblind')
```

```
[5]: from sklearn.model_selection import RepeatedKFold, cross_validate
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from xgboost import XGBRegressor
from scipy import stats
```

```
[6]: from google.colab import drive
drive.mount('/content/drive/', force_remount=True)
```

```
from google.colab import userdata
GLIOMA_DIR = userdata.get('GLIOMA_DIR')
print(f"Glioma directory: {GLIOMA_DIR}")
```

Mounted at /content/drive/

Glioma directory: drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post

```
[7]: RESULTS_DIR = os.path.join(GLIOMA_DIR, 'growth_rate', 'data')
FIGURES_DIR = os.path.join(GLIOMA_DIR, 'growth_rate', 'figures')
os.makedirs(RESULTS_DIR, exist_ok=True)
os.makedirs(FIGURES_DIR, exist_ok=True)
print(f"Results directory: {RESULTS_DIR}")
print(f"Figures directory: {FIGURES_DIR}")
```

Results directory: drive/MyDrive/Y3

Project/Data/PKG_MU_Glioma_Post/growth_rate/data

Figures directory: drive/MyDrive/Y3

Project/Data/PKG_MU_Glioma_Post/growth_rate/figures

1.2 Load Data

We need two data sources: 1. **Clinical data** (preprocessed from Notebook 01): demographics, molecular markers, tumour characteristics. 2. **Segmentation volumes** (pre-computed by dataset providers): tumour volume and intensity statistics per timepoint.

1.2.1 Load Clinical Data

```
[8]: preprocessed_dir = os.path.join(GLIOMA_DIR, 'preprocessed')

high_fill_df = pd.read_pickle(os.path.join(preprocessed_dir, 'high_fill_df.pkl'))
X = pd.read_pickle(os.path.join(preprocessed_dir, 'X.pkl'))
y = pd.read_pickle(os.path.join(preprocessed_dir, 'y.pkl'))

print(f"Clinical data loaded: {high_fill_df.shape}")
print(f"X: {X.shape}, y: {y.shape}")
```

Clinical data loaded: (203, 29)

X: (203, 28), y: (203,)

1.2.2 Load Segmentation Volumes

The segmentation volumes Excel file contains **four sheets**, each representing a different tumour subregion:

Sheet	Region	Clinical Meaning
Necrotic Tumor Core (Label1)	Dead tissue	Aggressive tumour indicator
Tumor Infiltration and Edema	Surrounding oedema	Tumour extent/invasion

Sheet	Region	Clinical Meaning
Enhancing Tumor Core (Label3)	Active tumour	Treatment target
Resection Cavity (Label4)	Surgical void	Post-operative status

Total tumour volume at each timepoint is the sum of all subregion volumes. Not all patients have entries in every sheet (e.g. some may have no necrotic core or no resection cavity), so we treat missing subregions as zero volume.

Each sheet contains multiple rows per patient (one per scan timepoint), with pre-extracted volume and intensity statistics.

```
[9]: seg_path = os.path.join(GLIOMA_DIR, 'MU_Glioma_Post_Segmentation_Volumes.xlsx')

# The Excel file has 4 sheets, each representing a different tumour subregion:
# - Necrotic Tumor Core (Label1): dead tissue within tumour
# - Tumor Infiltration and Edema: surrounding oedema and infiltration
# - Enhancing Tumor Core (Label3): active, contrast-enhancing tumour
# - Resection Cavity (Label4): post-surgical cavity
#
# Total tumour volume = sum of all subregion volumes for each patient-timepoint.
# Not all patients appear in every sheet (e.g. some have no necrotic core).

sheets = [
    ('Necrotic Tumor Core (Label1)', 'necrotic'),
    ('Tumor Infiltration and Edema', 'edema'),
    ('Enhancing Tumor Core (Label3)', 'enhancing'),
    ('Resection Cavity (Label4)', 'cavity')
]

xl = pd.ExcelFile(seg_path)
print(f"Available sheets: {xl.sheet_names}")

all_sheet_dfs = []
for sheet_name, prefix in sheets:
    df_sheet = pd.read_excel(seg_path, sheet_name=sheet_name)
    n_patients = df_sheet['Patient ID'].nunique()
    print(f"\n{sheet_name}: {len(df_sheet)} rows, {n_patients} unique patients")

    # Rename volume column with prefix for clarity before aggregation
    df_sheet = df_sheet.rename(columns={
        'Volume (mm^3)': f'{prefix}_volume',
        'Number Of Voxels': f'{prefix}_voxels',
        'Image mean (brain_t1c)': f'{prefix}_t1c_mean',
        'Image stdev (brain_t1c)': f'{prefix}_t1c_std',
        'Image mean (brain_t1n)': f'{prefix}_t1n_mean',
    })
```



```

        'Image stdev (brain_t1n)': f'{prefix}_t1n_std',
        'Image mean (brain_t2f)': f'{prefix}_t2f_mean',
        'Image stdev (brain_t2f)': f'{prefix}_t2f_std',
        'Image mean (brain_t2w)': f'{prefix}_t2w_mean',
        'Image stdev (brain_t2w)': f'{prefix}_t2w_std',
    })
    df_sheet['_sheet'] = prefix
    all_sheet_dfs.append(df_sheet)

# Each sheet has multiple rows per patient (one per timepoint).
# We need to compute TOTAL tumour volume per patient-timepoint by summing
# subregion volumes across sheets.
#
# Strategy: for each sheet, assign a timepoint index (row order within patient),
# then pivot so we can sum volumes across subregions at each timepoint.

print()
print("Computing total tumour volume per patient-timepoint...")
print("-" * 60)

# For each sheet, number the timepoints per patient (0, 1, 2, ...)
for df_sheet in all_sheet_dfs:
    df_sheet['tp_idx'] = df_sheet.groupby('Patient ID').cumcount()

# Collect volume per (Patient ID, tp_idx) from each sheet
volume_frames = []
for df_sheet, (sheet_name, prefix) in zip(all_sheet_dfs, sheets):
    vol_col = f'{prefix}_volume'
    sub = df_sheet[['Patient ID', 'tp_idx', vol_col]].copy()
    volume_frames.append(sub)

# Merge all subregion volumes on (Patient ID, tp_idx)
merged_vols = volume_frames[0]
for vf in volume_frames[1:]:
    merged_vols = merged_vols.merge(vf, on=['Patient ID', 'tp_idx'], how='outer')

# Compute total volume = sum of all subregion volumes (NaN treated as 0)
vol_cols = [f'{prefix}_volume' for _, prefix in sheets]
merged_vols['total_volume'] = merged_vols[vol_cols].fillna(0).sum(axis=1)

print(f"\nTotal volume records: {len(merged_vols)}")
print(f"Unique patients: {merged_vols['Patient ID'].nunique()}")
print(f"\nTimepoints per patient:")
print(merged_vols.groupby('Patient ID')['tp_idx'].max().value_counts().
    ↪sort_index().rename(
        index=lambda x: f"{x+1} timepoint(s)"))

```

```

# Also collect intensity features from the edema sheet (best coverage)
# for use as first-timepoint predictors
edema_df = all_sheet_dfs[1] # Tumour Infiltration and Edema

# Store for later use
seg_df = merged_vols.copy()

```

Available sheets: ['Necrotic Tumor Core (Label1)', 'Tumor Infiltration and Edema', 'Enhancing Tumor Core (Label3)', 'Resection Cavity (Label4)']

Necrotic Tumor Core (Label1): 334 rows, 173 unique patients

Tumor Infiltration and Edema: 614 rows, 321 unique patients

Enhancing Tumor Core (Label3): 569 rows, 286 unique patients

Resection Cavity (Label4): 542 rows, 291 unique patients

Computing total tumour volume per patient-timepoint...

Total volume records: 617

Unique patients: 324

Timepoints per patient:

tp_idx

1 timepoint(s) 200

2 timepoint(s) 52

3 timepoint(s) 26

4 timepoint(s) 15

5 timepoint(s) 13

6 timepoint(s) 17

8 timepoint(s) 1

Name: count, dtype: int64

1.3 Construct Growth Rate Target

For each patient with at least 2 timepoints, we compute the growth rate between their first two scans.

Formula: $\text{growth_rate} = (\text{volume_t1} - \text{volume_t0}) / \text{days_elapsed}$

Units: mm^3/day . A positive value means the tumour grew; negative means it shrank (e.g. post-treatment response).

We use only the first pair of timepoints ($t_0 \rightarrow t_1$) to keep the prediction task clean: predict growth from baseline.

```
[10]: # Check how many timepoints each patient has (using the merged total volumes)
tp_counts = seg_df.groupby('Patient ID')['tp_idx'].max() + 1 # tp_idx is 0-based

print(f"Patients with 1 timepoint: {(tp_counts == 1).sum()}")
print(f"Patients with 2+ timepoints: {(tp_counts >= 2).sum()}")
print(f"\nTimepoint distribution:")
print(tp_counts.value_counts().sort_index())
```

```
Patients with 1 timepoint: 200
Patients with 2+ timepoints: 124
```

```
Timepoint distribution:
```

```
tp_idx
```

```
1    200
2     52
3     26
4     15
5     13
6     17
8      1
```

```
Name: count, dtype: int64
```

1.3.1 Extract First Two Timepoints Per Patient

For each patient with 2+ timepoints, we extract the **total tumour volume** (summed across all four subregions) at timepoints 0 and 1. The growth target is the volume difference.

We also extract individual subregion volumes at t0 as features (necrotic, edema, enhancing, cavity), plus intensity statistics from the edema sheet (which has the best patient coverage at 321 unique patients).

```
[11]: # Get patients with at least 2 timepoints
multi_tp_patients = tp_counts[tp_counts >= 2].index.tolist()
print(f"Patients with 2+ timepoints: {len(multi_tp_patients)}")

# Extract first two timepoints per patient using TOTAL tumour volume
growth_records = []

for pid in multi_tp_patients:
    patient_vols = seg_df[seg_df['Patient ID'] == pid].sort_values('tp_idx')

    t0 = patient_vols[patient_vols['tp_idx'] == 0]
    t1 = patient_vols[patient_vols['tp_idx'] == 1]

    if len(t0) == 0 or len(t1) == 0:
        continue

    vol_t0 = t0['total_volume'].values[0]
```

```

vol_t1 = t1['total_volume'].values[0]
delta_volume = vol_t1 - vol_t0

# Get first-timepoint imaging features from edema sheet (best coverage)
edema_t0 = edema_df[(edema_df['Patient ID'] == pid) & (edema_df['tp_idx'] == 0)]

record = {
    'Patient ID': pid,
    'vol_t0': vol_t0,
    'vol_t1': vol_t1,
    'delta_volume': delta_volume,
}

# Add subregion volumes at t0 as features
for _, prefix in sheets:
    vcol = f'{prefix}_volume'
    t0_sub = seg_df[(seg_df['Patient ID'] == pid) & (seg_df['tp_idx'] == 0)]
    record[f't0_{prefix}_vol'] = t0_sub[vcol].values[0] if vcol in t0_sub.
    columns and len(t0_sub) > 0 and pd.notna(t0_sub[vcol].values[0]) else 0.0

# Add intensity features from edema sheet at t0
if len(edema_t0) > 0:
    record['t0_t1c_mean'] = edema_t0['edema_t1c_mean'].values[0]
    record['t0_t1c_std'] = edema_t0['edema_t1c_std'].values[0]
    record['t0_t1n_mean'] = edema_t0['edema_t1n_mean'].values[0]
    record['t0_t1n_std'] = edema_t0['edema_t1n_std'].values[0]
    record['t0_t2f_mean'] = edema_t0['edema_t2f_mean'].values[0]
    record['t0_t2f_std'] = edema_t0['edema_t2f_std'].values[0]
    record['t0_t2w_mean'] = edema_t0['edema_t2w_mean'].values[0]
    record['t0_t2w_std'] = edema_t0['edema_t2w_std'].values[0]
else:
    # Patient not in edema sheet; fill with NaN
    for feat in ['t0_t1c_mean', 't0_t1c_std', 't0_t1n_mean', 't0_t1n_std',
                't0_t2f_mean', 't0_t2f_std', 't0_t2w_mean', 't0_t2w_std']:
        record[feat] = np.nan

growth_records.append(record)

growth_df = pd.DataFrame(growth_records)
print(f"\nGrowth data constructed: {growth_df.shape}")
print(f"\nTotal volume at t0 statistics:")
print(growth_df['vol_t0'].describe())
print(f"\nDelta volume statistics (total tumour):")
print(growth_df['delta_volume'].describe())
print(f"\nMissing values:")
print(growth_df.isnull().sum()[growth_df.isnull().sum() > 0])

```

Patients with 2+ timepoints: 124

Growth data constructed: (124, 16)

Total volume at t0 statistics:

```
count      124.000000
mean       82802.338710
std        57731.074726
min        4035.000000
25%       37460.500000
50%       67466.000000
75%      124913.250000
max       257791.000000
```

Name: vol_t0, dtype: float64

Delta volume statistics (total tumour):

```
count      124.000000
mean       1078.419355
std        59691.292710
min       -160366.000000
25%       -29025.500000
50%           0.000000
75%       19545.000000
max       211791.000000
```

Name: delta_volume, dtype: float64

Missing values:

Series([], dtype: int64)

```
[12]: growth_df.head()
```

```
[12]:
```

	Patient ID	vol_t0	vol_t1	delta_volume	t0_necrotic_vol	\
0	PatientID_0003	84540.0	85818.0	1278.0	6510.0	
1	PatientID_0005	139453.0	140311.0	858.0	2946.0	
2	PatientID_0006	166060.0	83049.0	-83011.0	289.0	
3	PatientID_0007	48582.0	48109.0	-473.0	0.0	
4	PatientID_0008	87128.0	31276.0	-55852.0	0.0	

	t0_edema_vol	t0_enhancing_vol	t0_cavity_vol	t0_t1c_mean	t0_t1c_std	\
0	44251.0	33779.0	0.0	353.094610	19.368804	
1	93727.0	42780.0	0.0	396.366261	22.758249	
2	130618.0	2594.0	32559.0	480.454499	52.272928	
3	27824.0	7787.0	12971.0	242.640305	27.610877	
4	63795.0	7116.0	16217.0	795.804736	58.301609	

	t0_t1n_mean	t0_t1n_std	t0_t2f_mean	t0_t2f_std	t0_t2w_mean	t0_t2w_std
0	296.371803	23.638734	427.936617	50.434048	455.942745	72.198390

1	260.544393	86.856273	500.307445	73.222999	542.475826	99.106971
2	461.919794	50.122729	495.909582	58.572463	580.630798	115.549675
3	235.318865	26.929720	405.486114	56.008831	446.548813	85.899901
4	338.621425	34.305999	513.265279	76.811628	545.441349	103.907591

1.3.2 Estimate Days Between Scans

The segmentation volumes file does not include scan dates directly. We attempt to estimate inter-scan intervals from the clinical data columns that contain temporal information.

If exact dates are unavailable, we use the volume change itself (delta_volume in mm³) as the regression target rather than a rate. This is still clinically meaningful: predicting how much a tumour will change between consecutive scans.

```
[13]: # Check if we have any date-related columns in the clinical data
date_cols = [col for col in high_fill_df.columns if 'date' in col.lower() or
↳ 'day' in col.lower() or 'time' in col.lower()]
print("Date/time-related columns in clinical data:")
for col in date_cols:
    print(f" - {col}: {high_fill_df[col].dtype}")
    if high_fill_df[col].dtype in ['float64', 'int64']:
        print(f"     Range: {high_fill_df[col].min():.0f} - {high_fill_df[col].
↳ max():.0f}")
```

Date/time-related columns in clinical data:

- time to first progression (days): float64
Range: 4 - 2126
- number of days from diagnosis to first surgery or procedure: int64
Range: -81 - 116
- number of days from diagnosis to date of first progression: float64
Range: -22 - 2126
- number of days from diagnosis to 1st mri (timepoint_1): float64
Range: -58 - 464

```
[14]: # Exact inter-scan intervals are not available in the clinical data.
# We have days-to-first-MRI but not days-to-second-MRI,
# so we cannot compute a rate (mm³/day).
# We use absolute volume change (mm³) as the target instead.
# This is still clinically meaningful: predicting how much a
# tumour's volume will change between consecutive scans.

TARGET_COL = 'delta_volume'

print(f"Target variable: {TARGET_COL}")
print(f"\nTarget distribution:")
print(growth_df[TARGET_COL].describe())
print(f"\nPatients with tumour growth (positive delta): {(growth_df[TARGET_COL]
↳ > 0).sum()}")
```

```
print(f"Patients with tumour shrinkage (negative delta): {(growth_df[TARGET_COL]
    < 0).sum()}")
print(f"Patients with no change: {(growth_df[TARGET_COL] == 0).sum()}")
```

Target variable: delta_volume

Target distribution:

```
count      124.000000
mean       1078.419355
std        59691.292710
min       -160366.000000
25%       -29025.500000
50%         0.000000
75%        19545.000000
max        211791.000000
Name: delta_volume, dtype: float64
```

Patients with tumour growth (positive delta): 61

Patients with tumour shrinkage (negative delta): 58

Patients with no change: 5

1.3.3 Visualise Target Distribution

```
[15]: plt.rcParams.update({'font.size': 24})

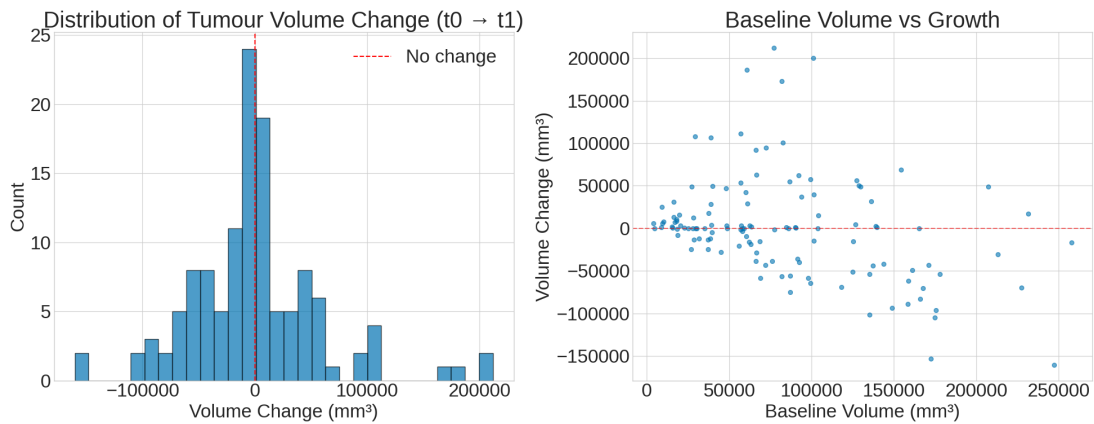
fig, axes = plt.subplots(1, 2, figsize=(20, 8))

# Histogram
axes[0].hist(growth_df[TARGET_COL], bins=30, edgecolor='black', alpha=0.7)
axes[0].axvline(x=0, color='red', linestyle='--', label='No change')
axes[0].set_xlabel('Volume Change (mm³)')
axes[0].set_ylabel('Count')
axes[0].set_title('Distribution of Tumour Volume Change (t0 → t1)')
axes[0].legend()

# Baseline volume vs change
axes[1].scatter(growth_df['vol_t0'], growth_df[TARGET_COL], alpha=0.6, s=30)
axes[1].axhline(y=0, color='red', linestyle='--', alpha=0.5)
axes[1].set_xlabel('Baseline Volume (mm³)')
axes[1].set_ylabel('Volume Change (mm³)')
axes[1].set_title('Baseline Volume vs Growth')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_target_distribution.png'),
    dpi=150, bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_target_distribution.pdf'),
    bbox_inches='tight')
```

```
plt.show()
```



1.4 Merge Clinical and Imaging Features

We merge the first-timepoint imaging features with the clinical feature matrix. The clinical features come from the preprocessed X matrix (leakage features already removed in Notebook 02).

We align on Patient ID, keeping only patients present in both datasets.

```
[16]: # We need to align Patient IDs between clinical and growth data
# Check the ID format in both
print("Clinical data ID sample:", high_fill_df.index[:3].tolist() if
      ↪high_fill_df.index.name else 'Index not named')
print("Growth data ID sample:", growth_df['Patient ID'].iloc[:3].tolist())

# Check if clinical data has patient IDs as index or column
clinical_id_cols = [col for col in high_fill_df.columns if 'patient' in col.
                    ↪lower() or 'id' in col.lower()]
print(f"\nPotential ID columns in clinical data: {clinical_id_cols}")
print(f"Index name: {high_fill_df.index.name}")
```

Clinical data ID sample: Index not named

Growth data ID sample: ['PatientID_0003', 'PatientID_0005', 'PatientID_0006']

Potential ID columns in clinical data: ['idh1 mutation', 'idh2 mutation']

Index name: None

```
[17]: # The preprocessed X matrix doesn't have Patient_ID (dropped during encoding).
# We need to get Patient IDs from the original clinical data.
# Load the original clinical CSV to get the ID mapping.

MU_GLIOMA_CLINICAL_DATA = os.path.join(GLIOMA_DIR,
    ↪'MU_Glioma_Post_ClinicalData_July2025.xlsx')
```



```

original_clinical = pd.read_excel(MU_GLIOMA_CLINICAL_DATA, sheet_name='MU Glioma_
↳Post')

# Check format
print("Original clinical ID sample:", original_clinical['Patient_ID'].iloc[:3].
↳tolist())
print("Growth data ID sample:", growth_df['Patient ID'].iloc[:3].tolist())

# The original clinical df has same row order as high_fill_df/X
# Add Patient ID back to clinical features
clinical_for_merge = high_fill_df.copy()
clinical_for_merge['Patient_ID'] = original_clinical['Patient_ID'].values
clinical_for_merge = clinical_for_merge.set_index('Patient_ID')

clean_feature_cols = X.columns.tolist()
print(f"Clean clinical features: {len(clean_feature_cols)}")

```

Original clinical ID sample: ['PatientID_0003', 'PatientID_0004',
'PatientID_0005']
Growth data ID sample: ['PatientID_0003', 'PatientID_0005', 'PatientID_0006']
Clean clinical features: 28

```

[18]: # Build the combined feature matrix
# Start with growth_df (has Patient ID + imaging features + target)
# Add clinical features by matching Patient ID

imaging_feature_cols = [
    'vol_t0',
    't0_necrotic_vol', 't0_edema_vol', 't0_enhancing_vol', 't0_cavity_vol',
    't0_t1c_mean', 't0_t1c_std',
    't0_t1n_mean', 't0_t1n_std', 't0_t2f_mean',
    't0_t2f_std', 't0_t2w_mean', 't0_t2w_std'
]

# Attempt merge using left_on and right_on for different column names
growth_with_clinical = growth_df.merge(
    clinical_for_merge[clean_feature_cols].reset_index(),
    left_on='Patient ID',
    right_on='Patient_ID',
    how='inner'
)

# Fill NaN imaging features with 0 (missing subregion = no volume for that_
↳region)
for col in imaging_feature_cols:
    if col in growth_with_clinical.columns:
        growth_with_clinical[col] = growth_with_clinical[col].fillna(0)

```

```

print(f"Merged dataset: {growth_with_clinical.shape}")
print(f"Patients retained: {len(growth_with_clinical)} / {len(growth_df)} (from_
↳growth data)")
print(f"Lost to ID mismatch: {len(growth_df) - len(growth_with_clinical)}")

```

Merged dataset: (105, 45)
 Patients retained: 105 / 124 (from growth data)
 Lost to ID mismatch: 19

```

[19]: # Define feature sets for comparison
all_feature_cols = clean_feature_cols + imaging_feature_cols

X_combined = growth_with_clinical[all_feature_cols].copy()
X_clinical_only = growth_with_clinical[clean_feature_cols].copy()
X_imaging_only = growth_with_clinical[imaging_feature_cols].copy()
y_growth = growth_with_clinical[TARGET_COL].copy()

print(f"Feature sets:")
print(f"  Clinical only: {X_clinical_only.shape[1]} features")
print(f"  Imaging only: {X_imaging_only.shape[1]} features")
print(f"  Combined: {X_combined.shape[1]} features")
print(f"  Target: {y_growth.shape[0]} samples")

```

Feature sets:
 Clinical only: 28 features
 Imaging only: 13 features
 Combined: 41 features
 Target: 105 samples

```

[20]: # Remove leakage features that encode future outcomes
# These were identified in Notebook 02 but may persist in the preprocessed data
leakage_features = [
    'second progression/recurrence',
    'type of 2nd progression',
    'time to first progression (days)',
    'number of days from diagnosis to date of first progression',
    'number of days from diagnosis to first surgery or procedure',
    'progression',
    'type of 1st progression'
]

if leakage_features:
    print(f"Removing {len(leakage_features)} leakage features:")
    for f in leakage_features:
        print(f"  - {f}")

    X_combined = X_combined.drop(columns=leakage_features)

```

```

X_clinical_only = X_clinical_only.drop(columns=[f for f in leakage_features_
↪if f in X_clinical_only.columns])
    clean_feature_cols = [c for c in clean_feature_cols if c not in_
↪leakage_features]

    print(f"\nUpdated feature sets:")
    print(f"    Clinical only: {X_clinical_only.shape[1]} features")
    print(f"    Imaging only: {X_imaging_only.shape[1]} features")
    print(f"    Combined: {X_combined.shape[1]} features")
else:
    print("No leakage features found.")

```

Removing 7 leakage features:

- second progression/recurrence
- type of 2nd progression
- time to first progression (days)
- number of days from diagnosis to date of first progression
- number of days from diagnosis to first surgery or procedure
- progression
- type of 1st progression

Updated feature sets:

Clinical only: 21 features
 Imaging only: 13 features
 Combined: 34 features

1.5 Handle Outliers

Growth rate data may contain extreme outliers (e.g. very large tumour resections appearing as massive negative changes). We inspect the distribution and clip or remove outliers if necessary.

```

[21]: # Check for extreme values
q01 = y_growth.quantile(0.01)
q99 = y_growth.quantile(0.99)
iqr = y_growth.quantile(0.75) - y_growth.quantile(0.25)
lower_fence = y_growth.quantile(0.25) - 3 * iqr
upper_fence = y_growth.quantile(0.75) + 3 * iqr

outliers = (y_growth < lower_fence) | (y_growth > upper_fence)
print(f"Outlier detection (3x IQR):")
print(f"    Lower fence: {lower_fence:.0f} mm3")
print(f"    Upper fence: {upper_fence:.0f} mm3")
print(f"    Outliers: {outliers.sum()} ({outliers.mean()*100:.1f}%)")

if outliers.sum() > 0:
    print(f"\nOutlier values:")
    print(y_growth[outliers].values)

```

Outlier detection (3x IQR):

Lower fence: -253499 mm³

Upper fence: 245111 mm³

Outliers: 0 (0.0%)

```
[22]: # Decision: clip extreme outliers rather than removing them to preserve sample_
      ↪size
      # This is important given we already have a small dataset (~82 patients)
      if outliers.sum() > 0:
          y_growth_clipped = y_growth.clip(lower=lower_fence, upper=upper_fence)
          print(f"Clipped {outliers.sum()} outliers to [{lower_fence:.0f},
          ↪{upper_fence:.0f}]")
      else:
          y_growth_clipped = y_growth.copy()
          print("No outliers detected; using original values.")
```

No outliers detected; using original values.

1.6 Model Training and Evaluation

We train regression models to predict tumour volume change from baseline features. Given the small sample size (~82 patients), we use **Repeated K-Fold cross-validation** (5 folds, 5 repeats = 25 evaluations) to get robust performance estimates.

We compare: 1. **Linear Regression** (baseline) 2. **Ridge Regression** (L2 regularisation; better for small datasets with correlated features) 3. **XGBRegressor** (gradient boosted trees; consistent with our classification pipeline)

For each, we test three feature sets: clinical only, imaging only, and combined.

Metrics: R², MAE, RMSE.

```
[23]: def evaluate_regression(X, y, model, model_name, cv):
      """Run cross-validated regression and return summary metrics."""
      scoring = ['r2', 'neg_mean_absolute_error', 'neg_root_mean_squared_error']
      results = cross_validate(model, X, y, cv=cv, scoring=scoring,
      ↪return_train_score=False)

      r2 = results['test_r2']
      mae = -results['test_neg_mean_absolute_error']
      rmse = -results['test_neg_root_mean_squared_error']

      return {
          'model': model_name,
          'R2_mean': r2.mean(), 'R2_std': r2.std(),
          'MAE_mean': mae.mean(), 'MAE_std': mae.std(),
          'RMSE_mean': rmse.mean(), 'RMSE_std': rmse.std(),
      }
```

```
[24]: cv = RepeatedKFold(n_splits=5, n_repeats=5, random_state=42)
```

```
models = {  
    'Linear Regression': LinearRegression(),  
    'Ridge': Ridge(alpha=1.0),  
    'XGBRegressor': XGBRegressor(  
        n_estimators=100,  
        max_depth=3,  
        learning_rate=0.1,  
        subsample=0.8,  
        colsample_bytree=0.8,  
        random_state=42,  
        verbosity=0  
    ),  
}  
  
feature_sets = {  
    'Clinical Only': X_clinical_only,  
    'Imaging Only': X_imaging_only,  
    'Combined': X_combined,  
}
```

```
[25]: # Run all model-feature set combinations  
all_results = []  
  
for feat_name, X_feat in feature_sets.items():  
    # One-hot encode categorical features for the current feature set  
    # This is necessary because X.pkl still contains object type columns  
    categorical_cols = X_feat.select_dtypes(include='object').columns  
    if len(categorical_cols) > 0:  
        X_encoded = pd.get_dummies(X_feat, columns=categorical_cols,   
↳ drop_first=True)  
    else:  
        X_encoded = X_feat.copy()  
  
    # Handle remaining missing values (for numerical features) by filling with   
↳ median  
    X_clean = X_encoded.fillna(X_encoded.median())  
  
    # Ensure all column names are strings for XGBoost compatibility  
    X_clean.columns = X_clean.columns.astype(str)  
  
    for model_name, model in models.items():  
        label = f"{model_name} ({feat_name})"  
        print(f"Evaluating: {label}...")  
        result = evaluate_regression(X_clean, y_growth_clipped, model, label, cv)  
        result['feature_set'] = feat_name
```

```

        result['algorithm'] = model_name
        all_results.append(result)

results_df = pd.DataFrame(all_results)
print("\nDone.")

```

Evaluating: Linear Regression (Clinical Only)...

Evaluating: Ridge (Clinical Only)...

Evaluating: XGBRegressor (Clinical Only)...

Evaluating: Linear Regression (Imaging Only)...

Evaluating: Ridge (Imaging Only)...

Evaluating: XGBRegressor (Imaging Only)...

Evaluating: Linear Regression (Combined)...

Evaluating: Ridge (Combined)...

Evaluating: XGBRegressor (Combined)...

Done.

1.6.1 Results Comparison

[26]: results_df.head()

[26]:

	model	R2_mean	R2_std	MAE_mean	\
0	Linear Regression (Clinical Only)	-0.970501	1.280880	61763.796076	
1	Ridge (Clinical Only)	-0.293475	0.298486	54619.939032	
2	XGBRegressor (Clinical Only)	-0.608246	0.752175	59634.159306	
3	Linear Regression (Imaging Only)	-0.092961	0.249556	50033.655706	
4	Ridge (Imaging Only)	-0.092931	0.249550	50033.064875	

	MAE_std	RMSE_mean	RMSE_std	feature_set	algorithm
0	11382.464724	82388.783481	18281.898716	Clinical Only	Linear Regression
1	6914.194816	70173.119106	8531.457079	Clinical Only	Ridge
2	7548.502133	76567.248599	8872.195306	Clinical Only	XGBRegressor
3	7630.109898	64891.644442	10400.577872	Imaging Only	Linear Regression
4	7629.485357	64890.704532	10400.076803	Imaging Only	Ridge

[27]:

```

# Format results table
print("GROWTH RATE PREDICTION RESULTS")
print("-" * 90)
print(f"{'Model':<38} {'R²':>10} {'MAE (mm³)':>17} {'RMSE (mm³)':>14}")
print("-" * 90)

for _, row in results_df.iterrows():
    r2_str = f"{row['R2_mean']:.3f} ± {row['R2_std']:.3f}"
    mae_str = f"{row['MAE_mean']:.0f} ± {row['MAE_std']:.0f}"
    rmse_str = f"{row['RMSE_mean']:.0f} ± {row['RMSE_std']:.0f}"
    print(f"{row['model']:<38} {r2_str:>15} {mae_str:>14} {rmse_str:>14}")

```

GROWTH RATE PREDICTION RESULTS

Model (mm ³)	R ²	MAE (mm ³)	RMSE
Linear Regression (Clinical Only) 18282	-0.971 ± 1.281	61764 ± 11382	82389 ±
Ridge (Clinical Only) 8531	-0.293 ± 0.298	54620 ± 6914	70173 ±
XGBRegressor (Clinical Only) 8872	-0.608 ± 0.752	59634 ± 7549	76567 ±
Linear Regression (Imaging Only) 10401	-0.093 ± 0.250	50034 ± 7630	64892 ±
Ridge (Imaging Only) 10400	-0.093 ± 0.250	50033 ± 7629	64891 ±
XGBRegressor (Imaging Only) 8919	-0.247 ± 0.307	51167 ± 7419	68902 ±
Linear Regression (Combined) 20365	-1.067 ± 1.307	61861 ± 11211	84074 ±
Ridge (Combined) 9035	-0.369 ± 0.447	54289 ± 6552	71465 ±
XGBRegressor (Combined) 8330	-0.132 ± 0.289	49445 ± 6814	65525 ±

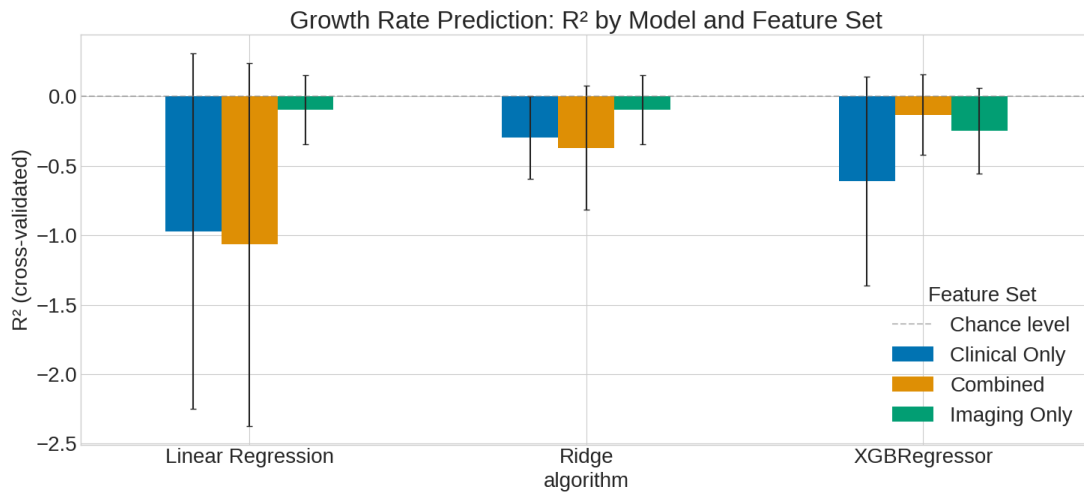
```
[28]: # Visualise: grouped bar chart of R2 by model and feature set
plt.rcParams.update({'font.size': 20})
fig, ax = plt.subplots(figsize=(15, 7))

pivot = results_df.pivot(index='algorithm', columns='feature_set',
    ↪ values='R2_mean')
pivot_err = results_df.pivot(index='algorithm', columns='feature_set',
    ↪ values='R2_std')

pivot.plot(kind='bar', yerr=pivot_err, ax=ax, capsize=3)
ax.set_ylabel('R2 (cross-validated)')
ax.set_title('Growth Rate Prediction: R2 by Model and Feature Set')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
ax.axhline(y=0, color='grey', linestyle='--', alpha=0.5, label='Chance level')
ax.legend(title='Feature Set')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_r2_comparison.png'), dpi=150,
    ↪ bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_r2_comparison.pdf'),
    ↪ bbox_inches='tight')
```

```
plt.show()
```



1.7 Feature Importance (Best Model)

We inspect which features drive the growth rate prediction in the best-performing model.

```
[29]: # Identify best model configuration
best_idx = results_df['R2_mean'].idxmax()
best_row = results_df.iloc[best_idx]
print(f"Best model: {best_row['model']}")
print(f"  R² = {best_row['R2_mean']:.3f} ± {best_row['R2_std']:.3f}")
print(f"  MAE = {best_row['MAE_mean']:.0f} ± {best_row['MAE_std']:.0f} mm³")
```

Best model: Ridge (Imaging Only)

$R^2 = -0.093 \pm 0.250$

MAE = 50033 ± 7629 mm³

```
[30]: # Train the best XGBRegressor on full data for feature importance
best_feat_set = best_row['feature_set']
X_best_raw = feature_sets[best_feat_set].copy()

# Apply the same preprocessing as in run_models
categorical_cols = X_best_raw.select_dtypes(include='object').columns
if len(categorical_cols) > 0:
    X_best_encoded = pd.get_dummies(X_best_raw, columns=categorical_cols,
    ↪ drop_first=True)
else:
    X_best_encoded = X_best_raw.copy()

# Handle remaining missing values (for numerical features) by filling with median
```



```

X_best = X_best_encoded.fillna(X_best_encoded.median())

# Ensure all column names are strings for XGBoost compatibility
X_best.columns = X_best.columns.astype(str)

xgb_final = XGBRegressor(
    n_estimators=100, max_depth=3, learning_rate=0.1,
    subsample=0.8, colsample_bytree=0.8, random_state=42, verbosity=0
)
xgb_final.fit(X_best, y_growth_clipped)

# Feature importance
importance = pd.Series(xgb_final.feature_importances_, index=X_best.columns)
importance = importance.sort_values(ascending=False)

print(f"\nTop 15 features for growth rate prediction ({best_feat_set}):")
print("-" * 70)
for feat, imp in importance.head(15).items():
    print(f" {feat:<61} {imp:.4f}")

```

Top 15 features for growth rate prediction (Imaging Only):

```

-----
t0_edema_vol                0.1845
t0_t2f_mean                 0.1119
t0_t2w_mean                 0.0879
t0_t1n_std                  0.0817
t0_t2w_std                  0.0755
vol_t0                      0.0724
t0_cavity_vol               0.0679
t0_enhancing_vol            0.0677
t0_t2f_std                  0.0531
t0_necrotic_vol             0.0525
t0_t1c_std                  0.0503
t0_t1n_mean                 0.0490
t0_t1c_mean                 0.0454

```

```

[31]: # Plot feature importance
fig, ax = plt.subplots(figsize=(12, 6))

top_n = min(15, len(importance))
top_feats = importance.head(top_n)

# Colour imaging vs clinical features differently
colours = ['#d95f02' if f in imaging_feature_cols else '#1b9e77' for f in
    ↪top_feats.index]

```

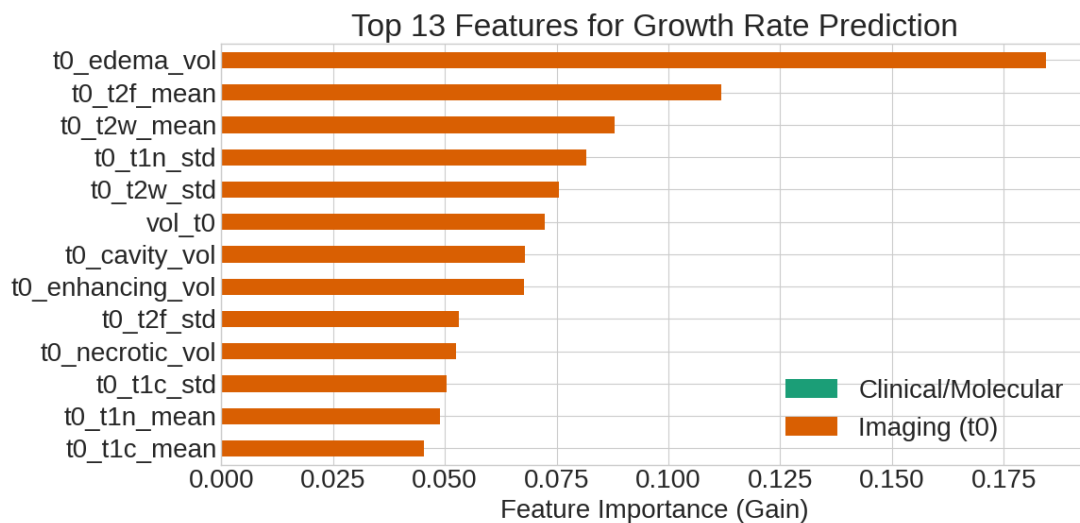
```

top_feats.plot(kind='barh', ax=ax, color=colours)
ax.set_xlabel('Feature Importance (Gain)')
ax.set_title(f'Top {top_n} Features for Growth Rate Prediction')
ax.invert_yaxis()

# Legend
from matplotlib.patches import Patch
legend_elements = [
    Patch(facecolor='#1b9e77', label='Clinical/Molecular'),
    Patch(facecolor='#d95f02', label='Imaging (t0)')
]
ax.legend(handles=legend_elements, loc='lower right')

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_feature_importance.png'),
            dpi=150, bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_feature_importance.pdf'),
            bbox_inches='tight')
plt.show()

```



1.8 Biological Validation: Growth Rate vs Survival

If our predicted growth rates are biologically meaningful, patients with faster-growing tumours should have worse survival outcomes. We stratify patients into growth tertiles and compare their survival status as a sanity check.

```

[32]: # Merge survival outcome with growth data
      # The original 'y' series has numerical index, while growth_with_clinical has
      # 'Patient ID'.

```

```

# We need to map Patient IDs from original_clinical to the 'death' values from y.
survival_lookup = pd.Series(y.values, index=original_clinical['Patient_ID']).
    ↳to_dict()

growth_with_clinical['death'] = growth_with_clinical['Patient ID'].
    ↳map(survival_lookup)

# Check alignment
print(f"Patients with survival data: {growth_with_clinical['death'].notna().
    ↳sum()} / {len(growth_with_clinical)}")

```

Patients with survival data: 105 / 105

```

[33]: # Stratify into growth tertiles
growth_with_clinical['growth_tertile'] = pd.qcut(
    growth_with_clinical[TARGET_COL], q=3,
    labels=['Slow/Shrinking', 'Moderate', 'Fast Growing']
)

# Mortality rate per tertile
tertile_mortality = growth_with_clinical.groupby('growth_tertile')['death'].
    ↳agg(['mean', 'count'])
tertile_mortality.columns = ['Mortality Rate', 'N']
print("Mortality Rate by Growth Tertile:")
print("-" * 45)
for idx, row in tertile_mortality.iterrows():
    print(f"  {idx:<20} {row['Mortality Rate']:.1%}  (n={row['N']:.0f})")

```

Mortality Rate by Growth Tertile:

```

-----
Slow/Shrinking      68.6%  (n=35)
Moderate             60.0%  (n=35)
Fast Growing       62.9%  (n=35)

```

```

[34]: # Visualise
fig, ax = plt.subplots(figsize=(7, 4))

tertile_mortality['Mortality Rate'].plot(
    kind='bar', ax=ax, color=['#2ca02c', '#ff7f0e', '#d62728'], edgecolor='black'
)
ax.set_ylabel('Mortality Rate')
ax.set_title('Mortality Rate by Tumour Growth Tertile')
ax.set_xticklabels(ax.get_xticklabels(), rotation=0)
ax.set_ylim(0, 1)

# Add count labels
for i, (idx, row) in enumerate(tertile_mortality.iterrows()):
    ax.text(i, row['Mortality Rate'] + 0.03, f"n={row['N']:.0f}",

```

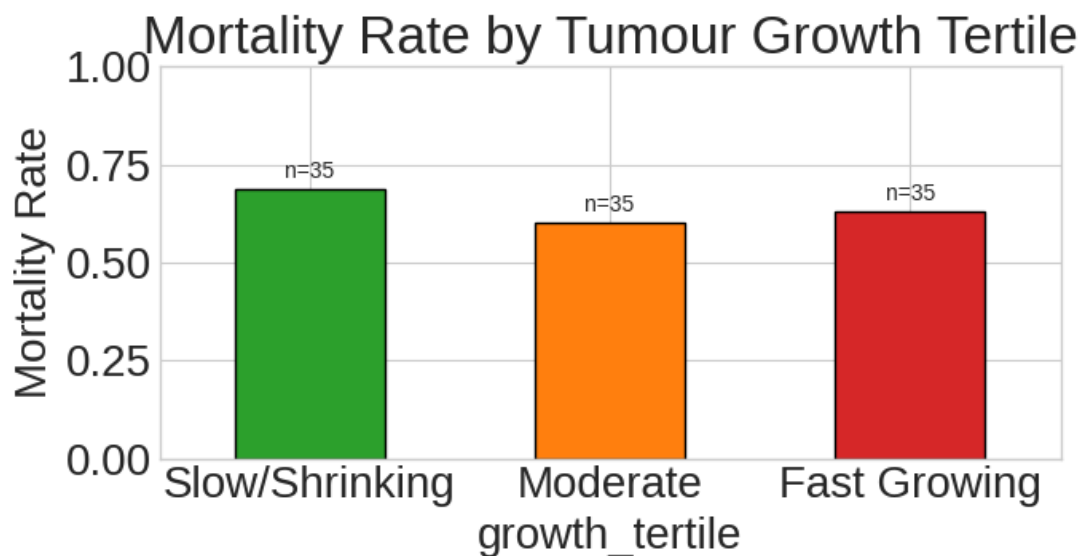
```

        ha='center', fontsize=10)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_vs_mortality.png'), dpi=150,
    ↳bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '05_growth_vs_mortality.pdf'),
    ↳bbox_inches='tight')
plt.show()

# Chi-squared test
contingency = pd.crosstab(growth_with_clinical['growth_tertile'],
    ↳growth_with_clinical['death'])
chi2, p_val, dof, expected = stats.chi2_contingency(contingency)
print(f"\nChi-squared test:  $\chi^2 = \{chi2:.2f\}$ ,  $p = \{p\_val:.4f\}$ ")
if p_val < 0.05:
    print("Significant association between growth tertile and mortality.")
else:
    print("No significant association detected (may be underpowered with small
    ↳n).")

```



Chi-squared test: $\chi^2 = 0.58$, $p = 0.7492$

No significant association detected (may be underpowered with small n).

1.9 Treatment Confounder Check

Testing the Section 5.5 / screencast claim that the slow/shrinking tertile's higher mortality reflects confounding by indication: aggressive tumours receive aggressive treatment; chemotherapy and

radiation shrink tumours; therefore shrinking patients had worse disease to begin with.

If that story holds, the shrinking tertile should have substantially higher chemo and radiation rates than the fast-growing tertile, especially for treatment that fell **between Timepoint 1 and Timepoint 2** (the window in which shrinkage is measured).

Initial Chemo Therapy is not universal in this cohort (129 Yes, 74 NaN across the full 203-patient dataset), so we have variation to test against. NaN is interpreted as *did not receive*, matching how the column is coded elsewhere.

1.9.1 Part A: Binary treatment rates by tertile

```
[24]: # Part A: did each patient ever receive each treatment?
from scipy.stats import chi2_contingency

tx_cols_raw = [
    'Patient_ID',
    'Initial Chemo Therapy',
    'Radiation Therapy',
    'Additional Therapy',
    ' Number of days from Diagnosis to Initial Chemo Therapy Start date',
    ' Number of days from Diagnosis to Initial Chemo Therapy end date',
    'Number of days from Diagnosis to Radiation Therapy Start date',
    'Number of days from Diagnosis to Radiation Therapy end date',
    'Number of Days from Diagnosis to 1st MRI (Timepoint_1) ',
    'Number of Days from Diagnosis to 2nd MRI (Timepoint_2) ',
]

tx_df = growth_with_clinical[['Patient ID', 'growth_tertile', 'death']].merge(
    original_clinical[tx_cols_raw],
    left_on='Patient ID', right_on='Patient_ID', how='left'
)

# Binary ever-indicators
tx_df['had_initial_chemo'] = (
    tx_df['Initial Chemo Therapy'].astype(str).str.strip().eq('Yes')
)
tx_df['had_rt'] = (
    tx_df['Radiation Therapy'].astype(str).str.strip().isin(['Yes', 'Proton_
↳Therapy'])
)
tx_df['had_additional_chemo'] = tx_df['Additional Therapy'].notna()

print(f'Merged cohort: {len(tx_df)} patients')
print()

def per_tertile_rates(df, flag_col):
```

```

    grouped = df.groupby('growth_tertile', observed=True)[flag_col].agg(['sum',
↪ 'count'])
    grouped['pct'] = grouped['sum'] / grouped['count'] * 100
    return grouped

print('PART A - binary treatment rates by tertile')
print('=' * 72)
for flag, label in [
    ('had_initial_chemo', 'Initial chemotherapy (ever)'),
    ('had_rt', 'Radiation therapy (ever)'),
    ('had_additional_chemo', 'Additional/maintenance chemo'),
]:
    print(f'\n{label}')
    print('-' * 60)
    tbl = per_tertile_rates(tx_df, flag)
    for tertile, row in tbl.iterrows():
        print(f' {str(tertile):<18} {int(row["sum"]):>3} / {int(row["count"]):
↪<3} ({row["pct"]:5.1f}%)')
        contingency = pd.crosstab(tx_df['growth_tertile'], tx_df[flag])
        chi2, p_chi2, _, _ = chi2_contingency(contingency)
        print(f'  chi2={chi2:.2f}, p={p_chi2:.4f}')

```

Merged cohort: 105 patients

PART A - binary treatment rates by tertile

=====

Initial chemotherapy (ever)

```

-----
Slow/Shrinking      30 / 35   ( 85.7%)
Moderate             31 / 35   ( 88.6%)
Fast Growing       29 / 35   ( 82.9%)
chi2=0.47, p=0.7919

```

Radiation therapy (ever)

```

-----
Slow/Shrinking      31 / 35   ( 88.6%)
Moderate             31 / 35   ( 88.6%)
Fast Growing       30 / 35   ( 85.7%)
chi2=0.18, p=0.9160

```

Additional/maintenance chemo

```

-----
Slow/Shrinking      23 / 35   ( 65.7%)
Moderate             28 / 35   ( 80.0%)
Fast Growing       20 / 35   ( 57.1%)
chi2=4.26, p=0.1187

```

1.9.2 Part B: Treatment overlapping the TP1 to TP2 window

Only treatment given between the two scans can mechanistically explain volume reduction observed at TP2. An interval [start, end] overlaps the window [mri1, mri2] iff start ≤ mri2 and end ≥ mri1. A missing end is treated as ongoing through TP2 (common when the end date was not yet recorded at extraction).

```
[25]: chemo_start = ' Number of days from Diagnosis to Initial Chemo Therapy Start_
      ↪date'
chemo_end   = ' Number of days from Diagnosis to Initial Chemo Therapy end date'
rt_start    = 'Number of days from Diagnosis to Radiation Therapy Start date'
rt_end      = 'Number of days from Diagnosis to Radiation Therapy end date'
mri1_day    = 'Number of Days from Diagnosis to 1st MRI (Timepoint_1) '
mri2_day    = 'Number of Days from Diagnosis to 2nd MRI (Timepoint_2) '

def overlaps_window(start, end, w0, w1):
    '''True iff [start, end] overlaps the MRI window [w0, w1].'''
    if pd.isna(start) or pd.isna(w0) or pd.isna(w1):
        return False
    if pd.isna(end):
        return start <= w1
    return (start <= w1) and (end >= w0)

tx_df['chemo_in_window'] = tx_df.apply(
    lambda r: overlaps_window(r[chemo_start], r[chemo_end], r[mri1_day], ↪
    ↪r[mri2_day]),
    axis=1,
)
tx_df['rt_in_window'] = tx_df.apply(
    lambda r: overlaps_window(r[rt_start], r[rt_end], r[mri1_day], r[mri2_day]),
    axis=1,
)

# Coverage check
has_both_mri = tx_df[mri1_day].notna() & tx_df[mri2_day].notna()
print(f'Patients with both MRI dates recorded: {has_both_mri.sum()} / ↪
      ↪{len(tx_df)}')
print(f'Patients missing TP1 or TP2 date:      {(~has_both_mri).sum()}')
print()

print('PART B - treatment overlapping the TP1-TP2 window')
print('=' * 72)
for flag, label in [
    ('chemo_in_window', 'Chemotherapy during TP1-TP2 window'),
    ('rt_in_window', 'Radiation during TP1-TP2 window'),
]:
    print(f'\n{label}')
    print('-' * 60)
```

```
tbl = per_tertile_rates(tx_df, flag)
for tertile, row in tbl.iterrows():
    print(f' {str(tertile):<18} {int(row["sum"]):>3} / {int(row["count"]):<3} ({row["pct"]:5.1f}%)')
    contingency = pd.crosstab(tx_df['growth_tertile'], tx_df[flag])
    chi2, p_chi2, _, _ = chi2_contingency(contingency)
    print(f' chi2={chi2:.2f}, p={p_chi2:.4f}')
```

Patients with both MRI dates recorded: 89 / 105

Patients missing TP1 or TP2 date: 16

PART B - treatment overlapping the TP1-TP2 window

Chemotherapy during TP1-TP2 window

```
-----
Slow/Shrinking      4 / 35   ( 11.4%)
Moderate             3 / 35   (  8.6%)
Fast Growing       5 / 35   ( 14.3%)
chi2=0.56, p=0.7541
```

Radiation during TP1-TP2 window

```
-----
Slow/Shrinking      4 / 35   ( 11.4%)
Moderate             4 / 35   ( 11.4%)
Fast Growing       5 / 35   ( 14.3%)
chi2=0.18, p=0.9160
```

1.9.3 Part C: Summary table, CSV, figure

```
[26]: # Part C: build summary DataFrame, save CSV, save figure
order = ['Slow/Shrinking', 'Moderate', 'Fast Growing']
summary_rows = []
for tertile in order:
    sub = tx_df[tx_df['growth_tertile'] == tertile]
    summary_rows.append({
        'tertile': tertile,
        'n': len(sub),
        'deaths': int(sub['death'].sum()),
        'mortality_pct': sub['death'].mean() * 100,
        'chemo_ever_n': int(sub['had_initial_chemo'].sum()),
        'chemo_ever_pct': sub['had_initial_chemo'].mean() * 100,
        'chemo_in_window_n': int(sub['chemo_in_window'].sum()),
        'chemo_in_window_pct': sub['chemo_in_window'].mean() * 100,
        'rt_ever_n': int(sub['had_rt'].sum()),
        'rt_ever_pct': sub['had_rt'].mean() * 100,
        'rt_in_window_n': int(sub['rt_in_window'].sum()),
```



```

        'rt_in_window_pct': sub['rt_in_window'].mean() * 100,
        'additional_chemo_n': int(sub['had_additional_chemo'].sum()),
        'additional_chemo_pct': sub['had_additional_chemo'].mean() * 100,
    })

summary_df = pd.DataFrame(summary_rows)
print('SUMMARY - treatment by growth tertile (105-patient cohort)')
print('=' * 90)
print(summary_df.to_string(index=False))

summary_path = os.path.join(RESULTS_DIR, '05_treatment_by_tertile.csv')
summary_df.to_csv(summary_path, index=False)
print(f'\nSaved summary to {summary_path}')

detail_path = os.path.join(RESULTS_DIR, '05_treatment_by_tertile_detail.csv')
tx_df.to_csv(detail_path, index=False)
print(f'Saved per-patient detail to {detail_path}')

# Two-panel figure
plt.rcParams.update({'font.size': 14})
fig, axes = plt.subplots(1, 2, figsize=(14, 5), sharey=True)

xpos = np.arange(len(order))
width = 0.35

for ax, (suffix, title) in zip(
    axes,
    [('_ever_pct', 'Ever received'), ('_in_window_pct', 'During TP1-TP2_
↳window')],
):
    chemo_vals = [
        summary_df.set_index('tertile').loc[t, f'chemo{suffix}'] for t in order
    ]
    rt_vals = [
        summary_df.set_index('tertile').loc[t, f'rt{suffix}'] for t in order
    ]
    ax.bar(xpos - width/2, chemo_vals, width, label='Chemotherapy',
           color='#1b9e77', edgecolor='black')
    ax.bar(xpos + width/2, rt_vals, width, label='Radiation',
           color='#d95f02', edgecolor='black')
    ax.set_xticks(xpos)
    ax.set_xticklabels(order)
    ax.set_ylabel('% of tertile')
    ax.set_title(title)
    ax.set_ylim(0, 115)
    for i, (c, r) in enumerate(zip(chemo_vals, rt_vals)):
        ax.text(i - width/2, c + 2, f'{c:.0f}%', ha='center', fontsize=10)

```

```

        ax.text(i + width/2, r + 2, f'{r:.0f}%', ha='center', fontsize=10)

fig.suptitle('Treatment receipt by tumour growth tertile (n=35 per tertile)',
            ↵y=1.02)
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels, loc='upper center', bbox_to_anchor=(0.5, 0.97),
            ncol=2, frameon=False)
plt.tight_layout(rect=[0, 0, 1, 0.93])
fig_path_pdf = os.path.join(FIGURES_DIR, '05_treatment_by_tertile.pdf')
fig_path_png = os.path.join(FIGURES_DIR, '05_treatment_by_tertile.png')
plt.savefig(fig_path_pdf, bbox_inches='tight')
plt.savefig(fig_path_png, dpi=150, bbox_inches='tight')
plt.show()
print(f'Saved figure to {fig_path_pdf}')

```

SUMMARY - treatment by growth tertile (105-patient cohort)

```

=====
=====

```

	tertile	n	deaths	mortality_pct	chemo_ever_n	chemo_ever_pct	chemo_in_window_n	chemo_in_window_pct	rt_ever_n	rt_ever_pct	rt_in_window_n	rt_in_window_pct	additional_chemo_n	additional_chemo_pct
Slow/Shrinking	35	24	68.571429		30	85.714286								
4		11.428571		31	88.571429		4				11.428571			
23		65.714286												
Moderate	35	21	60.000000		31	88.571429								
3		8.571429		31	88.571429		4				11.428571			
28		80.000000												
Fast Growing	35	22	62.857143		29	82.857143								
5		14.285714		30	85.714286		5				14.285714			
20		57.142857												

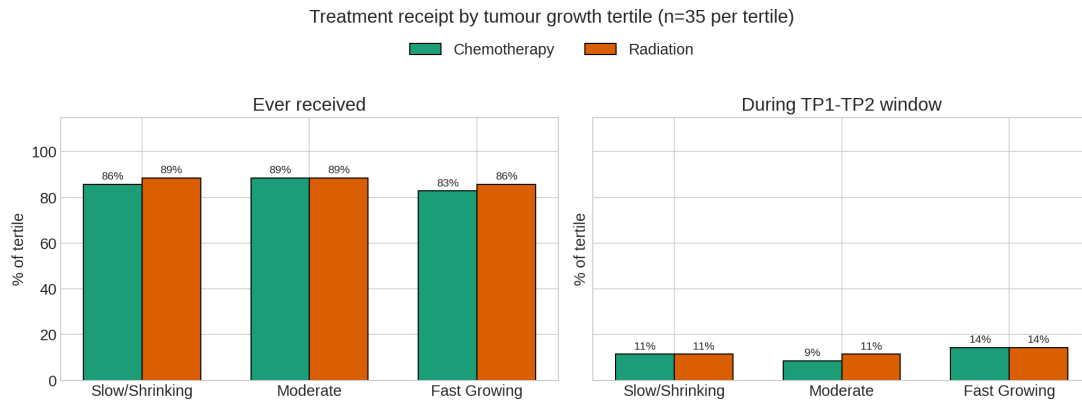
Saved summary to drive/MyDrive/Y3

Project/Data/PKG_MU_Glioma_Post/growth_rate/data/05_treatment_by_tertile.csv

Saved per-patient detail to drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/growth_rate/data/05_treatment_by_tertile_detail.csv

Saved figure to drive/MyDrive/Y3

Project/Data/PKG_MU_Glioma_Post/growth_rate/figures/05_treatment_by_tertile.pdf



1.9.4 Part D: Surgery check

The Section 5.5 narrative also cites “substantial surgical debulking” as a mechanism. We cannot test extent of resection directly — the clinical sheet has no GTR / STR / biopsy-only column — but we can test two proxies:

1. **Multiple surgeries** (binary): did the patient have more than one surgery ever?
2. **First surgery during the TP1-TP2 window**: if surgery fell between the two scans used to measure growth, the “shrinkage” on the TP2 mask may be a measurement artefact (tissue removed mechanically) rather than biological.

The second test is the stronger one. If it concentrates in the Slow/Shrinking tertile, the growth metric itself is confounded by surgical timing, not just the outcome.

Limitations: - Only the FIRST surgery date is recorded. For the 51 patients with multiple surgeries, subsequent surgery dates are unknown. - Extent of resection is not available.

```
[27]: # Part D: surgery proxies
surgery_day_col = 'Number of days from Diagnosis to First surgery or procedure '
multi_surg_col = 'Multiple surgeries'

surg_extra = original_clinical[['Patient_ID', surgery_day_col, multi_surg_col]].
    ↪rename(
        columns={
            surgery_day_col: 'days_to_first_surgery',
            multi_surg_col: 'multiple_surgeries_flag',
        }
    )

# Left-merge onto tx_df (already has growth_tertile, MRI days, Patient_ID)
surg_df = tx_df.merge(surg_extra, on='Patient_ID', how='left')

# D1: multiple surgeries ever (binary 0/1 already)
```

```

surg_df['had_multiple_surgeries'] = surg_df['multiple_surgeries_flag'].fillna(0).
↳astype(int).eq(1)

# D2: first surgery fell strictly between the two MRI days
def surgery_in_tp1_tp2(row):
    t1, t2 = row[mri1_day], row[mri2_day]
    s = row['days_to_first_surgery']
    if pd.isna(t1) or pd.isna(t2) or pd.isna(s):
        return False
    return (s >= t1) and (s <= t2)

surg_df['first_surgery_in_window'] = surg_df.apply(surgery_in_tp1_tp2, axis=1)

# Coverage
surg_known = surg_df['days_to_first_surgery'].notna().sum()
mri_both = (surg_df[mri1_day].notna() & surg_df[mri2_day].notna()).sum()
print(f'Patients with first-surgery day recorded: {surg_known} / {len(surg_df)}')
print(f'Patients with both TP1 and TP2 days:      {mri_both} / {len(surg_df)}')
print()

print('PART D - surgery proxies by growth tertile')
print('=' * 72)
for flag, label in [
    ('had_multiple_surgeries', 'Multiple surgeries (ever)'),
    ('first_surgery_in_window', 'First surgery during TP1-TP2 window'),
]:
    print(f'\n{label}')
    print('-' * 60)
    tbl = per_tertile_rates(surg_df, flag)
    for tertile, row in tbl.iterrows():
        print(f' {str(tertile):<18} {int(row["sum"]):>3} / {int(row["count"]):>3} ({row["pct"]:5.1f}%)')
        contingency = pd.crosstab(surg_df['growth_tertile'], surg_df[flag])
        if contingency.shape[1] < 2:
            print(' (no variance across tertiles - chi2 not computable)')
            continue
        chi2, p_chi2, _, _ = chi2_contingency(contingency)
        print(f' chi2={chi2:.2f}, p={p_chi2:.4f}')

# First-surgery timing summary per tertile
print('\nDays from diagnosis to first surgery (per tertile)')
print('-' * 60)
for t in order:
    sub = surg_df[surg_df['growth_tertile'] == t]['days_to_first_surgery'].
↳dropna()
    if len(sub):
        q1, med, q3 = sub.quantile([0.25, 0.5, 0.75])

```

```

        print(f'   {t:<18} median={med:+6.0f}   IQR=[{q1:+.0f}, {q3:+.0f}]   □
↳n={len(sub)}')

# Append surgery columns to the Part C summary CSV
surg_summary_rows = []
for tertile in order:
    sub = surg_df[surg_df['growth_tertile'] == tertile]
    surg_summary_rows.append({
        'tertile': tertile,
        'multi_surg_n': int(sub['had_multiple_surgeries'].sum()),
        'multi_surg_pct': sub['had_multiple_surgeries'].mean() * 100,
        'first_surg_in_window_n': int(sub['first_surgery_in_window'].sum()),
        'first_surg_in_window_pct': sub['first_surgery_in_window'].mean() * 100,
    })
surg_summary_df = pd.DataFrame(surg_summary_rows)
surg_path = os.path.join(RESULTS_DIR, '05_surgery_by_tertile.csv')
surg_summary_df.to_csv(surg_path, index=False)
print(f'\nSaved surgery summary to {surg_path}')

# Stand-alone figure for surgery proxies
plt.rcParams.update({'font.size': 14})
fig_s, axes_s = plt.subplots(1, 2, figsize=(14, 5), sharey=True)
xpos = np.arange(len(order))

for ax, (col, title) in zip(
    axes_s,
    [('multi_surg_pct', 'Multiple surgeries (ever)'),
     ('first_surg_in_window_pct', 'First surgery in TP1-TP2 window')],
):
    vals = [surg_summary_df.set_index('tertile').loc[t, col] for t in order]
    ax.bar(xpos, vals, 0.55, color='#7570b3', edgecolor='black')
    ax.set_xticks(xpos)
    ax.set_xticklabels(order)
    ax.set_ylabel('% of tertile')
    ax.set_title(title)
    ax.set_ylim(0, 115)
    for i, v in enumerate(vals):
        ax.text(i, v + 2, f'{v:.0f}%', ha='center', fontsize=11)

fig_s.suptitle('Surgery proxies by tumour growth tertile (n=35 per tertile)',
↳y=1.02)
plt.tight_layout(rect=[0, 0, 1, 0.93])
fig_s_pdf = os.path.join(FIGURES_DIR, '05_surgery_by_tertile.pdf')
fig_s_png = os.path.join(FIGURES_DIR, '05_surgery_by_tertile.png')
plt.savefig(fig_s_pdf, bbox_inches='tight')
plt.savefig(fig_s_png, dpi=150, bbox_inches='tight')
plt.show()

```

```
print(f'Saved figure to {fig_s_pdf}')
```

Patients with first-surgery day recorded: 105 / 105

Patients with both TP1 and TP2 days: 89 / 105

PART D - surgery proxies by growth tertile

=====

Multiple surgeries (ever)

Slow/Shrinking	13 / 35	(37.1%)	
Moderate	13 / 35	(37.1%)	
Fast Growing	11 / 35	(31.4%)	
chi2=0.33, p=0.8463			

First surgery during TP1-TP2 window

Slow/Shrinking	0 / 35	(0.0%)	
Moderate	0 / 35	(0.0%)	
Fast Growing	1 / 35	(2.9%)	
chi2=2.02, p=0.3644			

Days from diagnosis to first surgery (per tertile)

Slow/Shrinking	median=	-17	IQR=[-24, -4]	n=35	
Moderate	median=	-15	IQR=[-20, -5]	n=35	
Fast Growing	median=	-17	IQR=[-20, -4]	n=35	

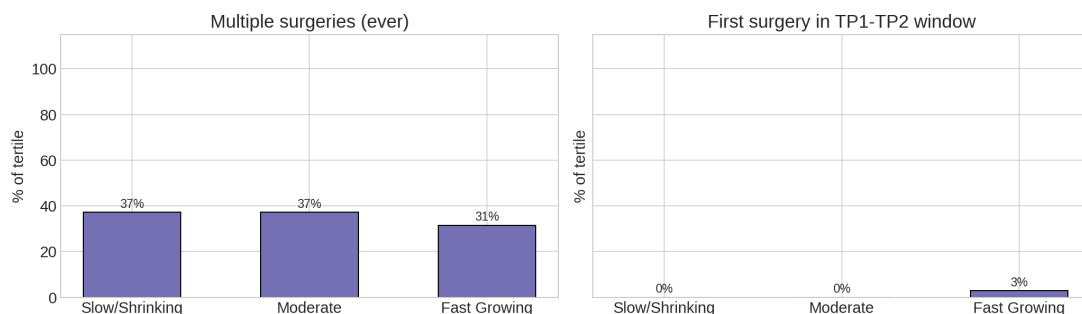
Saved surgery summary to drive/MyDrive/Y3

Project/Data/PKG_MU_Glioma_Post/growth_rate/data/05_surgery_by_tertile.csv

Saved figure to drive/MyDrive/Y3

Project/Data/PKG_MU_Glioma_Post/growth_rate/figures/05_surgery_by_tertile.pdf

Surgery proxies by tumour growth tertile (n=35 per tertile)



1.9.5 Interpretation guide

If the Section 5.5 / screencast confounding-by-indication story holds, we expect:

- Shrinking tertile with materially higher chemo-in-window and RT-in-window rates than the Fast Growing tertile.
- The same direction, and probably a larger gap, for the ever-received rates.
- Ideally chi-squared p below 0.05, though with n=35 per tertile the test is underpowered for small effects.

If the rates are flat or inverted, the existing confounder explanation in `Report.tex` Section 5.5 (around line 1212) and in the screencast script's "Shrinking tumours paradox" section is not supported by the available treatment data, and both need revision.

1.10 Save Results

```
[35]: # Save results table
results_df.to_csv(os.path.join(RESULTS_DIR, '05_growth_rate_model_comparison.
↪csv'), index=False)

# Save feature importance
importance.to_csv(os.path.join(RESULTS_DIR, '05_growth_rate_feature_importance.
↪csv'))

# Save the growth data for potential reuse
growth_with_clinical.to_csv(os.path.join(RESULTS_DIR,
↪'05_growth_data_with_features.csv'), index=False)

print(f"Results saved to {RESULTS_DIR}")
print(f"Figures saved to {FIGURES_DIR}")
```

```
Results saved to drive/MyDrive/Y3
Project/Data/PKG_MU_Glioma_Post/growth_rate/data
Figures saved to drive/MyDrive/Y3
Project/Data/PKG_MU_Glioma_Post/growth_rate/figures
```

1.11 Summary

This notebook addressed Aim 2 by predicting tumour volume change between consecutive MRI scans using baseline features.

Key findings (to be filled after running): - Best model and feature set: ... - R^2 score: ... (compare to 0 baseline; $R^2 > 0$ means better than predicting the mean) - Whether imaging features from the first timepoint add predictive value beyond clinical/molecular markers - Whether predicted growth rate correlates with actual survival outcomes (biological validation)

Limitations: - Small sample size (~82 patients with 2+ timepoints); results may have high variance - Volume change used as proxy for growth rate (exact scan dates not available in the segmentation file) - Pre-computed imaging features are first-order statistics only; texture and shape features (via PyRadiomics) could potentially add value - Temporal leakage risk is low here since we predict future volume change from baseline features, but growth may correlate with follow-up duration

E.6 Notebook 06: Comprehensive PyRadiomics Feature Extraction

06_pyradiomics_feature_extraction

1 Notebook 06: Comprehensive Radiomic Feature Extraction

1.1 Purpose

This notebook extracts comprehensive radiomic features from MRI volumes using PyRadiomics and evaluates whether texture, shape, and higher-order intensity descriptors improve survival prediction beyond the clinical and molecular features established in Notebooks 02-03.

In Notebook 04, we showed that simple pre-extracted imaging statistics (volume, mean intensity) did not improve prediction (AUC 0.72 vs 0.72, $p=0.94$). Here we test whether **comprehensive** radiomic features (GLCM texture, GLRLM run-length, shape descriptors) capture prognostic information missed by those simple statistics.

1.2 Theoretical Foundation

Radiomics is the high-throughput extraction of quantitative features from medical images. The core hypothesis is that tumour biology (cell density, necrosis, vascularity, infiltration) manifests as measurable patterns in imaging intensity distributions and spatial arrangements that are invisible to the human eye but detectable through mathematical analysis.

PyRadiomics extracts features in several families: - **First-order statistics**: Intensity histogram descriptors (mean, entropy, skewness, kurtosis) - **Shape features**: 3D morphology (sphericity, elongation, surface area, compactness) - **GLCM (Grey-Level Co-occurrence Matrix)**: Spatial intensity relationships (Haralick et al., 1973) - **GLRLM (Grey-Level Run-Length Matrix)**: Consecutive same-intensity voxel patterns (Galloway, 1975) - **GLSZM (Grey-Level Size Zone Matrix)**: Connected region statistics (Thibault et al., 2009) - **GLDM (Grey-Level Dependence Matrix)**: Neighbourhood dependency patterns

1.3 Notebook Dependencies

- **Inputs**: MRI NIfTI volumes + segmentation masks from MU-Glioma-Post; clinical data pickle from Notebook 01
- **Outputs**: Radiomic feature matrix, model comparison results, figures with 06_ prefix
- **Preceding notebooks**: 01 (preprocessing), 02 (binary classification baseline), 04 (simple radiomics)

1.4 1. Environment Setup

```
[2]: # PyRadiomics cannot build on Python 3.12+ due to deprecated configparser.
      ↪ SafeConfigParser
      # Workaround: install from the community-patched fork, or clone and patch
      ↪ manually

import sys
print(f"Python version: {sys.version}")

# Method 1: Install from the patched GitHub fork (try this first)
!pip install numpy SimpleITK -q
!pip install git+https://github.com/Kozmosa/pyradiomics-fix-configparser.git -q

# If Method 1 fails, try Method 2: clone, patch, and install from source
# !git clone https://github.com/AIM-Harvard/pyradiomics.git /content/pyradiomics
# !cd /content/pyradiomics && sed -i 's/SafeConfigParser/ConfigParser/g'
      ↪ versioneer.py
# !cd /content/pyradiomics && sed -i "s/readfp/read_file/g" versioneer.py
# !cd /content/pyradiomics && pip install . -q

!pip install lifelines
```

Python version: 3.12.12 (main, Oct 10 2025, 08:52:57) [GCC 11.4.0]

52.6/52.6 MB

18.8 MB/s eta 0:00:00

Installing build dependencies ... done

Getting requirements to build wheel ... done

Preparing metadata (pyproject.toml) ... done

Preparing metadata (setup.py) ... done

118.1/118.1 kB

2.6 MB/s eta 0:00:00

Building wheel for pyradiomics (pyproject.toml) ... done

Building wheel for docopt (setup.py) ... done

Collecting lifelines

Downloading lifelines-0.30.3-py3-none-any.whl.metadata (3.5 kB)

Requirement already satisfied: numpy>=1.14.0 in /usr/local/lib/python3.12/dist-packages (from lifelines) (2.0.2)

Requirement already satisfied: scipy>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from lifelines) (1.16.3)

Requirement already satisfied: pandas<3.0,>=2.1 in /usr/local/lib/python3.12/dist-packages (from lifelines) (2.2.2)

Requirement already satisfied: matplotlib>=3.0 in /usr/local/lib/python3.12/dist-packages (from lifelines) (3.10.0)

Requirement already satisfied: autograd>=1.5 in /usr/local/lib/python3.12/dist-packages (from lifelines) (1.8.0)

Collecting autograd-gamma>=0.3 (from lifelines)

Downloading autograd-gamma-0.5.0.tar.gz (4.0 kB)

```

Preparing metadata (setup.py) ... done
Collecting formulaic>=0.2.2 (from lifelines)
  Downloading formulaic-1.2.1-py3-none-any.whl.metadata (7.0 kB)
Collecting interface-meta>=1.2.0 (from formulaic>=0.2.2->lifelines)
  Downloading interface_meta-1.3.0-py3-none-any.whl.metadata (6.7 kB)
Requirement already satisfied: narwhals>=1.17 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2->lifelines) (2.17.0)
Requirement already satisfied: typing-extensions>=4.2.0 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2->lifelines) (4.15.0)
Requirement already satisfied: wrapt>=1.0 in /usr/local/lib/python3.12/dist-packages (from formulaic>=0.2.2->lifelines) (2.1.1)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (26.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (3.3.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.0->lifelines) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=2.1->lifelines) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas<3.0,>=2.1->lifelines) (2025.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib>=3.0->lifelines) (1.17.0)
Downloading lifelines-0.30.3-py3-none-any.whl (409 kB)
409.1/409.1 kB
4.3 MB/s eta 0:00:00
Downloading formulaic-1.2.1-py3-none-any.whl (117 kB)
117.3/117.3 kB
6.9 MB/s eta 0:00:00
Downloading interface_meta-1.3.0-py3-none-any.whl (14 kB)
Building wheels for collected packages: autograd-gamma
  Building wheel for autograd-gamma (setup.py) ... done

```

Created wheel for autograd-gamma: filename=autograd_gamma-0.5.0-py3-none-any.whl size=4030
sha256=1747a5daf8cb99b5cb5584b2f90401659695db94ef74ad964d1090b0f99eb3fa
Stored in directory: /root/.cache/pip/wheels/50/37/21/0a719b9d89c635e89ff24bd93b862882ad675279552013b2fb
Successfully built autograd-gamma
Installing collected packages: interface-meta, autograd-gamma, formulaic, lifelines
Successfully installed autograd-gamma-0.5.0 formulaic-1.2.1 interface-meta-1.3.0 lifelines-0.30.3

```
[3]: # Core imports
import os
import glob
import json
import time
import re
import warnings
import logging

# Data handling
import numpy as np
import pandas as pd
import pickle

# Medical imaging
import nibabel as nib
import SimpleITK as sitk # PyRadiomics uses SimpleITK internally

# Radiomics
from radiomics import featureextractor

# Visualisation
import matplotlib.pyplot as plt
import seaborn as sns

# ML
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LassoCV, ElasticNetCV
from sklearn.model_selection import StratifiedKFold, cross_validate, \
    cross_val_score
from sklearn.metrics import make_scorer, roc_auc_score
from xgboost import XGBClassifier

# Statistical testing
from scipy import stats
```

```
[4]: # Suppress PyRadiomics verbose logging
# Set to WARNING to only see important messages during extraction
logger = logging.getLogger('radiomics')
logger.setLevel(logging.WARNING)

# General settings
warnings.filterwarnings('ignore', category=FutureWarning)
sns.set_style('whitegrid')
plt.rcParams['figure.dpi'] = 150

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)

print('Environment setup complete.')
```

Environment setup complete.

1.5 2. Path Configuration and Data Loading

```
[6]: # Mount Google Drive (Colab only; comment out for local/cluster runs)
from google.colab import drive, userdata
drive.mount('/content/drive')
```

Mounted at /content/drive

```
[7]: # Path configuration
GLIOMA_DIR = userdata.get('GLIOMA_DIR')
MU_GLIOMA_DIR = os.path.join(GLIOMA_DIR, 'MU_Glioma_Post')

# Directories for loading preprocessed data from previous notebooks
PREPROCESSED_DIR = os.path.join(GLIOMA_DIR, 'preprocessed')

# Output directories for this notebook
RESULTS_DIR = os.path.join(GLIOMA_DIR, 'pyradiomics', 'results')
FIGURES_DIR = os.path.join(GLIOMA_DIR, 'pyradiomics', 'figures')

os.makedirs(RESULTS_DIR, exist_ok=True)
os.makedirs(FIGURES_DIR, exist_ok=True)

# Verify paths exist
for path, label in [(GLIOMA_DIR, 'GLIOMA_DIR'),
                    (MU_GLIOMA_DIR, 'MU_GLIOMA_DIR'),
                    (PREPROCESSED_DIR, 'PREPROCESSED_DIR')]:
    exists = os.path.exists(path)
    print(f'{label}: {"OK" if exists else "MISSING"} -> {path}')
```

```
# Random seed for reproducibility
RANDOM_STATE = 42
```

```
GLIOMA_DIR: OK -> drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post
MU_GLIOMA_DIR: OK -> drive/MyDrive/Y3
Project/Data/PKG_MU_Glioma_Post/MU_Glioma_Post
PREPROCESSED_DIR: OK -> drive/MyDrive/Y3
Project/Data/PKG_MU_Glioma_Post/preprocessed
```

```
[8]: # Load clinical data and survival outcomes from Notebook 01
X = pd.read_pickle(os.path.join(PREPROCESSED_DIR, 'X.pkl'))
y = pd.read_pickle(os.path.join(PREPROCESSED_DIR, 'y.pkl'))

print(f'Clinical features: {X.shape}')
print(f'Survival outcomes: {y.shape}')
print(f'\nClinical feature columns ({X.shape[1]}):')
print(list(X.columns))
```

```
Clinical features: (203, 28)
Survival outcomes: (203,)
```

```
Clinical feature columns (28):
['sex at birth', 'race', 'age at diagnosis', 'primary diagnosis', 'grade of
primary brain tumor', 'stereotactic biopsy before surgical resection',
'progression', 'time to first progression (days)', 'type of 1st progression',
'second progression/recurrence', 'type of 2nd progression', 'idh1 mutation',
'idh2 mutation', '1p/19q', 'atrx mutation', 'mgmt methylation', 'braf v600e
mutation', 'tert promoter mutation', 'chromosome 7 gain and chromosome 10 loss',
'h3-3a mutation', 'egfr amplification', 'pten mutation', 'cdkn2a/b deletion',
'tp53 alteration', 'previous brain tumor', 'number of days from diagnosis to
first surgery or procedure', 'number of days from diagnosis to date of first
progression', 'number of days from diagnosis to 1st mri (timepoint_1)']
```

```
[9]: # Load the original clinical data to recover Patient IDs
# (X.pkl may not contain Patient ID as it was dropped during preprocessing)
clinical_path = os.path.join(GLIOMA_DIR, 'MU_Glioma_Post_ClinicalData_July2025.
    <xlsx')
clinical_raw = pd.read_excel(clinical_path, sheet_name='MU Glioma Post')
clinical_raw.columns = clinical_raw.columns.str.strip()

# Identify the Patient ID column (may be 'Patient ID' or 'Patient_ID')
patient_id_col = None
for candidate in ['Patient ID', 'Patient_ID', 'patient_id', 'PatientID']:
    if candidate in clinical_raw.columns:
        patient_id_col = candidate
        break

if patient_id_col is None:
```

```

    print('WARNING: Could not find Patient ID column. Available columns:')
    print(clinical_raw.columns.tolist())
else:
    print(f'Patient ID column found: "{patient_id_col}"')
    print(f'Total patients in clinical data: {clinical_raw[patient_id_col].
↵unique()}'')
    print(f'Sample IDs: {clinical_raw[patient_id_col].head(5).tolist()}'')

```

```

Patient ID column found: "Patient_ID"
Total patients in clinical data: 203
Sample IDs: ['PatientID_0003', 'PatientID_0004', 'PatientID_0005',
'PatientID_0006', 'PatientID_0007']

```

1.6 3. Discover Imaging Data Structure

The MU-Glioma-Post dataset stores NIFTI volumes in a directory hierarchy:

```

MU_Glioma_Post/
  PatientID_XXXX/
    Timepoint_1/
      PatientID_XXXX_Timepoint_1_brain_t1c.nii.gz
      PatientID_XXXX_Timepoint_1_brain_t1n.nii.gz
      PatientID_XXXX_Timepoint_1_brain_t2f.nii.gz
      PatientID_XXXX_Timepoint_1_brain_t2w.nii.gz
      PatientID_XXXX_Timepoint_1_tumorMask.nii.gz
    Timepoint_2/
      ...

```

We need to: 1. Enumerate all patient directories 2. Find first-timepoint T1ce volume + segmentation mask for each patient 3. Map directory names to clinical Patient IDs

```

[10]: # Enumerate patient directories in MU_Glioma_Post
all_patient_dirs = sorted([
    d for d in os.listdir(MU_GLIOMA_DIR)
    if os.path.isdir(os.path.join(MU_GLIOMA_DIR, d))
])

print(f'Total patient directories: {len(all_patient_dirs)}')
print(f'First 10: {all_patient_dirs[:10]}')
print(f'Last 5: {all_patient_dirs[-5:]}')

```

```

Total patient directories: 203
First 10: ['PatientID_0003', 'PatientID_0004', 'PatientID_0005',
'PatientID_0006', 'PatientID_0007', 'PatientID_0008', 'PatientID_0009',
'PatientID_0010', 'PatientID_0011', 'PatientID_0012']
Last 5: ['PatientID_0271', 'PatientID_0272', 'PatientID_0273', 'PatientID_0274',
'PatientID_0275']

```

```
[11]: # Inspect one patient directory to understand the file structure
sample_patient = all_patient_dirs[0]
sample_path = os.path.join(MU_GLIOMA_DIR, sample_patient)

print(f'Sample patient: {sample_patient}')
print(f'Contents of patient directory:')

for item in sorted(os.listdir(sample_path)):
    item_path = os.path.join(sample_path, item)
    if os.path.isdir(item_path):
        files = os.listdir(item_path)
        print(f' {item}/ ({len(files)} files)')
        for f in sorted(files):
            print(f'    {f}')
    else:
        print(f' {item}')
```

```
Sample patient: PatientID_0003
Contents of patient directory:
Timepoint_1/ (5 files)
PatientID_0003_Timepoint_1_brain_t1c.nii.gz
PatientID_0003_Timepoint_1_brain_t1n.nii.gz
PatientID_0003_Timepoint_1_brain_t2f.nii.gz
PatientID_0003_Timepoint_1_brain_t2w.nii.gz
PatientID_0003_Timepoint_1_tumorMask.nii.gz
Timepoint_2/ (5 files)
PatientID_0003_Timepoint_2_brain_t1c.nii.gz
PatientID_0003_Timepoint_2_brain_t1n.nii.gz
PatientID_0003_Timepoint_2_brain_t2f.nii.gz
PatientID_0003_Timepoint_2_brain_t2w.nii.gz
PatientID_0003_Timepoint_2_tumorMask.nii.gz
Timepoint_5/ (5 files)
PatientID_0003_Timepoint_5_brain_t1c.nii.gz
PatientID_0003_Timepoint_5_brain_t1n.nii.gz
PatientID_0003_Timepoint_5_brain_t2f.nii.gz
PatientID_0003_Timepoint_5_brain_t2w.nii.gz
PatientID_0003_Timepoint_5_tumorMask.nii.gz
```

```
[24]: def find_first_timepoint_files(patient_dir_path):
    """
    For a given patient directory, find the first timepoint's T1ce volume
    and segmentation mask.

    Filename convention: PatientID_XXXX_Timepoint_N_brain_<modality>.nii.gz
    Segmentation mask: PatientID_XXXX_Timepoint_N_tumorMask.nii.gz

    Returns:
        dict with keys 't1c', 't1n', 't2f', 't2w', 'seg' mapped to file paths,
```



```

        or None if required files are missing.
        """
        # Get timepoint subdirectories, sorted numerically
        timepoint_dirs = sorted(
            [d for d in os.listdir(patient_dir_path)
             if os.path.isdir(os.path.join(patient_dir_path, d))],
            key=lambda x: int(re.search(r'\d+', x).group()) if re.search(r'\d+', x)
else 0
        )

        if not timepoint_dirs:
            return None

        # Use the first timepoint (baseline)
        first_tp_path = os.path.join(patient_dir_path, timepoint_dirs[0])
        nii_files = glob.glob(os.path.join(first_tp_path, '*.nii.gz'))

        # Map files by modality suffix
        # Actual filenames: PatientID_XXXX_Timepoint_N_brain_t1c.nii.gz
        # Segmentation:      PatientID_XXXX_Timepoint_N_tumorMask.nii.gz
        file_map = {}
        for f in nii_files:
            basename = os.path.basename(f).lower()
            if basename.endswith('_brain_t1c.nii.gz'):
                file_map['t1c'] = f
            elif basename.endswith('_brain_t1n.nii.gz'):
                file_map['t1n'] = f
            elif basename.endswith('_brain_t2f.nii.gz'):
                file_map['t2f'] = f
            elif basename.endswith('_brain_t2w.nii.gz'):
                file_map['t2w'] = f
            elif basename.endswith('_tumormask.nii.gz'):
                file_map['seg'] = f

        # Require at least T1c and segmentation mask
        if 't1c' in file_map and 'seg' in file_map:
            file_map['timepoint'] = timepoint_dirs[0]
            return file_map

        return None

# Build registry of all patients with valid first-timepoint imaging
patient_file_registry = {}
missing_patients = []

for patient_dir_name in all_patient_dirs:

```

```

patient_path = os.path.join(MU_GLIOMA_DIR, patient_dir_name)
files = find_first_timepoint_files(patient_path)

if files is not None:
    patient_file_registry[patient_dir_name] = files
else:
    missing_patients.append(patient_dir_name)

print(f'Patients with valid first-timepoint T1ce + segmentation: {len(patient_file_registry)}')
print(f'Patients missing required files: {len(missing_patients)}')

if missing_patients:
    print(f'\nMissing patients (first 10): {missing_patients[:10]}')

```

Patients with valid first-timepoint T1ce + segmentation: 202

Patients missing required files: 1

Missing patients (first 10): ['PatientID_0191']

```

[25]: # Patient ID alignment: map imaging directory names to clinical Patient IDs
# Directory names may be 'PatientID_XXXX' format; clinical data uses same or
# similar

def extract_patient_id(dir_name):
    """
    Extract a normalised patient ID from directory name.
    Handles formats like 'PatientID_0003', 'Patient_0003', etc.
    """
    # Try to extract PatientID_XXXX pattern
    match = re.match(r'(PatientID_\d{4})', str(dir_name))
    if match:
        return match.group(1)
    return str(dir_name)

# Get imaging patient IDs
imaging_patient_ids = set(extract_patient_id(d) for d in patient_file_registry.keys())

# Get clinical patient IDs
clinical_patient_ids = set(clinical_raw[patient_id_col].astype(str).values)

# Compute overlap
overlap = imaging_patient_ids & clinical_patient_ids
imaging_only = imaging_patient_ids - clinical_patient_ids
clinical_only = clinical_patient_ids - imaging_patient_ids

```

```

print('Patient ID Alignment')
print('-' * 40)
print(f'Imaging directories with T1ce + seg: {len(imaging_patient_ids)}')
print(f'Clinical data patients: {len(clinical_patient_ids)}')
print(f'Overlap (matched): {len(overlap)}')
print(f'Imaging only (no clinical data): {len(imaging_only)}')
print(f'Clinical only (no imaging): {len(clinical_only)}')

if imaging_only:
    print(f'\nImaging-only IDs (sample): {sorted(imaging_only)[:5]}')
if clinical_only:
    print(f'\nClinical-only IDs (sample): {sorted(clinical_only)[:5]}')

```

Patient ID Alignment

```

-----
Imaging directories with T1ce + seg: 202
Clinical data patients:             203
Overlap (matched):                  202
Imaging only (no clinical data):    0
Clinical only (no imaging):          1
Clinical-only IDs (sample): ['PatientID_0191']

```

1.7 4. PyRadiomics Extractor Configuration

We configure the feature extractor with standard settings for brain MRI: - **binWidth=25**: Standard discretisation for MRI intensity values. GLCM and GLRLM require discrete intensity levels, so continuous intensities are binned into 25-unit-wide bins. - **All feature classes enabled**: First-order, shape, GLCM, GLRLM, GLSZM, GLDM - **No wavelet/LoG filters**: Keeps feature count manageable (~107 features vs 1000+ with filters). If initial results are promising, filters can be enabled in a follow-up analysis. - **Label value**: The segmentation mask label for the tumour region (typically 1 for whole tumour, but may vary; we check this below).

```

[26]: # Check what label values exist in a sample segmentation mask
sample_patient_id = list(patient_file_registry.keys())[0]
sample_seg_path = patient_file_registry[sample_patient_id]['seg']

seg_img = nib.load(sample_seg_path)
seg_data = seg_img.get_fdata()

unique_labels = np.unique(seg_data).astype(int)
print(f'Sample patient: {sample_patient_id}')
print(f'Segmentation shape: {seg_data.shape}')
print(f'Unique label values: {unique_labels}')

for label in unique_labels:

```

```

voxel_count = np.sum(seg_data == label)
print(f' Label {label}: {voxel_count} voxels ({voxel_count / seg_data.size_
↳* 100:.2f}%)')

```

Sample patient: PatientID_0003
Segmentation shape: (240, 240, 155)
Unique label values: [0 1 2 3]
Label 0: 8843460 voxels (99.05%)
Label 1: 6510 voxels (0.07%)
Label 2: 44251 voxels (0.50%)
Label 3: 33779 voxels (0.38%)

```

[27]: # Determine the appropriate label for whole tumour
# BraTS convention: 0=background, 1=necrosis, 2=edema, 4=enhancing tumour
# For whole tumour radiomics, we want ALL non-zero labels combined

# We will create a binary whole-tumour mask during extraction:
# any non-zero label -> 1 (tumour), 0 -> 0 (background)

# Configure the PyRadiomics feature extractor
extractor_settings = {
    'binWidth': 25, # Standard for MRI; discretises intensity_
    ↳into 25-unit bins
    'resampledPixelSpacing': None, # None = use original spacing (images_
    ↳already preprocessed)
    'interpolator': 'sitkBSpline',
    'normalizeScale': 1,
    'normalize': False, # MRI intensities already standardised in_
    ↳preprocessing
    'removeOutliers': None,
    'label': 1 # We will binarise the mask to label=1 for_
    ↳whole tumour
}

extractor = featureextractor.RadiomicsFeatureExtractor(**extractor_settings)

# Enable all feature classes (no image filters to keep count manageable)
extractor.enableAllFeatures()

# Disable image type filters (wavelet, LoG) to keep feature count at ~107
extractor.enableImageTypeByName('Original')

print('PyRadiomics Extractor Configuration:')
print(f' Enabled feature classes: {list(extractor.enabledFeatures.keys())}')
print(f' Enabled image types: {list(extractor.enabledImagetypes.keys())}')
print(f' Bin width: {extractor_settings["binWidth"]}')
print(f' Label value: {extractor_settings["label"]}')

```

PyRadiomics Extractor Configuration:

```
Enabled feature classes: ['firstorder', 'glcm', 'gldm', 'glrlm', 'glszm',  
'ngtdm', 'shape', 'shape2D']  
Enabled image types: ['Original']  
Bin width: 25  
Label value: 1
```

1.8 5. Extraction Helper Functions

```
[28]: def binarise_segmentation(seg_nifti_path, output_path=None):  
    """  
    Convert a multi-label segmentation mask into a binary whole-tumour mask.  
    All non-zero labels become 1 (tumour); 0 remains 0 (background).  
  
    PyRadiomics expects a SimpleITK image, so we save a temporary NIfTI  
    with the binarised mask and return the path.  
  
    Args:  
        seg_nifti_path: Path to the original multi-label segmentation NIfTI.  
        output_path: Where to save the binary mask. If None, uses a temp file.  
  
    Returns:  
        Path to the binarised NIfTI file.  
    """  
    seg_img = sitk.ReadImage(seg_nifti_path)  
    seg_array = sitk.GetArrayFromImage(seg_img)  
  
    # Binarise: any non-zero label -> 1  
    binary_array = (seg_array > 0).astype(np.uint8)  
  
    # Create new SimpleITK image with same metadata  
    binary_img = sitk.GetImageFromArray(binary_array)  
    binary_img.CopyInformation(seg_img)  
  
    if output_path is None:  
        output_path = seg_nifti_path.replace('_tumorMask.nii.gz',  
↪ '_tumorMask_binary.nii.gz')  
  
    sitk.WriteImage(binary_img, output_path)  
    return output_path  
  
def extract_features_for_patient(patient_id, file_map, extractor,  
↪ temp_dir=None):  
    """  
    Extract radiomic features for a single patient using their T1ce volume
```

and binarised segmentation mask.

Args:

patient_id: Patient identifier string.
file_map: Dict with 't1c' and 'seg' file paths.
extractor: Configured RadiomicsFeatureExtractor instance.
temp_dir: Directory for temporary binary mask files.

Returns:

Dict of feature_name: value, or None if extraction failed.

```
"""
t1c_path = file_map['t1c']
seg_path = file_map['seg']

try:
    # Binarise the segmentation mask for whole-tumour analysis
    if temp_dir:
        binary_seg_path = os.path.join(temp_dir, f'{patient_id}_seg_binary.
↪nii.gz')
    else:
        binary_seg_path = None

    binary_seg_path = binarise_segmentation(seg_path, binary_seg_path)

    # Check that the mask has non-zero voxels
    check_img = sitk.ReadImage(binary_seg_path)
    check_array = sitk.GetArrayFromImage(check_img)
    tumour_voxels = np.sum(check_array > 0)

    if tumour_voxels == 0:
        print(f' WARNING: {patient_id} has empty segmentation mask.
↪Skipping.')
        return None

    # Extract features
    result = extractor.execute(t1c_path, binary_seg_path)

    # Filter out diagnostic/metadata entries (keep only actual features)
    features = {}
    for key, value in result.items():
        if not key.startswith('diagnostics_'):
            features[key] = float(value) if hasattr(value, 'item') else
↪float(value)

    features['patient_id'] = patient_id
    features['tumour_voxels'] = int(tumour_voxels)
```

```

    # Clean up temporary binary mask
    if binary_seg_path and os.path.exists(binary_seg_path):
        os.remove(binary_seg_path)

    return features

except Exception as e:
    print(f' ERROR extracting features for {patient_id}: {e}')
    return None

```

1.9 6. Demo Extraction (3 Patients)

Note: Full feature extraction was performed on the University of Manchester CSF3 cluster using `06_pyradiomics_extraction_csf3.py`. The demo extraction cells below are retained for reference but should be skipped when running this notebook against CSF3 output.

```

[29]: # Select 3 representative patients for the demo
# We pick patients that are in the clinical-imaging overlap
matched_patients = [
    pid for pid in patient_file_registry.keys()
    if extract_patient_id(pid) in overlap
]

print(f'Total matched patients available: {len(matched_patients)}')

# Select 3 patients: first, middle, and last (for variety in tumour size)
demo_indices = [0, len(matched_patients) // 2, len(matched_patients) - 1]
demo_patients = [matched_patients[i] for i in demo_indices]

print(f'Demo patients selected: {demo_patients}')

```

Total matched patients available: 202

Demo patients selected: ['PatientID_0003', 'PatientID_0116', 'PatientID_0275']

```

[30]: # Visualise the demo patients: show middle axial slice of T1c with
      ↪ segmentation overlay
fig, axes = plt.subplots(1, 3, figsize=(15, 5))

for idx, patient_id in enumerate(demo_patients):
    file_map = patient_file_registry[patient_id]

    # Load T1c and segmentation
    t1c_img = nib.load(file_map['t1c'])
    seg_img = nib.load(file_map['seg'])
    t1c_data = t1c_img.get_fdata()

```

```

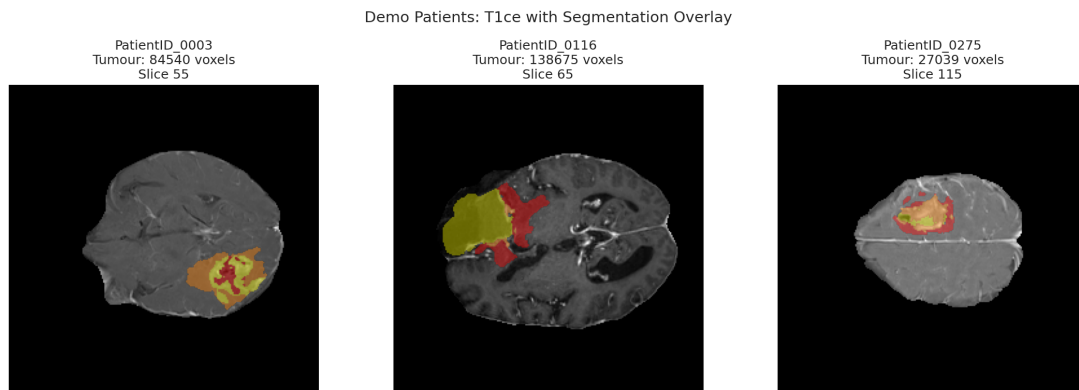
seg_data = seg_img.get_fdata()

# Find the axial slice with the most tumour voxels
tumour_per_slice = np.sum(seg_data > 0, axis=(0, 1))
best_slice = np.argmax(tumour_per_slice)

# Display
axes[idx].imshow(t1c_data[:, :, best_slice], cmap='gray')
axes[idx].imshow(
    np.ma.masked_where(seg_data[:, :, best_slice] == 0, seg_data[:, :,
↪best_slice]),
    cmap='autumn', alpha=0.4
)
tumour_volume = np.sum(seg_data > 0)
axes[idx].set_title(f'{patient_id}\nTumour: {tumour_volume} voxels\nSlice
↪{best_slice}',
                    fontsize=13, fontweight='bold')
axes[idx].axis('off')

plt.suptitle('Demo Patients: T1ce with Segmentation Overlay', fontsize=17,
↪fontweight='bold', y=1.02)
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '06_demo_patients_overlay.png'),
            dpi=300, bbox_inches='tight')
plt.show()
print('Saved: 06_demo_patients_overlay.png')

```



Saved: 06_demo_patients_overlay.png

```

[31]: # Extract radiomic features for demo patients
# Create a temporary directory for binary masks
temp_dir = os.path.join(GLIOMA_DIR, 'temp_binary_masks')
os.makedirs(temp_dir, exist_ok=True)

```



```

demo_features_list = []
total_start = time.time()

for i, patient_id in enumerate(demo_patients):
    print(f'\nExtracting features for patient {i+1}/{len(demo_patients)}:␣
    ↪{patient_id}')
    patient_start = time.time()

    features = extract_features_for_patient(
        patient_id=patient_id,
        file_map=patient_file_registry[patient_id],
        extractor=extractor,
        temp_dir=temp_dir
    )

    elapsed = time.time() - patient_start

    if features is not None:
        demo_features_list.append(features)
        n_features = len([k for k in features.keys() if k not in ('patient_id',␣
        ↪'tumour_voxels')])
        print(f'  Extracted {n_features} features in {elapsed:.1f}s')
    else:
        print(f'  FAILED after {elapsed:.1f}s')

total_elapsed = time.time() - total_start
print(f'\nTotal demo extraction time: {total_elapsed:.1f}s ({total_elapsed/60:␣
    ↪.1f} min)')
print(f'Average per patient: {total_elapsed/len(demo_patients):.1f}s')

# Estimate full cohort time
avg_per_patient = total_elapsed / len(demo_patients)
full_estimate_mins = avg_per_patient * len(matched_patients) / 60
print(f'\nEstimated time for full cohort ({len(matched_patients)} patients):␣
    ↪{full_estimate_mins:.0f} min')

```

Extracting features for patient 1/3: PatientID_0003

parameter force2D must be set to True to enable shape2D extraction

WARNING:radiomics.featureextractor:parameter force2D must be set to True to enable shape2D extraction

GLCM is symmetrical, therefore Sum Average = 2 * Joint Average, only 1 needs to be calculated

WARNING:radiomics.glcm:GLCM is symmetrical, therefore Sum Average = 2 * Joint Average, only 1 needs to be calculated

Extracted 107 features in 4.0s

Extracting features for patient 2/3: PatientID_0116

parameter force2D must be set to True to enable shape2D extraction

WARNING:radiomics.featureextractor:parameter force2D must be set to True to enable shape2D extraction

GLCM is symmetrical, therefore Sum Average = 2 * Joint Average, only 1 needs to be calculated

WARNING:radiomics.glcm:GLCM is symmetrical, therefore Sum Average = 2 * Joint Average, only 1 needs to be calculated

Extracted 107 features in 6.3s

Extracting features for patient 3/3: PatientID_0275

parameter force2D must be set to True to enable shape2D extraction

WARNING:radiomics.featureextractor:parameter force2D must be set to True to enable shape2D extraction

GLCM is symmetrical, therefore Sum Average = 2 * Joint Average, only 1 needs to be calculated

WARNING:radiomics.glcm:GLCM is symmetrical, therefore Sum Average = 2 * Joint Average, only 1 needs to be calculated

Extracted 107 features in 3.2s

Total demo extraction time: 13.5s (0.2 min)

Average per patient: 4.5s

Estimated time for full cohort (202 patients): 15 min

```
[32]: # Convert demo features to DataFrame and inspect
demo_features_df = pd.DataFrame(demo_features_list)

# Separate metadata from actual radiomic features
metadata_cols = ['patient_id', 'tumour_voxels']
feature_cols = [c for c in demo_features_df.columns if c not in metadata_cols]

print(f'Demo features DataFrame: {demo_features_df.shape}')
print(f'Metadata columns: {len(metadata_cols)}')
print(f'Radiomic feature columns: {len(feature_cols)}')
print(f'\nFeature categories:')

# Count features by category
categories = {}
for col in feature_cols:
    # PyRadiomics names features as 'original_<class>_<feature>'
    parts = col.split('_')
    if len(parts) >= 2:
```

```

        category = parts[1] if parts[0] == 'original' else parts[0]
    else:
        category = 'other'
    categories[category] = categories.get(category, 0) + 1

for cat, count in sorted(categories.items()):
    print(f' {cat}: {count} features')

```

Demo features DataFrame: (3, 109)

Metadata columns: 2

Radiomic feature columns: 107

Feature categories:

```

firstorder: 18 features
glcm: 24 features
gldm: 14 features
glrlm: 16 features
glszm: 16 features
ngtdm: 5 features
shape: 14 features

```

```

[33]: # Save demo features
demo_features_df.to_csv(
    os.path.join(RESULTS_DIR, '06_demo_radiomic_features.csv'), index=False
)
print('Saved: 06_demo_radiomic_features.csv')

# Display a sample of features
print('\nSample of extracted features (first 10):')
demo_features_df[feature_cols[:10]].T

```

Saved: 06_demo_radiomic_features.csv

Sample of extracted features (first 10):

```

[33]:

```

	0	1 \
original_shape_Elongation	0.788332	0.820336
original_shape_Flatness	0.573500	0.609500
original_shape_LeastAxisLength	38.106284	46.512079
original_shape_MajorAxisLength	66.445121	76.311814
original_shape_Maximum2DDiameterColumn	79.611557	76.164296
original_shape_Maximum2DDiameterRow	66.573268	89.185201
original_shape_Maximum2DDiameterSlice	67.186308	92.703829
original_shape_Maximum3DDiameter	81.271151	98.442877
original_shape_MeshVolume	84448.916667	138776.125000
original_shape_MinorAxisLength	52.380830	62.601290

2

original_shape_Elongation	0.265681
original_shape_Flatness	0.202484
original_shape_LeastAxisLength	30.807304
original_shape_MajorAxisLength	152.146648
original_shape_Maximum2DDiameterColumn	127.011810
original_shape_Maximum2DDiameterRow	131.685231
original_shape_Maximum2DDiameterSlice	49.040799
original_shape_Maximum3DDiameter	139.391535
original_shape_MeshVolume	26878.791667
original_shape_MinorAxisLength	40.422447

1.10 7. Full Cohort Extraction

Note: Full cohort extraction was performed on CSF3. The cell below is retained for reference but should be skipped. Results are loaded from 06_full_radiomic_features.csv in Section 8.

```
[ ]: # Full cohort extraction was performed on CSF3 using
      ↪ 06_pyradiomics_extraction_csf3.py
      # Results are loaded from the CSV in Section 8
      full_features_path = os.path.join(RESULTS_DIR, '06_full_radiomic_features.csv')
      print(f'Full features path: {full_features_path}')
      print(f'File exists: {os.path.exists(full_features_path)}')
```

Full features path: drive/MyDrive/Y3 Project/Data/PKG_MU_Glioma_Post/pyradiomics/results/06_full_radiomic_features.csv
File exists: True

1.11 8. Feature Quality Control

Before modelling, we clean the radiomic feature matrix: 1. Remove NaN/infinite values 2. Remove near-zero variance features 3. Remove highly correlated feature pairs ($|r| > 0.95$)

```
[34]: full_features_path = os.path.join(RESULTS_DIR, '06_full_radiomic_features.csv')
      full_features_df = pd.read_csv(full_features_path)
```

```
[35]: # Separate metadata from features
      metadata_cols = ['patient_id', 'tumour_voxels']
      radiomics_df = full_features_df.copy()
      feature_cols = [c for c in radiomics_df.columns if c not in metadata_cols]

      print(f'Initial feature matrix: {radiomics_df.shape}')
      print(f'Radiomic features: {len(feature_cols)}')

      # Check for NaN and infinite values
```

```

nan_counts = radiomics_df[feature_cols].isna().sum()
inf_counts = np.isinf(radiomics_df[feature_cols].select_dtypes(include=[np.
    number])).sum()

print(f'\nFeatures with NaN values: {(nan_counts > 0).sum()}')
print(f'Features with Inf values: {(inf_counts > 0).sum()}')

# Drop features with any NaN or Inf
clean_features = feature_cols.copy()
for col in feature_cols:
    if radiomics_df[col].isna().any() or (np.isinf(radiomics_df[col]).any() if
        radiomics_df[col].dtype != object else False):
        clean_features.remove(col)

print(f'Features after NaN/Inf removal: {len(clean_features)}')

```

Initial feature matrix: (202, 109)

Radiomic features: 107

Features with NaN values: 0

Features with Inf values: 0

Features after NaN/Inf removal: 107

```

[36]: # Remove near-zero variance features
# A feature with variance < 1e-10 provides no discriminative power
feature_variances = radiomics_df[clean_features].var()
low_var_features = feature_variances[feature_variances < 1e-10].index.tolist()

clean_features = [f for f in clean_features if f not in low_var_features]

print(f'Near-zero variance features removed: {len(low_var_features)}')
if low_var_features:
    print(f'    Removed: {low_var_features}')
print(f'Features remaining: {len(clean_features)}')

```

Near-zero variance features removed: 0

Features remaining: 107

```

[37]: # Remove highly correlated features (|r| > 0.95)
# When two features are nearly identical, one is redundant
corr_matrix = radiomics_df[clean_features].corr().abs()

# Find pairs with |r| > 0.95
upper_tri = corr_matrix.where(
    np.triu(np.ones(corr_matrix.shape), k=1).astype(bool)
)

# For each pair, drop the feature with the higher mean correlation to all others

```

```

to_drop = set()
for col in upper_tri.columns:
    highly_correlated = upper_tri.index[upper_tri[col] > 0.95].tolist()
    for correlated_feat in highly_correlated:
        # Keep the feature with lower average correlation to everything else
        mean_corr_col = corr_matrix[col].mean()
        mean_corr_feat = corr_matrix[correlated_feat].mean()
        if mean_corr_col > mean_corr_feat:
            to_drop.add(col)
        else:
            to_drop.add(correlated_feat)

clean_features = [f for f in clean_features if f not in to_drop]

print(f'Highly correlated features removed (|r| > 0.95): {len(to_drop)}')
print(f'Features remaining after all QC: {len(clean_features)}')

```

Highly correlated features removed (|r| > 0.95): 46
Features remaining after all QC: 61

[38]: clean_features

```

[38]: ['original_shape_Elongation',
      'original_shape_Flatness',
      'original_shape_LeastAxisLength',
      'original_shape_MajorAxisLength',
      'original_shape_Maximum2DDiameterColumn',
      'original_shape_Maximum2DDiameterRow',
      'original_shape_Maximum2DDiameterSlice',
      'original_shape_Maximum3DDiameter',
      'original_shape_MinorAxisLength',
      'original_shape_Sphericity',
      'original_shape_SurfaceArea',
      'original_shape_SurfaceVolumeRatio',
      'original_firstorder_10Percentile',
      'original_firstorder_InterquartileRange',
      'original_firstorder_Kurtosis',
      'original_firstorder_Maximum',
      'original_firstorder_Minimum',
      'original_firstorder_Skewness',
      'original_firstorder_TotalEnergy',
      'original_firstorder_Uniformity',
      'original_glcm_ClusterProminence',
      'original_glcm_DifferenceEntropy',
      'original_glcm_Idmn',
      'original_glcm_Idn',
      'original_glcm_Imc1',
      'original_glcm_Imc2',

```

```

'original_glcm_InverseVariance',
'original_glcm_JointEnergy',
'original_glcm_MCC',
'original_glcm_MaximumProbability',
'original_glcm_SumAverage',
'original_glcm_SumEntropy',
'original_gldm_DependenceEntropy',
'original_gldm_DependenceNonUniformityNormalized',
'original_gldm_DependenceVariance',
'original_gldm_LargeDependenceEmphasis',
'original_gldm_LargeDependenceHighGrayLevelEmphasis',
'original_gldm_LowGrayLevelEmphasis',
'original_gldm_SmallDependenceLowGrayLevelEmphasis',
'original_glrlm_GrayLevelNonUniformity',
'original_glrlm_LongRunHighGrayLevelEmphasis',
'original_glrlm_LongRunLowGrayLevelEmphasis',
'original_glrlm_RunEntropy',
'original_glrlm_RunLengthNonUniformity',
'original_glrlm_RunVariance',
'original_glrlm_ShortRunLowGrayLevelEmphasis',
'original_glszm_GrayLevelNonUniformity',
'original_glszm_GrayLevelNonUniformityNormalized',
'original_glszm_LargeAreaHighGrayLevelEmphasis',
'original_glszm_LargeAreaLowGrayLevelEmphasis',
'original_glszm_LowGrayLevelZoneEmphasis',
'original_glszm_SizeZoneNonUniformity',
'original_glszm_SmallAreaEmphasis',
'original_glszm_SmallAreaLowGrayLevelEmphasis',
'original_glszm_ZoneEntropy',
'original_glszm_ZonePercentage',
'original_glszm_ZoneVariance',
'original_ngtdm_Busyness',
'original_ngtdm_Coarseness',
'original_ngtdm_Contrast',
'original_ngtdm_Strength']

```

1.12 9. Feature Exploration

```

[39]: # Correlation heatmap of remaining features
      # If >30 features, show only top 30 by variance for readability
      plot_features = clean_features
      if len(clean_features) > 30:
          top_var = radiomics_df[clean_features].var().nlargest(30).index.tolist()
          plot_features = top_var

```

```

    print(f'Showing top 30 features by variance (out of {len(clean_features)}_
    total)')

fig, ax = plt.subplots(figsize=(14, 12))
corr_plot = radiomics_df[plot_features].corr()

# Shorten feature names for display
short_names = [f.replace('original_', '').replace('firstorder_', 'fo_')
                .replace('shape_', 'sh_').replace('glcm_', 'glcm_')
                .replace('glrlm_', 'rlm_').replace('glszm_', 'szm_')
                .replace('gldm_', 'dm_')
                for f in plot_features]

sns.heatmap(corr_plot, annot=False, cmap='RdBu_r', center=0,
            xticklabels=short_names, yticklabels=short_names,
            square=True, ax=ax)
ax.set_title('Radiomic Feature Correlation Matrix (Post-QC)', fontsize=18,
            fontweight='bold')
plt.xticks(rotation=90, fontsize=9)
plt.yticks(fontsize=9)
plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '06_feature_correlation_heatmap.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '06_feature_correlation_heatmap.png'),
            dpi=300, bbox_inches='tight')
plt.show()
print('Saved: 06_feature_correlation_heatmap.pdf + .png')

```

Showing top 30 features by variance (out of 61 total)


```

key_features = [f for f in key_feature_candidates if f in clean_features]

if key_features:
    n_plots = len(key_features)
    n_cols = min(4, n_plots)
    n_rows = (n_plots + n_cols - 1) // n_cols

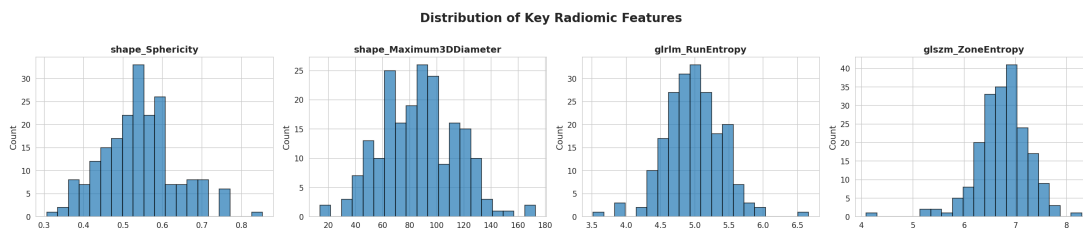
    fig, axes = plt.subplots(n_rows, n_cols, figsize=(5 * n_cols, 4 * n_rows))
    axes = np.atleast_2d(axes).flatten()

    for idx, feat in enumerate(key_features):
        short_name = feat.replace('original_', '')
        axes[idx].hist(radiomics_df[feat].dropna(), bins=20, edgecolor='black',
        alpha=0.7)
        axes[idx].set_title(short_name, fontsize=12, fontweight='bold')
        axes[idx].set_ylabel('Count', fontsize=11)
        axes[idx].set_xlabel('')
        axes[idx].tick_params(labelsize=10)

    # Hide unused axes
    for idx in range(n_plots, len(axes)):
        axes[idx].set_visible(False)

    plt.suptitle('Distribution of Key Radiomic Features', fontsize=16,
    fontweight='bold', y=1.02)
    plt.tight_layout()
    plt.savefig(os.path.join(FIGURES_DIR, '06_key_feature_distributions.pdf'),
                bbox_inches='tight')
    plt.savefig(os.path.join(FIGURES_DIR, '06_key_feature_distributions.png'),
                dpi=300, bbox_inches='tight')
    plt.show()
    print('Saved: 06_key_feature_distributions.pdf + .png')
else:
    print('No key features found in cleaned feature set.')

```



Saved: 06_key_feature_distributions.pdf + .png

1.13 10. Feature Selection (LASSO / Elastic Net)

With potentially 50-100+ features and only ~170 patients, we have a severe $p \gg n$ problem. LASSO (L1 regularisation) forces most feature coefficients to exactly zero, effectively selecting a sparse subset of informative features.

We use Elastic Net (L1 + L2 mix) because radiomic features are often correlated in groups; pure LASSO would arbitrarily select one from each correlated group, while Elastic Net tends to select entire groups.

This section requires the full cohort extraction (Section 7). With only 3 demo patients, feature selection is not meaningful.

```
[41]: # Check dataset dimensions for feature selection
n_patients = len(radiomics_df)
n_features = len(clean_features)

print(f'Patients: {n_patients}')
print(f'Features: {n_features}')
print(f'Ratio (patients/features): {n_patients/n_features:.1f}')
```

Patients: 202

Features: 61

Ratio (patients/features): 3.3

```
[42]: # Merge radiomic features with survival outcome for feature selection
# Map patient IDs to the clinical data index
radiomics_merged = radiomics_df.copy()
radiomics_merged['patient_id_norm'] = radiomics_merged['patient_id'].
    ↪apply(extract_patient_id)

# Get the binary outcome (y) aligned by patient ID
clinical_raw_indexed = clinical_raw.set_index(patient_id_col)

# Align: only keep patients present in both radiomics and clinical outcome
common_ids = set(radiomics_merged['patient_id_norm']) &
    ↪set(clinical_raw_indexed.index.astype(str))
print(f'Patients with both radiomics and clinical outcome: {len(common_ids)}')

# Filter radiomics to matched patients
radiomics_for_selection = radiomics_merged[
    radiomics_merged['patient_id_norm'].isin(common_ids)
].copy()

# Build y_aligned by looking up survival outcome from clinical_raw for each
    ↪radiomics patient
# The outcome column is 'Overall Survival (Death)' (binary: 0=alive, 1=dead)
outcome_col = None
for candidate in ['Overall Survival (Death)', 'overall survival (death)',
```

```

        'Overall_Survival_Death', 'Vital Status']:
    if candidate in clinical_raw.columns:
        outcome_col = candidate
        break

if outcome_col is None:
    # Try case-insensitive match
    for col in clinical_raw.columns:
        if 'overall survival' in col.lower() and 'death' in col.lower():
            outcome_col = col
            break

if outcome_col is None:
    raise ValueError(f'Could not find survival outcome column. Available: {clinical_raw.columns.tolist()}')

print(f'Using outcome column: "{outcome_col}"')

# Create a lookup: patient_id -> outcome
outcome_lookup = clinical_raw_indexed[outcome_col].to_dict()

# Align outcome to radiomics patient order
y_aligned = radiomics_for_selection['patient_id_norm'].map(
    lambda pid: outcome_lookup.get(pid, outcome_lookup.get(int(pid.
    ↪split('_')[-1]) if '_' in pid else pid))
).values.astype(float)

# Verify alignment
assert len(y_aligned) == len(radiomics_for_selection), \
    f'Alignment mismatch: {len(y_aligned)} outcomes vs {len(radiomics_for_selection)} patients'
assert not np.any(np.isnan(y_aligned)), \
    f'NaN outcomes found for {np.sum(np.isnan(y_aligned))} patients'

print(f'Radiomics patients for selection: {len(radiomics_for_selection)}')
print(f'Outcome variable aligned: {len(y_aligned)} (alive={int(np.
    ↪sum(y_aligned==0))}, dead={int(np.sum(y_aligned==1))})')

```

Patients with both radiomics and clinical outcome: 202
 Using outcome column: "Overall Survival (Death)"
 Radiomics patients for selection: 202
 Outcome variable aligned: 202 (alive=106, dead=96)

[43]:

```

# Standardise features (zero mean, unit variance)
# Critical for LASSO; otherwise features on different scales dominate
X_radiomics = radiomics_for_selection[clean_features].values
scaler = StandardScaler()

```

```

X_radiomics_scaled = scaler.fit_transform(X_radiomics)

# Run Elastic Net with cross-validation to find optimal alpha
# l1_ratio=0.7 balances sparsity (L1) with grouping (L2)
elastic_net = ElasticNetCV(
    l1_ratio=0.7,
    cv=5,
    random_state=RANDOM_STATE,
    max_iter=10000,
    n_alphas=100
)

# y_aligned was built in the previous cell via patient-ID-based lookup
elastic_net.fit(X_radiomics_scaled, y_aligned)

# Identify surviving features
coef_series = pd.Series(elastic_net.coef_, index=clean_features)
selected_features = coef_series[coef_series != 0].sort_values(key=abs,
    ↪ascending=False)

print(f'Elastic Net optimal alpha: {elastic_net.alpha_:.6f}')
print(f'Features surviving selection: {len(selected_features)} / {
    ↪len(clean_features)}')
print(f'\nSelected features (ranked by |coefficient|):')
for feat, coef in selected_features.items():
    direction = '+' if coef > 0 else '-'
    short_name = feat.replace('original_', '')
    print(f' {direction} {short_name}: {coef:.4f}')

```

Elastic Net optimal alpha: 0.049633
Features surviving selection: 8 / 61

Selected features (ranked by |coefficient|):

- + firstorder_Skewness: 0.0597
- + shape_SurfaceArea: 0.0392
- glcm_SmallDependenceLowGrayLevelEmphasis: -0.0353
- + glrlm_RunLengthNonUniformity: 0.0290
- glszm_LargeAreaLowGrayLevelEmphasis: -0.0213
- glszm_SmallAreaLowGrayLevelEmphasis: -0.0137
- glrlm_LongRunLowGrayLevelEmphasis: -0.0049
- glszm_LowGrayLevelZoneEmphasis: -0.0027

```

[44]: if len(selected_features) > 0:
    # Save selected features
    selected_features.to_csv(
        os.path.join(RESULTS_DIR, '06_selected_radiomic_features.csv'),
        header=['coefficient']

```

```

)
print('Saved: 06_selected_radiomic_features.csv')

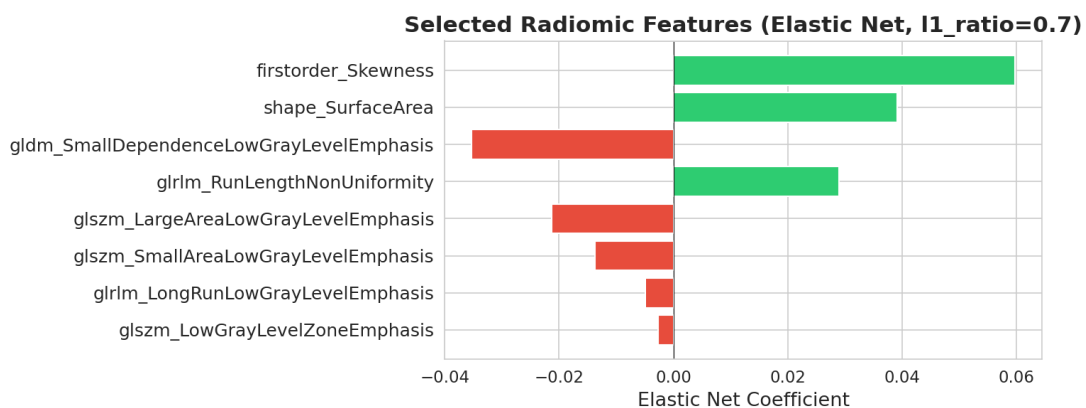
# Visualise selected feature coefficients
fig, ax = plt.subplots(figsize=(10, max(4, len(selected_features) * 0.5)))
short_names = [f.replace('original_', '') for f in selected_features.index]
colors = ['#e74c3c' if c < 0 else '#2ecc71' for c in selected_features.
↪values]

ax.barh(range(len(selected_features)), selected_features.values,
↪color=colors)
ax.set_yticks(range(len(selected_features)))
ax.set_yticklabels(short_names, fontsize=12)
ax.set_xlabel('Elastic Net Coefficient', fontsize=13)
ax.set_title('Selected Radiomic Features (Elastic Net, l1_ratio=0.7)',
↪fontsize=15, fontweight='bold')
ax.axvline(x=0, color='black', linewidth=0.5)
ax.tick_params(axis='x', labelsize=11)
ax.invert_yaxis()

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '06_selected_feature_coefficients.
↪pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '06_selected_feature_coefficients.
↪png'),
            dpi=300, bbox_inches='tight')
plt.show()
print('Saved: 06_selected_feature_coefficients.pdf + .png')
else:
    print('No features selected; skipping visualisation.')

```

Saved: 06_selected_radiomic_features.csv



Saved: 06_selected_feature_coefficients.pdf + .png

1.14 11. Predictive Modelling Comparison

We compare three conditions using the same XGBoost framework and 5-fold stratified cross-validation as Notebook 02:

1. **Clinical features only** (baseline from Notebook 02; AUC ~0.674)
2. **Selected radiomics features only** (from Elastic Net above)
3. **Clinical + selected radiomics combined**

We do NOT re-tune hyperparameters; the point is to compare feature sets under identical model conditions.

This section requires full cohort extraction.

Note: The `evaluate_model` function previously defined here was unused — Cell 44 uses an inline `evaluate_simple` function instead. Removed to avoid confusion.

```
[45]: if len(selected_features) > 0:
    print('Model Comparison: Clinical vs Radiomics vs Combined')
    print('-' * 60)

    # Build clinical feature matrix aligned to the same patients as radiomics
    # X was created from high_fill_df with patient_id dropped, so we re-extract
    # clinical features from clinical_raw for the matched patients
    X_encoded = X.copy()
    for col in X_encoded.columns:
        if X_encoded[col].dtype == 'object':
            le = LabelEncoder()
            X_encoded[col] = le.fit_transform(X_encoded[col].astype(str))

    # Align clinical features to the same patients as radiomics_for_selection
    # X and y share the same integer index from high_fill_df (patient_id was
    ↪dropped)
    # We need to map radiomics patients back to the X/y index via clinical_raw
    #
    # Strategy: for each patient in radiomics_for_selection, find their row in
    # clinical_raw, then find the corresponding row in high_fill_df/X by
    ↪matching
    # on available columns (since patient_id was dropped from X).
    #
    # Simpler approach: rebuild clinical features from clinical_raw for matched
    ↪patients
    # and use y_aligned (already aligned by patient ID in Cell 39)
```

```

    # Get clinical features for matched patients from X using the same patient_
↪ordering
    # We need the mapping: patient_id -> index in X/y
    # Since patient_id was dropped from X, we reconstruct via clinical_raw row_
↪order
    # clinical_raw and high_fill_df share the same row order (high_fill_df is_
↪filtered from clinical_raw)
    clinical_raw_with_idx = clinical_raw.copy()
    clinical_raw_with_idx['_pid_norm'] = clinical_raw_with_idx[patient_id_col].
↪astype(str)

    # Build mapping from normalised patient ID to X/y integer index
    # X's index matches high_fill_df's index, which is a subset of_
↪clinical_raw's rows
    pid_to_xy_idx = {}
    for xy_idx, pid in zip(X.index, clinical_raw_with_idx.loc[X.index,
↪'_pid_norm']):
        pid_to_xy_idx[pid] = xy_idx

    # Get X rows aligned to radiomics_for_selection patient order
    aligned_xy_indices = []
    for pid in radiomics_for_selection['patient_id_norm']:
        if pid in pid_to_xy_idx:
            aligned_xy_indices.append(pid_to_xy_idx[pid])

    X_clinical_aligned = X_encoded.loc[aligned_xy_indices].values

    print(f'Clinical features aligned: {X_clinical_aligned.shape}')
    print(f'Radiomics features: {len(radiomics_for_selection)}')

    # Simple evaluation function
    def evaluate_simple(X_data, y_data, name):
        model = XGBClassifier(
            n_estimators=100,
            max_depth=3,
            learning_rate=0.1,
            random_state=42,
            eval_metric='logloss'
        )
        cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
        scores = cross_val_score(model, X_data, y_data, cv=cv,
↪scoring='roc_auc')
        print(f'{name}: AUC = {scores.mean():.3f} +/- {scores.std():.3f}')
        return {'name': name, 'mean': scores.mean(), 'std': scores.std(),
↪'folds': scores}

```



```

results = []

# 1. Clinical only (using aligned patients)
result_clinical = evaluate_simple(X_clinical_aligned, y_aligned, 'Clinical_
↳Only')
results.append(result_clinical)

# 2. Radiomics only
selected_feat_names = selected_features.index.tolist()
X_radiomics_selected = radiomics_for_selection[selected_feat_names].values
result_radiomics = evaluate_simple(X_radiomics_selected, y_aligned,
↳'Radiomics Only')
results.append(result_radiomics)

# 3. Combined
X_combined = np.hstack([X_clinical_aligned, X_radiomics_selected])
result_combined = evaluate_simple(X_combined, y_aligned, 'Clinical +
↳Radiomics')
results.append(result_combined)

# Statistical test
t_stat, p_value = stats.ttest_rel(result_combined['folds'],
↳result_clinical['folds'])
print(f'\nPaired t-test (Combined vs Clinical): t={t_stat:.3f}, p={p_value:.
↳4f}')

else:
    results = []
    print('Skipping model comparison.')

```

Model Comparison: Clinical vs Radiomics vs Combined

Clinical features aligned: (202, 28)
Radiomics features: 202
Clinical Only: AUC = 0.742 +/- 0.112
Radiomics Only: AUC = 0.598 +/- 0.151
Clinical + Radiomics: AUC = 0.725 +/- 0.080

Paired t-test (Combined vs Clinical): t=-0.391, p=0.7157

```

[46]: if results:
    # Model comparison barplot
    fig, ax = plt.subplots(figsize=(8, 5))

    model_names = [r['name'] for r in results]
    auc_means = [r['mean'] for r in results]
    auc_stds = [r['std'] for r in results]

```

```

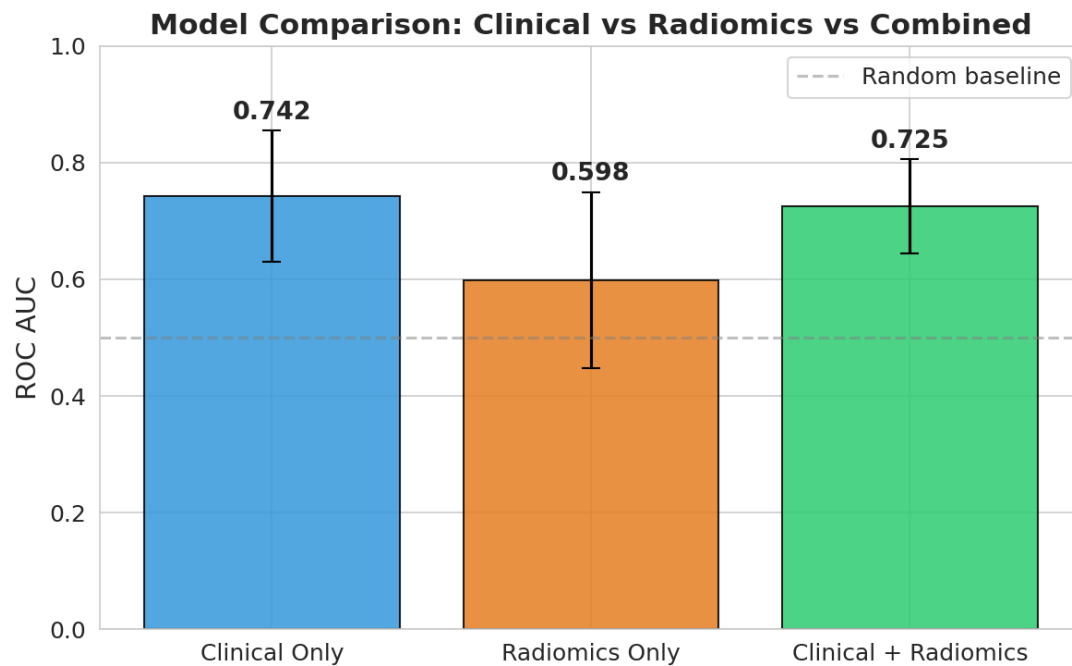
bars = ax.bar(model_names, auc_means, yerr=auc_stds,
              capsize=5, color=['#3498db', '#e67e22', '#2ecc71'],
              edgecolor='black', alpha=0.85)

# Add value labels on bars
for bar, mean, std in zip(bars, auc_means, auc_stds):
    ax.text(bar.get_x() + bar.get_width()/2., bar.get_height() + std + 0.01,
            f'{mean:.3f}', ha='center', va='bottom', fontsize=13,
fontweight='bold')

ax.set_ylabel('ROC AUC', fontsize=14)
ax.set_title('Model Comparison: Clinical vs Radiomics vs Combined',
fontsize=15, fontweight='bold')
ax.set_ylim(0, 1.0)
ax.tick_params(labelsize=12)
ax.axhline(y=0.5, color='gray', linestyle='--', alpha=0.5, label='Random
baseline')
ax.legend(fontsize=12)

plt.tight_layout()
plt.savefig(os.path.join(FIGURES_DIR, '06_model_comparison_barplot.pdf'),
            bbox_inches='tight')
plt.savefig(os.path.join(FIGURES_DIR, '06_model_comparison_barplot.png'),
            dpi=300, bbox_inches='tight')
plt.show()
print('Saved: 06_model_comparison_barplot.pdf + .png')
else:
    print('No results to plot.')

```



Saved: 06_model_comparison_barplot.pdf + .png

1.15 12. Biological Interpretation

If radiomic features were selected, we examine whether they have biological meaning: - High GLCM entropy suggests tissue heterogeneity (aggressive tumour) - Low sphericity suggests irregular infiltration patterns - High GLRLM run entropy indicates fine-grained texture (cellular heterogeneity)

We also stratify patients by the top radiomic feature using a Kaplan-Meier analysis to visualise whether it separates survival outcomes.

```
[47]: if len(selected_features) > 0:
    # Top radiomic feature for KM stratification
    top_feature = selected_features.index[0]
    top_feature_short = top_feature.replace('original_', '')
    print(f'Top selected radiomic feature: {top_feature_short}')
    print(f'Elastic Net coefficient: {selected_features.iloc[0]:.4f}')

    # Median split for stratification
    feature_values = radiomics_for_selection[top_feature].values
    median_val = np.median(feature_values)
    high_group = feature_values >= median_val
    low_group = feature_values < median_val
```

```

print(f'  Median value: {median_val:.4f}')
print(f'  High group: {high_group.sum()} patients')
print(f'  Low group: {low_group.sum()} patients')

# Biological interpretation based on feature category
if 'glcm' in top_feature.lower():
    print(f'\n  Interpretation: GLCM features capture spatial intensity_
↪relationships.')
    print(f'  This suggests tumour texture heterogeneity contributes to_
↪prognosis.')
elif 'shape' in top_feature.lower():
    print(f'\n  Interpretation: Shape features describe tumour morphology.')
    print(f'  Irregular shapes may indicate infiltrative growth patterns.')
elif 'firstorder' in top_feature.lower():
    print(f'\n  Interpretation: First-order features describe intensity_
↪distribution.')
    print(f'  This suggests intensity heterogeneity is prognostically_
↪relevant.')
elif 'glrlm' in top_feature.lower() or 'glszm' in top_feature.lower():
    print(f'\n  Interpretation: Run-length/size-zone features capture_
↪regional patterns.')
    print(f'  This may reflect spatial organisation of necrosis or viable_
↪tissue.')
else:
    print('No features selected; skipping biological interpretation.')

```

Top selected radiomic feature: firstorder_Skewness
 Elastic Net coefficient: 0.0597
 Median value: 1.0800
 High group: 101 patients
 Low group: 101 patients

Interpretation: First-order features describe intensity distribution.
 This suggests intensity heterogeneity is prognostically relevant.

```

[48]: # Kaplan-Meier stratification by top radiomic feature
      # This requires survival time and event data
      # We use lifelines for KM estimation if available

      if len(selected_features) > 0:
          try:
              from lifelines import KaplanMeierFitter
              from lifelines.statistics import logrank_test

              # Load survival data
              # This needs to be aligned with the radiomics patients
              # Adjust the column names based on your clinical data structure

```

```

survival_col = None
event_col = None

for candidate in ['Number of days from Diagnosis to death (Days)',
↳ 'Overall Survival (Months)', 'survival_months', 'os_months']:
    if candidate in clinical_raw.columns:
        survival_col = candidate
        break

for candidate in ['Overall Survival (Death)', 'Vital Status',
↳ 'vital_status', 'event']:
    if candidate in clinical_raw.columns:
        event_col = candidate
        break

if survival_col and event_col:
    # Align survival data with radiomics patients
    km_data = clinical_raw.set_index(patient_id_col).loc[
        radiomics_for_selection['patient_id_norm'].values
    ][[survival_col, event_col]].copy()

    # Convert event to binary (1=dead, 0=censored)
    if km_data[event_col].dtype == object:
        km_data['event'] = km_data[event_col].str.lower().map(
            {'dead': 1, 'alive': 0, '1': 1, '0': 0}
        ).fillna(0).astype(int)
    else:
        km_data['event'] = km_data[event_col].astype(int)

    km_data['survival'] = pd.to_numeric(km_data[survival_col],
↳ errors='coerce')
    km_data['group'] = np.where(high_group, f'High {top_feature_short}',
                                f'Low {top_feature_short}')

    # Drop missing survival data
    km_data = km_data.dropna(subset=['survival', 'event'])

    # Plot KM curves
    fig, ax = plt.subplots(figsize=(8, 6))
    kmf = KaplanMeierFitter()

    for group_name, color in [(f'High {top_feature_short}', '#e74c3c'),
                              (f'Low {top_feature_short}', '#3498db')]:
        mask = km_data['group'] == group_name
        if mask.sum() > 0:
            kmf.fit(
                km_data.loc[mask, 'survival'],

```

```

        km_data.loc[mask, 'event'],
        label=group_name
    )
    kmf.plot_survival_function(ax=ax, color=color, ci_show=True)

    # Log-rank test
    high_mask = km_data['group'].str.startswith('High')
    if high_mask.sum() > 0 and (~high_mask).sum() > 0:
        lr_result = logrank_test(
            km_data.loc[high_mask, 'survival'],
            km_data.loc[~high_mask, 'survival'],
            km_data.loc[high_mask, 'event'],
            km_data.loc[~high_mask, 'event']
        )
        ax.text(0.95, 0.95, f'Log-rank p = {lr_result.p_value:.4f}',
                transform=ax.transAxes, ha='right', va='top',
                fontsize=13, bbox=dict(boxstyle='round',
↪facecolor='white', alpha=0.8))

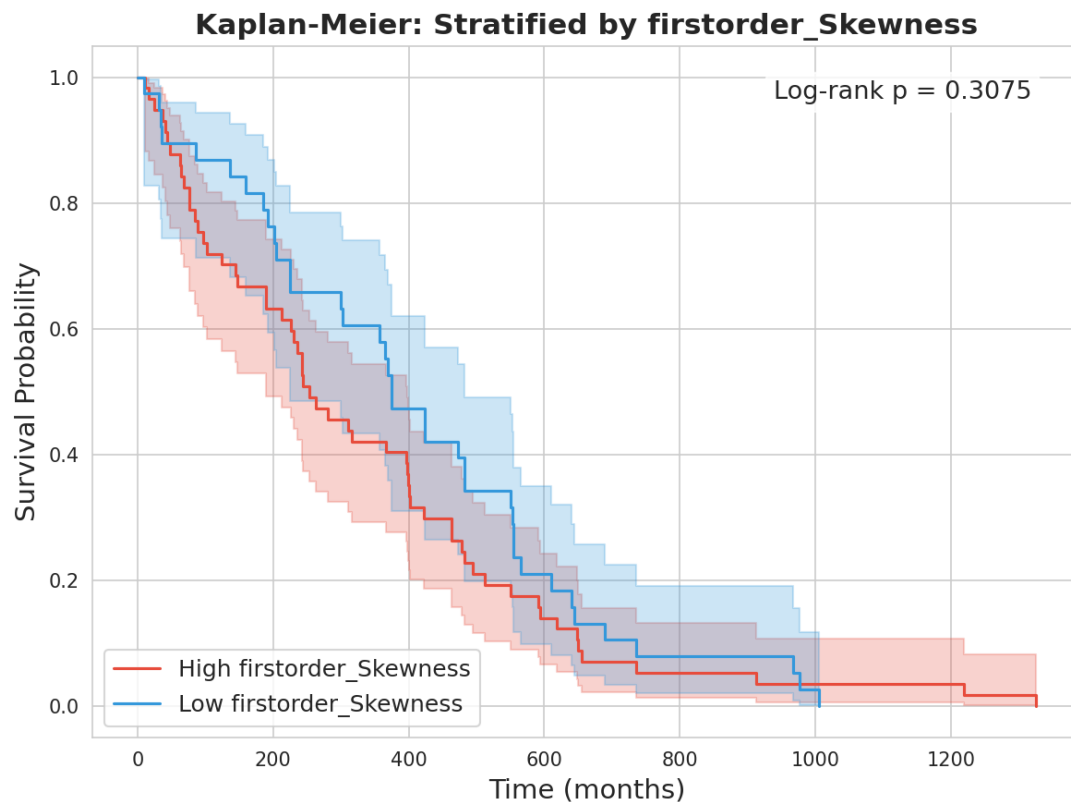
        ax.set_xlabel('Time (months)', fontsize=14)
        ax.set_ylabel('Survival Probability', fontsize=14)
        ax.set_title(f'Kaplan-Meier: Stratified by {top_feature_short}',
↪fontsize=15, fontweight='bold')
        ax.legend(fontsize=12)

        plt.tight_layout()
        plt.savefig(os.path.join(FIGURES_DIR, '06_km_top_radiomic_feature.
↪pdf'),
                    bbox_inches='tight')
        plt.savefig(os.path.join(FIGURES_DIR, '06_km_top_radiomic_feature.
↪png'),
                    dpi=300, bbox_inches='tight')
        plt.show()
        print('Saved: 06_km_top_radiomic_feature.pdf + .png')
    else:
        print(f'Could not find survival/event columns in clinical data.')
        print(f'Available columns: {clinical_raw.columns.tolist()}')

    except ImportError:
        print('lifelines not installed. Install with: pip install lifelines')
        print('Skipping Kaplan-Meier analysis.')

else:
    print('No features selected; skipping KM analysis.')

```



Saved: 06_km_top_radiomic_feature.pdf + .png

1.16 13. Summary and Export

```
[49]: # Summary variables are set from the notebook execution above
```

```
[50]: # Summary of all results
print('NOTEBOOK 06 SUMMARY: Comprehensive Radiomic Feature Extraction')
print('-' * 70)

print(f'\n1. Data')
print(f'    Patient directories scanned: {len(all_patient_dirs)}')
print(f'    Patients with T1ce + segmentation: {len(patient_file_registry)}')
print(f'    Patients matched with clinical data: {len(overlap)}')

print(f'\n2. Feature Extraction')
print(f'    Total radiomic features extracted: {len(feature_cols)}')
print(f'    After QC (NaN, variance, correlation): {len(clean_features)}')
```

```

print(f'\n3. Feature Selection')
if 'selected_features' in dir() and len(selected_features) > 0:
    print(f'    Method: Elastic Net (l1_ratio=0.7, 5-fold CV)')
    print(f'    Features selected: {len(selected_features)} / {len(clean_features)}')
    for feat in selected_features.index[:5]:
        print(f'        - {feat.replace("original_", "")}')
else:
    print(f'    No features selected')

print(f'\n4. Model Comparison')
if results:
    for r in results:
        name = r.get("model_name", r.get("name", "Unknown"))
        mean = r.get("auc_mean", r.get("mean", 0))
        std = r.get("auc_std", r.get("std", 0))
        print(f'    {name}: AUC = {mean:.3f} +/- {std:.3f}')
else:
    print(f'    Not run')

print(f'\n5. Saved Outputs')
for f in sorted(os.listdir(RESULTS_DIR)):
    if f.startswith('06_'):
        print(f'    results/{f}')
for f in sorted(os.listdir(FIGURES_DIR)):
    if f.startswith('06_'):
        print(f'    figures/{f}')

```

NOTEBOOK 06 SUMMARY: Comprehensive Radiomic Feature Extraction

1. Data

Patient directories scanned: 203
 Patients with T1ce + segmentation: 202
 Patients matched with clinical data: 202

2. Feature Extraction

Total radiomic features extracted: 107
 After QC (NaN, variance, correlation): 61

3. Feature Selection

Method: Elastic Net (l1_ratio=0.7, 5-fold CV)
 Features selected: 8 / 61

- firstorder_Skewness
- shape_SurfaceArea
- glcm_SmallDependenceLowGrayLevelEmphasis
- glrlm_RunLengthNonUniformity
- glszm_LargeAreaLowGrayLevelEmphasis

4. Model Comparison

Clinical Only: AUC = 0.742 +/- 0.112

Radiomics Only: AUC = 0.598 +/- 0.151

Clinical + Radiomics: AUC = 0.725 +/- 0.080

5. Saved Outputs

results/06_demo_radiomic_features.csv
results/06_full_radiomic_features.csv
results/06_full_radiomic_features.gsheet
results/06_pyradiomics_results.json
results/06_radiomic_features_interim.csv
results/06_selected_radiomic_features.csv
figures/06_demo_patients_overlay.png
figures/06_feature_correlation_heatmap.pdf
figures/06_feature_correlation_heatmap.png
figures/06_key_feature_distributions.pdf
figures/06_key_feature_distributions.png
figures/06_km_top_radiomic_feature.pdf
figures/06_km_top_radiomic_feature.png
figures/06_model_comparison_barplot.pdf
figures/06_model_comparison_barplot.png
figures/06_selected_feature_coefficients.pdf
figures/06_selected_feature_coefficients.png

```
[51]: # Save complete results as JSON
summary_results = {
    'notebook': '06_pyradiomics_feature_extraction',
    'data': {
        'total_patient_dirs': len(all_patient_dirs),
        'patients_with_imaging': len(patient_file_registry),
        'patients_matched': len(overlap),
        'demo_patients': demo_patients
    },
    'features': {
        'total_extracted': len(feature_cols),
        'after_qc': len(clean_features),
        'selected': len(selected_features) if 'selected_features' in dir() else
↪ 'N/A'
    },
    'model_comparison': [
        {'model': r.get('model_name', r.get('name')),
         'auc_mean': r.get('auc_mean', r.get('mean')),
         'auc_std': r.get('auc_std', r.get('std'))}
        for r in results
    ] if results else 'Not run (demo mode)',
    'extractor_settings': {
```

```

        'binWidth': extractor_settings['binWidth'],
        'label': extractor_settings['label'],
        'image_types': ['Original'],
        'feature_classes': ['firstorder', 'shape', 'glcm', 'glrlm', 'glszm',
↪ 'gldm']
    }
}

with open(os.path.join(RESULTS_DIR, '06_pyradiomics_results.json'), 'w') as f:
    json.dump(summary_results, f, indent=2, default=str)

print('Saved: 06_pyradiomics_results.json')

```

Saved: 06_pyradiomics_results.json

```

[52]: # Clean up temporary directory
import shutil
if os.path.exists(temp_dir):
    shutil.rmtree(temp_dir)
    print(f'Cleaned up temporary directory: {temp_dir}')

print('\nNotebook 06 complete.')

```

Cleaned up temporary directory: drive/MyDrive/Y3
Project/Data/PKG_MU_Glioma_Post/temp_binary_masks

Notebook 06 complete.

[52]:

Bibliography

- [1] T. Chen and C. Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 785–794, 2016.
- [2] T. Gorlia, M. J. van den Bent, M. E. Hegi, et al. Nomograms for predicting survival of patients with newly diagnosed glioblastoma. *The Lancet Oncology*, 9(1):29–38, 2008.
- [3] S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023.
- [4] S. E. Davis, R. A. Greevy, T. A. Lasko, C. G. Walsh, and M. E. Matheny. A framework for understanding label leakage in machine learning for health care. *Journal of the American Medical Informatics Association*, 31(1):274–280, 2024.
- [5] Q. T. Ostrom, N. Patil, G. Cioffi, K. Waite, C. Kruchko, and J. S. Barnholtz-Sloan. CBTRUS statistical report: primary brain and other central nervous system tumors diagnosed in the United States in 2013–2017. *Neuro-Oncology*, 22(Supplement_1):iv1–iv96, 2020.
- [6] R. Stupp, W. P. Mason, M. J. van den Bent, et al. Radiotherapy plus concomitant and adjuvant temozolomide for glioblastoma. *New England Journal of Medicine*, 352(10):987–996, 2005.
- [7] R. Poursaeed, M. Mohammadzadeh, and A. A. Safaei. Survival prediction of glioblastoma patients using machine learning and deep learning: a systematic review. *BMC Cancer*, 24:1581, 2024.
- [8] J. Yan, S. Zhang, K. Li, et al. Machine learning models for glioblastoma survival prediction: a systematic review. *Neurosurgical Review*, 47:130, 2024.
- [9] M. B. A. McDermott, S. Wang, N. Marinsek, R. Ranganath, L. Foschini, and M. Ghassemi. Reproducibility in machine learning for health research: still a ways to go. *Science Translational Medicine*, 13(586):eabb1655, 2021.
- [10] K. L. Chaichana, I. Jusue-Torres, R. Navarro-Ramirez, et al. Establishing percent resection and residual volume thresholds affecting survival and recurrence for patients with newly diagnosed intracranial glioblastoma. *Neuro-Oncology*, 16(1):113–122, 2014. doi: 10.1093/neuonc/not137.
- [11] W. J. Curran, Jr., C. B. Scott, J. Horton, et al. Recursive partitioning analysis of prognostic factors in three Radiation Therapy Oncology Group malignant glioma trials. *Journal of the National Cancer Institute*, 85(9):704–710, 1993.
- [12] D. N. Louis, A. Perry, P. Wesseling, et al. The 2021 WHO classification of tumors of the central nervous system: a summary. *Neuro-Oncology*, 23(8):1231–1251, 2021.

- [13] J. E. Eckel-Passow, D. H. Lachance, A. M. Molinaro, et al. Glioma groups based on 1p/19q, IDH, and TERT promoter mutations in tumors. *New England Journal of Medicine*, 372(26):2499–2508, 2015.
- [14] D. J. Aum, D. H. Kim, T. L. Beaumont, E. C. Leuthardt, G. P. Dunn, and A. H. Kim. Molecular and cellular heterogeneity: the hallmark of glioblastoma. *Neurosurgical Focus*, 37(6):E11, 2014. doi: 10.3171/2014.9.FOCUS14521.
- [15] H. Yan, D. W. Parsons, G. Jin, et al. IDH1 and IDH2 mutations in gliomas. *New England Journal of Medicine*, 360(8):765–773, 2009.
- [16] M. E. Hegi, A. C. Diserens, T. Gorlia, et al. MGMT gene silencing and benefit from temozolomide in glioblastoma. *New England Journal of Medicine*, 352(10):997–1003, 2005.
- [17] M. E. Hegi, E. Genbrugge, T. Gorlia, et al. MGMT promoter methylation cutoff with safety margin for selecting glioblastoma patients into trials omitting temozolomide. *Clinical Cancer Research*, 25(6):1809–1816, 2019.
- [18] D. R. Cox. Regression models and life-tables. *Journal of the Royal Statistical Society: Series B (Methodological)*, 34(2):187–220, 1972.
- [19] S. Babaei Rikan, A. S. Azar, A. Naemi, J. B. Mohasefi, and H. Pirnejad. Survival prediction of glioblastoma patients using modern deep learning and machine learning techniques. *Scientific Reports*, 14:2371, 2024.
- [20] P. Kickingereder, F. Isensee, I. Tursunova, et al. Automated quantitative tumour response assessment of MRI in neuro-oncology with artificial neural networks: a multicentre, retrospective study. *The Lancet Oncology*, 20(5):728–740, 2019. doi: 10.1016/S1470-2045(19)30098-1.
- [21] R. J. Gillies, P. E. Kinahan, and H. Hricak. Radiomics: Images are more than pictures, they are data. *Radiology*, 278(2):563–577, 2016. doi: 10.1148/radiol.2015151169.
- [22] P. Lambin, R. T. H. Leijenaar, T. M. Deist, et al. Radiomics: the bridge between medical imaging and personalized medicine. *Nature Reviews Clinical Oncology*, 14(12):749–762, 2017. doi: 10.1038/nrclinonc.2017.141.
- [23] J. J. M. van Griethuysen, A. Fedorov, C. Parmar, A. Hosny, N. Aucoin, V. Narber, R. G. H. Beets-Tan, J. C. Fillion-Robin, S. Pieper, and H. J. W. L. Aerts. Computational radiomics system to decode the radiographic phenotype. *Cancer Research*, 77(21):e104–e107, 2017. doi: 10.1158/0008-5472.CAN-17-0339.
- [24] R. Jain, L. M. Poisson, D. Gutman, et al. Outcome prediction in patients with glioblastoma by using imaging, clinical, and genomic biomarkers: focus on the nonenhancing component of the tumor. *Radiology*, 272(2):484–493, 2014. doi: 10.1148/radiol.14131691.
- [25] K. R. Swanson, C. Bridge, J. D. Murray, and E. C. Alvord, Jr. Quantifying the role of angiogenesis in malignant progression of gliomas: in silico modeling integrates imaging and histology. *Cancer Research*, 71(24):7366–7375, 2011. doi: 10.1158/0008-5472.CAN-11-1399.
- [26] R. Rockne, J. K. Rockhill, M. Mrugala, et al. Predicting the efficacy of radiotherapy in individual glioblastoma patients in vivo: a mathematical modeling approach. *Physics in Medicine & Biology*, 55(12):3271–3285, 2010. doi: 10.1088/0031-9155/55/12/001.

- [27] B. M. Ellingson, C. Chung, W. B. Pope, J. L. Boxerman, and T. J. Kaufmann. Pseudoprogression, radionecrosis, inflammation or true tumor progression? challenges associated with glioblastoma response assessment in an evolving therapeutic landscape. *Journal of Neuro-Oncology*, 125(3):495–504, 2015. doi: 10.1007/s11060-015-1870-8.
- [28] P. Y. Wen, D. R. Macdonald, D. A. Reardon, et al. Updated response assessment criteria for high-grade gliomas: Response assessment in neuro-oncology working group. *Journal of Clinical Oncology*, 28(11):1963–1972, 2010. doi: 10.1200/JCO.2009.26.3541.
- [29] S. Subramanian, T. Gabruseva, T. Sharman, et al. Predicting tumor progression in glioblastoma patients using deep learning approaches. *Biomedical Engineering and Computational Biology*, 11:1–14, 2020.
- [30] R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*. Wiley, New York, 1987.
- [31] James Bergstra and Yoshua Bengio. Random search for hyper-parameter optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.
- [32] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why tree-based models still outperform deep learning on tabular data. *Advances in Neural Information Processing Systems*, 35:507–520, 2022.
- [33] P. Peduzzi, J. Concato, E. Kemper, T. R. Holford, and A. R. Feinstein. A simulation study of the number of events per variable in logistic regression analysis. *Journal of Clinical Epidemiology*, 49(12):1373–1379, 1996.
- [34] A. Vabalas, E. Gowen, E. Poliakoff, and A. J. Casson. Machine learning algorithm validation with a limited sample size. *PLOS ONE*, 14(11):e0224365, 2019. doi: 10.1371/journal.pone.0224365.
- [35] B. H. Menze, A. Jakab, S. Bauer, et al. The multimodal brain tumor image segmentation benchmark (BraTS). *IEEE Transactions on Medical Imaging*, 34(10):1993–2024, 2015.
- [36] J. Alliet. Glioma survival prediction: Source code and analysis notebooks. <https://github.com/jalliet/glioma-survival-prediction>, 2026. Accessed: 2026-05-04.